

IBM Z NetView
Version 6 Release 3

IP Management



Note

Before using this information and the product it supports, read the information in [“Notices” on page 161.](#)

This edition applies to version 6, release 3 of IBM Z NetView (product number 5697-NV6) and to all subsequent versions, releases, and modifications until otherwise indicated in new editions.

This edition replaces SC27-2855-02.

© **Copyright International Business Machines Corporation 2009, 2019.**

US Government Users Restricted Rights – Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Contents

Figures.....	vii
About this publication.....	xiii
Intended audience.....	xiii
Publications.....	xiii
IBM Z NetView library.....	xiii
Related publications	xiv
Terminology in this Library.....	xv
Using IBM Z NetView online help.....	xv
Accessing publications online.....	xv
Ordering publications	xvi
Accessibility	xvi
Tivoli user groups.....	xvi
Support information.....	xvi
Conventions used in this publication.....	xvi
Typeface conventions	xvii
Operating system-dependent variables and paths.....	xvii
Syntax diagrams.....	xvii
 Part 1. Introduction to IP Network Management.....	 1
Chapter 1. IBM Z NetView IP Management Overview.....	3
 Part 2. Sysplex Management.....	 9
Chapter 2. Sysplex Management Overview.....	11
Chapter 3. Managing the NetView Program as a Sysplex Application By Using XCF.....	13
Master NetView Program and NetView XCF Groups.....	13
Master-capable and Basic NetView Programs.....	13
Master Role Takeover.....	14
User-defined XCF Groups.....	15
Commands and Pipe Stages for XCF.....	16
DVIPA Support for XCF.....	16
Turning Off XCF Support.....	16
Chapter 4. Managing the Enterprise by Using the NetView Program.....	19
Chapter 5. Managing Resources That are Discovered by the Discovery Manager.....	21
Using Multiple NetView Programs on a Single Z Image.....	22
Using 3270 Commands.....	22
Using NetView Enterprise Management Agent Workspaces.....	23
HyperSockets Configuration and Status Workspace.....	23
NetView Applications Workspace.....	24
OSA Channels and Ports Workspace.....	25
Stack Configuration and Status Workspace.....	26
Telnet Server Configuration and Status Workspace.....	27
Using NetView Management Console Topology.....	28
Sysplex and z Systems Objects.....	29
Viewing Sysplex Topology.....	32
Using z Systems Topology.....	38

Chapter 6. Managing Dynamic Virtual IP Addresses.....	45
Updating DVIPA Information by Using Events	45
Using Distributed DVIPA Statistics.....	46
Using 3270 Commands.....	48
Using NetView Enterprise Management Agent Workspaces.....	49
Distributed DVIPA Connection Routing Workspace.....	49
Distributed DVIPA Server Health Workspace.....	50
Distributed DVIPA Targets Workspace.....	51
DVIPA Connections Workspace.....	52
DVIPA Definition and Status Workspace.....	53
DVIPA Sysplex Distributors Workspace.....	54
VIPA Routes Workspace.....	55
DVIPA Stack Summary Workspace.....	56

Part 3. IP Management Functions.....59

Chapter 7. Controlling and Viewing IP Traces.....	61
Defining Packet Trace Data Collection.....	61
Controlling, Collecting, and Formatting Packet Trace Data.....	61
Running IP Traces.....	62
Issuing the IPTRACE Command.....	62
IP Packet Tracing.....	63
OSA Packet Tracing.....	77
Component Tracing.....	80
Chapter 8. Managing Connections.....	85
Collecting Connection Data.....	85
Viewing Connection Data.....	86
Viewing Encryption Data.....	86
Monitoring Connections.....	87
Checking Connection Status (IPSTAT)	88
Displaying the Connections for a Stack.....	88
Displaying Connection Information.....	90
Determining If a Connection Is Stopped.....	91
Setting Connection Management Filters.....	91
Launching IP Packet Trace.....	93
Chapter 9. Discovering IP Network Topology with the MultiSystem Manager IBM Tivoli Network Manager Agent.....	95
IP View Objects.....	95
Finding Resources.....	97
Navigating Network Views.....	98
Viewing IP Networks.....	98
IP Networks View.....	99
User-defined Views.....	100
IP Subnetwork Views.....	102
Router, Bridge, Hub, Host, and Switch Views.....	103
Chapter 10. Monitoring and Managing IP Resources.....	105
Using PING.....	105
Using TRACERTE.....	106
Using SNMP Management.....	107
Using the Get Command.....	108
Using the Set Command.....	109
Using the Walk Command.....	109
Using the Group Command.....	110

Using Extended SNMP Groups.....	112
Using IP Resource Management.....	113
IP Resource Management Main Panel.....	113
IP Resource Management Add Panel.....	114
IP Resource Management Change Panel.....	115
IP Resource Management Filters Panel.....	116
IP Resource Management Command Window.....	117
Monitoring Critical Ports.....	117
Managing SNA over IP.....	118
 Chapter 11. Basic IP Services.....	119
IPLOG.....	119
PING.....	119
REXEC.....	119
RMTCMD.....	119
RSH.....	119
SNMP.....	120
SOCKET.....	120
TN3270.....	120
TRACERTE.....	120
 Part 4. Applications for IP Network Management.....	121
 Chapter 12. Handling Events.....	123
Handling SNMP Traps.....	123
Sending SNMP Traps.....	123
Receiving SNMP Traps.....	123
Handling Event Integration Facility Events.....	124
Sending EIF Events.....	124
Receiving EIF Events.....	124
Handling SNA Alerts.....	124
Sending SNA Alerts.....	124
Receiving SNA Alerts.....	125
Responding to Intrusions.....	125
 Chapter 13. Using Z NetView Socket.....	127
Using the SOCKET Interface.....	127
Initializing a Socket Interface.....	127
Ending a Socket Interface.....	128
Making Synchronous and Asynchronous Requests.....	128
Making Global and Socket-specific Requests.....	128
Retrieving Information.....	129
Waiting for Activity on Sockets.....	130
Handling TCP/IP Errors.....	131
Canceling Outstanding Asynchronous Requests.....	131
Giving and Taking Sockets.....	132
Using the SOCKET Command to Create TCP/IP Applications.....	133
Creating Datagram Applications.....	133
Creating Stream Applications.....	137
 Chapter 14. Verifying and Converting IP Addresses.....	145
IPXLATE() REXX Function.....	145
IPXLATE() Example: Verification.....	145
IPXLATE() Example: Conversion of One Presentation Form to Another.....	146
Cnmipxl C Macro.....	147
Cnmipxl Example: Verification.....	147
Cnmipxl Examples: Conversion of One Presentation Form to Another.....	147

Cnmipxl Examples: Conversion of Presentation Form to Binary.....	148
Cnmipxl Examples: Conversion of Binary to Presentation Form.....	149
CNMIPXLATE NetView PL/I Macro.....	151
CNMIPXLATE Example: Verification.....	151
CNMIPXLATE Examples: Conversion of One Presentation Form to Another.....	151
CNMIPXLATE Examples: Conversion of Presentation Form to Binary.....	152
CNMIPXLATE Examples: Conversion of Binary to Presentation Form.....	153
Chapter 15. Using IP Servers.....	155
Appendix A. Discovery Manager Data.....	157
Appendix B. Customizing the SNMP Group Definition File (FKXSNMP).....	159
Notices.....	161
Programming Interfaces.....	162
Trademarks.....	162
Privacy policy considerations.....	163
Index.....	165

Figures

1. Sysplex Configuration.....	14
2. Sysplex Configuration after Master Takeover.....	15
3. NVC as an Enterprise Master NetView Program.....	19
4. HiperSockets Configuration and Status Workspace.....	24
5. NetView Applications Workspace.....	25
6. OSA Channels and Ports Workspace.....	26
7. Stack Configuration and Status Workspace.....	27
8. Telnet Server Configuration and Status Workspace.....	28
9. Network Views View.....	29
10. Types of SysplexView and SystemzView Objects.....	31
11. SysplexView View.....	32
12. NETVPLEX More Detail Logical View.....	33
13. TIVMVS21 More Detail Logical View.....	33
14. STACKS More Detail Logical View.....	34
15. TCPIPB More Detail Logical View.....	34
16. NETVIEWS More Detail Logical View.....	35
17. TELNETSERVERS More Detail Logical View.....	36
18. TN3270B.PORTS More Detail Logical View.....	36
19. NETVPLEX.TIVMVS21.TN3270B.823 (Port Peers) View.....	37
20. NETVPLEX-LOG (Sysplex Logical Only) View.....	37
21. EZBTCPCS More Detail Logical View.....	38
22. SystemzView View.....	39
23. VM-TOKEN More Detail Physical View.....	39

24. CSS01 More Detail Physical View.....	40
25. TIVMVS21 More Detail Physical View (z/OS Image Aggregate).....	40
26. TIVMVS21 More Detail Physical View (IP Stack Aggregate).....	41
27. TCPIP More Detail Physical View.....	41
28. TCPIPLINK More Detail Physical View.....	42
29. OSA00B0 More Detail Physical View.....	42
30. PORT0.RP563B0 More Detail Physical View.....	43
31. PORT0.RP63B0-PAR View.....	43
32. IQDIO1 More Detail Physical View.....	44
33. HIPERSOCKETSD1 More Detail Physical View.....	44
34. Distributed DVIPA Connection Routing Workspace.....	50
35. Distributed DVIPA Server Health Workspace.....	51
36. Distributed DVIPA Targets Workspace.....	52
37. DVIPA Connections Workspace.....	53
38. DVIPA Definition and Status Workspace.....	54
39. DVIPA Sysplex Distributors Workspace.....	55
40. VIPA Routes Workspace.....	56
41. DVIPA Stack Summary Workspace.....	57
42. TCP/IP IPTrace Control Center Panel.....	62
43. IPTrace Control Center Panel.....	63
44. Packet Trace Control Panel.....	64
45. PKTTRACE Control Panel.....	65
46. NetView PKTS Management Panel for IP Packet Trace.....	66
47. Display Packet Control Panel.....	66
48. PKTTRACE SUMMARY Panel.....	67

49. PKTTRACE Summary - Save Packets Panel.....	68
50. Display Packet Control - Save Packets panel.....	68
51. Packet Trace Analysis Panel.....	69
52. Packet Trace Analysis TCP Sessions Panel.....	70
53. Session Analysis Panel.....	71
54. Session Analysis Packets Panel.....	71
55. Packet Detail for the Packet Selected in Session Analysis Packets.....	72
56. Session Analysis - Actions Panel.....	73
57. Packet Trace Analysis UDP Sessions Panel.....	74
58. UDP Session Report (Packet Detail for the Selected UDP Session).....	75
59. Saved Packet Traces Panel.....	76
60. Saved Packet Trace Details Panel (Viewing Saved Data).....	76
61. FMTPACKET Session Detail Report.....	77
62. OSATRACE Control Panel.....	77
63. OSATRACE Filters Panel.....	78
64. NetView PKTS Management Panel for OSA Packet Trace.....	79
65. Display OSA Packet Control Panel.....	79
66. OSA TRACE PACKETS SUMMARY Panel.....	80
67. Packet Detail for the Selected OSA Packet.....	80
68. CTRACE Control Panel (FKXK2A12).....	81
69. CTRACE Control Panel (FKXK2A11).....	82
70. CTRACE Control Panel (FKXK2A1A).....	82
71. CTRACE Control Panel (FKXK2A14).....	83
72. CTRACE Control Panel (FKXK2A10).....	84
73. Connection Status: Main Panel.....	88

74. Connection Management: Selected Stacks.....	89
75. Connection Management: Connections for Selected Stack.....	90
76. Connection Management: Connection Information and Commands.....	90
77. Connection Details Output.....	91
78. Connection Management Filters Panel.....	92
79. Connection Management: Filtered Connections.....	92
80. TCP/IP for z/OS Connection Management Panel (FKXK2221).....	93
81. Display Packet Control Panel (FKXK2A24).....	94
82. Types of IP View Objects for the IBM Tivoli Network Manager Agent.....	97
83. NetView Management Console Default Network View (MultiSysView): ITNM_IP_Networks.....	98
84. ITNM_IP_Networks IP Networks View.....	99
85. ITNM_IP_Networks User-Defined View: NCOMS Object.....	100
86. ITNM_IP_Networks User-Defined View: Subnets and Device_Classes Objects.....	101
87. ITNM_IP_Networks User-Defined View: fe80___/10 and 9.0.0.0_/8 Objects.....	101
88. ITNM_IP_Networks User-Defined View: 9.0.0.0/255.0.0.0 Object.....	102
89. ITNM_IP_Networks IP Subnetwork View.....	103
90. ITNM_IP_Networks Host View.....	104
91. NetView IP Management Functions Menu Panel.....	105
92. TCP/IP Automation: Ping Panel.....	106
93. TCP/IP Automation: Trace Route Panel.....	107
94. TCP/IP Automation: Trace Route Output.....	107
95. TCP/IP SNMP Menu Panel.....	108
96. TCP/IP SNMP Commands Panel: Get Command.....	108
97. TCP/IP SNMP Commands Panel: Options.....	109
98. TCP/IP SNMP Commands Panel: Set Command.....	109

99. TCP/IP SNMP Commands Panel: Walk Command.....	110
100. TCP/IP SNMP Groups Panel.....	110
101. TCP/IP SNMP Groups Description Panel: LIST.....	111
102. TCP/IP SNMP Groups Description Panel: WALK.....	111
103. TCP/IP SNMP Groups Description Panel: TABLE.....	111
104. TCP/IP SNMP Group Extensions Panel.....	112
105. SNMP GET to LOCAL Output.....	112
106. TCP/IP Resource Management Main Panel.....	113
107. TCP/IP Resource Management Add Panel.....	115
108. TCP/IP Resource Management Change Panel.....	116
109. IP Resource Management Filters Panel.....	116
110. TCP/IP Resource Management Panel: Commands Window.....	117

About this publication

The IBM Z® NetView® product provides advanced capabilities that you can use to maintain the highest degree of availability of your complex, multi-platform, multi-vendor networks and systems from a single point of control. This publication, *IBM Z NetView IP Management*, provides information about using the NetView program to manage IP networks.

Intended audience

This publication is for network operators and network system programmers who use the NetView program to manage IP networks.

Publications

This section lists publications in the IBM Z NetView library and related documents. It also describes how to access NetView publications online and how to order NetView publications.

IBM Z NetView library

The following documents are available in the IBM Z NetView library:

- *Administration Reference*, SC27-2869, describes the NetView program definition statements required for system administration.
- *Application Programmer's Guide*, SC27-2870, describes the NetView program-to-program interface (PPI) and how to use the NetView application programming interfaces (APIs).
- *Automation Guide*, SC27-2846, describes how to use automated operations to improve system and network efficiency and operator productivity.
- *Command Reference Volume 1 (A-N)*, SC27-2847, and *Command Reference Volume 2 (O-Z)*, SC27-2848, describe the NetView commands, which can be used for network and system operation and in command lists and command procedures.
- *Installation: Configuring Additional Components*, GC27-2851, describes how to configure NetView functions beyond the base functions.
- *Installation: Configuring the NetView Enterprise Management Agent*, GC27-2853, describes how to install and configure the IBM Z NetView Enterprise Management Agent.
- *Installation: Getting Started*, GI11-9443, describes how to install and configure the base NetView program.
- *Installation: Migration Guide*, GC27-2854, describes the new functions that are provided by the current release of the NetView product and the migration of the base functions from a previous release.
- *IP Management*, SC27-2855, describes how to use the NetView product to manage IP networks.
- *Messages and Codes Volume 1 (AAU-DSI)*, GC27-2856, and *Messages and Codes Volume 2 (DUI-IHS)*, GC27-2857, describe the messages for the NetView product, the NetView abend codes, the sense codes that are included in NetView messages, and generic alert code points.
- *Programming: Pipes*, SC27-2859, describes how to use the NetView pipelines to customize a NetView installation.
- *Programming: REXX and the NetView Command List Language*, SC27-2861, describes how to write command lists for the NetView product using the Restructured Extended Executor language (REXX) or the NetView command list language.
- *Security Reference*, SC27-2863, describes how to implement authorization checking for the NetView environment.

- *Troubleshooting Guide*, GC27-2865, provides information about documenting, diagnosing, and solving problems that occur in the NetView product.
- *Tuning Guide*, SC27-2874, provides tuning information to help achieve certain performance goals for the NetView product and the network environment.
- *User's Guide: Automated Operations Network*, SC27-2866, describes how to use the NetView Automated Operations Network (AON) component, which provides event-driven network automation, to improve system and network efficiency. It also describes how to tailor and extend the automated operations capabilities of the AON component.
- *User's Guide: NetView*, SC27-2867, describes how to use the NetView product to manage complex, multivendor networks and systems from a single point.
- *User's Guide: NetView Enterprise Management Agent*, SC27-2876, describes how to use the NetView Enterprise Management Agent.
- *Using Tivoli System Automation for GDPS/PPRC HyperSwap Manager with NetView*, GI11-4704, provides information about the Tivoli® System Automation for GDPS®/PPRC HyperSwap® Manager with NetView feature, which supports the GDPS and Peer-to-Peer Remote Copy (PPRC) HyperSwap Manager services offering.
- *Licensed Program Specifications*, GC31-8848, provides the license information for the NetView product.
- *Program Directory for IBM Z NetView US English*, GI11-9444, contains information about the material and procedures that are associated with installing the NetView product.
- *Program Directory for IBM Z NetView Japanese*, GI11-9445, contains information about the material and procedures that are associated with installing the NetView product.
- *Program Directory for IBM Z NetView Enterprise Management Agent*, GI11-9446, contains information about the material and procedures that are associated with installing the IBM Z NetView Enterprise Management Agent.

The following books are archived:

- *Customization Guide*, SC27-2849, describes how to customize the NetView product and points to sources of related information.
- *Data Model Reference*, SC27-2850, provides information about the Graphic Monitor Facility host subsystem (GMFHS), SNA topology manager, and MultiSystem Manager data models.
- *Installation: Configuring Graphical Components*, GC27-2852, describes how to install and configure the NetView graphics components.
- *Programming: Assembler*, SC27-2858, describes how to write exit routines, command processors, and subtasks for the NetView product using assembler language.
- *Programming: PL/I and C*, SC27-2860, describes how to write command processors and installation exit routines for the NetView product using PL/I or C.
- *Resource Object Data Manager and GMFHS Programmer's Guide*, SC27-2862, describes the NetView Resource Object Data Manager (RODM), including how to define your non-SNA network to RODM and use RODM for network automation and for application programming.
- *SNA Topology Manager Implementation Guide*, SC27-2864, describes planning for and implementing the NetView SNA topology manager, which can be used to manage subarea, Advanced Peer-to-Peer Networking, and TN3270 resources.
- *User's Guide: NetView Management Console*, SC27-2868, provides information about the NetView management console interface of the NetView product.

Related publications

You can find additional product information on the IBM Z NetView web site at <https://www.ibm.com/us-en/marketplace/ibm-tivoli-netview-for-zos>.

For information about the NetView Bridge function, see *Tivoli NetView for OS/390® Bridge Implementation*, SC31-8238-03 (available only in the V1R4 library).

Terminology in this Library

The following terms are used in this library:

CNMCMD

For the CNMCMD member and the members that are included in it using the %INCLUDE statement

CNMSTYLE

For the CNMSTYLE member and the members that are included in it using the %INCLUDE statement

DSIOPF

For the DSIOPF member and the members that are included in it using the %INCLUDE statement

IBM® Tivoli Netcool®/OMNIbus

For either of these products:

- IBM Tivoli Netcool/OMNIbus
- IBM Tivoli OMNIbus and Network Manager

MVS™

For z/OS® operating systems

MVS element

For the base control program (BCP) element of the z/OS operating system

NetView

For the following products:

- IBM Z NetView version 6 release 3
- IBM Tivoli NetView for z/OS version 6 release 2 modification 1
- NetView releases that are no longer supported

PARMLIB

For SYS1.PARMLIB and other data sets in the concatenation sequence

VTAM®

For Communications Server - SNA Services

Unless otherwise indicated, topics to programs indicate the latest version and release of the programs. If only a version is indicated, the topic is to all releases within that version.

When a topic is made about using a personal computer or workstation, any programmable workstation can be used.

Using IBM Z NetView online help

The following types of IBM Z NetView mainframe online help are available, depending on your installation and configuration:

- General help and component information
- Command help
- Message help
- Sense code information
- Recommended actions

Accessing publications online

IBM posts publications for this and all other products, as they become available and whenever they are updated, to the IBM Knowledge Center at <https://www.ibm.com/support/knowledgecenter>. You can find IBM Z NetView documentation on [IBM Z NetView Knowledge Center](#).

Note: If you print PDF documents on other than letter-sized paper, set the option in the **Print** window that enables Adobe Reader to print letter-sized pages on your local paper.

Ordering publications

You can order many Tivoli publications online at <http://www.ibm.com/e-business/linkweb/publications/servlet/pbi.wss>

You can also order by telephone by calling one of these numbers:

- In the United States: 800-426-4968
- In Canada: 800-879-2755

In other countries, contact your software account representative to order Tivoli publications. To locate the telephone number of your local representative, perform the following steps:

1. Go to <http://www.ibm.com/e-business/linkweb/publications/servlet/pbi.wss>.
2. Select your country from the list and click the grey arrow button beside the list.
3. Click **About this site** to see an information page that includes the telephone number of your local representative.

Accessibility

Accessibility features help users with a physical disability, such as restricted mobility or limited vision, to use software products successfully. Standard shortcut and accelerator keys are used by the product and are documented by the operating system. Refer to the documentation provided by your operating system for more information.

For additional information, see the Accessibility appendix in the *User's Guide: NetView*.

Tivoli user groups

Tivoli user groups are independent, user-run membership organizations that provide Tivoli users with information to assist them in the implementation of Tivoli Software solutions. Through these groups, members can share information and learn from the knowledge and experience of other Tivoli users.

Support information

If you have a problem with your IBM software, you want to resolve it quickly. IBM provides the following ways for you to obtain the support you need:

Online

Please follow the instructions located in the support guide entry: <https://www.ibm.com/support/home/pages/support-guide/?product=4429363>.

Troubleshooting information

For more information about resolving problems with the IBM Z NetView product, see the *IBM Z NetView Troubleshooting Guide*. You can also discuss technical issues about the IBM Z NetView product through the NetView user group located at <https://groups.io/g/NetView>. This user group is for IBM Z NetView customers only, and registration is required. This forum is also monitored by interested parties within IBM who answer questions and provide guidance about the NetView product. When a problem with the code is found, you are asked to open an official case to obtain resolution.

Conventions used in this publication

This section describes the conventions that are used in this publication.

Typeface conventions

This publication uses the following typeface conventions:

Bold

- Lowercase commands and mixed case commands that are otherwise difficult to distinguish from surrounding text
- Interface controls (check boxes, push buttons, radio buttons, spin buttons, fields, folders, icons, list boxes, items inside list boxes, multicolumn lists, containers, menu choices, menu names, tabs, property sheets), labels (such as **Tip:**, and **Operating system considerations:**)
- Keywords and parameters in text

Italic

- Citations (examples: titles of publications, diskettes, and CDs)
- Words defined in text (example: a nonswitched line is called a *point-to-point line*)
- Emphasis of words and letters (words as words example: "Use the word *that* to introduce a restrictive clause."; letters as letters example: "The LUN address must start with the letter *L*.")
- New terms in text (except in a definition list): a *view* is a frame in a workspace that contains data.
- Variables and values you must provide: ... where *myname* represents...

Monospace

- Examples and code examples
- File names, programming keywords, and other elements that are difficult to distinguish from surrounding text
- Message text and prompts addressed to the user
- Text that the user must type
- Values for arguments or command options

Operating system-dependent variables and paths

For workstation components, this publication uses the UNIX convention for specifying environment variables and for directory notation.

When using the Windows command line, replace *\$variable* with *%variable%* for environment variables and replace each forward slash (/) with a backslash (\) in directory paths. The names of environment variables are not always the same in the Windows and UNIX environments. For example, %TEMP% in Windows environments is equivalent to \$TMPDIR in UNIX environments.

Note: If you are using the bash shell on a Windows system, you can use the UNIX conventions.

Syntax diagrams

The following syntax elements are shown in syntax diagrams. Read syntax diagrams from left-to-right, top-to-bottom, following the horizontal line (the main path).

- [“Symbols” on page xvii](#)
- [“Parameters” on page xviii](#)
- [“Punctuation and parentheses” on page xviii](#)
- [“Abbreviations” on page xix](#)

For examples of syntax, see [“Syntax examples” on page xix](#).

Symbols

The following symbols are used in syntax diagrams:



Marks the beginning of the command syntax.



Marks the end of the command syntax.



Indicates that the command syntax is continued on the next line.



Indicates that a statement is continued from the previous line.



Marks the beginning and end of a fragment or part of the command syntax.

Parameters

The following types of parameters are used in syntax diagrams:

Required

Required parameters are shown on the main path.

Optional

Optional parameters are shown below the main path.

Default

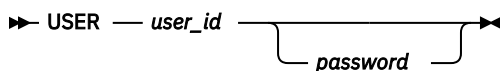
Default parameters are shown above the main path. In parameter descriptions, default parameters are underlined.

Syntax diagrams do not rely on highlighting, brackets, or braces. In syntax diagrams, the position of the elements relative to the main syntax line indicates whether an element is required, optional, or the default value.

When you issue a command, spaces are required between the parameters unless a different separator, such as a comma, is specified in the syntax.

Parameters are classified as keywords or variables. Keywords are shown in uppercase letters. Variables, which represent names or values that you supply, are shown in lowercase letters and are either italicized or, in NetView help, displayed in a differentiating color.

In the following example, the `USER` command is a keyword, the `user_id` parameter is a required variable, and the `password` parameter is an optional variable.



Punctuation and parentheses

You must include all punctuation that is shown in the syntax diagram, such as colons, semicolons, commas, minus signs, and both single and double quotation marks.

When an operand can have more than one value, the values are typically enclosed in parentheses and separated by commas. For a single value, the parentheses typically can be omitted. For more information, see [“Multiple operands or values” on page xx](#).

If a command requires positional commas to separate keywords and variables, the commas are shown before the keywords or variables.

When examples of commands are shown, commas are also used to indicate the absence of a positional operand. For example, the second comma indicates that an optional operand is not being used:

```
COMMAND_NAME opt_variable_1,,opt_variable_3
```

You do not need to specify the trailing positional commas. Trailing positional and non-positional commas either are ignored or cause a command to be rejected. Restrictions for each command state whether trailing commas cause the command to be rejected.

Abbreviations

Command and keyword abbreviations are listed in synonym tables after each command description.

Syntax examples

The following examples show the different uses of syntax elements:

- “Required syntax elements” on page xix
- “Optional syntax elements” on page xix
- “Default keywords and values” on page xix
- “Multiple operands or values” on page xx
- “Syntax that is longer than one line” on page xx
- “Syntax fragments” on page xx

Required syntax elements

Required keywords and variables are shown on the main syntax line. You must code required keywords and variables.

➤ REQUIRED_KEYWORD — *required_variable* ➤

A required choice (two or more items) is shown in a vertical stack on the main path. The items are shown in alphanumeric order.

➤ REQUIRED_OPERAND_OR_VALUE_1
 REQUIRED_OPERAND_OR_VALUE_2 ➤

Optional syntax elements

Optional keywords and variables are shown below the main syntax line. You can choose not to code optional keywords and variables.

➤ _____ ➤
 OPTIONAL_OPERAND

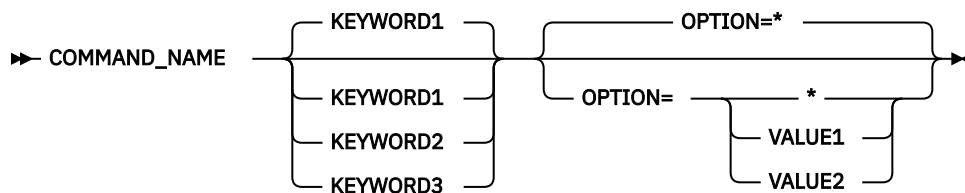
A required choice (two or more items) is shown in a vertical stack below the main path. The items are shown in alphanumeric order.

➤ _____ ➤
 OPTIONAL_OPERAND_OR_VALUE_1
 OPTIONAL_OPERAND_OR_VALUE_2

Default keywords and values

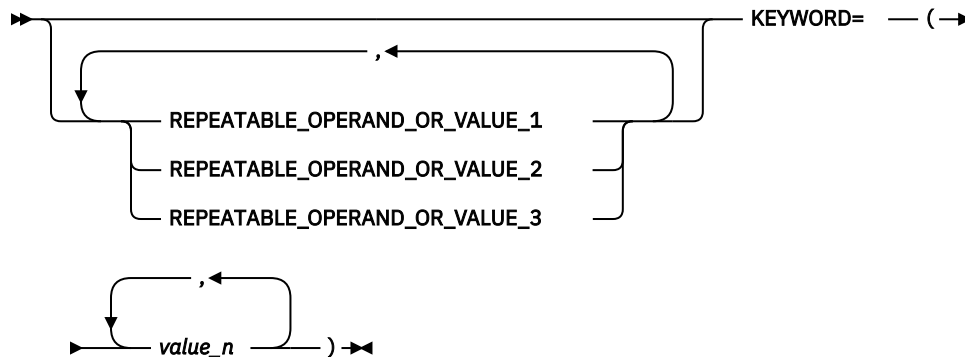
Default keywords and values are shown above the main syntax line in one of the following ways:

- A default keyword is shown only above the main syntax line. You can specify this keyword or allow it to default. The following syntax example shows the default keyword KEYWORD1 above the main syntax line and the rest of the optional keywords below the main syntax line.
- If an operand has a default value, the operand is shown both above and below the main syntax line. A value below the main syntax line indicates that if you specify the operand, you must also specify either the default value or another value shown. If you do not specify the operand, the default value above the main syntax line is used. The following syntax example shows the default values for operand OPTION=* above and below the main syntax line.



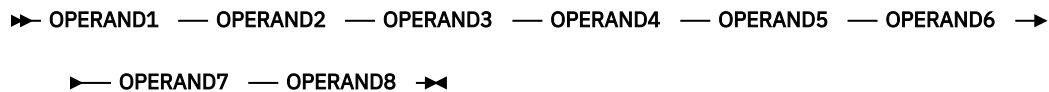
Multiple operands or values

An arrow returning to the left above a group of operands or values indicates that more than one can be selected or that a single one can be repeated.



Syntax that is longer than one line

If a diagram is longer than one line, each line that is to be continued ends with a single arrowhead and the following line begins with a single arrowhead.



Syntax fragments

Some syntax diagrams contain syntax fragments, which are used for lengthy, complex, or repeated sections of syntax. Syntax fragments follow the main diagram. Each syntax fragment name is mixed case and is shown in the main diagram and in the heading of the fragment. The following syntax example shows a syntax diagram with two fragments that are identified as Fragment1 and Fragment2.



Fragment1



Fragment2



Part 1. Introduction to IP Network Management

Chapter 1. IBM Z NetView IP Management Overview

The NetView program provides functions to help maintain the highest degree of availability for IBM Z networks. An extensive set of tools is included to manage and maintain complex, multi-vendor, multi-platform networks and systems from a single point of control. The NetView program provides advanced correlation facilities to automate any network or system event and provides support for both TCP/IP and SNA networks. The program also provides a set of user interfaces to meet the needs of any user and management functions that work with other products to provide a complete picture of your networks and systems.

TCP/IP management is an integral part of the IBM Z NetView program. A full array of management functions is provided, including the following functions.

- Management of SNA over IP

The Enterprise Extender technology enables the transport of SNA traffic over an IP network. This technology routes SNA path information units (PIUs) over Advanced Peer-to-Peer Networking nodes using high-performance routing (HPR) and, subsequently, across the IP network using User Datagram Protocol (UDP) packets. The routing provided by Enterprise Extender is more complex and requires additional information about the paths to session partners. To help discover congestion and broken links, the NetView program can locate specific Enterprise Extender sessions passing through a particular link station and provide extensive information about the path to the session partner.

- Support for dynamic virtual IP addresses (DVIPA)

The use of DVIPAs is a vital technique for eliminating application connection failures because the physical adapter is removed as a point of failure. Distributed DVIPAs distribute the workload of connection requests and provide additional fail-safe precautions in the event of a server failure. The NetView program provides the information that is needed to manage DVIPAs, including the following information:

- DVIPA definition and status information, including the differentiation of application-instance, stack-defined, and distributed DVIPAs, so that you can ensure that the characteristics of each DVIPA are what you want, such as the following characteristics:
 - The status of the DVIPA on the TCP/IP stack
 - The XCF group that the DVIPA is part of
 - The TCP/IP host to which the DVIPA is defined
 - The origin (how it is configured to the TCP/IP stack)
 - The mobility (the ways that it can be moved to another TCP/IP stack)
 - The rank of the stack to which the DVIPA is defined in the chain of backup stacks
- Distributed DVIPA information, including sysplex distributors, distributed targets, application server health statistics for distributed targets, and statistics on workload balancing
- DVIPA connection information, including the number of active connections and abundant information about the current state of the connection
- DVIPA routing information, including VIPA routes and distributed DVIPA connection routing
- Historical DVIPA information

- Discovery manager resource discovery

The discovery manager provides a comprehensive set of monitoring tools for your sysplex, and a view of your physical configuration. The following types of resources can be discovered:

- Central processor complex (CPC)
- Channel subsystem identifier
- Logical partition (LPAR)

- Sysplex
- Coupling facility
- z/OS image
- TCP/IP stack
- TCP/IP subplex
- IP interfaces
- NetView applications
- Telnet servers and ports
- Open Systems Adapter (OSA) channels and ports
- HiperSockets adapter

In addition to using this information to manage and monitor your sysplex from the master NetView program, you can view this information at the enterprise master NetView program. For more information about the master and enterprise master NetView programs, see [Chapter 2, “Sysplex Management Overview,”](#) on page 11.

- Connection management

The NetView program manages both active and historical connections, including stack name, local and remote addresses and ports, start time, end time and termination code (for connections that have ended), traffic information such as sent and received byte and segment counts, retransmit counts, and information about the connection state, the interface, the host, TN3270, and more. With this information, for example, connections that end but should still be active or connections with unexpectedly low activity can easily be identified. Data is available both in a form for you to read and in binary form for programming use. Host name translation and IPv4 or IPv6 addresses are supported. In addition, the cross-domain capabilities of the NetView program enable the viewing of connection data at remote z/OS hosts, thus enabling centralized management. The NetView program can also provide details about encryption protocols being used on active connections by exploiting Communication Server's z/OS Encryption Readiness Technology (zERT).

- Packet trace collection and formatting at the stack and Open Systems Adapter (OSA) level

The examination of packet content is sometimes necessary to debug a problem. The NetView program provides real-time capture and formatting of IP packet or OSA trace data, including both headers and payloads. The formatting is the same as that under IPCS, so that you do not have to learn a new format. Because the formatter is directly integrated with the TCP/IP stack, no translation mismatches can occur. Highly flexible tracing and formatting options are available so that you can filter out unwanted data. Both IPv4 and IPv6 packets are supported, and the data is also available in binary (unformatted) form for use by automation routines. In addition to providing data, the NetView program provides analysis to help locate problem areas. For example, the analysis can indicate how many of your IP connections (TCP, UDP or ICMP) have errors and the kinds of errors (delayed ACKs, resets, retransmissions, and zero window size). With the NetView program, this information is readily available. If necessary, you can run multiple IP packet traces simultaneously. Packet traces can be saved in CTRACE and Sniffer trace format for additional analysis.

- Command support

Monitoring a network or system can have limited value when you cannot take action on problems that you find. The NetView program provides extensive support for IP-related commands, giving you the control capabilities you need to manage IP resources. The following commands can be issued from the NetView command line directly, in REXX procedures, and in other automation routines:

- The TN3270 command logs in to remote TCP/IP-connected systems, either from the NetView command line or from the NetView management console.
- The SNMP command sends requests to SNMP agents to retrieve or set information in the management information bases (MIBs) maintained by those agents. Besides supporting SNMPv1 and SNMPv2c, this command also supports SNMPv3 authentication and encryption from the command line, REXX, and command lists, with switches for the following settings:

- The authentication protocol (MD5 or SHA) used for authenticating SNMPv3 messages
- The authentication pass phrase used for authenticating SNMPv3 messages
- The privacy pass phrase used for encrypted SNMPv3 messages.
- The encryption (privacy) algorithm (DES or AES) used for encrypting and decrypting SNMPv3 messages.
- Any UNIX System Services command can be issued directly from the NetView command line or used in REXX procedures and other automation routines.
- The SOCKET command retrieves information about TCP/IP stacks or can be used to run stack services. It can be used in TCP/IP applications based on the NetView program.
- The TRACERTE command traces the routes of data packets to a specified IP host from the TCP/IP stack on the host on which the NetView program is running. Use this command to determine connectivity with or routing to a particular endpoint, roundtrip times between the NetView and target hosts, and routers along the way.
- The PING command tests connectivity to an IP host.
- The RMTCMD command sends system, subsystem, and network commands to a remote NetView host for processing.
- The REXEC command sends a command over IP to a remote host for processing and displays the resulting output. The standard UNIX RSH protocol is used. The remote host must have an REXEC server listening at the specified or default port for the command to work.
- The RSH command sends a command over IP to a remote host for processing. The output can be displayed as line-mode output or in a panel that is placed on the NetView roll stack. If the remote host provides support, additional commands can be issued from the panel where the output is displayed.
- The IPLOG command sends a message to the syslog daemon on a remote host for processing.

- Detection of hung listeners

System administrators often have no way to proactively monitor for and manage hung listeners. They find out that a listener is hung only when a user complains that an application is unavailable. The NetView program provides for automated monitoring of customer-designated ports to detect those that refuse connections. If a port that refuses connections is hung, a message that can be automated is issued, thus enabling recovery. If a port that refuses connections is not hung, the connection is ended.

- IP server support

In addition to client function, the NetView program also provides server function for the following TCP/IP services:

- REXEC
- RSH
- Syslogd

- NetView web services gateway

This function provides an open interface to the NetView program for issuing commands and receiving responses. SOAP is used for communications, and HTTP or HTTPS is used as the transport mechanism.

- Automated responses to intrusions

Firewalls are not impenetrable. Even within a firewall, systems can be vulnerable to attack or misuse, whether accidental or malicious. Working in conjunction with the Intrusion Detection Services of z/OS Communications Server, the NetView program offers a variety of automated responses to an intrusion, responses that can eliminate delays that are required if you have to wait for human intervention:

- Send a notification. Send an e-mail to security administrators, an alert to the NetView console, or a message to designated NetView operators.
- Take action. Issue UNIX System Services, NetView, or z/OS commands to collect more data or take other actions, for example, shut down a port that is under attack.

- Collect statistics. Collect statistics and generate a report to send by e-mail to security administrators.

- Events

The NetView program supports a variety of event types:

- SNMP traps. The NetView program can emit SNMPv1, SNMPv2c, and SNMPv3 traps. The program can also receive and process SNMP traps through one of the following mechanisms:
 - SNMPv1, SNMPv2c, and SNMPv3 traps, including encrypted and authenticated SNMPv3 traps, can be received for automation processing through NetView base services.
 - SNMPv1 traps can be received through the NetView Event/Automation Service (E/AS), which converts them to alerts for processing.
- Alerts and messages. The NetView Event/Automation Service (E/AS) can convert alerts to SNMPv1 traps and send them to a trap manager. To avoid flooding the trap manager, this conversion is recommended only for alerts that require human intervention.
- Event Integration Facility (EIF) events. The NetView E/AS can both send and receive EIF events, which enables centralized event management from either a mainframe or a distributed platform.

- IP resource discovery

The MultiSystem Manager topology manager collects topology and status information about the IP resources in your network and stores the information in the Resource Object Data Manager (RODM) component. After the information is in RODM, you can manage your network resources from graphical displays in the NetView management console.

- Topology correlation

Topology correlation automatically ties together resources that are managed by different types of topology functions such as IP and open topology. Topology correlation is provided for any resource that is stored in RODM, including resources that are discovered by the discovery manager, by MultiSystem Manager topology functions, by the NetView SNA Topology Manager, and by customer or vendor applications that use the Graphic Monitor Facility host subsystem data model.

- IPv6 support

In support of the growing adoption of IPv6 addresses, the NetView program provides for IPv6 connectivity, allows IPv6 addresses in command input, and displays IPv6 addresses in messages, views, and most other places where an IP address can be shown.

- Security

To prevent unauthorized connections to the NetView program from a TCP/IP host, you can restrict access using the NetView command authorization table, the NETCMDS class of an SAF product, and sample definition members that are part of the NetView program. You can also protect IP addresses for command security and span of control. For more information about security, see the *IBM Z NetView Security Reference*.

The NetView program can also collect security data for active TCP/IP connections. Data includes the state of IP filtering on the connection, the types of encryption protocols being used on the connection (if any), and information on any digital certificates being used on the connection. For each encryption protocol (TLS/SSL, SSH or IPsec), protocol-related information such as encryption algorithms, cipher suites, message authentication algorithms and FIPS140 support is collected. Data is displayed in the Tivoli Enterprise Portal and is available in 3270 messages.

- AON IP functions

Most IP-related functions that were previously implemented as part of the NetView Automated Operations Network (AON) component are now implemented as base NetView services and no longer require AON enablement and configuration. The following functions are exceptions and still require AON; for more information about these functions, see the *IBM Z NetView User's Guide: Automated Operations Network*.

- IP server management (managing TSO server sessions)
- Issuing of line mode TSO and UNIX commands without logging on to TSO

- SNMPView
- CISCOWorks Blue Inter-network Status Monitor

For information about installing and enabling the various IP functions, including a "Getting Started" guide for enabling basic IP management, see *IBM Z NetView Installation: Configuring Additional Components*.

Part 2. Sysplex Management

Chapter 2. Sysplex Management Overview

The increasing complexity of managing a sysplex environment has led to the need for management from a single point of control. The NetView program provides high availability sysplex management to ease complex system interactions and to maximize operational effectiveness. A master NetView program is automatically available for you to use in managing and displaying information about your sysplex. Automatic failover to another NetView program that can monitor the sysplex in the event of an outage is also provided. Monitoring of sysplex and system resources, including sysplexes, coupling facilities, z/OS images, TCP/IP stacks, IP interfaces, dynamic virtual IP addresses (DVIPAs), Telnet servers and ports, central processor complexes, logical partitions (LPARs), Open Systems Adapter (OSA) and HiperSockets adapters, is available with this powerful management capability.

A master NetView program can also provide management for systems outside of the sysplex and for another sysplex. The NetView program in this role is known as an enterprise master NetView program. Additional configuration is needed for the enterprise master NetView program to manage systems that are outside of the sysplex. DVIPA information is restricted to sysplex management.

The NetView program provides the following high availability sysplex management capabilities:

- Discovery, using XCF, of the operational status of all NetView programs in the sysplex, which allows one NetView program to fail over to another NetView program in the event of an outage.
- Monitoring of sysplex and system resources using a combination of sampling and event automation, with discovery being split between discovery manager and DVIPA. To have a complete view of resources in the sysplex, a NetView program must run on each z/OS image with discovery enabled.
- Commands that can be issued from a 3270 command line to collect real-time data about sysplex and system resources
- IBM Z NetView Enterprise Management Agent workspaces in the Tivoli Enterprise Portal
- Logical sysplex and physical IBM Z topology in the NetView management console

The Resource Object Data Manager (RODM) component is the data cache for NetView discovery manager sysplex management capabilities. Beginning with NetView V5R4, RODM is no longer used as a repository only for data that is displayed through the NetView management console. Sysplex and IBM Z management information is stored in RODM, whether or not the Network management console is used. The following kinds of sysplex and IBM Z management information are stored in RODM:

- Central processor complex (CPC)
- Channel subsystem identifier
- Logical partition (LPAR)
- Sysplex
- z/OS image
- TCP/IP stack
- TCP/IP subplex
- NetView application
- Telnet server and port
- Open Systems Adapter (OSA)
- HiperSockets adapter

Notes:

1. Display of OSA and HiperSockets information in any NetView user interface requires RODM.
2. Discovery of interfaces and OSA requires SNMP. OSA also requires the SNMP OSA subagent (IOBSNMP).

Chapter 3. Managing the NetView Program as a Sysplex Application By Using XCF

A sysplex application, where multiple instances of the application are running on different systems, can share status information and communicate with other instances of the application. As a sysplex application, the NetView program uses the z/OS cross-system coupling facility (XCF) services to communicate with other NetView programs.

Master NetView Program and NetView XCF Groups

In sysplex management, the NetView program uses z/OS Cross-Systems Coupling Facility (XCF) services to facilitate resource discovery, forward management data, and implement a controlling NetView program. The controlling NetView program, known as the master NetView program, has a complete view of the sysplex. The master program is a logical point for interacting with the NetView user interfaces, the NetView management console, and the Tivoli Enterprise Portal. The master program is also a logical point from which to manage a Resource Object Data Manager (RODM) data cache for the sysplex.

XCF group and signaling services enable the NetView programs in the sysplex to inform each other about status and to send messages to or receive messages from each other. XCF services enable the NetView programs to be notified when NetView programs enter or leave the sysplex, to be informed about which NetView program is the master program, and to exchange critical control information about the relationship between the master program and other NetView programs. The use of XCF services makes the NetView program a high availability sysplex application.

An XCF group is a set of related applications that register with z/OS XCF services by using the same group name. At any given time, only one NetView program can be the master program in an XCF group. NetView XCF groups are named DSIPLXnn, where nn is the value of the XCF.GROUPNUM statement in the CNMSTYLE member. The DSIXCFMT task processes commands for joining and leaving XCF groups and information provided by XCF exit routines to handle messages and group status changes. For more information about configuring NetView XCF groups, see *IBM Z NetView Installation: Configuring Additional Components*.

Master-capable and Basic NetView Programs

Other NetView programs in the sysplex that are members of the same DSIPLXnn group can be defined as backups for the master NetView program and are known as master-capable. If the master NetView program is unavailable, one of the master-capable NetView programs can take over the role of master, depending on the rank defined for each master-capable program.

The master and master-capable NetView programs are defined by using the XCF.RANK statement in the CNMSTUSR or CxxSTGEN member. For a high degree of control, these statements must be coordinated throughout the sysplex. Other statements in the CNMSTUSR or CxxSTGEN member can be used to define, for example, a list of procedures that is associated with the master NetView program or the time that a master-capable NetView program must wait before taking on the master role if no master program is detected at initialization. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*. For information about the statements, see the *IBM Z NetView Administration Reference*.

NetView programs that are neither master nor master-capable are known as basic NetView programs. Basic NetView programs forward status to the master NetView program but cannot assume the role of master.

Figure 1 on page 14 shows an example configuration consisting of a sysplex with three systems, each running a single NetView program. NVA is the master NetView program, with NVB (a master-capable

NetView program) and NVC (a basic NetView program) forwarding data to NVA. The master NetView program provides data to the NetView management console and the Tivoli Enterprise Portal and stores data in RODM. The Z NetView Enterprise Management Agent connects to a z/OS Tivoli Enterprise Management Server on the same LPAR. All three NetView programs belong to the same XCF group and use z/OS XCF support.

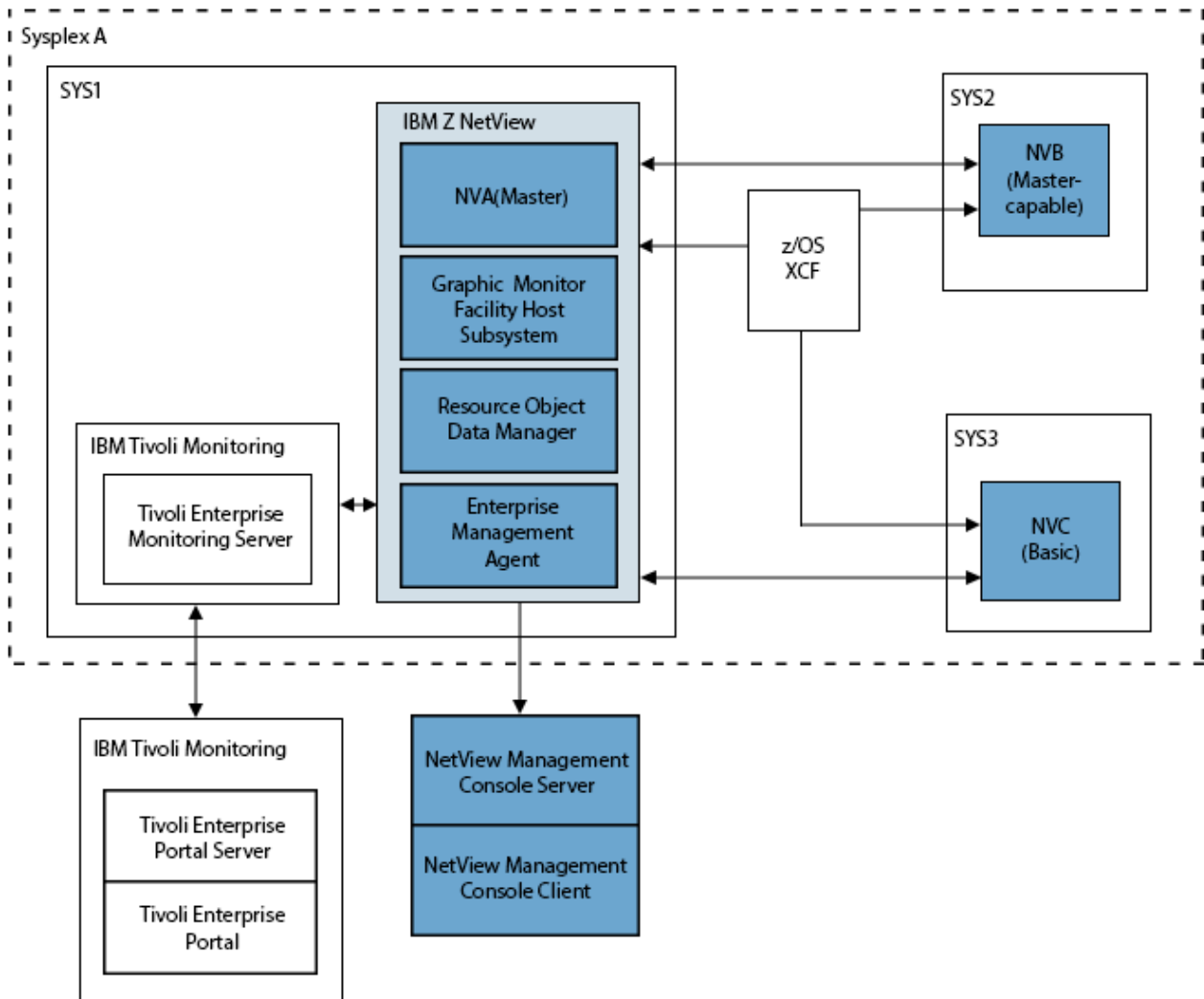


Figure 1. Sysplex Configuration

Master Role Takeover

The takeover of the master role can be initiated by XCF exits or by an operator command. If a master NetView program stops or an operator issues a NetView command that causes the master NetView program to leave the XCF group or give up the master role, XCF exits are run at the other DSIPLXnn group members. The other NetView programs determine which one should take over as the new master NetView program. An operator at a master-capable NetView program can also issue a NetView command to take over the master role.

The following takeover example is based on the configuration that is shown in [Figure 1](#) on page 14:

1. NVA becomes inactive, and the z/OS XCF support detects the outage.
2. XCF runs exits at NVB and NVC to report the outage.
3. NVB determines that it should become the master and updates the XCF state field accordingly.
CNMSTYLE definition statements can be used to facilitate the automatic startup of any jobs that need to be started, such as RODM or the Z NetView Enterprise Management Agent. The NetView program

can also be configured to end NETCONV connections at the old master NetView program and to restart them at the new master NetView program.

4. XCF informs NVC that NVB has assumed the master role.
5. NVB sends RMTCMD commands to NVC to establish data forwarding connectivity and to initiate data discovery at other systems to obtain a current view of the sysplex.

As data is forwarded to the new master NetView program, Tivoli Enterprise Portal and NetView management console views are updated.

Figure 2 on page 15 shows the results of NVB becoming the master NetView program when NVA becomes inactive. CNMSTYLE statements result in GMFHS, RODM, Z NetView Enterprise Management Agent, and Tivoli Enterprise Monitoring Server becoming active on SYS2. The NETCONV connection to the NetView management console server is switched to NVB, and status is forwarded from NVC to NVB.

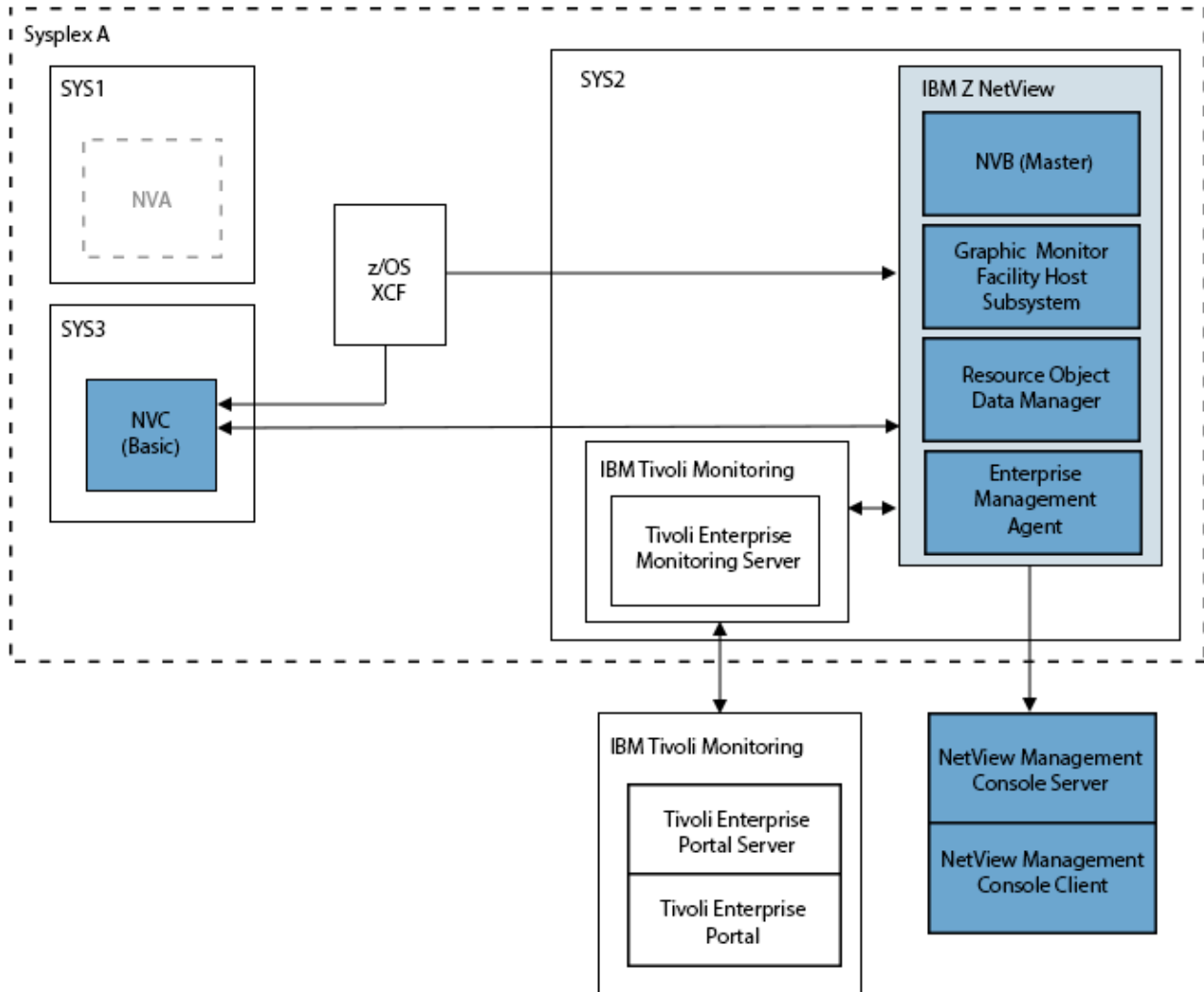


Figure 2. Sysplex Configuration after Master Takeover

User-defined XCF Groups

In addition to the DSIPLXnn group, you can use NetView commands, messages, and pipe stages to define your own XCF groups. The NetView program supports user applications by processing XCF group and message events for the user group and then generating messages that the application can use to process the events. NetView messages can be processed by using automation. The XCFMSG pipe stage enables a user application to process XCF messages that are sent to it in the pipe.

The NetView program generates messages when XCF applications enter or leave an XCF group, change XCF status, or update the 32-byte user field that is associated with a group member, or when an XCF message for a group member is received.

The following NetView commands, messages, and pipe stages are involved in defining a user-defined group:

- START XCFGROUP
- STOP XCFGROUP
- BNH589I GROUP *grp* MEMBER *mem* STATUS CHANGE. NEW STATUS=*newstat*. PREVIOUS STATUS=*oldstat*
- BNH590I XCF MESSAGE RECEIVED FROM *mem* IN GROUP *grp*
- BNH592I MEMBER *mem* HAS JOINED GROUP *grp*
- BNH593I MEMBER *mem* HAS LEFT GROUP *grp*
- PIPE XCFMSG
- PIPE XCFTABLE

Commands and Pipe Stages for XCF

The following commands can be used with XCF groups. For detailed information about these commands, see the NetView online help.

- LIST STATUS = XCFGRPS returns a list of z/OS XCF groups in which the NetView program participates.
- PLEXCTL changes the rank of the NetView program in the DSIPLXnn group in which it participates.
- START XCFGROUP and STOP XCFGROUP specify the name of the XCF group that the NetView program is to join or leave, respectively.

The XCFMSG PIPE stage enables the sending and receiving of XCF messages between group members. One receive pipe stage for a group can be active at any given time. The XCFTABLE and XCFQUERY PIPE stages enable user data to be associated with a group member, support the retrieving and setting of the user state field that XCF maintains for members, and enable issuing XCF queries. For information about these PIPE stages, see *IBM Z NetView Programming: Pipes*.

DVIPA Support for XCF

A dynamic virtual IP address (DVIPA) can be defined for the master NetView program. This address is associated with the master NetView program, so that connection requests to this address always go to the master program. This address can be used by applications to contact the master NetView program regardless of the system that it is running on within the sysplex.

When the master program moves in the sysplex, the address is dropped from the stack of the old master program and is dynamically defined on the stack of the new master program. Moving the data transport from a master NetView program to a master-capable NetView program becomes more flexible because it is no longer necessary to know the specific IP address of the master-capable NetView program. Using a DVIPA to represent the NetView application requires TCP/IP configuration to define a NetView port reservation and a block of IP addresses (VIPARANGE). For information about configuring the master NetView DVIPA, see *IBM Z NetView Installation: Configuring Additional Components*.

Turning Off XCF Support

XCF support is not mandatory. By default, the NetView program participates in an XCF group named DSIPLX01 with a rank of 1 (master-capable). If the rank is set to -1 in the CNMSTYLE member, a NetView program does not participate in an XCF group. For information about how to set the rank, see *IBM Z NetView Installation: Configuring Additional Components*.

Note: The rank can be reset by using the RESTYLE command.

If the NetView program does not participate in an XCF group, it cannot receive status information from other NetView programs and it displays only local status information.

Chapter 4. Managing the Enterprise by Using the NetView Program

An enterprise consists of a combination of sysplexes and stand-alone systems. With some additional system definition, a sysplex master NetView program can manage systems and sysplexes beyond the scope of the sysplex where it is located.

For example, the NVC NetView program that is shown in [Figure 3 on page 19](#) is an enterprise master program. In this configuration, the NVA, NVB, and NVC NetView programs are in Sysplex A on the SYS1, SYS2, and SYS3 systems, respectively; the NVD, NVE, and NVF NetView programs are in Sysplex B on the SYS4, SYS5, and SYS6 systems, respectively; and the NVLCL NetView program is on the stand-alone SYS7 system. With this type of configuration, a single enterprise-wide Resource Object Data Manager (RODM) can be used to store data that is forwarded to the enterprise master NetView program.

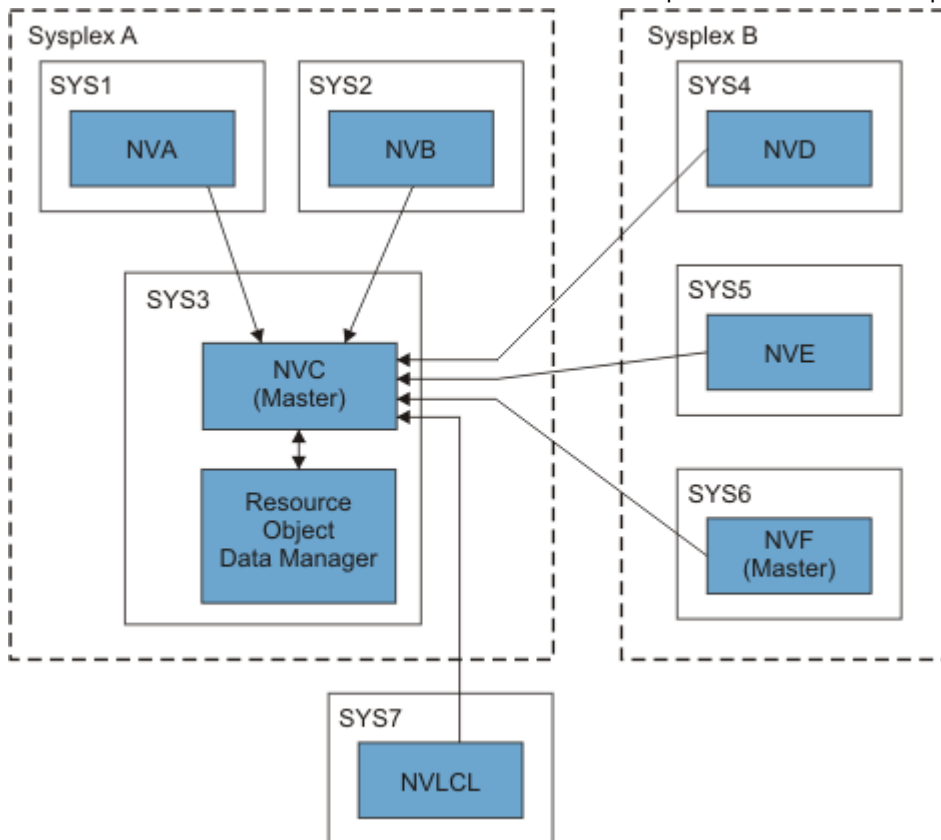


Figure 3. NVC as an Enterprise Master NetView Program

If a master-capable NetView program within the sysplex assumes the role of sysplex master, it also becomes an enterprise master, provided that the CNMSTYLE definitions for the enterprise are synchronized across the master and master-capable NetView programs within the sysplex. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*. For information about the statements, see the *IBM Z NetView Administration Reference*.

The GDPS Continuous Availability solution expands the scope of the enterprise master to enable both discovery manager data and Active/Active data to be forwarded to an enterprise master. You can have different enterprise masters for Active/Active data and discovery manager data, depending on the NetView configuration. If you have multiple enterprise masters, they must reside in different sysplexes or on separate stand-alone systems. For more information, see *IBM Z NetView for Continuous Availability Configuring and Using the GDPS Continuous Availability Solution*.

The following steps provide an overview of how an enterprise master NetView program is set up and operates:

1. Communication between the enterprise master NetView program and the other NetView programs in the enterprise must be set up. XCF services are available only within a sysplex, so the enterprise master NetView program cannot use XCF to learn about other managed systems. Because of this, you must provide CNMSTYLE definition statements (ENT.SYSTEMS.name) to define managed systems or sysplexes. Definitions are required for stand-alone NetView systems and at least one NetView program within a sysplex. The required definitions include RMTCMD aliases to the remote systems that the enterprise master can use to initially contact them.
2. When a NetView program assumes the sysplex master role in the sysplex and CNMSTYLE statements define other systems, the NetView program uses the RMTCMD command to notify the NetView programs in the other sysplexes and systems that an enterprise master NetView program exists. If CNMSTYLE statements for the enterprise are not defined or cannot be processed successfully, no enterprise master exists. Only one NetView program within a sysplex XCF group needs to be contacted. The contacted system forwards configuration information about the XCF group to which it belongs to the enterprise master NetView program, which saves the information and uses it to establish RMTCMD sessions to other NetView programs in the sysplex. The master NetView program within the sysplex group forwards updates to the enterprise master as NetView programs enter or leave the group.
3. If this enterprise master is handling discovery manager data, the enterprise master NetView program initiates resource discovery across the enterprise, and, after processing the data that is forwarded to it, updates the Tivoli Enterprise Portal and the NetView management console. You can define CNMSTYLE statements to pace the flow of discovery commands to the managed systems. Not all data that is forwarded within a sysplex is sent to an enterprise master NetView program. Discovery manager data is forwarded, but DVIPA data is not.

For information about configuring an enterprise RODM, see *IBM Z NetView Installation: Configuring Additional Components*.

Chapter 5. Managing Resources That are Discovered by the Discovery Manager

The discovery manager provides a comprehensive set of monitoring tools for your sysplex, and a view of your physical configuration. Before NetView V5R4, Sysplex IP Stack Manager managed TCP/IP stack information, including monitoring of the sysplex, and z/OS image. The discovery manager provides information that you can use to manage and monitor your sysplex from the master NetView program. Additionally, information that is discovered by the discovery manager can be viewed at the enterprise master NetView program.

The following kinds of resources can be monitored by the discovery manager:

- Central processor complex (CPC)
- Channel subsystem identifier
- Logical partition (LPAR)
- Sysplex
- Coupling facility
- z/OS image
- TCP/IP stack
- TCP/IP subplex
- IP interfaces
- NetView applications
- Telnet servers and ports
- Open Systems Adapter (OSA) channels and ports
- HiperSockets adapter

Notes:

1. Display of OSA and HiperSockets information in any NetView user interface requires RODM.
2. Discovery of interfaces and OSA requires SNMP. OSA also requires the SNMP OSA subagent (IOBSNMP).

The DISCOVERY tower, which is required by the discovery manager, is enabled by default, and discovery is started during NetView initialization. For information about configuring this discovery, see *IBM Z NetView Installation: Configuring Additional Components*.

Users can effect discovery of these resources in the following ways:

- Manually issue the CNMEERSC command, which initiates rediscovery.
- Modify the sampling intervals in the CNMSTUSR or CxxSTGEN member for IP interfaces (including OSA and HiperSockets interfaces), Telnet servers and ports, and NetView applications and issue a RESTYLE command. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*.
- Issue the COLLCTL command to start or stop data collection for IP interfaces (including OSA and HiperSockets interfaces), Telnet servers and ports, and NetView applications.

For detailed information about the data that is collected, see [Appendix A, “Discovery Manager Data,” on page 157](#).

Information about resources that are discovered by the discovery manager can be viewed in the following ways:

- [“Using 3270 Commands” on page 22](#)

- [“Using NetView Enterprise Management Agent Workspaces” on page 23](#)
- [“Using NetView Management Console Topology” on page 28](#)

Using Multiple NetView Programs on a Single Z Image

By default, when more than one instance of the NetView program is running on a Z image, each NetView instance can collect sysplex data when discovery is activated in the CNMSTYLE member. Specify DISCOVERY.NetViewOnly=YES in the CNMSTUSR or CxxSTGEN member for the NetView instances for which you do not want to collect all of the sysplex data. With this option specified, the NetView program reports only its own status. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*.

Using 3270 Commands

The discovery manager provides 3270 commands that collect real-time information about NetView applications, TCP/IP stacks, IP interfaces, Telnet servers, Telnet server ports, Open Systems Adapter (OSA) channels and ports, and HiperSockets configuration. These commands, which are shown in [Table 1 on page 22](#), can be issued from non-master NetView programs to view local information. These commands can be issued for all the domains that are in the scope of the master NetView program. For example, a sysplex master NetView program can issue these commands to other NetView domains in the sysplex in which the NetView programs are either master-capable or basic NetView programs. However, a sysplex master NetView program cannot issue these commands to an enterprise master NetView program, unless it is also the enterprise master NetView program. The enterprise master NetView program can issue these commands to all the domains that are in the scope of the enterprise master NetView program.

Table 1. Discovery Manager Data Collector Commands			
Command	Type of Data Collected	Sample to View Data	Messages Returned by the Collector
HIPERSOC	HiperSockets configuration and status information	CNMSHIPR	BNH598I
IFSTAT	IP interface information	CNMSIFST	BNH498I
NVSTAT	Configuration and status information about the NetView domains that are known to this NetView program	CNMSNVST	BNH495I
OSAPORT	OSA channel and port information	CNMSOSAP	BNH597I
STACSTAT	Configuration and status information about TCP/IP stacks	CNMSSTAC	BNH845I
TELNSTAT	Configuration and status information for Telnet servers	CNMSTNST	BNH496I
TNPTSTAT	Configuration and status information for Telnet server ports	CNMSTPST	BNH497I

These data collector commands are intended for use in application programs. When a data collector command completes successfully, it returns the data in a particular multiline message. To view the data in a format that can be read, use the associated sample. To display this data in a different format, modify the applicable sample or create your own application to format the data that is returned by the specified commands. You can use input parameters with a sample to specify the data to collect. The input parameters for a sample are the same as those for the NetView command that is called by the sample. For information about the returned messages, the command parameters, or other information about the data collector commands, see the IBM Z NetView online message and command help.

Notes:

1. You can access a menu of discovery manager data collector commands from the command facility:
 - a. Enter NETVIP on the command line. The NetView IP Management Functions Menu panel is displayed (Figure 91 on page 105).
 - b. Select Manage Sysplex. A menu panel from which you can select a data collector command is displayed.
2. Because the data records that are displayed can be quite wide, the samples are based on the WINDOW command. You can scroll to the right to view the data that is not currently visible in a 3270 session.
3. These samples are also used by the Z NetView Enterprise Management Agent take action commands.

Using NetView Enterprise Management Agent Workspaces

Z NetView Enterprise Management Agent workspaces are available for NetView applications, TCP/IP stacks, Telnet servers and ports, Open Systems Adapter (OSA) channels and ports, and HiperSockets adapters.

HiperSockets Configuration and Status Workspace

The HiperSockets Configuration and Status workspace displays configuration and status information for HiperSockets. The data collection for this information requires Resource Object Data Manager (RODM).

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **HiperSockets**.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

This workspace provides data that is updated by using sampling.

Historical data is not enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 4 on page 24](#).

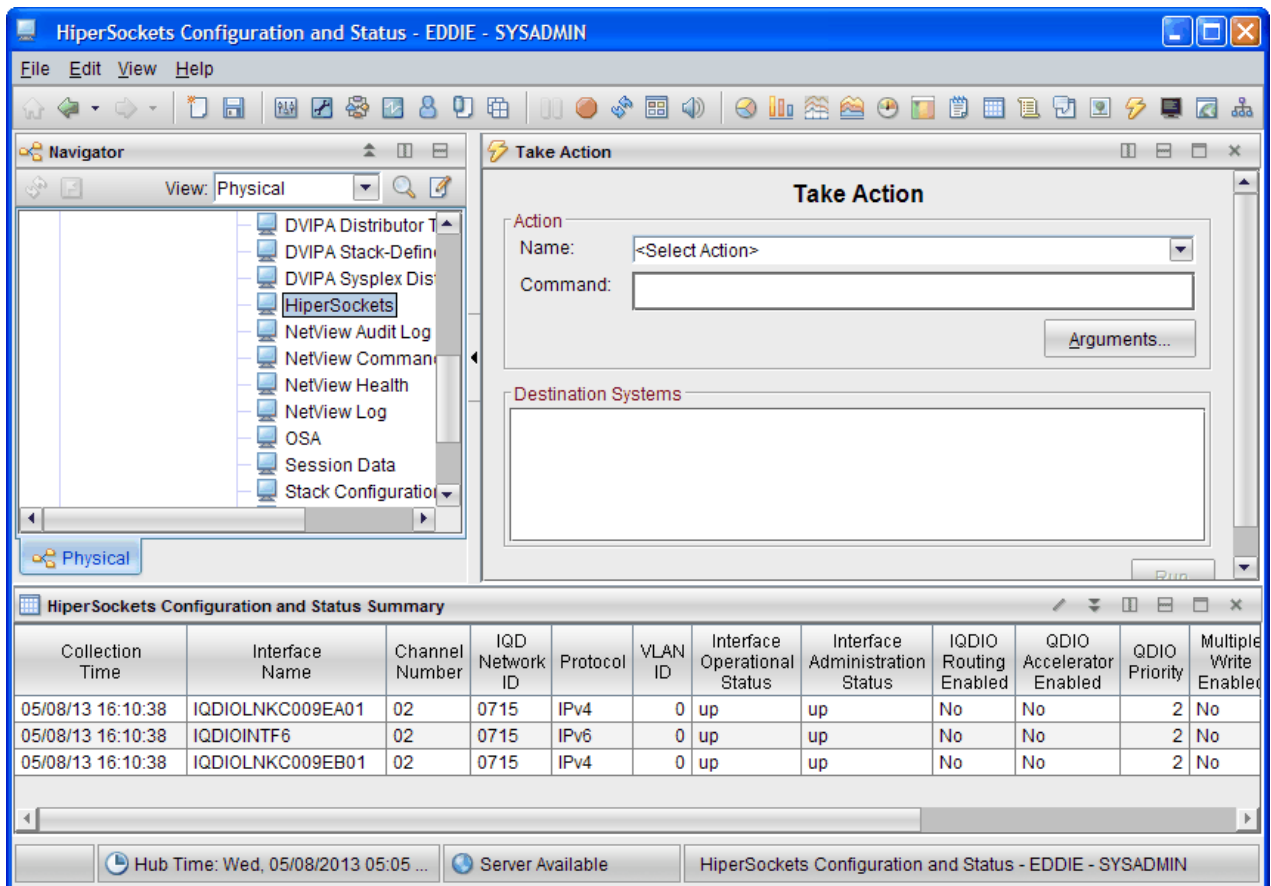


Figure 4. HiperSockets Configuration and Status Workspace

NetView Applications Workspace

The NetView Applications workspace displays information about the NetView applications.

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **NetView Health**.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

This workspace provides data that is updated by using sampling and events.

Historical data is enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 5](#) on page 25.

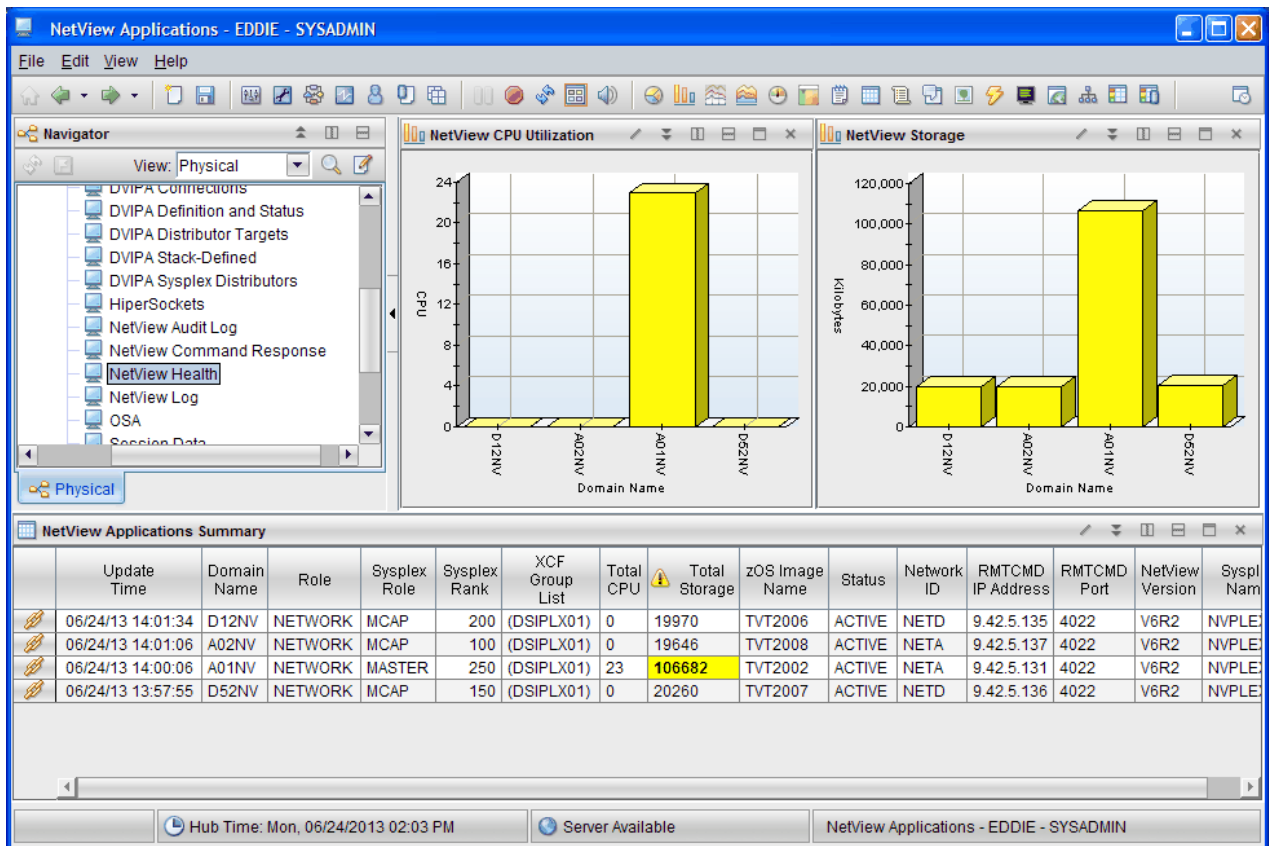


Figure 5. NetView Applications Workspace

OSA Channels and Ports Workspace

The OSA Channels and Ports workspace displays the configuration and status data for the OSA channels and ports. The data collection for this information requires Resource Object Data Manager (RODM).

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **OSA**.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

This workspace provides data that is updated by using sampling.

Historical data is not enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 6 on page 26](#).

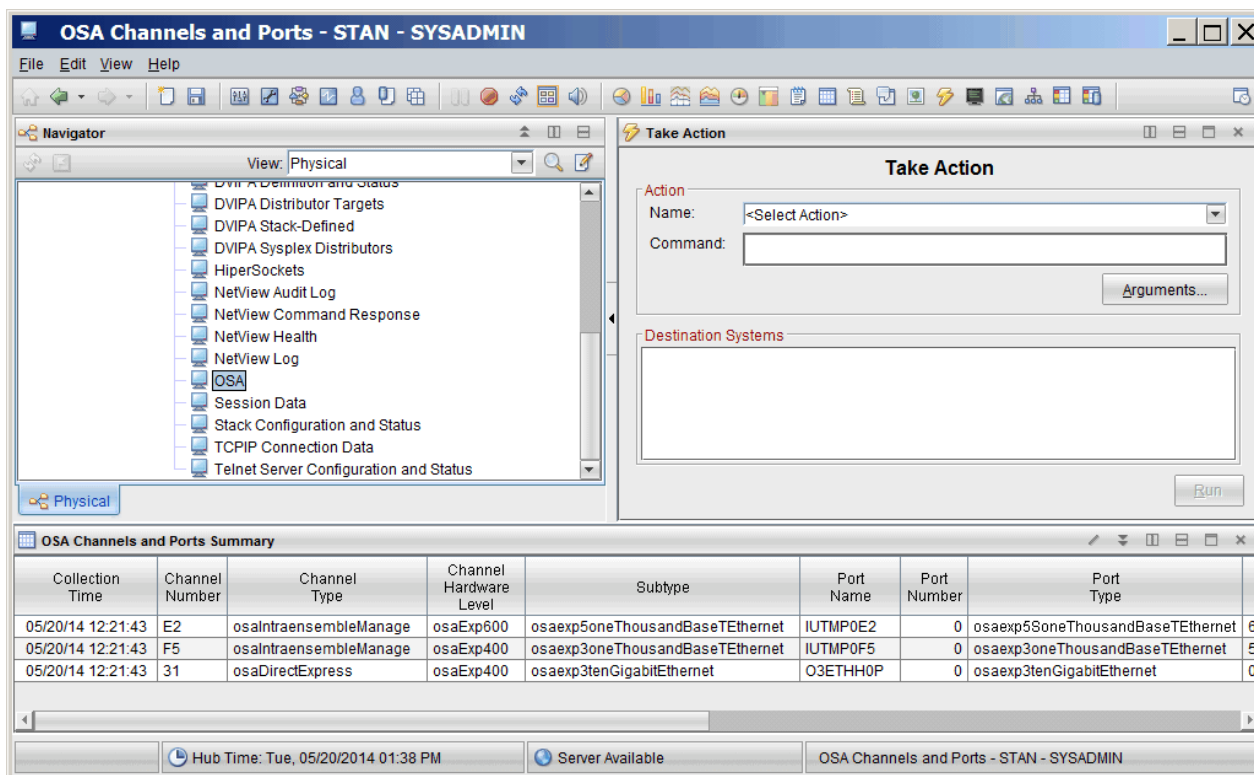


Figure 6. OSA Channels and Ports Workspace

Stack Configuration and Status Workspace

The Stack Configuration and Status workspace displays information about the z/OS Communications Server stacks.

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **Stack Configuration and Status**.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

Historical data is not enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 7 on page 27](#).

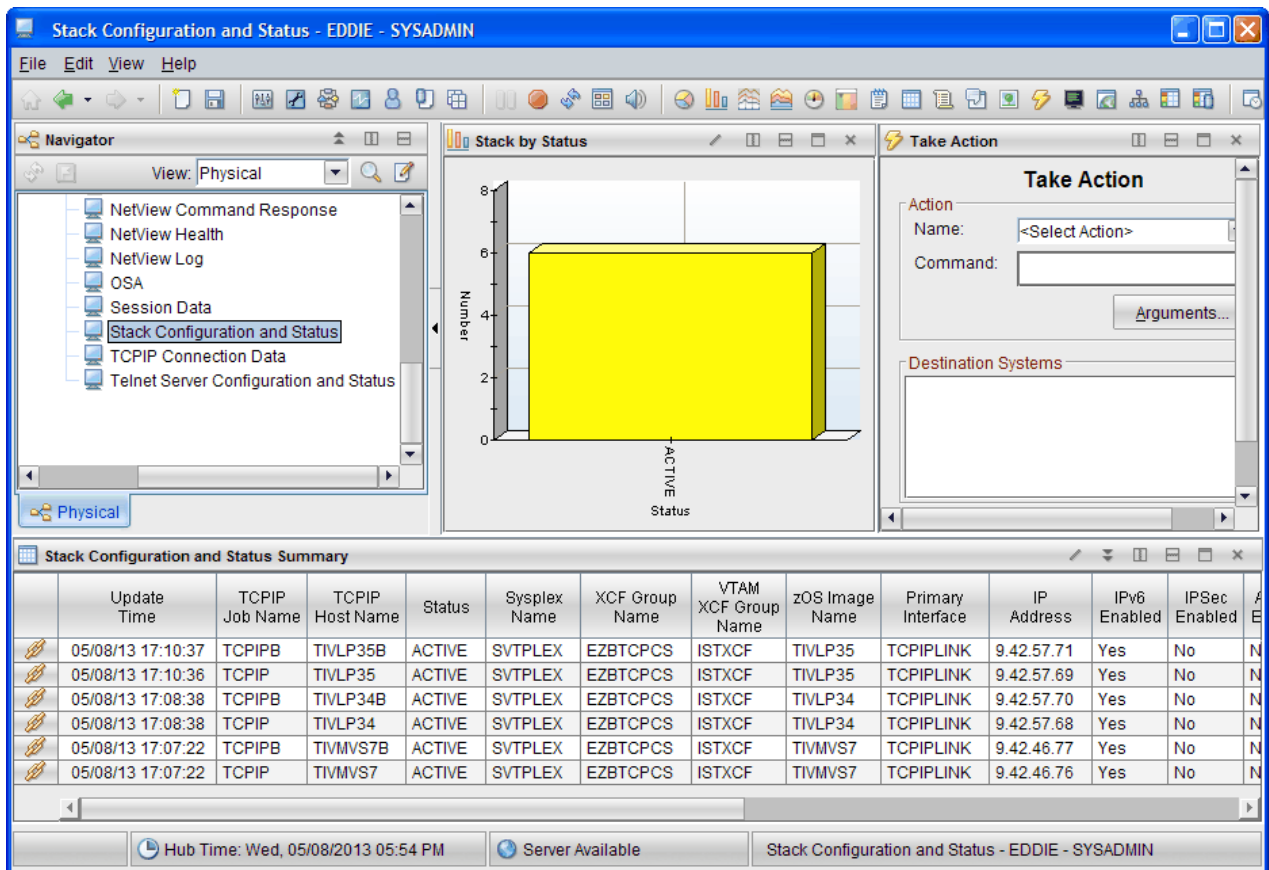


Figure 7. Stack Configuration and Status Workspace

Telnet Server Configuration and Status Workspace

The Telnet Server Configuration and Status workspace displays information about Telnet servers and Telnet server ports.

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **Telnet Server Configuration and Status**.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

This workspace provides data that is updated by using sampling and events.

Historical data is not enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 8 on page 28](#).

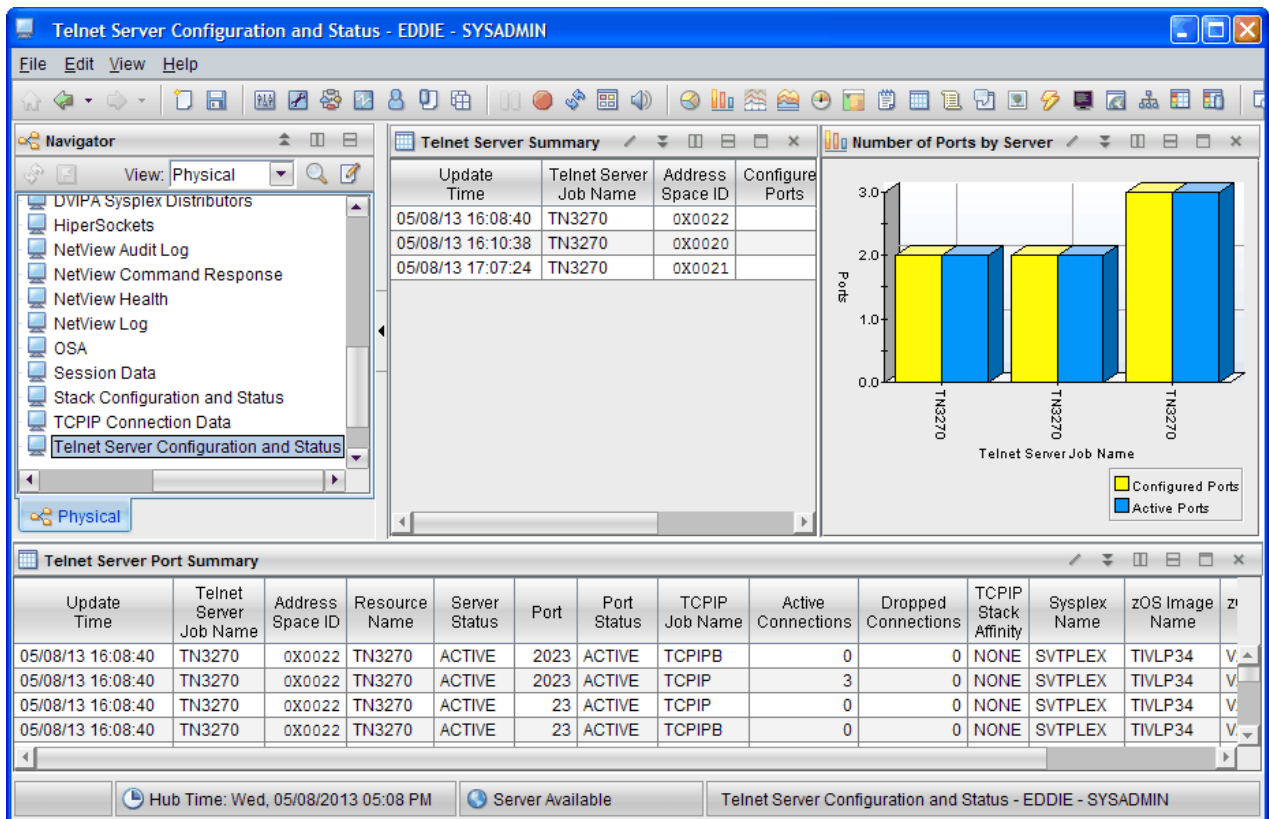


Figure 8. Telnet Server Configuration and Status Workspace

Using NetView Management Console Topology

The NetView management console displays the resources that are discovered by the discovery manager in the SysplexView and SystemzView topological views.

The Network Views view, which is similar to the view that is shown in [Figure 9 on page 29](#), contains the SysplexView and SystemzView objects.

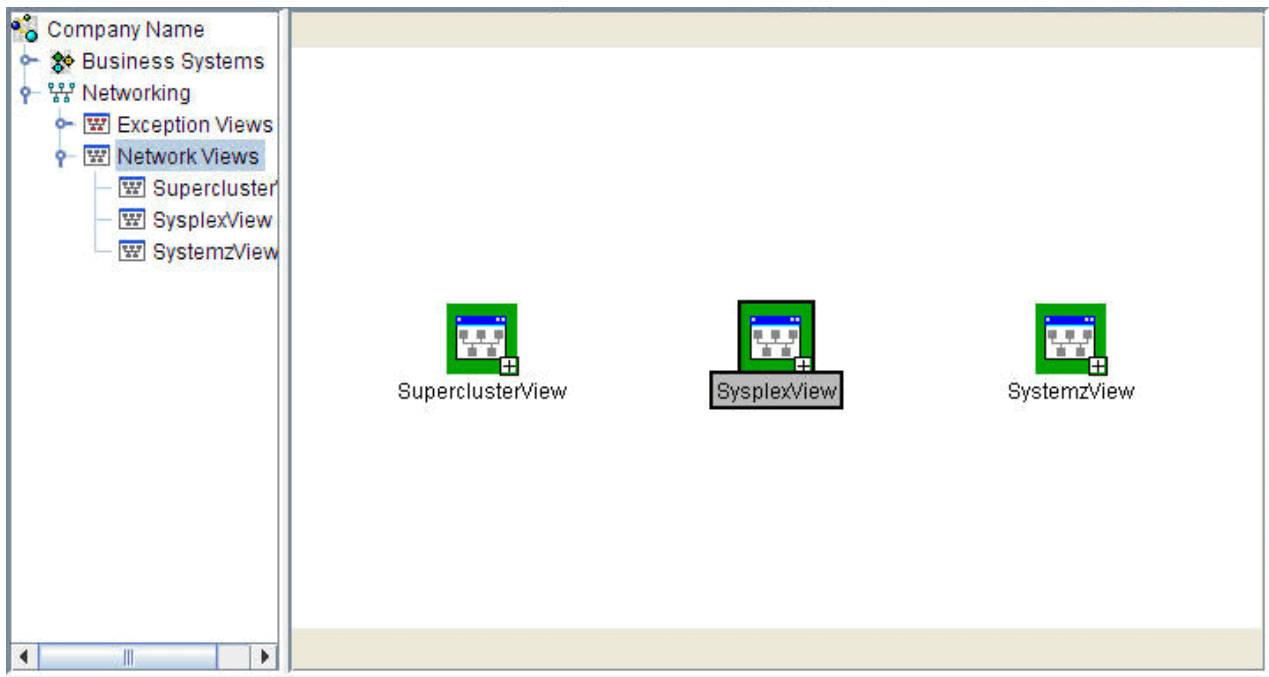


Figure 9. Network Views View

Starting with this view, you can view the sysplex topology (“Viewing Sysplex Topology” on page 32) or the z Systems® topology (“Using z Systems Topology” on page 38). For information about the objects that are shown in either topology, see [“Sysplex and z Systems Objects” on page 29](#).

Sysplex and z Systems Objects

The following types of objects are displayed in the sysplex and z Systems topology views:

- Aggregate objects that represent the following resources:
 - Sysplex
 - z Systems host
 - NetView applications
 - IP subplex
 - TCP/IP stacks
 - TCP/IP stack
 - Telnet server
 - Telnet server port
 - Central processor complex (CPC)
 - Channel subsystem
 - Logical partition (LPAR)
 - IP interface
 - Open Systems Adapter (OSA)
 - OSA port
 - HiperSockets adapter
- Real objects that represent the following resources:
 - z Systems host
 - Coupling facility
 - NetView program

- TCP/IP stack
- Telnet server
- Telnet server port
- IP interface
- Virtual local area network (VLAN)
- OSA port
- HiperSockets adapter

[Figure 10 on page 31](#) shows geometric shapes and icons for the various types of objects that are displayed in sysplex and z Systems views.

Note: The NetView management console, by default, displays icons in views. To display geometric shapes instead of icons, click **View > Show Icons**.

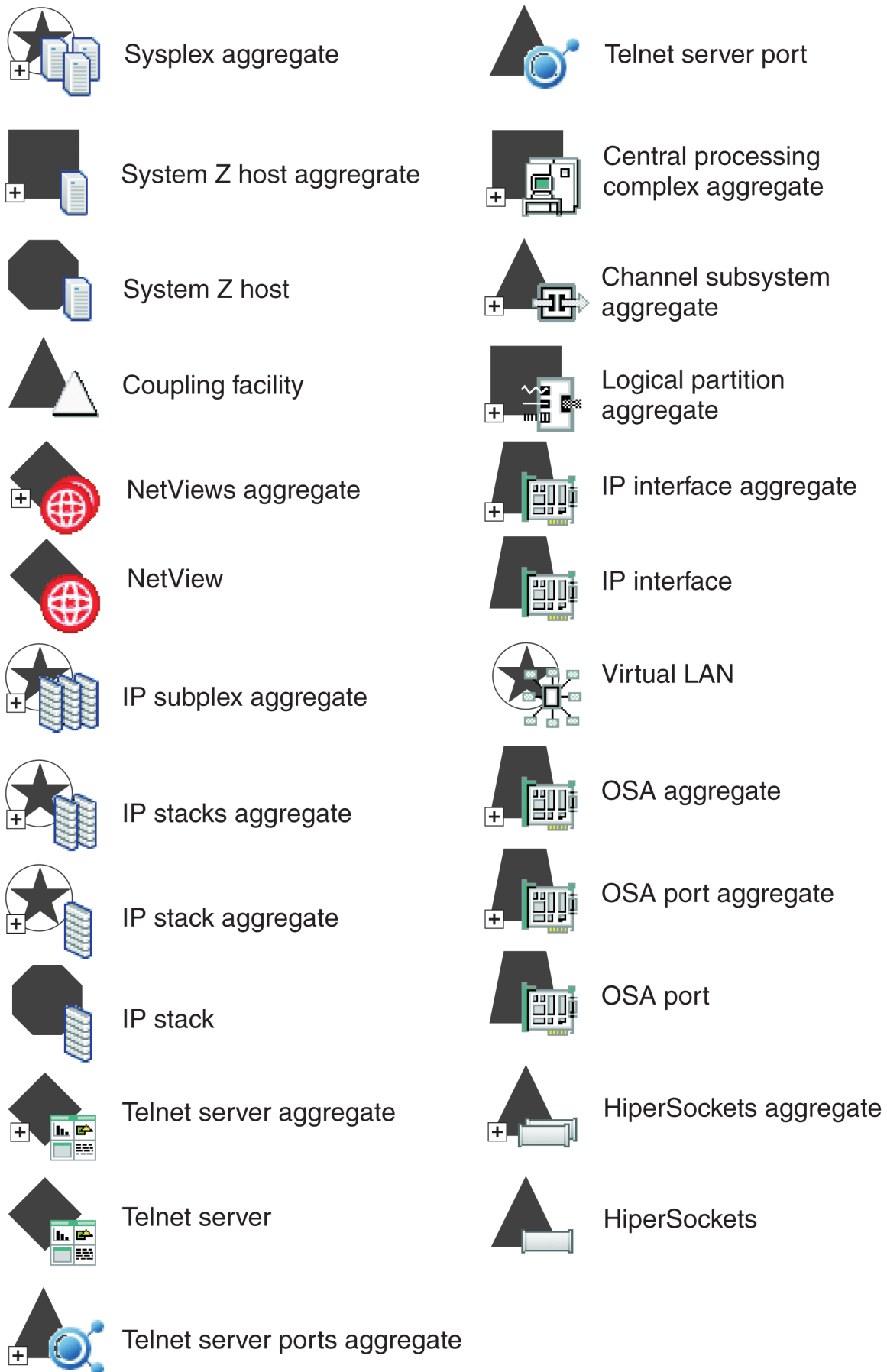


Figure 10. Types of SysplexView and SystemzView Objects

Viewing Sysplex Topology

The SysplexView view provides a logical topology that starts at the sysplex and shows the topology for applications, including TCP/IP stacks, Telnet servers and ports, and NetView applications.

This example shows how to navigate the NetView management console views to manage your sysplexes. For this example, data collection is enabled for all the resources that can be discovered by the discovery manager.

In the NetView Views view (Figure 9 on page 29), double-click the SysplexView object. The resulting view, which is similar to the view that is shown in Figure 11 on page 32, is the SysplexView view. This view shows all the sysplexes. The NETVPLEX object is a sysplex aggregate object.

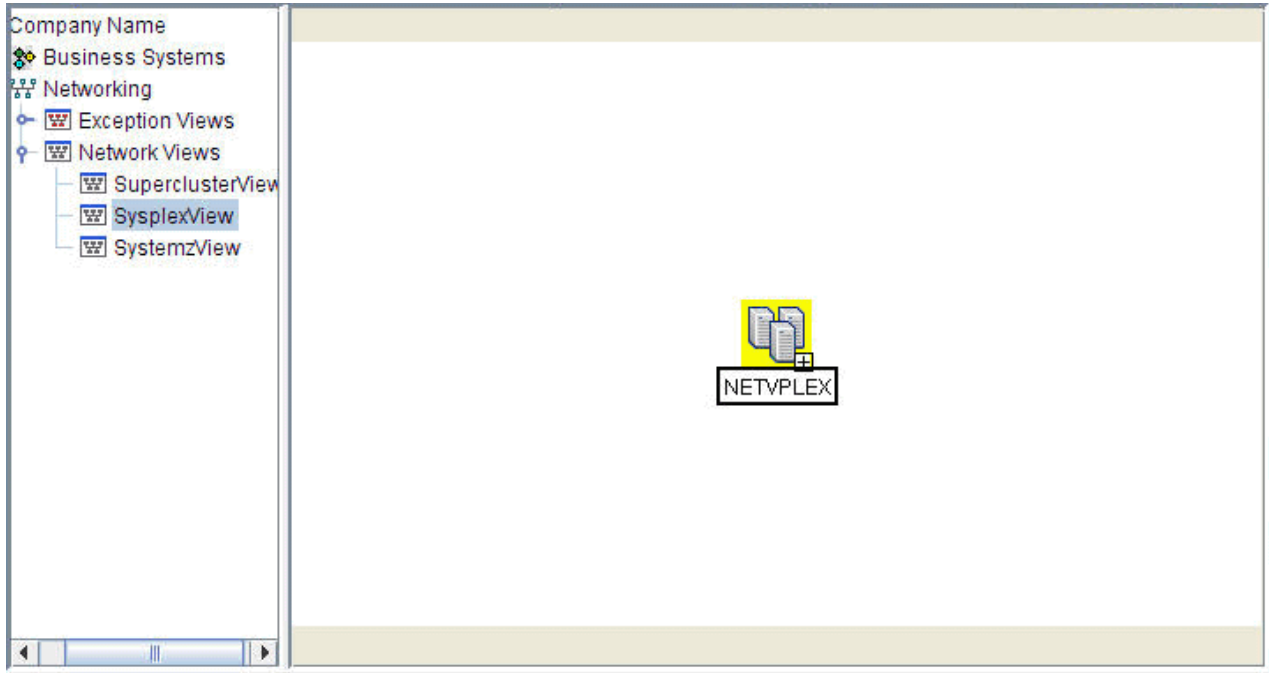


Figure 11. SysplexView View

Double-click the NETVPLEX object. The resulting view, which is similar to the view that is shown in Figure 12 on page 33, is the NETVPLEX More Detail Logical view. This view shows all of the z/OS images that belong to the NETVPLEX sysplex. The NETVPLEX More Detail Logical view contains system aggregate objects for the TIVMVS21, TIVMVS22, and TIVMVS23 systems and a single real object for the coupling facility (RALNSCFF). On the TIVMVS21 system, the discovery of both Open Systems Adapter (OSA) and HiperSockets resources is enabled. Thus, both logical and physical information is available for the TIVMVS21 system.

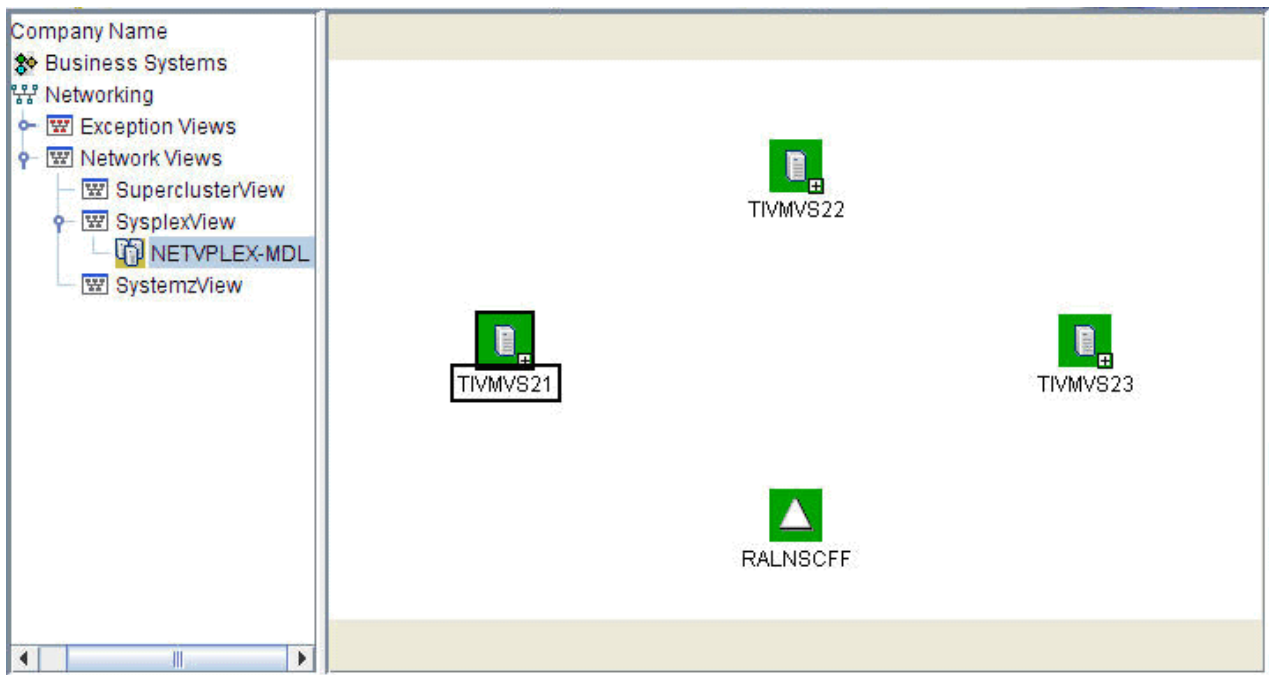


Figure 12. NETVPLEX More Detail Logical View

To see the logical information, double-click the TIVMVS21 object, and, in the View Selection window that is displayed, select **TIVMVS21-MDL** and click **OK**. The resulting view, which is similar to the view that is shown in Figure 13 on page 33, is the TIVMVS21 More Detail Logical view. This view shows a real object for the z/OS image (the TIVMVS21 object) and aggregate objects for NetView applications, TCP/IP stacks, and Telnet servers (the NETVIEWS, STACKS, and TELNETSERVERS objects). The TIVMVS21 object is a z/OS image object, the NETVIEWS object is a NetViews aggregate object, the STACKS object is a TCP/IP stacks aggregate object, and the TELNETSERVERS object is a Telnet server aggregate object.

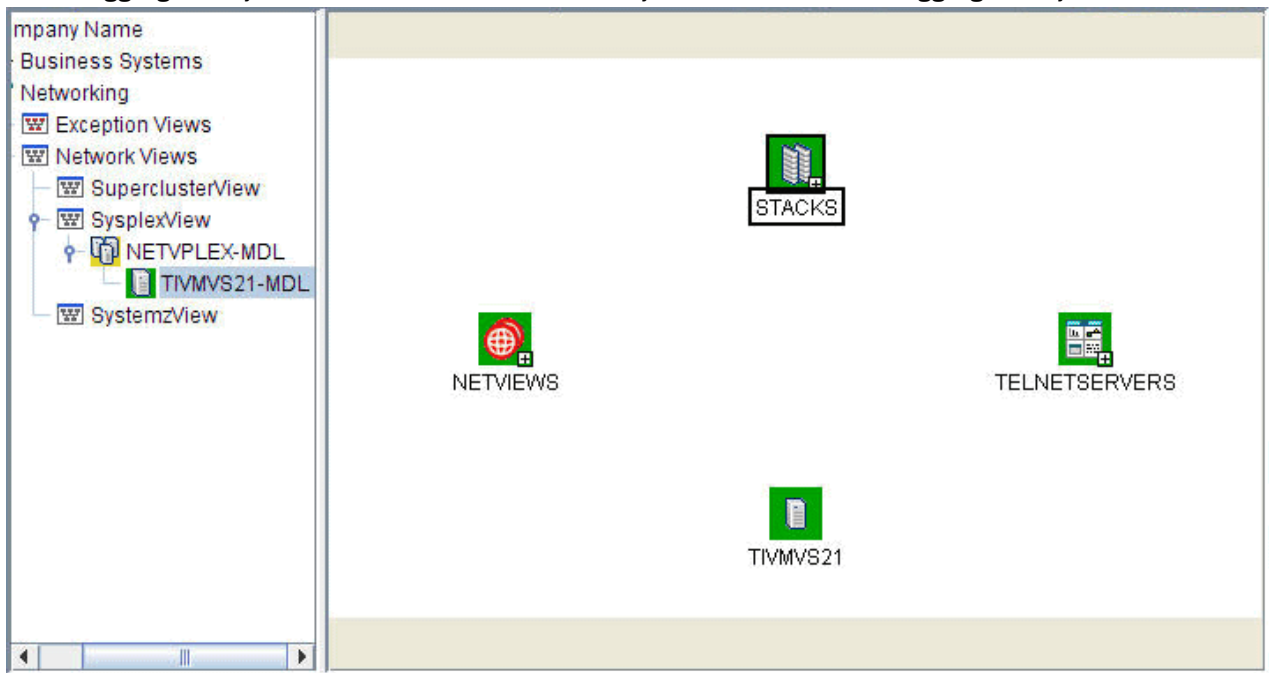


Figure 13. TIVMVS21 More Detail Logical View

Viewing TCP/IP Stack Information

In the TIVMVS21 More Detail Logical view (Figure 13 on page 33), double-click the STACKS object. The resulting view, which is similar to the view that is shown in Figure 14 on page 34, is the STACKS More

Detail Logical view. This view shows all of the TCP/IP stacks that were discovered for this system, in this case, the TCPIP and TCPIPB stacks, which are TCP/IP stack aggregate objects.

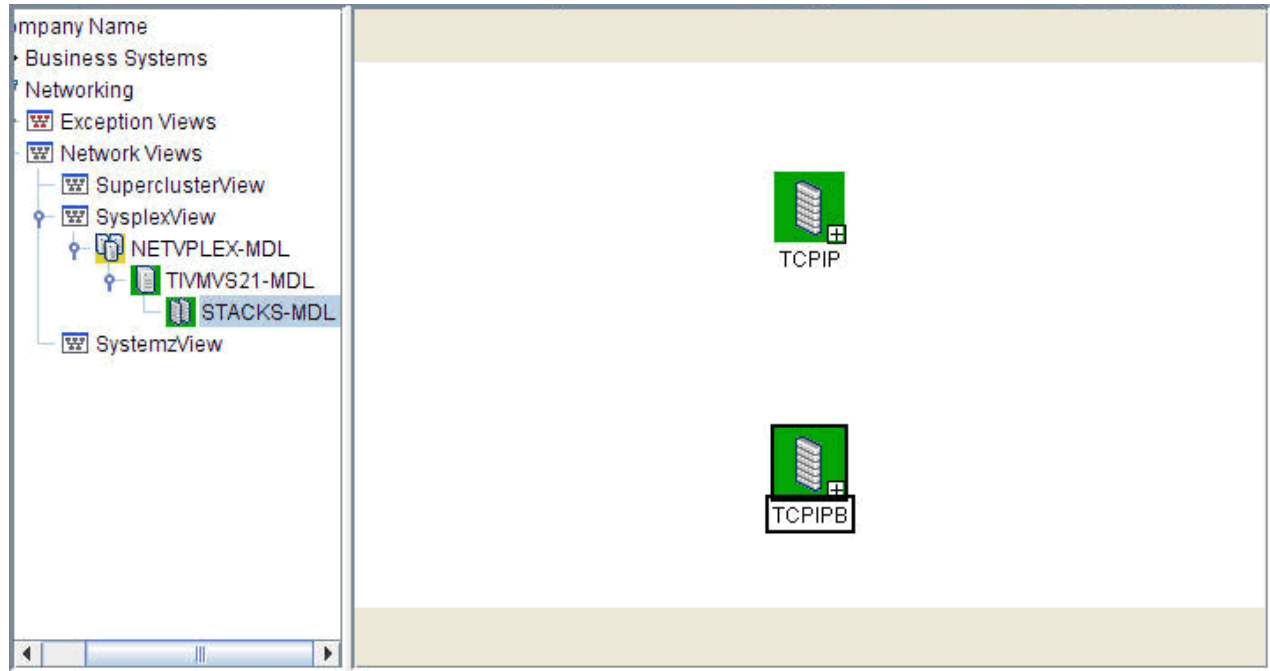


Figure 14. STACKS More Detail Logical View

Double-click the TCPIPB object. The resulting view, which is similar to the view that is shown in Figure 15 on page 34, is the TCPIPB More Detail Logical view. This view shows a real object for the stack, the TCPIPB object (which is a TCP/IP stack object) and several real objects that represent the IP interfaces that were discovered for this stack. Note that the TCPIPLINK1 and LVIPA1 objects indicate IP interfaces that are not active.

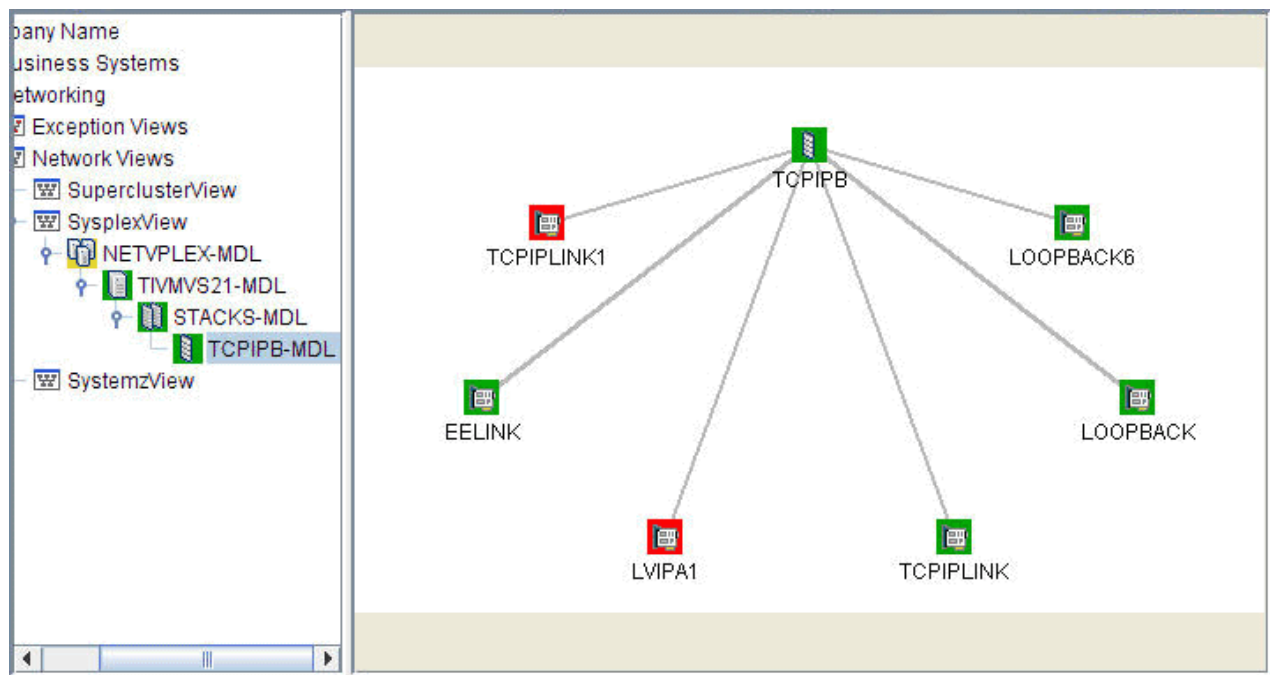


Figure 15. TCPIPB More Detail Logical View

Viewing NetView Information

To see information about the NetView applications, return to the TIVMVS21 More Detail Logical view (Figure 13 on page 33). You can also display this view by clicking **TIVMVS21-MDL** in the business tree.

In the TIVMVS21 More Detail Logical view (Figure 13 on page 33), double-click the NETVIEWS object. The resulting view, which is similar to the view that is shown in Figure 16 on page 35, is the NETVIEWS More Detail Logical view. This view shows all of the NetView instances for this z/OS image. In this case, a single NetView instance is shown, the CNM30 object, which is a NetView object.

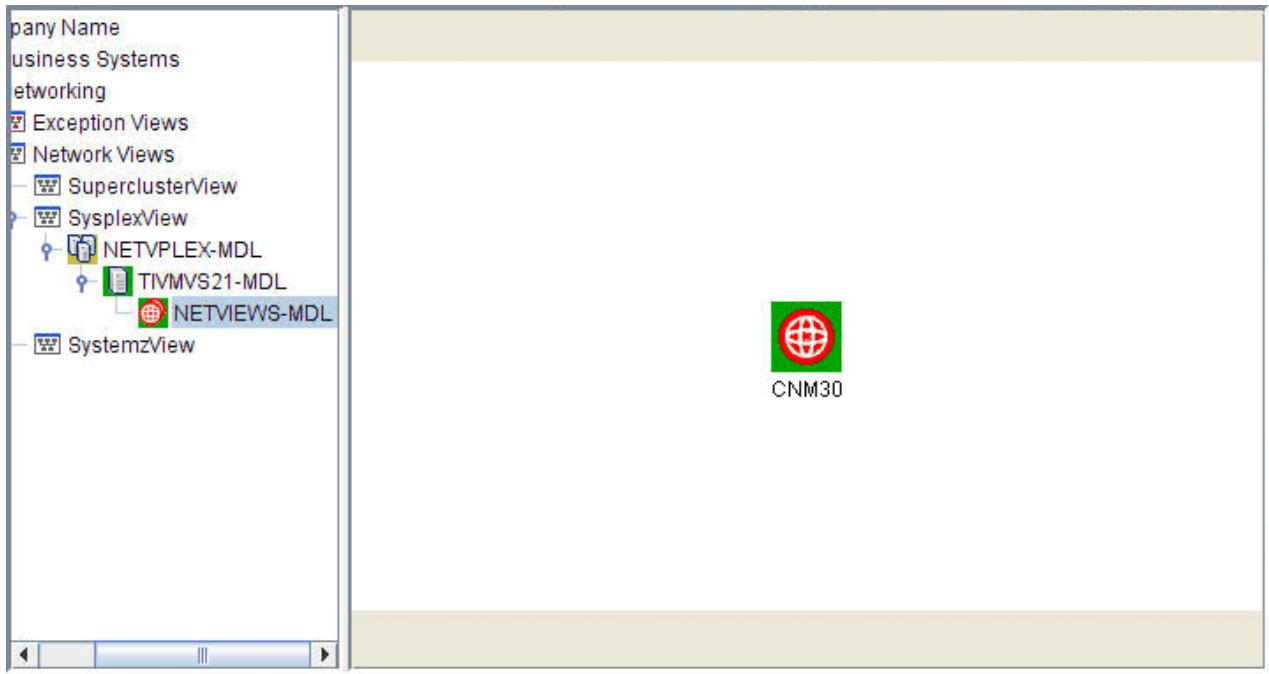


Figure 16. NETVIEWS More Detail Logical View

Viewing Telnet Server Information

To see information about the Telnet servers, return to the TIVMVS21 More Detail Logical view (Figure 13 on page 33). You can also display this view by clicking **TIVMVS21-MDL** in the business tree.

In the TIVMVS21 More Detail Logical view, double-click the TELNETSERVERS object. The resulting view, which is similar to the view that is shown in Figure 17 on page 36, is the TELNETSERVERS More Detail Logical view. This view shows the real objects for the Telnet servers on this system. Each Telnet server icon shows an aggregate object for the Telnet ports that are known to the Telnet server.

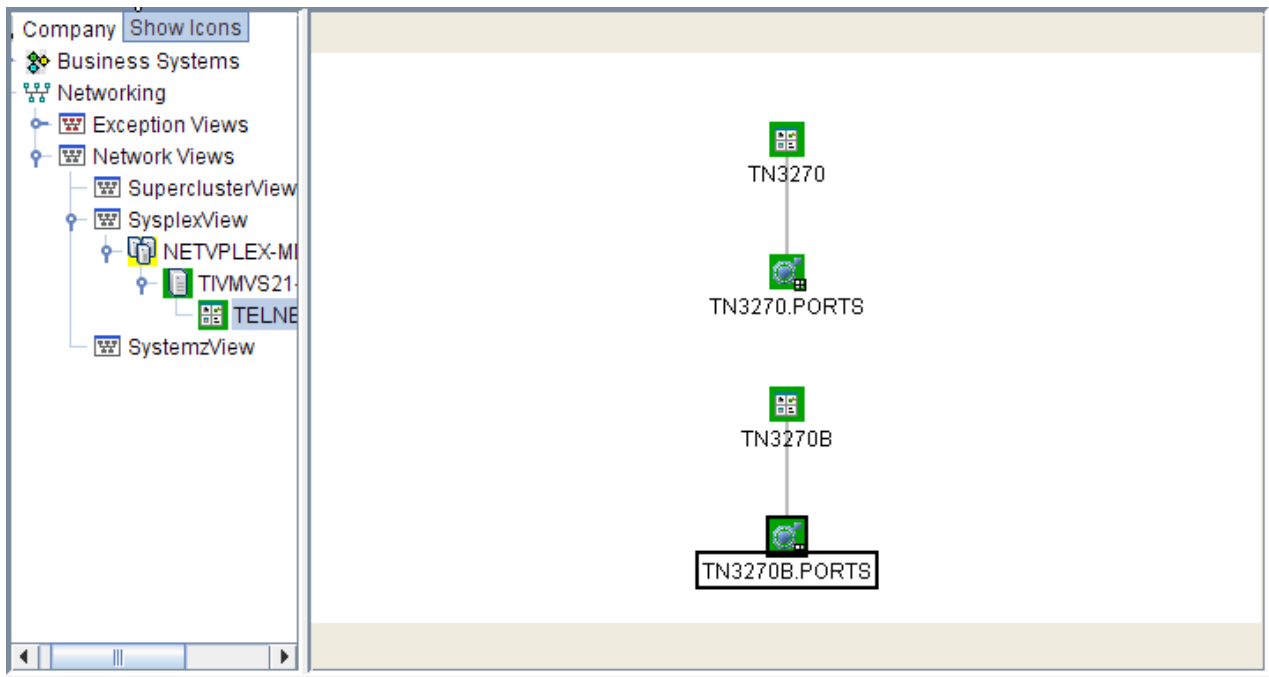


Figure 17. TELNETSERVERS More Detail Logical View

In the TELNETSERVERS More Detail Logical view, double-click the Telnet port aggregate object, in this case, the TN3270B.PORTS object. The resulting view, which is similar to the view that is shown in Figure 18 on page 36, is the TN3270B.PORTS More Detail Logical view. This view shows the real Telnet port object, in this case, the TN3270B.PORT823.

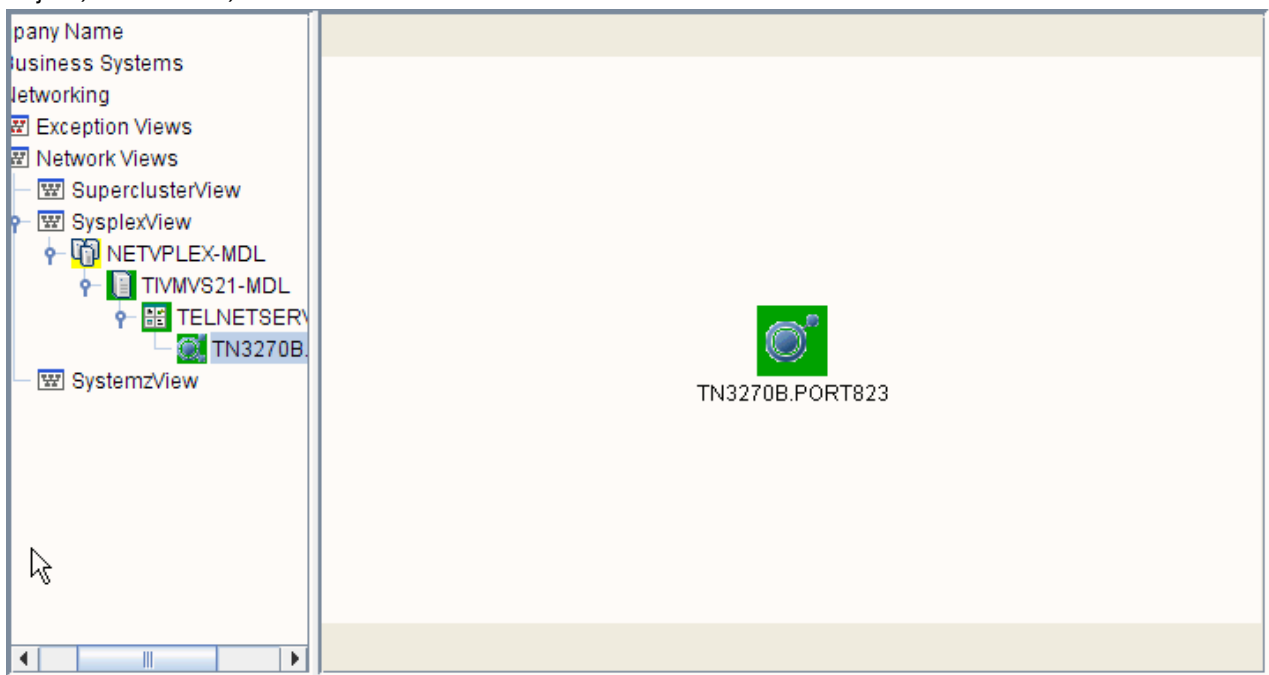


Figure 18. TN3270B.PORTS More Detail Logical View

From this view, you can navigate to the Telnet port peer view for this Telnet port by selecting the object and clicking **Configuration > Peers**. The resulting view, which is similar to the view that is shown in Figure 19 on page 37, is the NETVPLEX.TIVMVS21.TN3270B.823 view. Telnet ports can be linked to by one or more TCP/IP stacks. This port peer view shows the TCP/IP stacks that are listening on the port. In this case, the TN3270B.PORT823 Telnet port is linked to by the TCP/IP and TCP/IPB TCP/IP stacks.

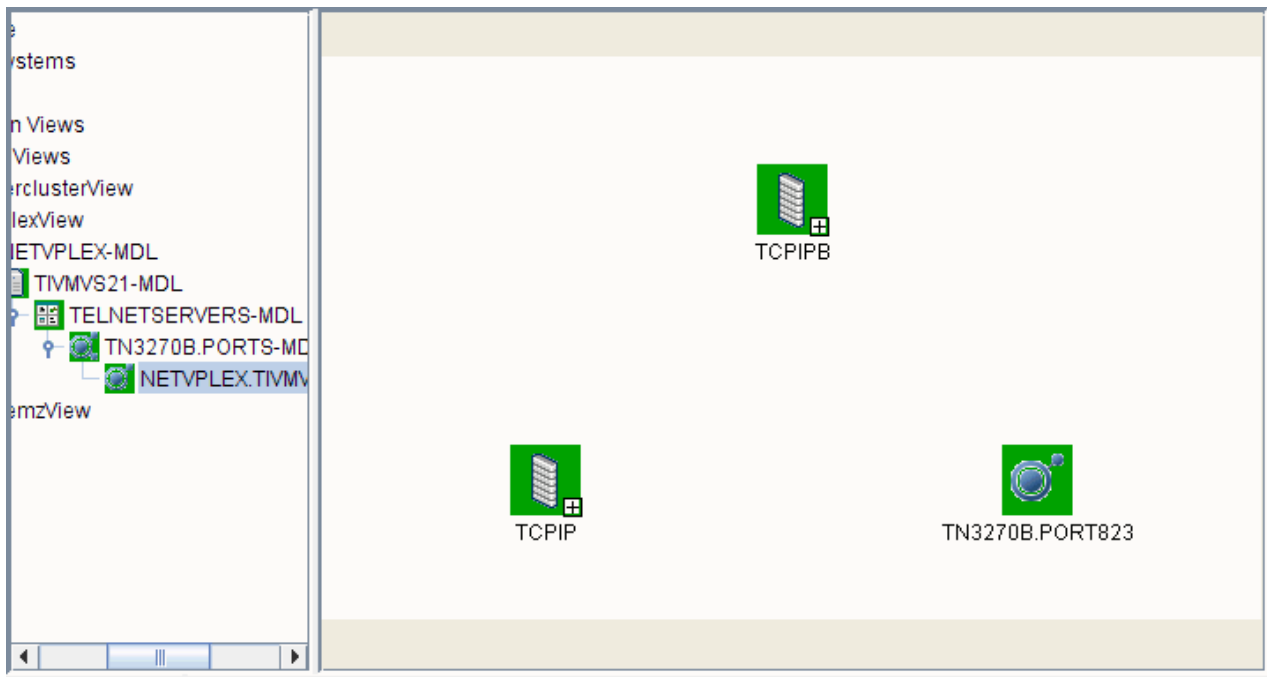


Figure 19. NETVPLEX.TIVMVS21.TN3270B.823 (Port Peers) View

Viewing Subplex Information

A sysplex can have one or more TCP/IP subplexes defined to enable partitioning of TCP/IP stacks for different security or functional attributes. To see the configuration of the subplexes in the sysplex, start at the SysplexView view, which is shown in Figure 11 on page 32. From this view, select the NETVPLEX object, and click **Configuration > Logical Only**. The resulting view, which is similar to the view that is shown in Figure 20 on page 37, is the sysplex logical only view, in this case, the NETVPLEX-LOG view. This view shows a connection between the sysplex and the subplexes that were discovered within the sysplex. The subplexes are shown as a TCP/IP subplex aggregate object, in this case, the EZBTCPCS object.

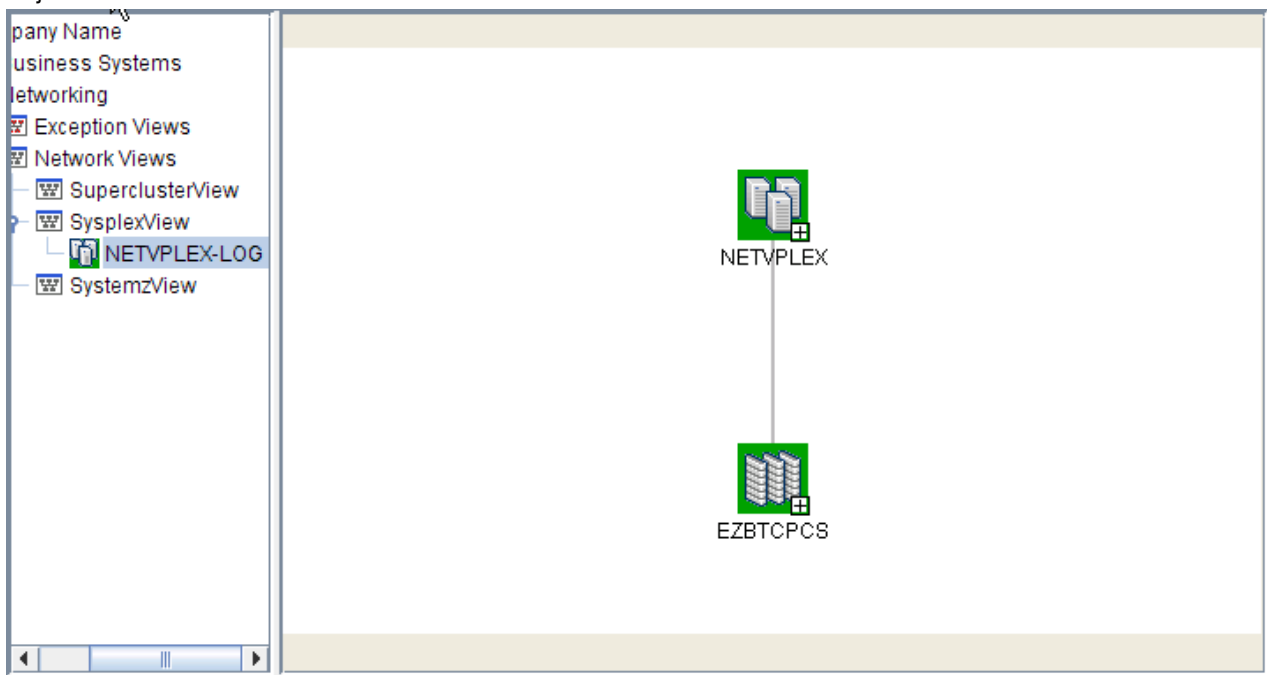


Figure 20. NETVPLEX-LOG (Sysplex Logical Only) View

To see a more detail logical view for a subplex, double-click the subplex object. The resulting view, which is similar to the view that is shown in Figure 21 on page 38, is the subplex more detail logical view, in this case, the EZBTCPCS More Detail Logical view. This view shows the aggregate objects for the TCP/IP stacks that are members of the subplex.

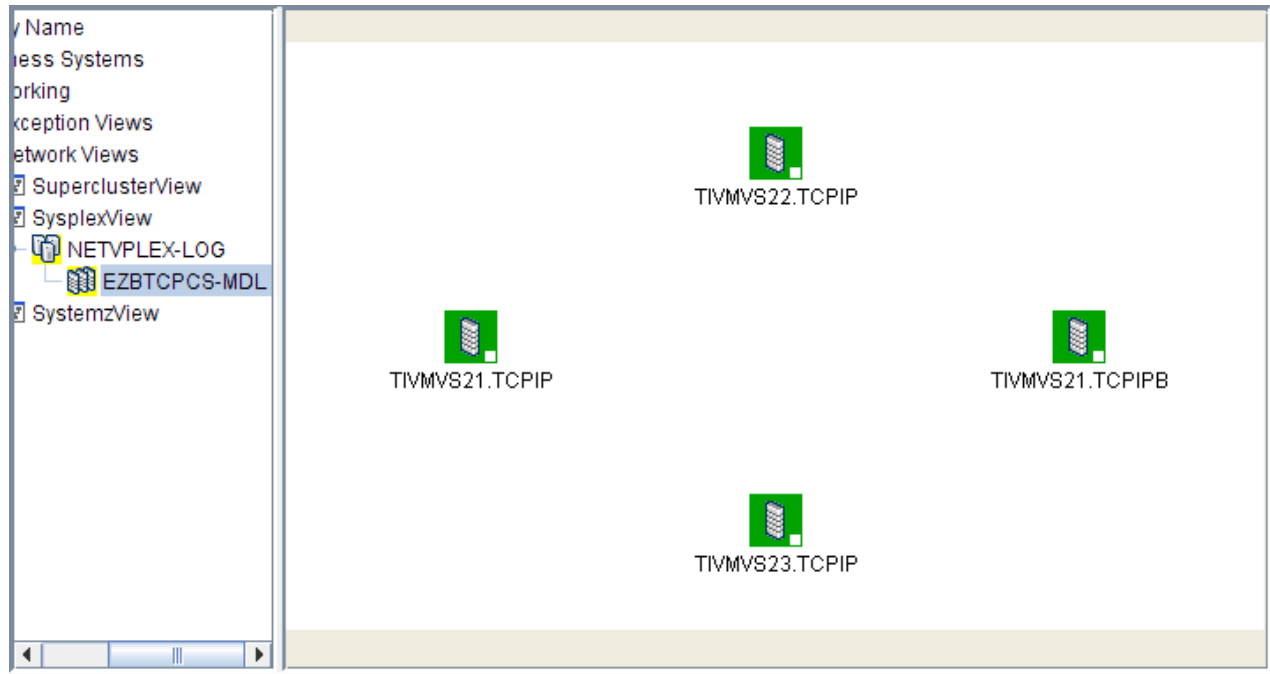


Figure 21. EZBTCPCS More Detail Logical View

Using z Systems Topology

The SystemzView view provides a physical topology that starts with the central processor complex (CPC) and emphasizes the Open Systems Adapter (OSA) and HiperSockets adapter configuration. The major benefit of this topology is in looking at the configuration parent topology views from the OSA or HiperSockets adapter, in which you can see how these adapters are shared across systems and TCP/IP stacks within a CPC.

This example shows how to navigate the NetView management console views to manage your OSA and HiperSockets resources. For this example, data collection is enabled for all the resources that can be discovered by the discovery manager.

In the NetView Views view (Figure 9 on page 29), double-click the SystemzView object. The resulting view, which is similar to the view that is shown in Figure 22 on page 39, is the SystemzView view. This view shows a central processor complex (CPC) aggregate named VM-TOKEN.

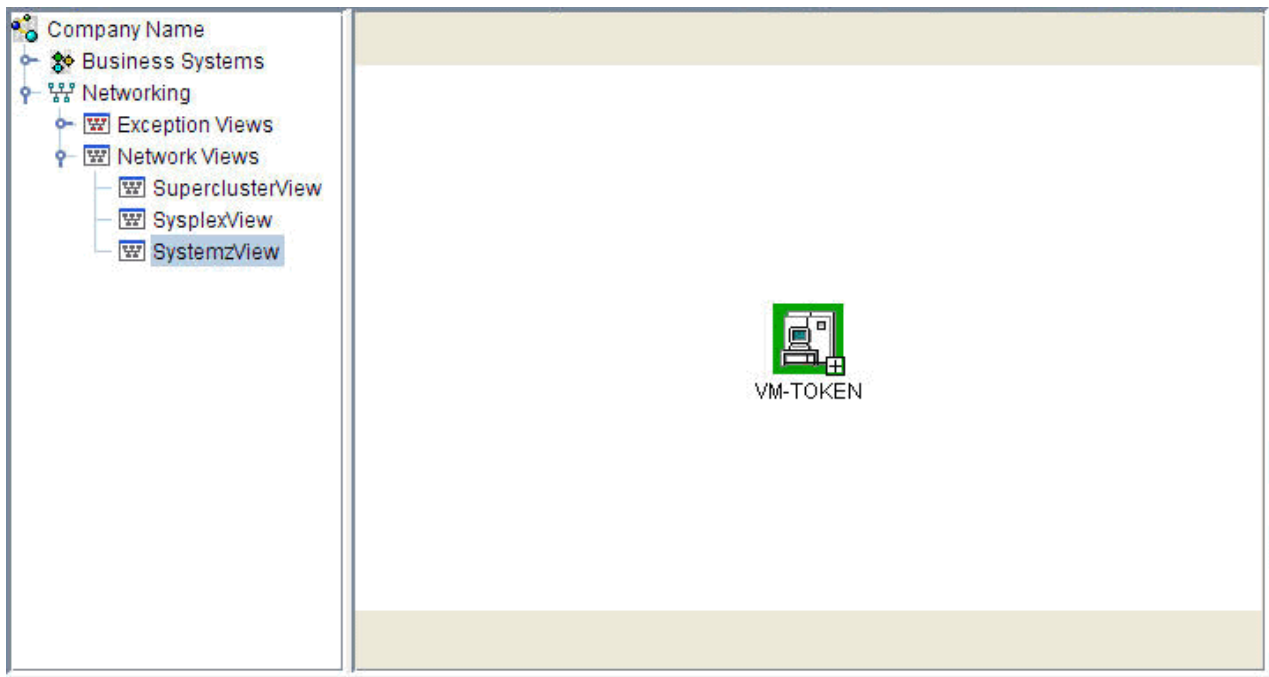


Figure 22. SystemzView View

Double-click VM-TOKEN. The resulting view, which is similar to the view that is shown in [Figure 23](#) on page 39, is the VM-TOKEN More Detail Physical view. This view shows the channel subsystem (CSS) identifiers that are contained in the VM-TOKEN central processor complex. The CSS01 object is a channel subsystem identifier aggregate object.

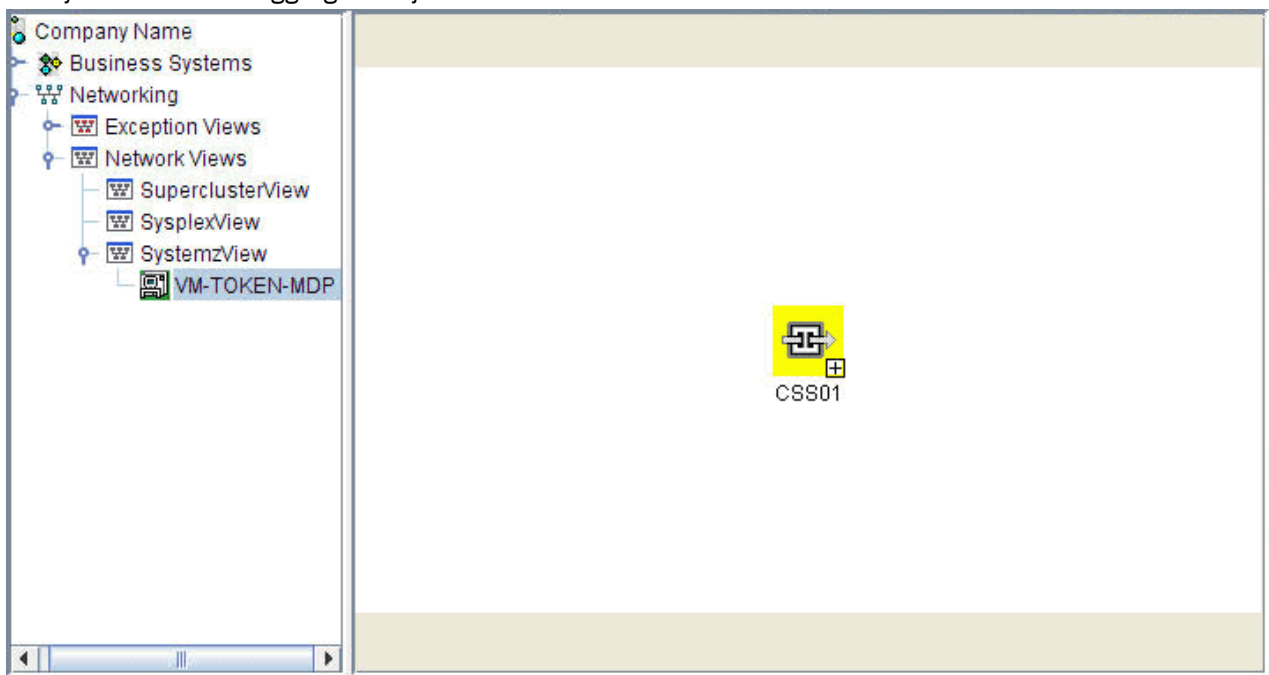


Figure 23. VM-TOKEN More Detail Physical View

Double-click the CSS01 object. The resulting view, which is similar to the view that is shown in [Figure 24](#) on page 40, is the CSS01 More Detail Physical view. This view shows the logical partitions (LPARs) of the CSS01 channel subsystem identifier. The three LPARs that are part of this complex (TIVMVS21, TIVMVS22, and TIVMVS23) are shown as LPAR aggregate objects.

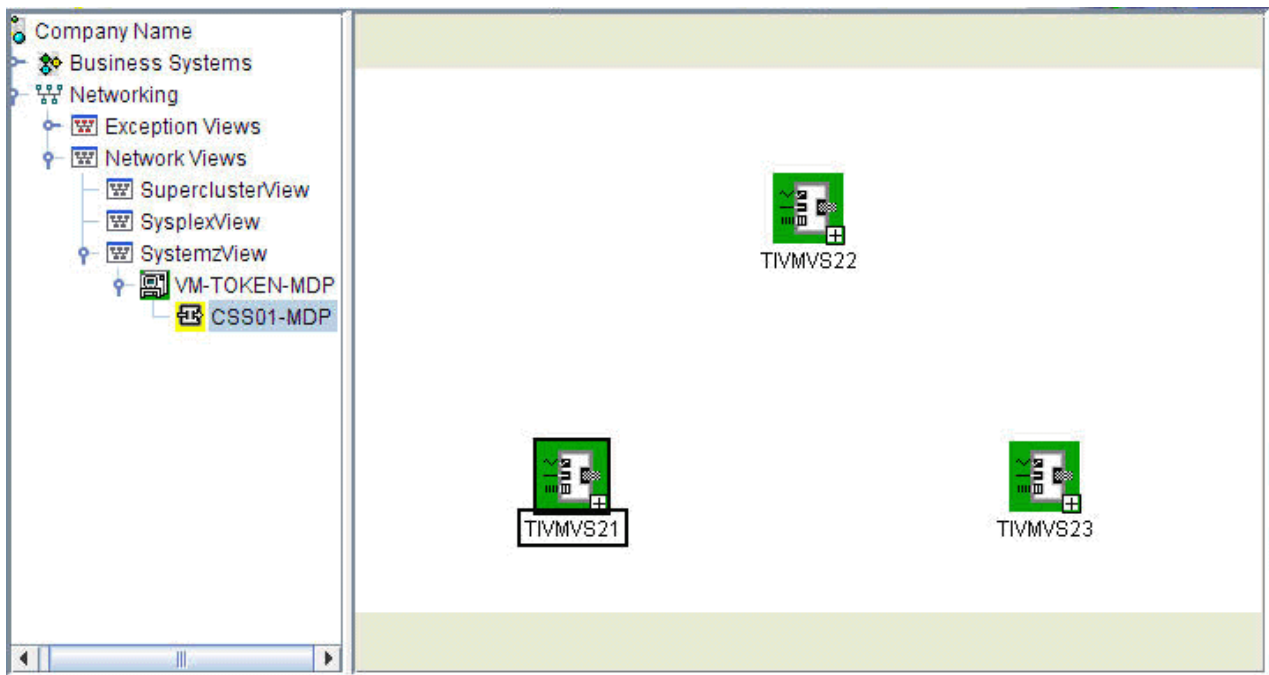


Figure 24. CSS01 More Detail Physical View

Double-click the TIVMVS21 LPAR aggregate object. The resulting view, which is similar to the view that is shown in Figure 25 on page 40, is the TIVMVS21 More Detail Physical view that shows a z/OS image aggregate object for the systems that are on the TIVMVS21 LPAR.

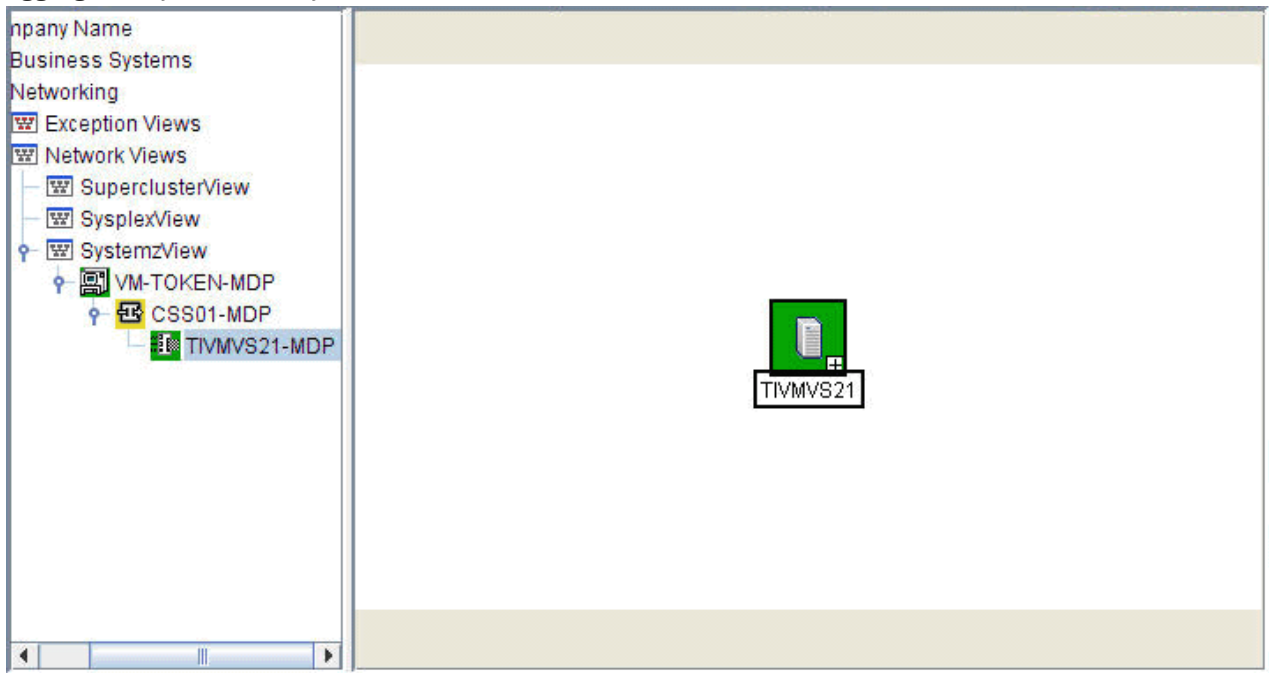


Figure 25. TIVMVS21 More Detail Physical View (z/OS Image Aggregate)

Double-click the TIVMVS21 z/OS image aggregate object. Because this system has both logical and physical topology, a View Selection window is displayed; in that window, select **More Detail Resource - Physical** and click **OK**. The resulting view, which is similar to the view that is shown in Figure 26 on page 41, is the TIVMVS21 More Detail Physical view that shows the TCP/IP stacks aggregates on the system that have physical topology. Physical topology is determined by the existence of OSA or HiperSockets resources that are associated with this stack.

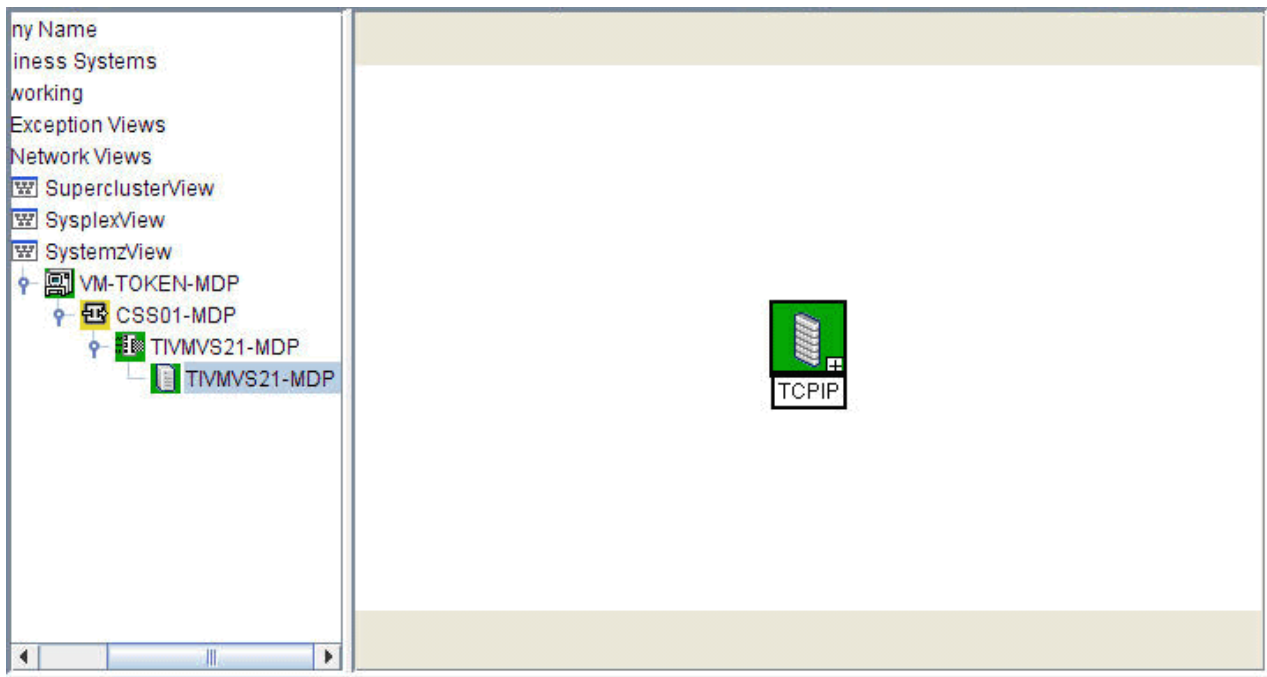


Figure 26. TIVMVS21 More Detail Physical View (IP Stack Aggregate)

Double-click the TCPIP stack aggregate object. Because this stack has both logical and physical topology, a View Selection window is displayed. In that window, select **More Detail Resource - Physical** and click **OK**. The resulting view, which is similar to the view that is shown in Figure 27 on page 41, is the TCPIP More Detail Physical view. This view shows the physical interfaces as aggregate objects. These aggregate objects represent the OSA or HiperSockets interfaces that are known to the TCP/IP stack.

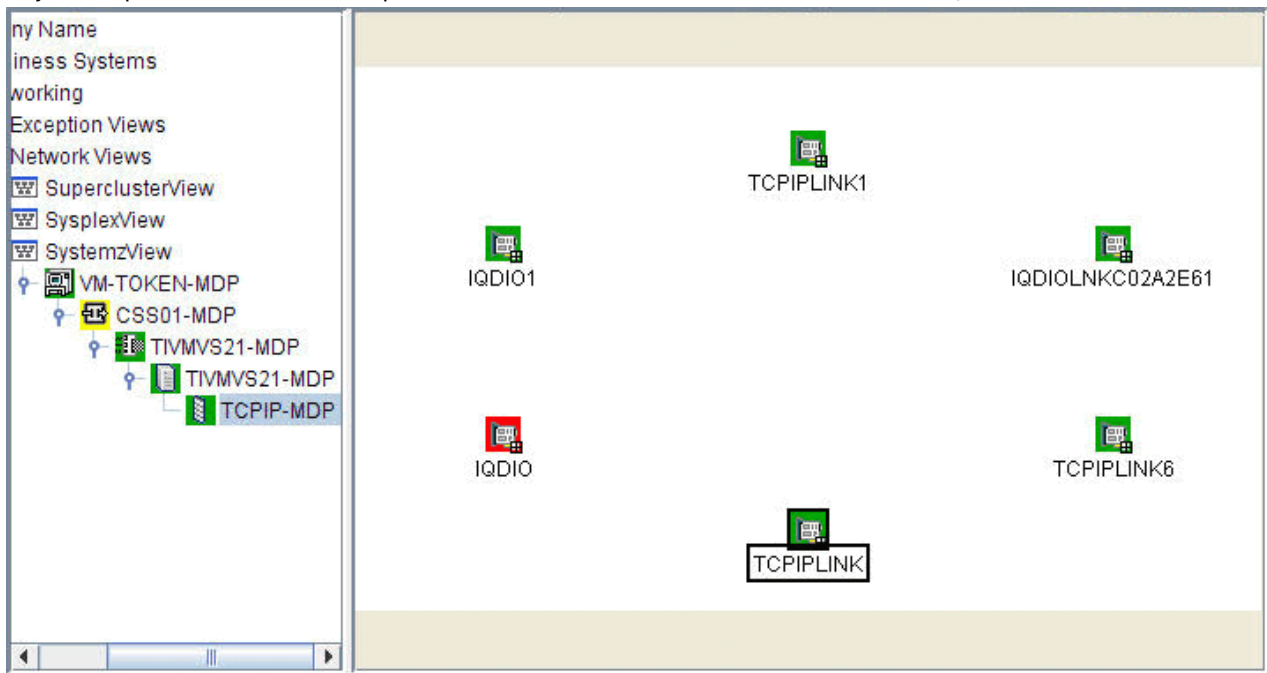


Figure 27. TCPIP More Detail Physical View

Viewing OSA Information

Double-click the TCIPLINK object. The resulting view, which is similar to the view that is shown in Figure 28 on page 42, is the TCIPLINK More Detail Physical view. This view shows the OSA00B0 aggregate object.

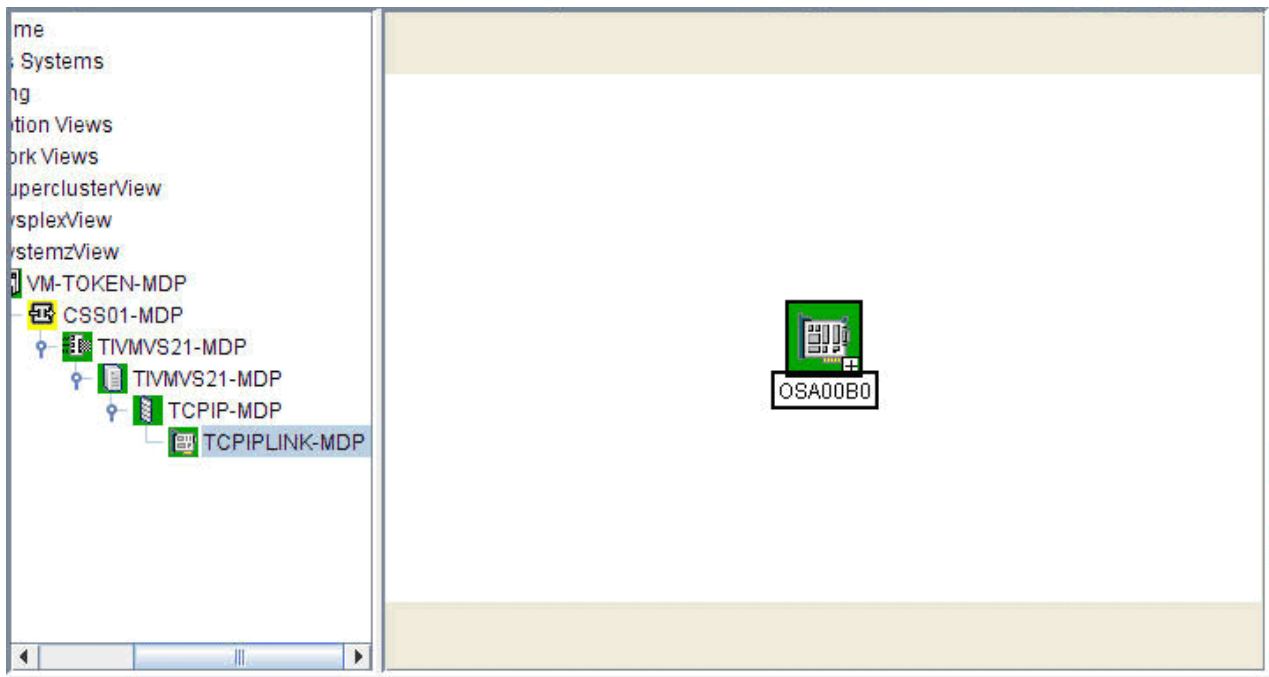


Figure 28. TCPIPLINK More Detail Physical View

Double-click the OSA00B0 aggregate object. The resulting view, which is similar to the view that is shown in Figure 29 on page 42, is the OSA00B0 More Detail Physical view. This view shows the PORT0.RP563B0 object, which represents the OSA ports as aggregate resources.

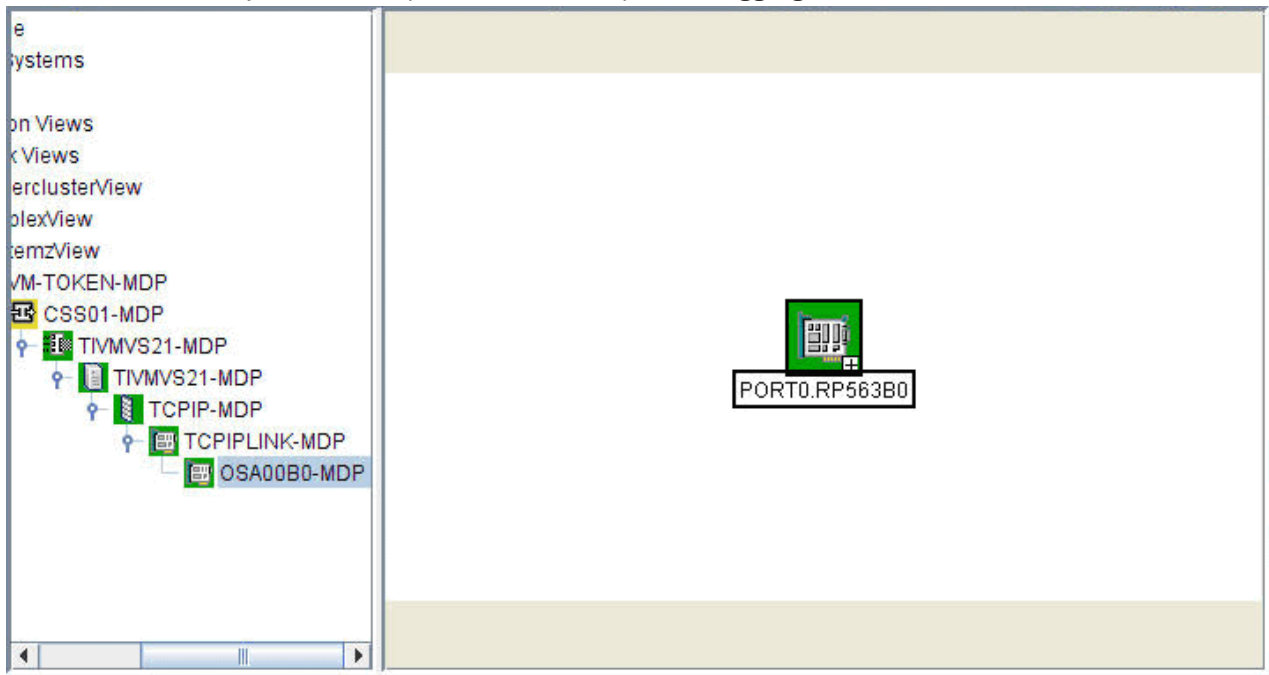


Figure 29. OSA00B0 More Detail Physical View

Double-click the PORT0.RP563B0 OSA port aggregate object. The resulting view, which is similar to the view that is shown in Figure 30 on page 43, is the PORT0.RP563B0 More Detail Physical view. This view shows the OSA port real objects and virtual local area networks (VLANs) that are associated with the OSA interface. PORT0.RP563B0 is an OSA port object, and VLAN510 is a virtual LAN object.

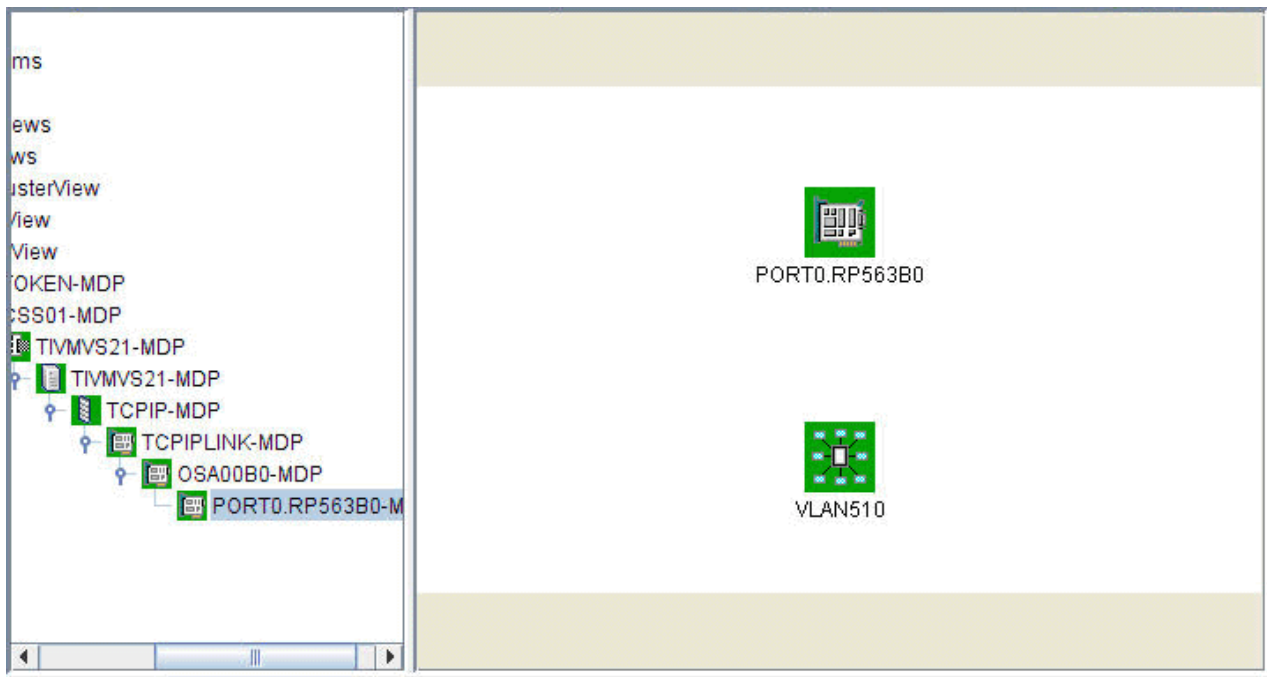


Figure 30. PORT0.RP563B0 More Detail Physical View

From this view, you can navigate to the configuration parents view for this OSA port by right-clicking the PORT0.RP63B0 object and clicking **Configuration > Parents**. The resulting view, which is similar to the view that is shown in Figure 31 on page 43, is the PORT0.RP63B0-PAR view. This view shows the full topology and connectivity for the OSA port.

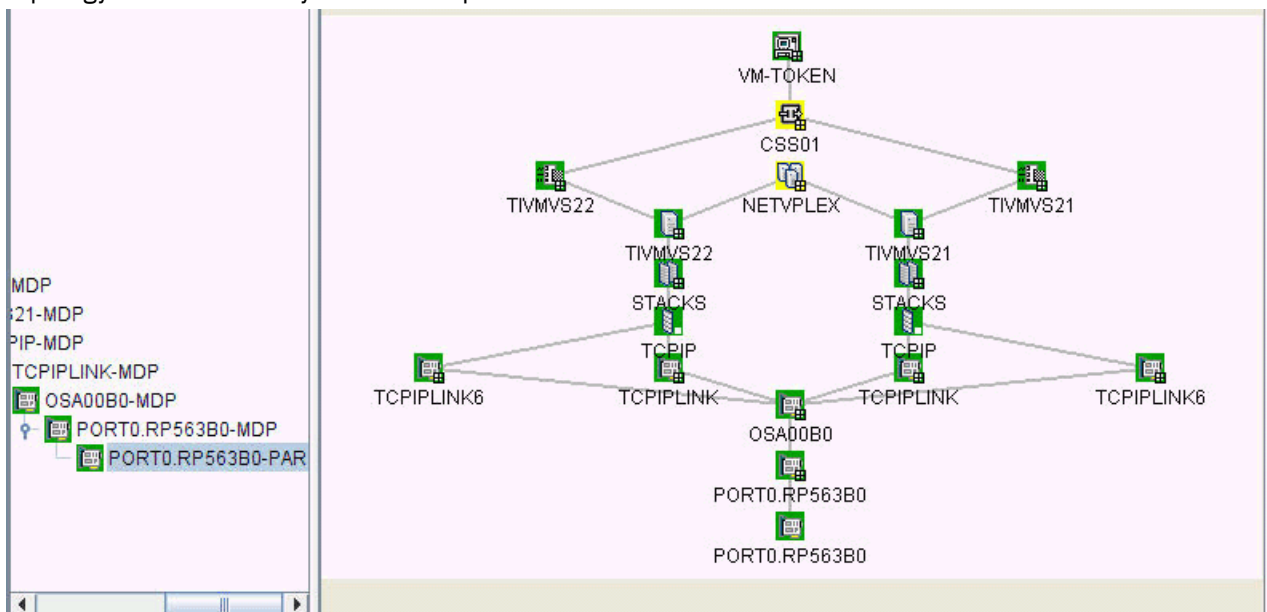


Figure 31. PORT0.RP63B0-PAR View

Viewing HiperSockets Information

To view the HiperSockets adapter topology, you can return to the TCPIP More Detail Physical view (Figure 27 on page 41). You can also display this view by selecting TCPIP-MDP in the business tree.

In this example, the HiperSockets interface names start with IQDIO. In the TCPIP More Detail Physical view, double-click the IQDIO1 HiperSockets interface aggregate object to view the HiperSockets adapters for this interface. The resulting view, which is similar to the view that is shown in Figure 32 on page 44,

is the IQDIO1 More Detail Physical view. This view shows aggregate objects for the HiperSockets adapters on this interface, namely, the HIPERSOCKETSD1 HiperSockets aggregate object.

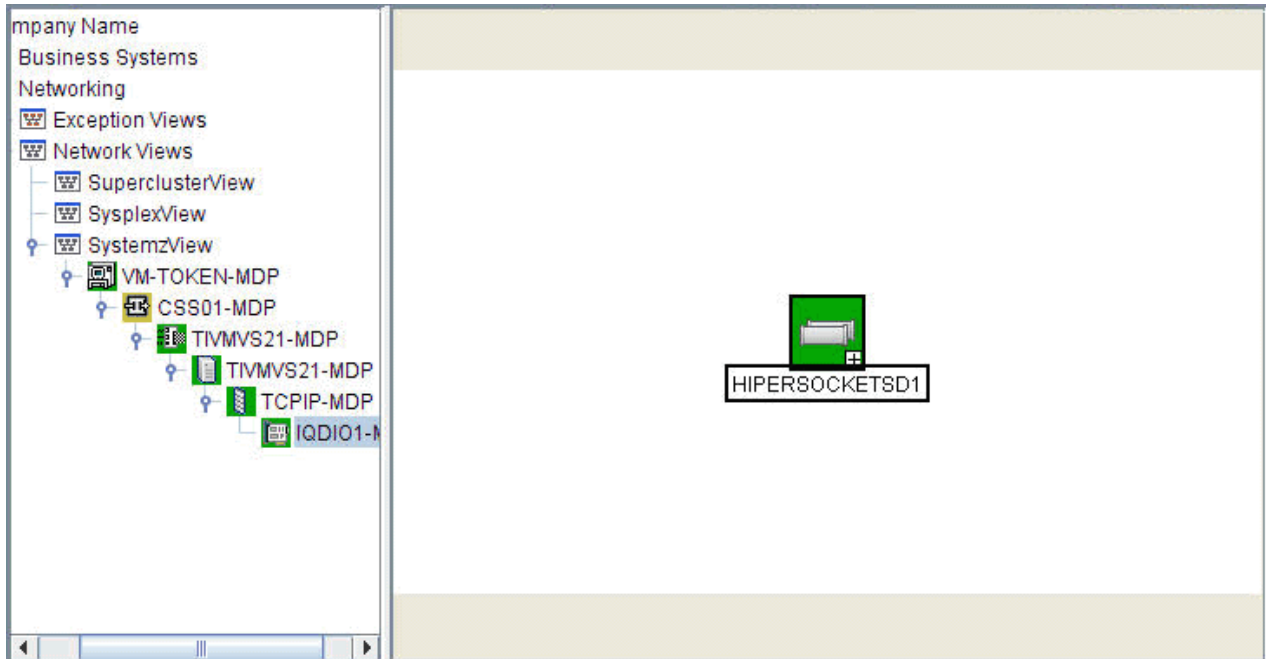


Figure 32. IQDIO1 More Detail Physical View

Double-click the HIPERSOCKETSD1 object. The resulting view, which is similar to the view that is shown in Figure 33 on page 44, is the HIPERSOCKETSD1 More Detail Physical view. This view shows the HiperSockets adapters and virtual local area networks (VLANs) that are specified on the selected HiperSockets interface. This view includes the HIPERSOCKETSD1 HiperSockets object and the VLAN510 virtual LAN object.

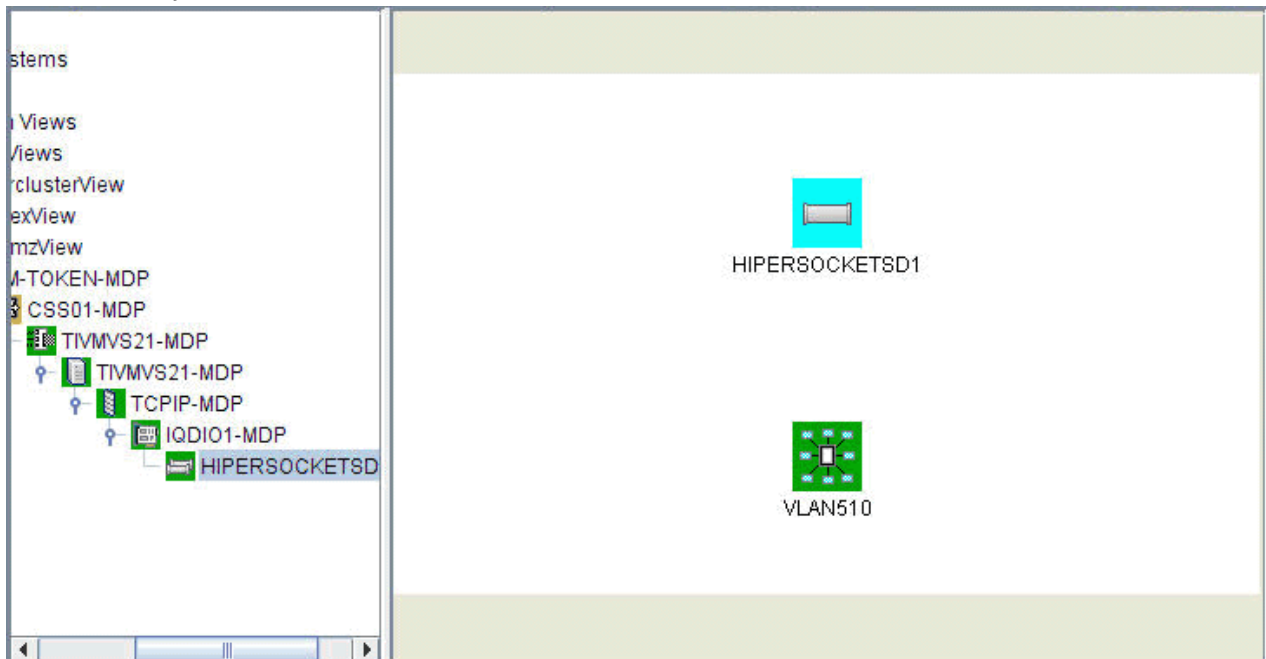


Figure 33. HIPERSOCKETSD1 More Detail Physical View

Chapter 6. Managing Dynamic Virtual IP Addresses

z/OS Communications Server uses dynamic virtual IP addresses (DVIPAs), along with sysplex distributors (also known as distributed DVIPAs), to provide high availability and load balancing in a sysplex. The NetView program provides a comprehensive set of DVIPA information to use in managing and diagnosing problems in your sysplex.

The following DVIPA information is provided by the NetView program in a sampled basis:

- DVIPA definition and status
- Distributed DVIPA
- DVIPA connections
- VIPA routes
- Distributed DVIPA connection routing

Changes within your sysplex that affect DVIPAs can generate events that enable the NetView program to provide up-to-date DVIPA information. Distributed DVIPA statistics are provided to assist with workload balancing. The statistics can be used to monitor historical trends and for problem determination.

DVIPA discovery is not enabled at NetView initialization. To collect DVIPA information, you must uncomment the DVIPA tower and any additional subtowers. For information about configuring this discovery, see *IBM Z NetView Installation: Configuring Additional Components*.

Users can effect the discovery of DVIPA information in the following ways:

- Manually issue the CNMEDRCL command, which initiates immediate rediscovery.
- Modify the sampling intervals in the CNMSTUSR or CxxSTGEN member for DVIPA subtowers, and issue a RESTYLE command. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*.
- Issue the COLLCTL command to start or stop data collection for DVIPA subtowers.

The discovered DVIPA information can be viewed in the following ways:

- [“Using NetView Enterprise Management Agent Workspaces” on page 49](#)
- [“Using 3270 Commands” on page 48](#)

Updating DVIPA Information by Using Events

The NetView program uses DVIPA events and sampling to update the DVIPA information, including the following kinds of events.

- z/OS Communications Server profile updates
- z/OS Communications Server real-time DVIPA updates
- Some z/OS Communications Server sysplex monitoring messages

When an event is received, the DVIPA.Event.Delay timer is set. When the timer expires, local DVIPA rediscovery begins if the NetView program is not the master NetView program. In a master NetView program environment, a message is sent to the master NetView program indicating that rediscovery for the local system is necessary.

When the master NetView program receives notification from a local NetView program that rediscovery is necessary, it sets the DVIPA.Mast.Disc.Delay timer. When this timer expires, rediscovery requests are sent to the systems in the sysplex that had previously notified the master NetView program of the need for rediscovery.

The NetView program uses the algorithm in Table 2 on page 46 to determine the number of systems that have DVIPA information rediscovered. Depending on the size of the sysplex, not all systems are discovered. This helps performance.

<i>Table 2. DVIPA Rediscovery Algorithm</i>		
Number of Systems Using DVIPAs	Number of Systems that Received Events	Number of Systems Rediscovered
2	1	All
3	1	Partial
3	2	All
4	<= .5	Partial
4	> .5	All
5	<= .5	Partial
5	> .5	All
6-11	<= .66	Partial
6-11	> .66	All
12-16	<= .75	Partial
12-16	> .75	All
> 16	n/a	Partial

Usage :

1. For event-related rediscovery to occur, enable the DVIPA discovery function. In a master NetView environment, the master NetView program keeps the CNMDVPST in-storage table that indicates what information has been received from systems in the sysplex. This table is used to determine what rediscovery occurs for each NetView program.
2. Use caution if you set the DVIPA.Event.Delay and DVIPA.Mast.Disc.Delay statements to 0 in the CNMSTYLE member. If a burst of events comes in within a few seconds, rediscovery is done for every event that the NetView program receives, which can cause performance problems.

For more information about enabling DVIPA support, see *IBM Z NetView Installation: Configuring Additional Components*.

Using Distributed DVIPA Statistics

Use distributed DVIPA statistics to evaluate how the workload is being distributed to distributed DVIPA targets. This information is collected each time distributed DVIPA data collection is run (enabled with the DVIPA.DVTAD subtower) and logged to sequential data sets CNMDVIPP (primary) and CNMDVIPS (secondary). You can also use the information for problem diagnosis or for historical purposes.

You can start logging distributed DVIPA statistics at NetView initialization by using the INIT.DVIPASStats statement in the CNMSTUSR or CxxSTGEN member, or by dynamically issuing the DVIPALOG command. You can filter the data that is logged by using the DVIPA.STATS statements in the CNMSTUSR or CxxSTGEN member or by using the DVIPALOG command. When data is logged, you can issue the CNMSDVST sample command to display the information from both data sets. You can also browse the data sets. The data is in a readable format, except for the first field, which is the STCK value. You can control the local logging and forwarding of these statistics by using the DVIPA.STATS.Logto statement in the CNMSTUSR or CxxSTGEN member. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*.

You can also write your own tools and reports to extract the information from the data sets. If you write tools, consider using the FORMAT=NO option in the CNMSDVST sample to return sequential records. The records are combined from both data sets. For information about filtering keywords, see the CNMSDVST sample. The CNM482I and CNM483I messages are issued when logging is switched from one data set to another.

Each data line in the data set represents a distributed DVIPA target and contains the following information:

Note: The format shown is for Format = NO, and the data is right-justified.

Columns	Description
---------	-------------

1–16	STCK value when the record was written to the data set.
-------------	---

18–25	Date of the distributed DVIPA data collection.
--------------	--

27–34	Time of the distributed DVIPA data collection.
--------------	--

36–43	Local z/OS operating system name.
--------------	-----------------------------------

45–52	Local TCP/IP job name.
--------------	------------------------

54–98	Distributed DVIPA.
--------------	--------------------

100–104	Distributed DVIPA port.
----------------	-------------------------

106–113	Distributed DVIPA target z/OS operating system.
----------------	---

115–122	Distributed DVIPA target TCP/IP job name.
----------------	---

124	Distribution method used for incoming connection requests. The following distribution method values are valid:
------------	--

1	BASEWLM
----------	---------

2	ROUNDROBIN
----------	------------

3	SERVERWLM
----------	-----------

4	WEIGHTEDACTIVE
----------	----------------

5	TARGCONTROLLED
----------	----------------

6	HOTSTANDBY
----------	------------

If the distribution method is not BASEWLM or SERVERWLM, you do not see values for WLM Weight and Percentage of WLM Weight. For more information about the distribution methods, see the *z/OS Communications Server IP Configuration Reference*.

126–135	Number of connections for the DVIPA and port that were distributed to the target stack.
----------------	---

137–146

Number of connections for the DVIPA and port that were distributed to the target stack in the most recent time interval.

148–157

Workload manager (WLM) weight value for either the z/OS images on which the target TCP/IP stack is located or the specific server on the target stack.

159–161

Sysplex distributor percentage of new connections for this distributed target in an interval.

163–165

Percentage WLM weight value indicating the percentage of new connections that each distributed target receives in an interval from WLM.

Using 3270 Commands

DVIPA management provides 3270 commands that collect real-time information about DVIPA definitions and status, distributed DVIPAs, DVIPA connections, and DVIPA routing information. The commands that are shown in Table 3 on page 48 are used to collect DVIPA data. These commands can be issued for all the domains that are in the scope of the master NetView program. For example, a sysplex master NetView program can issue these commands to other NetView domains in the sysplex in which the NetView programs are either master-capable or basic NetView programs. However, a sysplex master NetView program cannot issue these commands to an enterprise master NetView program, unless it is also the enterprise master NetView program. The enterprise master NetView program can issue these commands to all domains that are in the scope of the enterprise master NetView program.

<i>Table 3. DVIPA Data Collector Commands</i>			
Command¹	Type of Data Collected	Sample to View Data	Messages Returned by the Collector
CNMSDVST ²	Distributed DVIPA statistics	CNMSDVST	BNH867I
DVIPCONN	DVIPA connections	CNMSDVPC	BNH849I
DVIPDDCR	Distributed DVIPA connection routing information	CNMSDDCR	BNH815I
DVIPHLTH	Distributed DVIPA server health information	CNMSDVPH	BNH814I
DVIPPLEX	DVIPA sysplex distributor information	CNMSPLEX	BNH847I
DVIPSTAT	DVIPA definition and status information	CNMSDVIP	BNH846I
DVIPTARG	Distributed DVIPA target information	CNMSTARG	BNH813I
VIPAROUT	VIPA route information	CNMSVPRT	BNH824I
Notes: 1. The TEMA DVIPA subtower is not required to run these commands. 2. This is a sample only. For more information about the CNMSDVST sample, see the sample preface.			

These data collector commands are intended for use in application programs. When a data collector command completes successfully, it returns the data in a particular multiline message. To view the data in a format that can be read, use the associated sample. To display this data in a different format, modify the applicable sample or create your own application to format the data that is returned by the specified commands. You can use input parameters with a sample to specify the data to collect. The input parameters for a sample are the same as those for the NetView command that is called by the sample. For information about the returned messages, the command parameters, or other information about the data collector commands, see the IBM Z NetView online message and command help.

Notes:

1. You can access a menu of DVIPA data collector commands from the command facility:
 - a. Enter NETVIP on the command line. The NetView IP Management Functions Menu panel is displayed (Figure 91 on page 105).
 - b. Select Manage DVIPA. A menu panel from which you can select a data collector command is displayed.
2. Because the data records that are displayed can be quite wide, the samples are based on the WINDOW command. You can scroll to the right to view the data that is not currently visible in a 3270 session.
3. These samples are also used by the Z NetView Enterprise Management Agent take action commands.

For information about setting up DVIPA support for the NetView workspaces, see *IBM Z NetView Installation: Configuring Additional Components*. For more information about DVIPAs and distributed DVIPAs, see the z/OS Communications Server publications.

Using NetView Enterprise Management Agent Workspaces

You can use the Z NetView Enterprise Management Agent to view information in Tivoli Enterprise Portal for the dynamic virtual IP addresses (DVIPAs). You can view information about DVIPA definition and status, distributed DVIPAs (including sysplex distributors, distributed targets, and application server health), DVIPA connections, VIPA routes, and distributed DVIPA connection routing.

Distributed DVIPA Connection Routing Workspace

The Distributed DVIPA Connection Routing workspace displays detailed information about distributed DVIPA connection routing.

To display this workspace, click the link icon from a row in one of the following views:

- DVIPA Connections and Filtered DVIPA Connections workspaces, the DVIPA Connections Summary table. See “[DVIPA Connections Workspace](#)” on page 52.
- DVIPA Sysplex Distributors and Filtered Sysplex Distributors workspaces, the DVIPA Sysplex Distributors Summary table. See “[DVIPA Sysplex Distributors Workspace](#)” on page 54.

This workspace provides data that is updated by using sampling.

Historical data is not enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 34 on page 50](#).

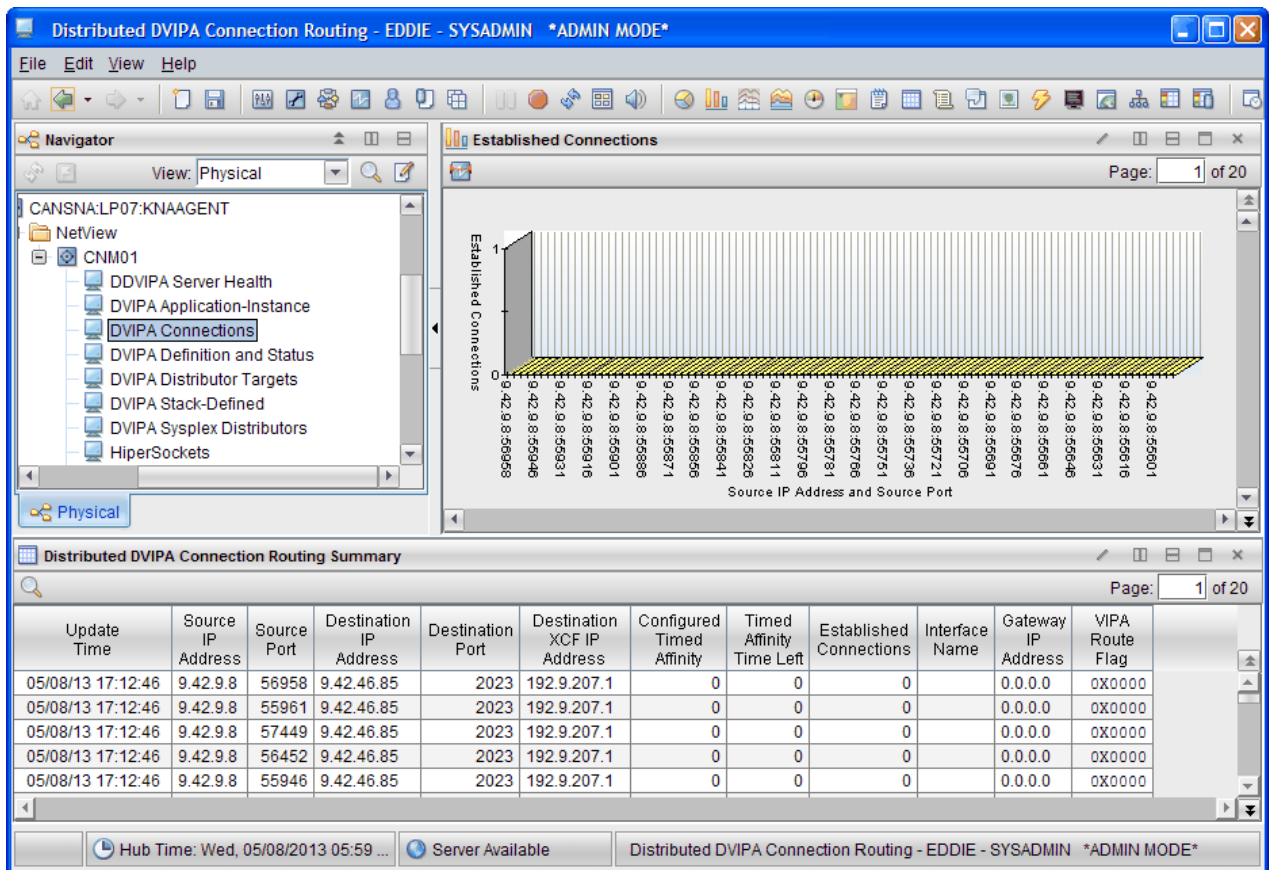


Figure 34. Distributed DVIPA Connection Routing Workspace

Distributed DVIPA Server Health Workspace

The Distributed DVIPA Server Health workspace displays health statistics for all application servers that reside on distributed DVIPA targets.

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **DDVIPA Server Health**.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

This workspace provides data that is updated by using sampling and events.

Historical data is enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 35 on page 51](#).

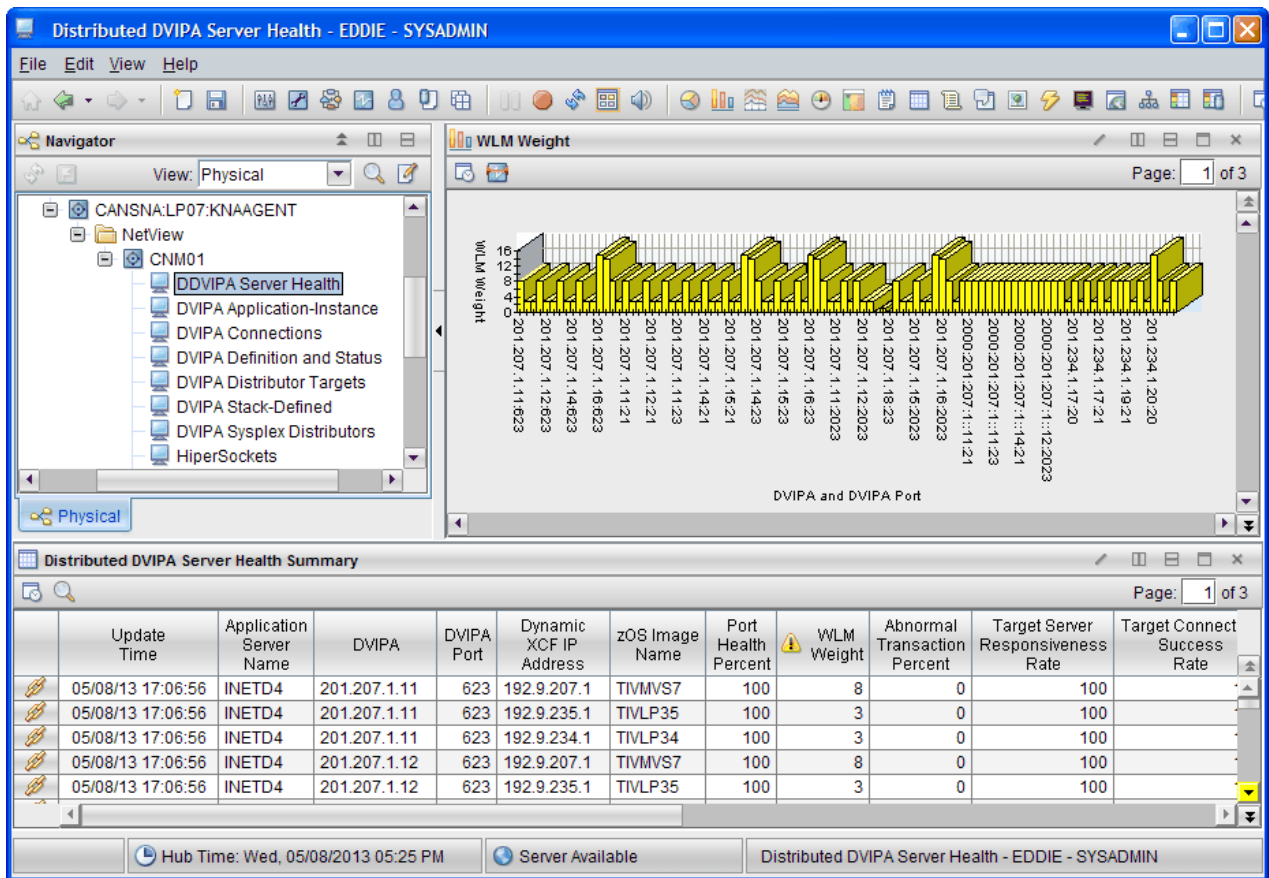


Figure 35. Distributed DVIPA Server Health Workspace

Distributed DVIPA Targets Workspace

The Distributed DVIPA Targets workspace displays information about the distributed DVIPA targets.

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **DVIPA Distributor Targets**.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

This workspace provides data that is updated by using sampling and events.

Historical data is enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 36 on page 52](#).

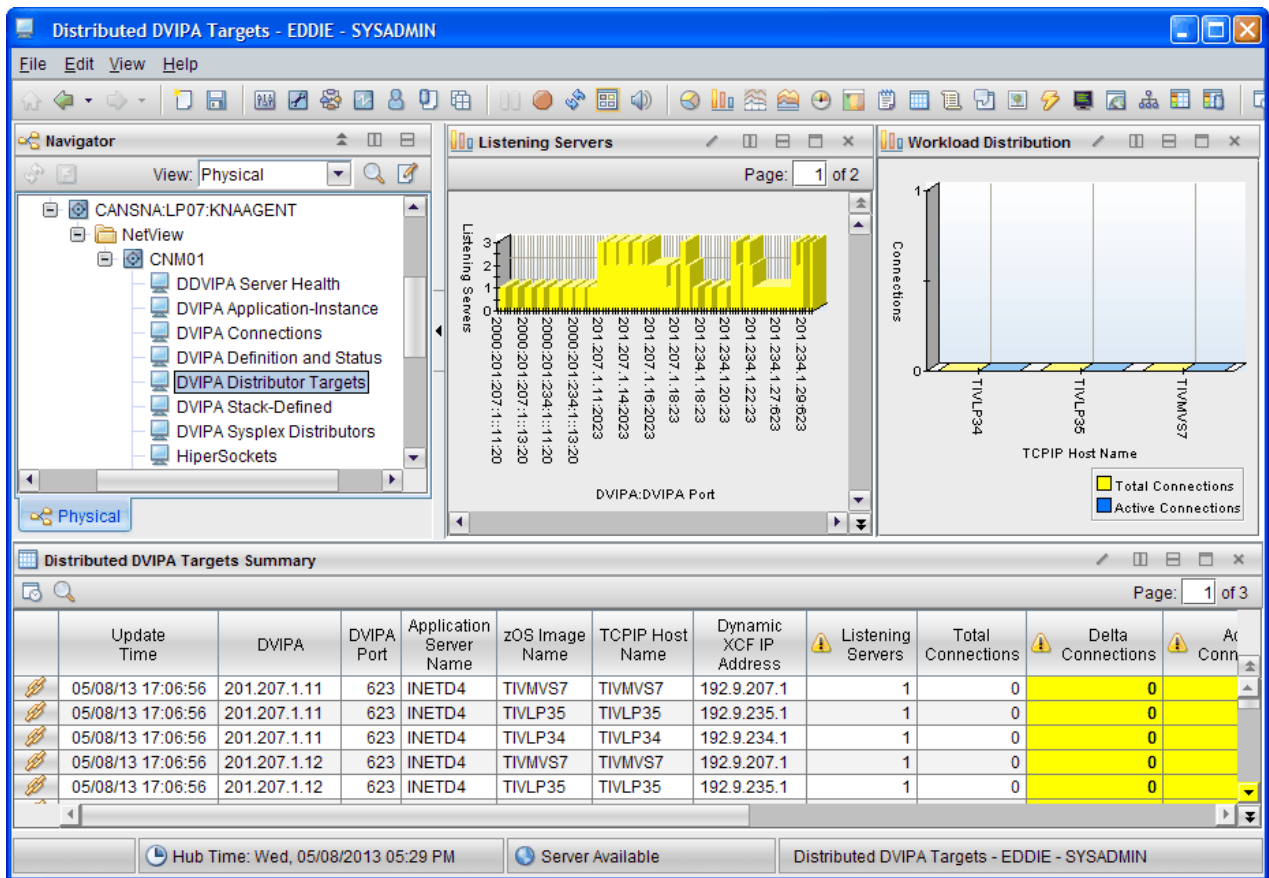


Figure 36. Distributed DVIPA Targets Workspace

DVIPA Connections Workspace

The DVIPA Connections workspace displays connections involving a dynamic virtual IP address (DVIPA) and a DVIPA port.

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **DVIPA Connections**.

This workspace provides data that is updated by using sampling.

The queries for this workspace use the Byte Rate attribute to filter the rows that can be retrieved for display. Because of the default filter, you might not see all your connections. You can modify the query to display more or fewer connections than the default filter allows.

Historical data is not enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 37](#) on page 53.

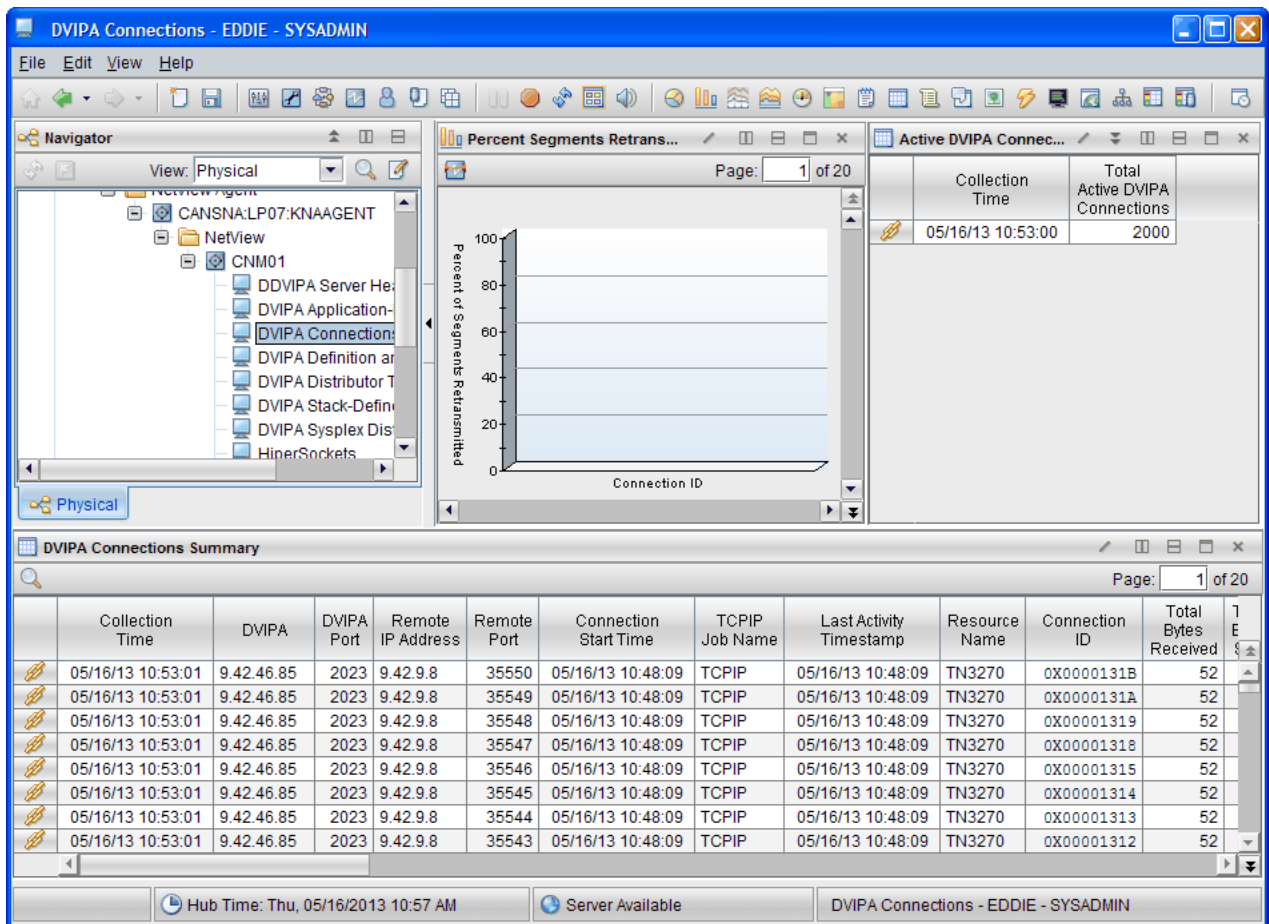


Figure 37. DVIPA Connections Workspace

DVIPA Definition and Status Workspace

The DVIPA Definition and Status workspace displays the definition and status of the dynamic virtual IP addresses (DVIPAs).

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **DVIPA Definition and Status**.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

This workspace provides data that is updated by using sampling and events.

Historical data is not enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 38 on page 54](#).

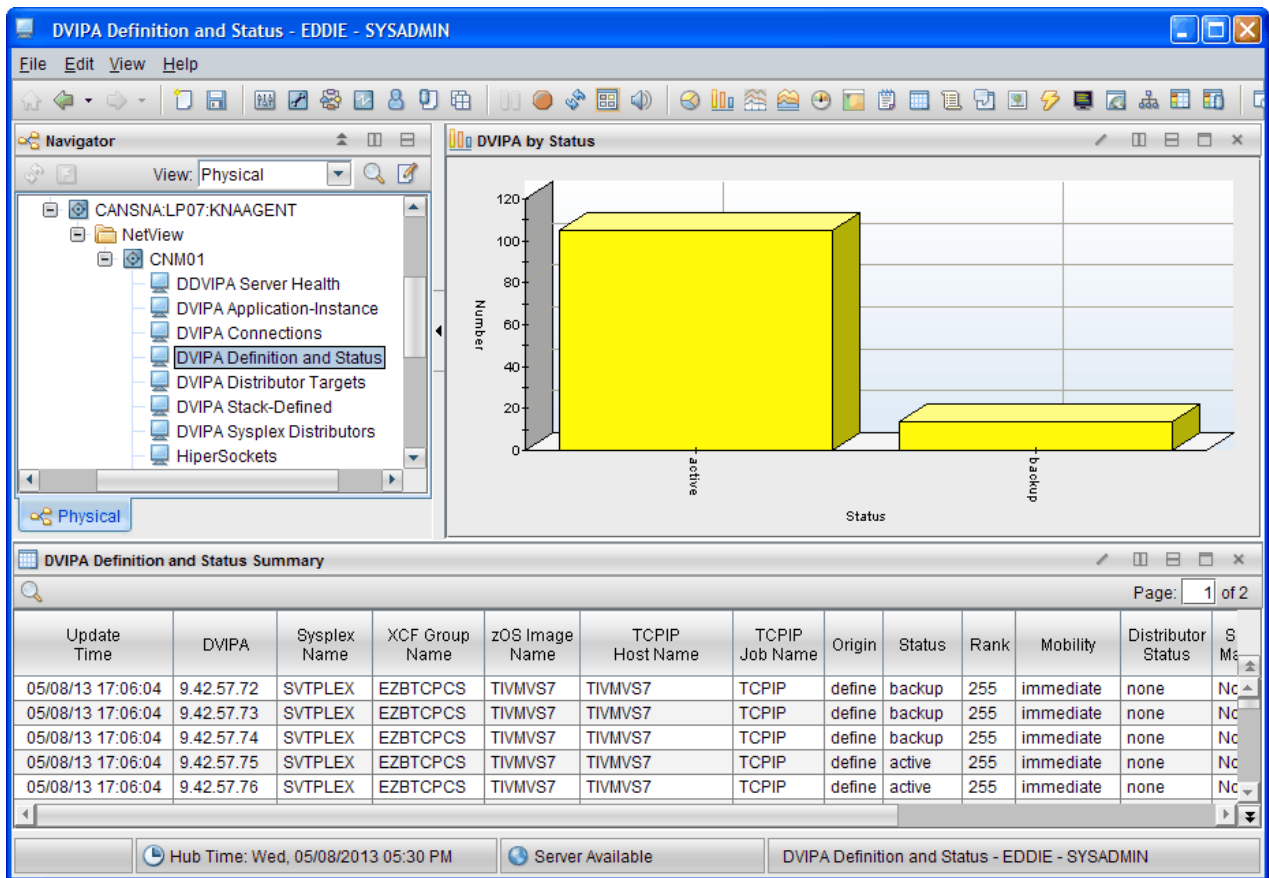


Figure 38. DVIPA Definition and Status Workspace

DVIPA Sysplex Distributors Workspace

The DVIPA Sysplex Distributors workspace displays information about DVIPA sysplex distributors.

To display this workspace, expand **z/OS Systems > system > NetView Agent > subagent > NetView > domain**, and then click **DVIPA Sysplex Distributors**.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

This workspace provides data that is updated by using sampling and events.

Historical data is enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 39 on page 55](#).

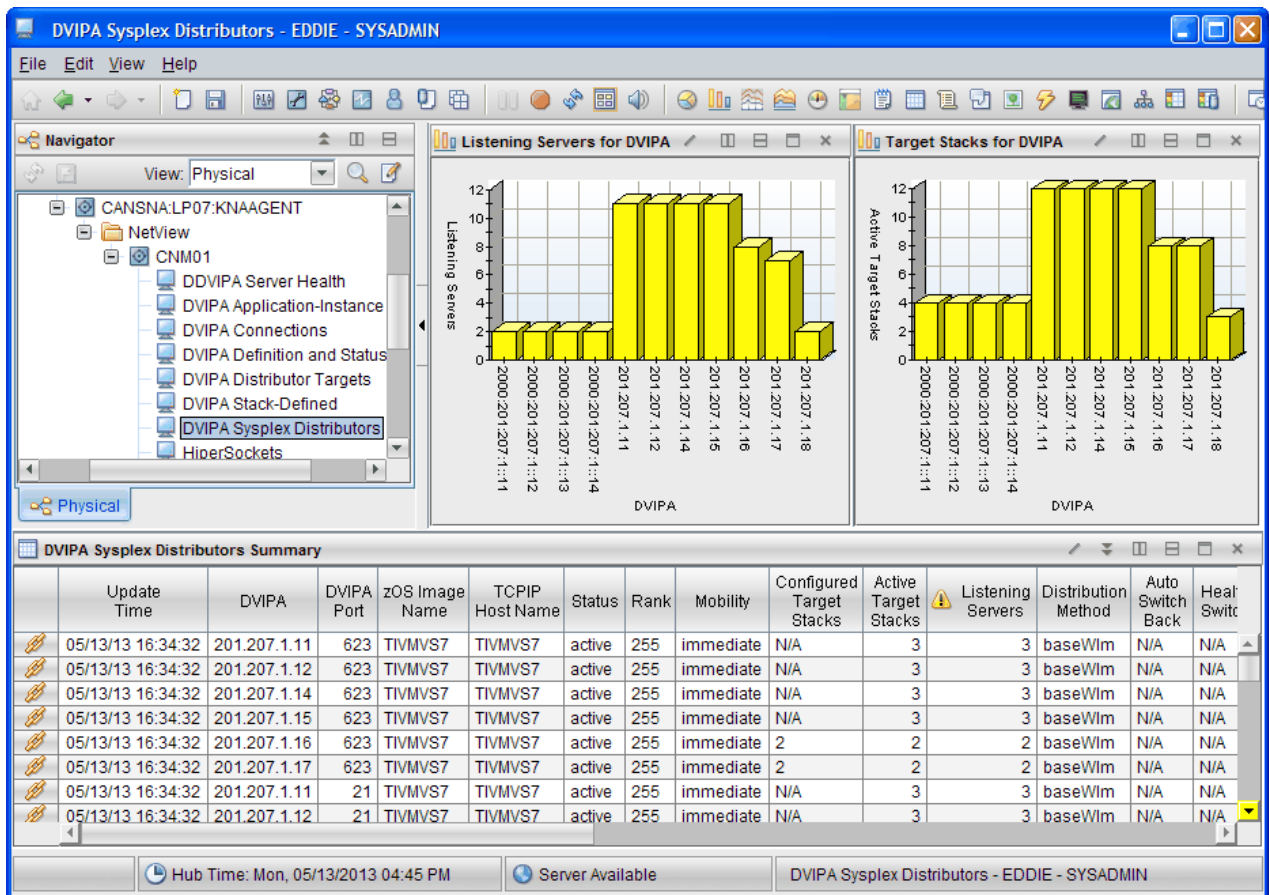


Figure 39. DVIPA Sysplex Distributors Workspace

VIPA Routes Workspace

The VIPA Routes workspace displays VIPA route information for a specific stack and z/OS image name.

To display this workspace, click the link icon from a row in the DVIPA Sysplex Distributors workspace, the DVIPA Sysplex Distributors Summary table. See “DVIPA Sysplex Distributors Workspace” on page 54.

If the NetView domain node that is shown in the navigator view represents the master NetView program, this workspace can contain sysplex-wide data. Otherwise, the workspace contains data for the z/OS image on which the NetView program resides.

This workspace provides data that is updated by using sampling and events.

Historical data is not enabled in this workspace.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in Figure 40 on page 56.

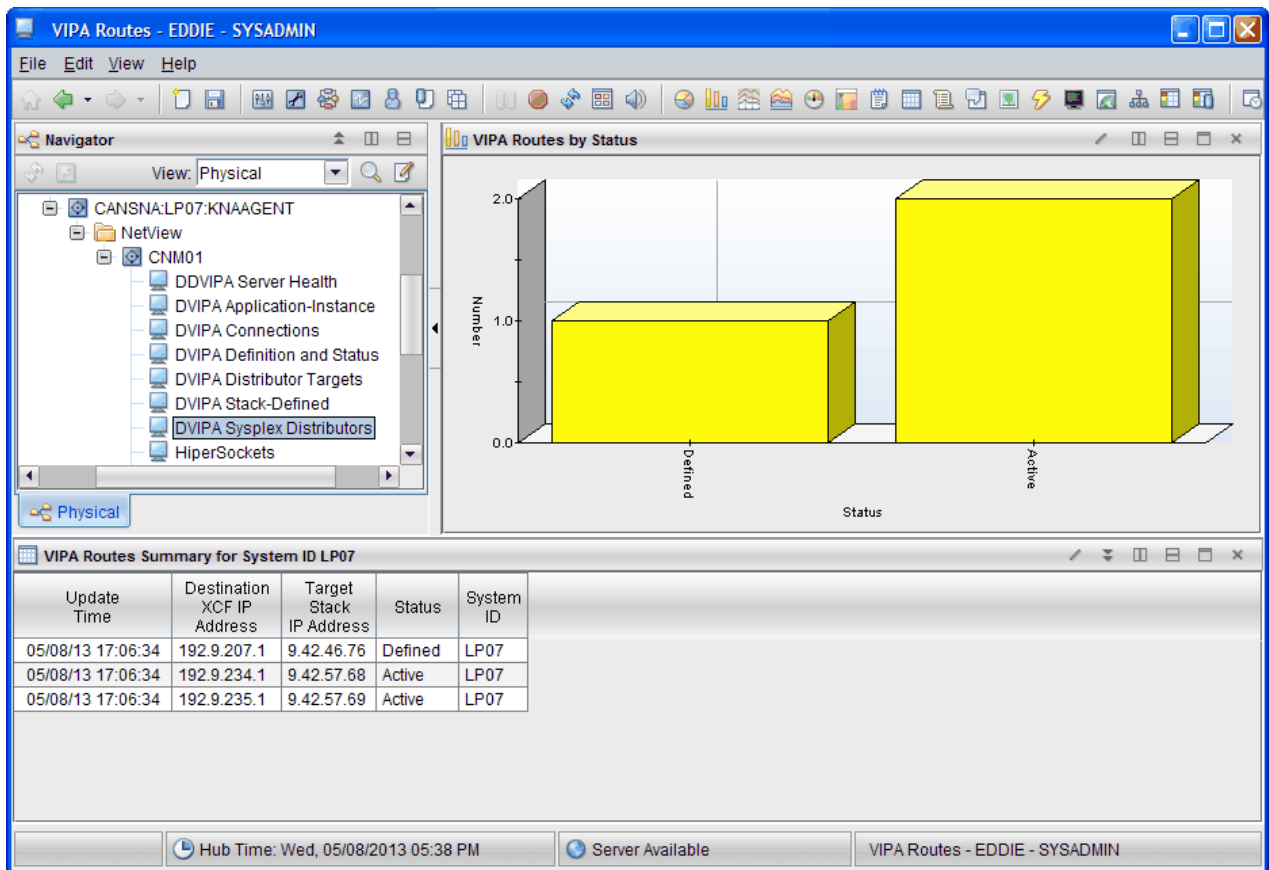


Figure 40. VIPA Routes Workspace

DVIPA Stack Summary Workspace

The DVIPA Stack Summary workspace displays DVIPA information for a specific stack (TCPIP Job Name) and z/OS image name.

To display this workspace, click the link icon from a row in the Stack Configuration and Status workspace, the Stack Configuration and Status Summary table. See [“Stack Configuration and Status Workspace”](#) on page 26.

This workspace provides data that is updated by using sampling and events.

Historical data is enabled in this workspace for the Sysplex Distributors Defined and Local Distributed Targets Defined table views.

For information about data collection and data display for this workspace, see the table in *IBM Z NetView Installation: Configuring Additional Components*.

This workspace is shown in [Figure 41](#) on page 57.

Navigator

- NetView Audit Log
- NetView Command Response
- NetView Health
- NetView Log
- OSA
- Session Data
- Stack Configuration and Status**
- TCPIP Connection Data
- Telnet Server Configuration and

DVIPA Defined for TCPIP J...

	Update Time	DVIPA	Time Active
	05/08/13 17:09:46	9.42.57.72	
	05/08/13 17:09:46	9.42.57.73	
	05/08/13 17:09:46	9.42.57.74	
	05/08/13 17:09:46	9.42.57.79	
	05/08/13 17:09:46	9.42.57.80	05/03/13
	05/08/13 17:09:46	201.235.1.2	05/03/13
	05/08/13 17:09:46	201.235.1.3	05/03/13
	05/08/13 17:09:46	201.235.1.4	05/03/13
	05/08/13 17:09:46	201.235.1.5	05/03/13
	05/08/13 17:09:46	201.235.1.6	05/03/13
	05/08/13 17:09:46	201.235.1.7	05/03/13
	05/08/13 17:09:46	201.235.1.8	05/03/13

Sysplex Distributors Defin...

	Update Time	DVIPA	DVIPA Port	S
	05/08/13 17:10:21	201.235.1.17	21	a
	05/08/13 17:10:21	201.235.1.16	20	a
	05/08/13 17:10:21	201.235.1.17	20	a
	05/08/13 17:10:21	201.235.1.16	21	a
	05/08/13 17:10:21	201.235.1.17	23	a
	05/08/13 17:10:21	201.235.1.16	23	a
	05/08/13 17:10:21	201.235.1.12	20	a
	05/08/13 17:10:21	201.235.1.15	23	a
	05/08/13 17:10:21	201.235.1.12	21	a
	05/08/13 17:10:21	201.235.1.14	23	a
	05/08/13 17:10:21	201.235.1.15	21	a
	05/08/13 17:10:21	201.235.1.14	20	a

Local Distributed Targets Defined for TCPIP Job Name TCPIP and z/OS Image Name TIVLP35

	Update Time	DVIPA	DVIPA Port	Dynamic XCF IP Address	Listening Servers	Total Connections	Active Connections	Hot Standby Server Type	Hot Standby Server Status	Hot Standby Rank	Distribution Port Function
	05/08/13 17:06:56	201.207.1.11	623	192.9.235.1	1	0	0	N/A	N/A	N/A	0X80
	05/08/13 17:06:56	201.207.1.12	623	192.9.235.1	1	0	0	N/A	N/A	N/A	0X80
	05/08/13 17:06:56	201.207.1.14	623	192.9.235.1	1	0	0	N/A	N/A	N/A	0X80
	05/08/13 17:06:56	201.207.1.15	623	192.9.235.1	1	0	0	N/A	N/A	N/A	0X80
	05/08/13 17:06:56	201.207.1.16	623	192.9.235.1	1	0	0	N/A	N/A	N/A	0X80
	05/08/13 17:06:56	201.207.1.17	623	192.9.235.1	1	0	0	N/A	N/A	N/A	0X80
	05/08/13 17:06:56	201.207.1.11	21	192.9.235.1	1	0	0	N/A	N/A	N/A	0X80
	05/08/13 17:06:56	201.207.1.12	21	192.9.235.1	1	0	0	N/A	N/A	N/A	0X80

Hub Time: Wed, 05/08/2013 05:18 PM Server Available DVIPA Stack Summary - EDDIE - SYSADMIN

Figure 41. DVIPA Stack Summary Workspace

Part 3. IP Management Functions

Chapter 7. Controlling and Viewing IP Traces

The examination of packet content is sometimes necessary to debug a problem. The NetView program provides real-time capture and formatting of IP packet and Open Systems Adapter (OSA) packet trace data, including both headers and payloads. Because the formatting is the same as that under Interactive Problem Control System (IPCS), you do not need to learn a new format. Because the formatter is directly integrated with the IP stack, no translation mismatches can occur. Highly flexible tracing and formatting options are available so that you can filter out unwanted data. Both IPv4 and IPv6 packets are supported, and the data is also available in binary (unformatted) form for use by automation routines.

Notes:

1. z/OS V2R1 Communications Server or later is required for some packet trace functions.
2. The OSA SNMP subagent must be active to control OSA traces.

Defining Packet Trace Data Collection

The PKTS START command starts a long-running process to collect IP or OSA packet trace data from z/OS Communications Server for the specified TCP/IP stack. The data-collection process persists as an autotask, which is defined by a FUNCTION.AUTOTASK statement (either FUNCTION.AUTOTASK.PKTS or FUNCTION.AUTOTASK.OPKT) in the CNMSTYLE member or a PKTS DEFINE command. To collect both IP and OSA packet trace data requires two autotasks. The TCIPCOLLECT.PKTS subtower must be enabled to collect the packet trace data. The PKTS STOP or STOPCOLL command stops the packet trace data collection.

Note: Both PKTS STOP and PKTS STOPCOLL stop the collection of packet data for the specified stack and source combination, but PKTS STOPCOLL keeps the data already collected available. You can view the collected data by using the IPTRACE command or by running PKTS QUERY or PKTS PURGE against the stack. If you issue the PKTS STOP command, the PKTS QUERY and PKTS PURGE commands are no longer valid for the stack.

For more information about defining packet trace collection, see *IBM Z NetView Installation: Configuring Additional Components*. For more information about the TCIPCOLLECT.PKTS subtower, see *IBM Z NetView Installation: Getting Started*. For more information about the FUNCTION.AUTOTASK statement, see the *IBM Z NetView Administration Reference*. For the syntax of and detailed information about the PKTS command, see the online help.

Controlling, Collecting, and Formatting Packet Trace Data

You can control and view IP and OSA packet trace information in a 3270 panel interface. To use the panel interface, issue the IPTRACE command from the NetView command line. You can specify a stack on the IPTRACE command; you can issue the IPTRACE command with no parameters, which displays the local stack; or, you can issue the IPTRACE * command, which displays a list of all known stacks from which you can select a stack. You can also start or stop the tracing by using the panel interface. For more information about using IPTRACE and the panel interface, see [“Running IP Traces” on page 62](#). For the syntax of and detailed information about the IPTRACE command, see the online help.

To view formatted packets, use the FMTPACKET command from the NetView command line. You can also use the FMTPACKET command to send packet trace data to a pipe stage. For the syntax of and detailed information about the FMTPACKET command, see the online help.

Running IP Traces

Note: This function no longer requires AON. The information about this function is included in both *IBM Z NetView IP Management* and the *IBM Z NetView User's Guide: Automated Operations Network*.

Use the IPTRACE command to start and view diagnostic traces to help resolve TCP/IP problems. The following traces are available:

- IP packet trace is used for IP data flow problems and copies IP packets as they are received or sent.
- OSA packet trace is used for Open Systems Adapter (OSA) data flow problems and copies OSA packets as they are received or sent.
- Component trace is used to trace data processing problems between the client and the server.

If you want to use an external writer, it must be established before you use IP trace. The trace data is written to the writer. See *z/OS MVS Diagnosis: Tools and Service Aids* for more information about creating source JCL for an external writer, or for more details about trace functions.

Issuing the IPTRACE Command

Issue the IPTRACE command from the NetView command line.

If you issue the IPTRACE * command, all known stacks are displayed in the TCP/IP for 390 IPTrace Control Center panel (FKXK2A00), which is shown in [Figure 42 on page 62](#). From this panel, you must select the stack that you want to trace by tabbing to the stack and pressing Enter. Active stacks are highlighted in green. Inactive stacks are displayed in blue and cannot be selected.

FKXK2A00		TCP/IP for 390 IPTrace Control Center	
Service Point/ Stack	Proc Name	NetView Domain	Trace Status
NMP101	TCPIP	LOCAL	PKT/ACT OSA/ACT
NMP217	TCPIP	NTVE1	OSA/ACT
Command ==>			
F1=Help	F2=Main Menu	F3=Return	F6=Roll
F7=Backward	F8=Forward		F12=Cancel

Figure 42. TCP/IP IPTrace Control Center Panel

If you issue IPTRACE with no parameters, information about the local stack is displayed in the IPTrace Control Center panel (FKXK2A01), which is shown in [Figure 43 on page 63](#). This panel is also displayed if you selected a stack on the TCP/IP for 390 IPTrace Control Center panel (FKXK2A00).

FKXK2A01			IPTrace Control Center			NTVE1	
Service Point: NMP217			Stack: TCPIP			Domain: Local	
			Status/Owner	Start		For	Writer
_	CTRACE	SYSTCPIP	NONE/NA	NA		NA	*NONE*
_	PKTTRACE	SYSTCPDA ACTIVE TRACE SESSIONS	NONE/NA	NA		NA	*NONE*
_	OSATRACE	SYSTCPOP	NONE/NA	NA		NA	*NONE*
Command ==>							
F1=Help		F2=Main Menu		F3=Return		F5=Refresh	
						F6=Roll F12=Cancel	

Figure 43. IPTrace Control Center Panel

This panel provides information about tracing on the selected stack. To select a trace, type any non-blank character next to the trace that you want and press Enter.

- If you select PKTTRACE (IP packet trace), see [“IP Packet Tracing”](#) on page 63.
- If you select OSATRACE (OSA packet trace), see [“OSA Packet Tracing”](#) on page 77.
- If you select CTRACE (component trace), see [“Component Tracing”](#) on page 80.

IP Packet Tracing

The NetView program can collect trace data through two interfaces:

instance trace

Allows multiple concurrent traces, each with a set of trace criteria. This type of trace is available with z/OS Communications Server V2.1. Each individual trace can be referred to as a *trace instance*.

global trace

Allows only one trace at a time, with one set of trace criteria. If a second trace, with its set of trace criteria, is started while a trace is already running, the second trace replaces the first trace.

A global trace and one or more trace instances can run simultaneously.

FKXK2C01		Packet Trace Control	NTVF9	
	Owner/Autotask	Status	Start Date/Time	Description
-	GLOBAL	ACTIVE	2019-08-20 08:52:37	
-	USER1/AUTTRA1	ACTIVE	02/12/19 07:00:00	STATUS FOR AP
-	USER1/AUTTRA3	DORMANT		
-	USER1/AUTTRA4	DORMANT		
-	USER1/AUTTRA2	DORMANT		
-	USER2/AUTTRA7	ACTIVE	02/12/19 12:24:12	SLOW ATMS
Command ==>				
F1=Help		F3=Return	F6=Roll	
F7=Backward		F8=Forward	F9=Create Instance	
			F12=Cancel	

Figure 44. Packet Trace Control Panel

From this panel, you can make the following selections:

- If you press F9 (Create Instance) or select a packet trace instance such as USER1/AUTTRA1, you can control a new or existing packet trace instance; see [“Starting and Stopping IP Packet Trace Instances”](#) on page 64.
- If you select GLOBAL, you can start or stop a global IP packet trace; see [“Starting and Stopping a Global IP Packet Trace”](#) on page 64.

Starting and Stopping IP Packet Trace Instances

When you press F9 or select a packet trace instance on the Packet Trace Control panel (FKXK2C01), the Packet Trace Details panel (FKXK2C02) is displayed. You can set filters for a new packet trace instance or you can view information for or control an active packet trace instance, as follows:

- If you select an active packet trace instance, details about the trace instance are displayed. The information that is displayed cannot be changed.
- If you select a dormant packet trace instance, details about the trace instance are displayed and can be changed.
- If you press F9, you can set the filters for a new packet trace instance.

The following options are available from the Packet Trace Details panel (FKXK2C02), depending on the status of the trace. On the command line, enter the option number that you want:

- 1=START: If the trace is stopped (DORMANT status), this option restarts the collection of packets. If you are starting a new packet trace instance, this is the only valid option.
- 2=STOP: If the trace is active (collecting packets), this option stops the collection. When a trace is stopped, you can view the packets that were collected, but no new packets are collected.
- 3=DISPLAY: This option displays the collected packets; see [“Viewing IP Packet Trace Data”](#) on page 66.
- 4=END: This option closes the trace, deletes all collected data, and releases storage.

Starting and Stopping a Global IP Packet Trace

The PKTTRACE Control panel (FKXK2A22), which is shown in [Figure 45 on page 65](#), is displayed as follows:

- When you are running z/OS Communications Server V2.1 or later and select GLOBAL on the Packet Trace Control panel (FKXK2C01)

- When you are running an earlier version of z/OS Communications Server and select PKTTRACE on the IPTrace Control Center panel (FKXK2A01)
- When you select command **11. Packet Trace** on the TCP/IP for z/OS Connection Management panel (FKXK2221), which is shown in [Figure 76 on page 90](#)

FKXK2A22

PKTTRACE Control - SYSTCPDA

LOCAL

Service Point: NMP217

Stack: TCPIP

Task: AUTOPKTS

PKTS: ACTIVE

SYSTCPDA: ACTIVE

GTF: NO

Start Time: 2019-04-28-07:57:45

Writer: *NONE*

Options: 1-START/ADD 2-STOP 3-VIEW PACKETS

Infrc/Link	Stat	Prot	IP	Address/Prefix	Src Port	Port Num	Dest Port	Record Count
- ALL		*	*		*	*	*	
- TCPIPLINK6	OFF	*	*		*	*	*	0
- TCPIPLINK	OFF	*	*		*	*	*	0
- EZASAMEMVS	OFF	*	*		*	*	*	0
- EZ6SAMEMVS	OFF	*	*		*	*	*	0
- VIPLC9020A3C	OFF	*	*		*	*	*	0
- VIPLC9020A3D	OFF	*	*		*	*	*	0
- VIPLC9020A3E	OFF	*	*		*	*	*	0
- VIPLC9020A3F	OFF	*	*		*	*	*	0

Command ==>

F1=Help

F2=Main Menu

F3=Return

F4=Stop SYSTCPDA

F5=Refresh

F6=Roll

F7=Backward

F8=Forward

F9=Assist

F10=PKTS Management

F12=Cancel

Figure 45. PKTTRACE Control Panel

Starting and stopping a global IP packet trace involves the following tasks:

- [“Managing the z/OS Communications Server IP Packet Trace” on page 65](#)
- [“Managing the NetView IP Packet Trace Collection Process” on page 65](#)

Managing the z/OS Communications Server IP Packet Trace

If SYSTCPDA is not shown as ACTIVE on the PKTTRACE Control panel (FKXK2A22), type any character next to at least one interface (for example, TCPIPLINK6 as shown in [Figure 45 on page 65](#)) and press F4 (Start SYSTCPDA) to start packet tracing by the SYSTCPDA component of z/OS Communications Server. After packet tracing starts, SYSTCPDA is shown as ACTIVE, the status of the interface changes to ON, and the options change.

Instead of tracing by interface name, you can select **ALL** to start tracing for all the interfaces on the stack. Status and record counts are reported by and displayed in the PKTTRACE Control panel (FKXK2A22) for each interface on the stack. If you select **ALL**, status and record counts are not totaled and are not displayed in the Stat and Record Count columns of the ALL line in the FKXK2A22 panel.

To set optional filters or see more detail for an interface, press F9 (Assist) on the PKTTRACE Control panel (FKXK2A22) to go to the PKTTRACE Control Assistant panel (FKXK2A29). On that panel, make any changes that are needed and press F3 to return.

On the PKTTRACE Control panel (FKXK2A22), note the Options line:

Options: 1-START/ADD 2-STOP 3-VIEW PACKETS

To start an IP packet trace filter for an interface, type 1 next to the interface name that you want to trace, and press Enter. To stop an IP packet trace filter, type 2 next to the interface name, and press Enter. To view the packets that were traced, type 3 next to the interface name, and press Enter; see [“Viewing IP Packet Trace Data” on page 66](#).

Managing the NetView IP Packet Trace Collection Process

To manage the packet collection process (the PKTS task), press F10 (PKTS Management). The NetView PKTS Management panel (FKXK2A23), which is shown in [Figure 46 on page 66](#), is displayed. Use this

panel to start and stop the collection of packet data. Make any changes that are needed, and press F3 to return to the PKTTRACE Control panel (FKXK2A22).

Note: Instead of using the NetView PKTS Management panel, you can use the PKTS command or statements in the CNMSTUSR or CxxSTGEN member; see [“Defining Packet Trace Data Collection”](#) on page 61.

```
FKXK2A23                                PKTS Management                                LOCAL
Start with Storage Size:                M                                PKTS Status: ACTIVE
- Stop
- Stopcoll
Define TCPName: TCPIP                    OPID: AUTOPKTS
- Purge
  Intfname: *
  LAddr:    *
  RAddr:    *
  LPort: *   RPort: *   Portnum: *
    Start Time: *
    End Time:  *
Protocol 1  1-All
            2-TCP
            3-UDP
            4-ICMP
            5-OSPF
            6      (Number)

Command ==>
F1=Help                                F3=Return                                F6=Roll
                                         F12=Cancel
```

Figure 46. NetView PKTS Management Panel for IP Packet Trace

Viewing IP Packet Trace Data

Display Packet Control panel (FKXK2A24)

Use the Display Packet Control panel (FKXK2A24), which is shown in Figure 47 on page 66, to specify additional filtering criteria. This panel is displayed by typing 3 next to the interface to be traced on the PKTTRACE Control panel (FKXK2A22).

```
FKXK2A24                                Display Packet Control                                LOCAL
                                                NMPIPL10

Stack: TCPIP                                Infc Name: ALL
LAddr *
RAddr *
Portnum: *   LPort: *   RPort: *   Protocol: _ ALL (default)
                                                - TCP
                                                - UDP
                                                - ICMP
                                                - OSPF
                                                -      (Number)

Start Time:  *
End Time:    *

MaxRecs: 1  1-Last   100      Truncate: 65535
          2-First

Command ==>
F1=Help  F2=Save Packets  F3=Return  F4=View Packets  F6=Roll
          F8=Extended Opts          F10=Analyze    F12=Cancel
```

Figure 47. Display Packet Control Panel

The interface name in the **Infc Name** field is carried over from the **Infc/Link** field on the PKTTRACE Control panel (FKXK2A22) but can be changed to search for packets that are associated with any valid interface. You can also use an asterisk (*) at the end of the interface name as a wildcard to see packets

that are from several interfaces that have similar names; for example, to see packets for all interfaces that have names that start with TCPIP, you can specify TCPIP* in the Infic Name field.

To save packets in CTRACE and Sniffer trace format, press F2 (Save Packets). The Display Packet Control - Save Packets pop-up panel (FKXK2S24) will be displayed, which is shown in [Figure 50 on page 68](#).

The NetView program issues an MVS START command to run the MVS procedure defined in PKTS.SNFPROC to copy or convert the CTRACE data set to the Sniffer format data set. The NetView user that issues the request must have the authority to issue the NetView MVS command.

To display packet trace data, set any display options and press F4 (View Packets). For displaying packets, you can make only one selection in the Protocol field. Pressing F4 opens the PKTTRACE SUMMARY panel (FKXK2A26); see [Figure 48 on page 67](#).

Note: You can request a more specific trace report by pressing F8 (Extended Options), which opens the Display Packets Control Extended Options panel (FKXK2A25). On that panel, you can set options for detailed packet data.

PKTTRACE SUMMARY panel (FKXK2A26)

The PKTTRACE SUMMARY panel (FKXK2A26) provides summary packet trace information for the packets requested. To navigate the panel, press F11 (Right) to scroll to the right or F8 (Forward) to scroll forward.

To save packets in CTRACE format and Sniffer trace format, press F2 (Save Packets). The PKTTRACE Summary - Save Packets pop-up panel (FKXK2S26) will be displayed, which is shown in [Figure 49 on page 68](#).

To see commands that are available for a specific packet, press F9 (Commands). A list of commands that you can use for the selected packet, such as Ping, TraceRte, and Hostnames, is displayed.

FKXK2A26 PKTTRACE SUMMARY							NTV70
							More:+
DP	Nr	hh:mm:ss.mmmmm	IpId	Seq_num	Ack_num	Wndw	Flags
OU1582418	12:42:23.053840	911E	protocol=UDP				
			ESP Spi: 3082015D	Seq: 02010104			
OU1582417	12:42:23.053296	911C	protocol=UDP				
			ESP Spi: 308201C8	Seq: 02010104			
IU1582416	12:42:23.053281	911D	protocol=UDP				
			30470201 0104076E	*.....>	0G.....n*		
OU1582415	12:42:23.052033	911A	protocol=UDP				
			ESP Spi: 308201EC	Seq: 02010104			
IU1582414	12:42:23.052017	911B	protocol=UDP				
			30470201 0104076E	*.....>	0G.....n*		
IU1582413	12:42:23.050783	9119	protocol=UDP				
			30470201 0104076E	*.....>	0G.....n*		
OU1582412	12:42:23.050075	9118	protocol=UDP				
			ESP Spi: 308201BC	Seq: 02010104			
IU1582411	12:42:23.049256	9117	protocol=UDP				
			30470201 0104076E	*.....>	0.....n*		
Command ==>							
F1=Help	F2=Save Packets	F3=Return	F4=Details	F5=Refresh	F6=Roll		
F7=Backward	F8=Forward	F9=Commands		F11=Right	F12=Cancel		

Figure 48. PKTTRACE SUMMARY Panel

```

FKXK2S26          PKTRACE  SUMMARY - Save Packets          NTV70
                                                           More:+
DP   Nr hh:mm:ss.mmmmmm IpId   Seq_num   Ack_num   Wndw  Flags
IT   710 05:40:57.570752 0046  2754826990  846188506 32767 ACK

: .....:
: CTRACE format data set name:      :
: NETVIEW.NMPIPL10.D030619.T083012.CTRACE :
: .....:
: Create Sniffer File (Y/N)  N      :
: Sniffer format data set name:    :
: NETVIEW.NMPIPL10.D030619.T083012.SNIFFER :
: .....:
: Press Enter to save packets to the data set(s) :
: F1=Help          F3=Return          F6=Roll          F12=Cancel :
: .....:
Command ===>

```

Figure 49. PKTRACE Summary - Save Packets Panel

Display Packet Control - Save Packets pop-up panel (FKXK2S24)

Use the Figure 50 on page 68 to save packet traces in CTRACE format and Sniffer trace format. If you have specified a PKTS.SAVEHLQ in CNMSTYLE, the data set names for CTRACE and Sniffer trace formats are generated by the NetView program, and are displayed in the fields. You can change the names as required by overtyping the fields.

Set the **Create Sniffer File** field to Y to run the process to generate a Sniffer trace format data set. If the packet traces are saved successfully in CTRACE format and Sniffer trace format, the DSI633I message is displayed.

```

FKXK2S24          Display Packet Control - Save Packets          LOCAL
                                                           NMPIPL10

Stack: TCPIP          Infrc Name: ALL

LAddr *
RAddr *

: .....:
: CTRACE format data set name:      :
: NETVIEW.NMPIPL10.D030619.T083012.CTRACE :
: .....:
: Create Sniffer File (Y/N)  N      :
: Sniffer format data set name:    :
: NETVIEW.NMPIPL10.D030619.T083012.SNIFFER :
: .....:
: Press Enter to save packets to the data set(s) :
: F1=Help          F3=Return          F6=Roll          F12=Cancel :
: .....:
Command ===>

```

Figure 50. Display Packet Control - Save Packets panel

Analyzing and Saving IP Packet Traces

If you press F10 (Analyze) in the Display Packet Control panel (FKXK2A24), the analysis function scans the TCP, UDP, and ICMP trace sessions in the collected trace data and analyzes the TCP sessions for a set of error conditions. To keep data from being lost, stop the trace collection before starting analysis. The amount of data that is analyzed is based on the options that are selected on the Display Packet Control panel (FKXK2A24).

Note: The UDP and ICMP sessions are listed so that you can view the session report for the individual sessions, but these sessions are not analyzed.

From the resulting display, you can request a list of the sessions that contain specific types of errors, and, from that list, you can request the detailed information for a specific session. The detailed data for a session contains all the packets in the collected trace data that is associated with the selected session.

For the analysis function, the rules for the Protocol field on the Display Packet Control panel (FKXK2A24) are as follows:

- The TCP, UDP, and ICMP protocols are supported.
- More than one protocol can be selected, and any combination of the supported protocols is allowed.
- Selecting ALL, or selecting ALL in combination with any of the supported protocols, processes all the supported protocols.
- The OSPF and numeric protocols are not supported. If these protocols are specified, either singly or in combination with other protocols (including ALL), an error message is displayed.

If you select only UDP or ICMP as the protocol on the Display Packet Control panel (FKXK2A24), when you press F10 (Analyze), the trace analysis panel for the selected protocol is displayed. See [“Viewing UDP and ICMP Sessions”](#) on page 73.

Otherwise, when you press F10 (Analyze), the Packet Trace Analysis panel (FKXK2B10), which is shown in [Figure 51 on page 69](#), lists all the sessions that were found, based on the options that you selected.

FKXK2B10

Packet Trace Analysis

LOCAL
NMP217

TCP Sessions 2

UDP Sessions 7

ICMP Sessions 0

TCP Sessions with error flags 2

Unacknowledged Syns 0

Window Probes 0

Retransmissions 0

Reset Flags 0

Duplicate Acks 2

Delayed Acks 0

Zero Window Size 0

Command ===>

F1=Help

F3=Return

F4=Sessions

F6=Roll

F12=Cancel

Figure 51. Packet Trace Analysis Panel

From this panel, you can select the TCP Sessions with error flags option to display the TCP sessions that have error flags, or select a specific TCP session error type (Retransmissions, Duplicate Acks, Zero Window Size, Window Probes, Reset Flags, or Delayed Acks) to display all sessions that have the selected error flag. After you make a selection (for example, Duplicate Acks), press F4 (Sessions) to display a list of the sessions that meet your specification on the Packet Trace Analysis TCP Sessions panel (FKXK2B22). For more information about TCP session analysis, see [“Analyzing TCP Sessions”](#) on page 69.

The following actions are also available from the Packet Trace Analysis panel (FKXK2B10):

- Place the cursor on TCP Sessions and press F4 (Sessions) to go to the Packet Trace Analysis All TCP Sessions panel (FKXK2B20), which lists all TCP sessions and the number of packets and error flags for each. For information about filtering and navigation on this panel, see the online help.
- Place the cursor on UDP Sessions or ICMP Sessions and press F4 (Sessions) to go to the trace analysis panel for the selected protocol. For more information about UDP and ICMP sessions, see [“Viewing UDP and ICMP Sessions”](#) on page 73.

Analyzing TCP Sessions

The Packet Trace Analysis TCP Sessions panel (FKXK2B22), which is shown in [Figure 52 on page 70](#), displays the sessions with the error type that you specified on the Packet Trace Analysis panel (FKXK2B10). The Trace Flag Type field shows the error type that you selected, in this case,

Duplicate Ack. Two lines are displayed for each session with packets that match the requested error flag type. The first line contains the number of packets in the session and the local IP address and port. The second line contains the remote IP address and port for the session.

FKXK2B22

Packet Trace Analysis TCP Sessions

NTVF9

Trace Flag Type: Duplicate Ack

	Packets	Flags	IP Address-Ports (Local and Remote)
-	90	2	9.42.45.101-1033 9.42.45.10-23
-	140	1	9.42.45.101-1035 9.42.45.10-23
-	96	2	9.42.45.101-1038 9.42.45.10-23
-	70	2	9.42.45.101-23 9.42.45.196-1026
-	14	1	9.42.45.101-1034 9.42.45.196-21
-	24	1	9.42.45.101-1037 9.42.45.196-21
-	22	1	9.42.45.101-1039 9.42.45.196-23

Command ==>

F1=Help

F3=Return

F9=Filters

F4=Select

F6=Roll

F12=Cancel

Figure 52. Packet Trace Analysis TCP Sessions Panel

In this panel, you can navigate through the session list by using commands to move to the beginning or the end of the session list or you can specify the number of sessions to scroll; for more information, see the online help. To filter the session list, press F9 (Filters); you can filter by local IP address and port, by local IP address, by local port, by remote IP address and port, by remote IP address, or by remote port. From the filtered session list that is displayed, you can return to the unfiltered list by pressing F9 (Unfilter).

If you select a session on the Packet Trace Analysis TCP Sessions panel (FKXK2B22) and press F4 (Select), the Session Analysis panel (FKXK2B51) is displayed. This panel is shown in Figure 53 on page 71. Note the following information on this panel about the selected session, all of which can be useful in diagnosing a connection or network problem:

- The resolved host names for the local and remote IP addresses
- The connection status (for this example, ESTABLISHED)
- The number of errors of each type for both inbound and outbound data
- The variations in window sizes for inbound and outbound data

```

FKXK2B51                      Session Analysis                      LOCAL

Local  IP  9.42.45.101
      Port 1026 Host Name  nmp101.tivlab.raleigh.ibm.com

Remote IP  9.42.45.10
      Port 23  Host Name  nmpipl10.tivlab.raleigh.ibm.com

Total Packets Summarized  30          Status  ESTABLISHED

  Flags      Inbound  Outbound  Window Size  Inbound  Outbound
  -----
Retransmissions  0          0          Largest      32768    32768
Duplicate Acks   0          1          Average      32759    32753
Reset           0          0          Smallest     32750    32707
Window Size 0    0          0
Window Probes    0          0
Delay Ack        1          2

Command ==>
F1=Help          F8=Packets      F3=Return      F10=Report      F6=Roll
                  F9=Actions                      F12=Cancel

```

Figure 53. Session Analysis Panel

From the Session Analysis panel (FKXK2B51), shown in [Figure 53 on page 71](#), you can continue your analysis as follows:

- To go to the Session Analysis Packets panel (FKXK2B53), press F8 (Packets); see [“Displaying Session Packets” on page 71](#).
- To go to the Session Analysis - Actions panel (FKXK2B52), press F9 (Actions); see [“Issuing Commands for Sessions” on page 72](#).
- To display a full z/OS Communications Server session report, press F10 (Report). For more information, see the z/OS Communications Server library.

Displaying Session Packets

The Session Analysis Packets panel (FKXK2B53), which is shown in [Figure 54 on page 71](#), displays the packets for the selected connection. To navigate the panel, press F11 (Right) to scroll to the right or F8 (Forward) to scroll forward.

```

FKXK2B53                      Session Analysis Packets                      LOCAL
                                More:+

Packet Summary
  TcpHdr  IO F      Seq      Ack  RcvWnd  Data  Delta Time      TimeStamp
  -----
  A  S    I    2258808397      0    32768    0    0.000000  13:15:56.397877
  A  S    I    2414361815  2258808398    32768    0    0.017182  13:15:56.415059
  A      u    2258808398  2414361816    32768    0    0.000071  13:15:56.415130
  AP     I    2414361816  2258808398    32768    3    0.042128  13:15:56.457258
  A      d    2258808398  2414361819    32765    0    0.277782  13:15:56.735040
  AP     .    2258808398  2414361819    32765    3    0.344952  13:15:57.079992
  AP     .    2258808401  2414361819    32765    3    0.000511  13:15:57.080503
  AP     I  a    2414361819  2258808404    32765    0    0.000861  13:15:57.081364
  AP     I    2414361819  2258808404    32765    3    0.011026  13:15:57.092390
  AP     d    2258808404  2414361822    32765    0    0.245366  13:15:57.337756
  AP     I    2414361822  2258808404    32765    6    0.000851  13:15:57.338607
  AP     +    2258808404  2414361828    32762   18    0.000643  13:15:57.339250
  AP     I    2414361828  2258808422    32750    3    0.016542  13:15:57.355792
  AP     +    2258808422  2414361831    32765    3    0.000281  13:15:57.356073
  AP     .    2258808425  2414361831    32765    3    0.000100  13:15:57.356173
  AP     I  +    2414361831  2258808425    32765    9    0.000855  13:15:57.357028

Command ==>
F1=Help          F3=Return      F4=Packet Detail      F6=Roll
F7=Backward F8=Forward      F11=Right             F12=Cancel

```

Figure 54. Session Analysis Packets Panel

To see the details of a packet, select a packet on the Session Analysis Packets panel (FKXK2B53 or FKXK2B54) and press F4. The Packet Detail window, which is shown in [Figure 55 on page 72](#), displays the full content of the selected packet. To return (to FKXK2B53 or FKXK2B54), press F3.

```

CNMKWIND OUTPUT FROM  Packet Detail                                LINE 0 OF 48
*----- Top of Data -----*
z/OS TCP/IP Packet Trace Formatter, Copyright IBM Corp. 2000, 2009; 2009.028

**** 2019/02/12
RcdNr Sysname Mnemonic Entry Id   Time Stamp   Description
-----
-----
971 NMP101  PACKET  00000004 13:15:56.415130 Packet Trace
To Interface : TCPIPLINK      Device: QDIO Ethernet   Full=52
Tod Clock    : 2019/02/12 13:15:56.415130 Intfx: 5
Segment #    : 0              Flags: Out
Source       : 9.42.45.101
Destination  : 9.42.45.196
Source Port   : 1029          Dest Port: 23   Asid: 002C TCB: 00000000
IpHeader: Version: 4          Header Length: 20
Tos          : 00             QOS: Routine Normal Service
Packet Length: 52            ID Number: 523A
Fragment     :               Offset: 0
TTL          : 64             Protocol: TCP      CheckSum: BB0D F
Source       : 9.42.45.101
Destination  : 9.42.45.196

TCP
Source Port   : 1029 ( )      Destination Port: 23 (telnet)
Sequence Number: 2258808398  Ack Number: 2414361816
Header Length : 32           Flags: Ack
Window Size   : 32768        CheckSum: C87B FFFF Urgent Data Pointer:
Option        : NOP
Option        : NOP
Option        : Timestamp    Len: 10 Value: 1EE2BE99 Echo: 1EE2BE98

Ip Header     : 20            IP: 9.42.45.101, 9.42.45.196 Offset: 0
000000 45000034 523A0000 4006BB0D 092A2D65 092A2DC4

Protocol Header : 32          Port: 1029, 23          Offset: 14
000000 04050017 86A2AE4E 8FE83CD8 80108000 C87B0000 0101080A 1EE2BE99 1EE2BE98
No packets required reassembly

=====
TO SEE YOUR KEY SETTINGS, ENTER 'DISPFK'
CMD==>

```

Figure 55. Packet Detail for the Packet Selected in Session Analysis Packets

Issuing Commands for Sessions

The Session Analysis - Actions panel (FKXK2B52), which is shown in [Figure 56 on page 73](#), displays commands that you can issue for a selected connection.

```

FKXK2B52          Session Analysis - Actions          LOCAL
                                                         NMPIPL10
Local  IP: 9.30.240.21  *****
      Port: 175      Host Name: NMPIPL  _ Command
Remote IP: 9.30.238.6      1. Drop Connection
      Port: 2339      Host Name: HSSVMA      2. Save Session Packets in
                                         NetView Format
Total Packets Summarized: 400      3. Save Session Packets in
                                         CTRACE format and
                                         SNIFFER format
Flags      Inbound      Outbound
-----
Retransmissions 0      0
Duplicate Acks  0      0
Reset           0      0
Window Size 0    0      0      F1=Help      F3=Return
Window Probes  0      0      0      F6=Roll    F12=Cancel
Delayed Acks   0      0
Selective Acks  0      0
*****
Command ==>

```

Figure 56. Session Analysis - Actions Panel

The following actions are available from the Session Analysis - Actions panel (FKXK2B52):

- To drop the active connection for this session, enter 1 (Drop Connection). The Session Analysis Drop Command Confirmation panel (FKXK2B5A) is displayed. This panel provides information about the connection that is to be dropped. Press F4 (Confirm Drop) to confirm that you want to drop the connection, or press F3 (Return) to cancel the drop request.
- To save the session packets in NetView format, enter 2. The Saved Packet Trace Details panel (FKXK2B62) is displayed. This panel provides details about the packet trace that you want to save. Provide a description (for example, Print server problem) and press F4 to save the trace. The FKX475I message indicates that the trace was saved. You can retrieve the trace later; see [“Retrieving Saved IP Packet Traces”](#) on page 75.
- To save the session packets in CTRACE format and Sniffer trace format, enter 3. The Session Analysis Data Set Name Input panel (FKXK2B55) is displayed so that you can specify the name of a data set in which to save the packets. If the save is successful, the DSI633I message is displayed.

Viewing UDP and ICMP Sessions

If you selected only **UDP** or **ICMP** as the protocol before pressing F10 (Analyze) on the Display Packet Control panel (FKXK2A24) or if you placed the cursor on **UDP Sessions** or **ICMP Sessions** before pressing F4 on the Packet Trace Analysis panel (FKXK2B10), the sessions are displayed in the Packet Trace Analysis UDP Sessions or the Packet Trace Analysis ICMP Sessions panel (FKXK2B23), which is shown in [Figure 57](#) on page 74.

FKXK2B23	Packet Trace Analysis UDP Sessions	NTVF9
	Packets IP Address-Ports (Local and Remote)	
- 2	9.42.45.101-9850	
	9.42.4.253-53	
- 4	9.42.45.101-9851	
	9.42.4.253-53	
- 4	9.42.45.101-9860	
	9.42.4.253-53	
- 4	9.42.45.101-9861	
	9.42.4.253-53	
Command ==>		
F1=Help	F3=Return F9=Filters	F4=Session Report F6=Roll F12=Cancel

Figure 57. Packet Trace Analysis UDP Sessions Panel

In this panel, the IP addresses and ports that are shown under **IP Address-Ports** are the connection partners for UDP sessions. For ICMP sessions, only IP addresses are shown.

Note: ICMP is a connectionless protocol that does not have actual sessions, but the Communications Server IP Trace Session Report collects the ICMP packets between two hosts and displays them in a session report; this is known as an ICMP session.

In this panel, you can navigate through the session list by using commands to move to the beginning or the end of the session list or you can specify the number of sessions to scroll; for more information, see the online help. To filter the session list, press F9 (Filters); you can filter by local IP address and port, by local IP address, by local port, by remote IP address and port, by remote IP address, or by remote port. From the filtered session list that is displayed, you can return to the unfiltered list by pressing F9 (Unfilter).

Select a session and press F4 to see the session report for that session, such as the UDP session report that is shown in the PACKET DETAIL window in [Figure 58 on page 75](#).

```

CNMKWIND OUTPUT FROM  PACKET DETAIL                                LINE 0 OF 50
*----- Top of Data -----*
BNH773I NUMBER OF PACKETS: 2 , MISSED BUFFERS: 0 , TCPNAME: TCPIP
z/OS TCP/IP Packet Trace Formatter, Copyright IBM Corp. 2000, 2009; 2009.028

**** 2019/04/16
      No packets required reassembly

=====
Interface Table Report
Index Count Link                      Address
   5      2 TCPIPLINK                9.42.45.101

UDP Sessions Report
      1 sessions found
-----
2 UDP packets summarized
Local IP address:                    9.42.45.101
Local port, app:                      9850
Remote IP address:                    9.42.4.253
Remote port, app:                      53 domain
Protocol:                             UDP
First TimeStamp:      2019/04/16 13:26:29.545709
Last TimeStamp:       2019/04/16 13:26:29.546448
Duration:              00:00:00.000739
Statistics:              Inbound,    Outbound
Packets:                1,          1
Fragments:              0,          0 (  0%), (  0%)
Bytes sent:              298,        49
Time spent sending:      00:00:00.0, 00:00:00.0 ( 100%), (  0%)
Time spent fragments:    00:00:00.0, 00:00:00.0 (  0%), (  0%)
Throughput:              454.711,    74.768 K/s
Largest segment size:    298,        49
Average segment size:    298,        49
Smallest segment size:   298,        49
Direction change:        1,          0
UDP Flags IO              Delta      Time DatLn  RcdNr  Inf  Ip_id  Froff  Drop
      0 00:00:00.000000 13:26:29.545709 49 2848 5 738C 0 0
      D I 00:00:00.000739 13:26:29.546448 298 2849 5 0000 0 0
16384 bytes of storage were used for the session report
TO SEE YOUR KEY SETTINGS, ENTER 'DISPFK'
CMD==>

```

Figure 58. UDP Session Report (Packet Detail for the Selected UDP Session)

The following actions are available from this window:

- To view the contents of a packet, move the cursor to a packet summary that is highlighted in green and press F4.
- To save the UDP or ICMP packet data, press F9.

Retrieving Saved IP Packet Traces

Use the LISTTRC command, which has no parameters, from the NetView command line to see a list of all of the traces that are saved. The Saved Packet Traces panel (FKXK2B61), which is shown in [Figure 59](#) on [page 76](#), displays any packet traces that you saved. This panel shows the print server problem trace that was saved in “[Issuing Commands for Sessions](#)” on [page 72](#).

```

FKXK2B61                                Saved Packet Traces

Collect Time      Operator Domain Stack      Description
-----
_ 09/09/19 12:40:37 PHK      LOCAL  TCPIP      TEST
_ 09/09/19 12:51:14 PHK      LOCAL  TCPIP      PRINT SERVER PROBLEM

Command ==>
F1=Help      F3=Return   F4=View Trace      F6=Roll
F7=Backward F8=Forward  F10=Details F11=Delete Trace  F12=Cancel

```

Figure 59. Saved Packet Traces Panel

You can select a trace and press F4 (View Trace) to display the content of this trace or press F11 (Delete Trace) to delete the data that was saved for this trace.

If you select the PRINT SERVER PROBLEM trace and press F10 (Details), the Saved Packet Trace Details panel (FKXK2B62), which is shown in [Figure 60 on page 76](#), displays information about the saved packet trace.

```

FKXK2B62                Saved Packet Trace Details                LOCAL
Service Point/Stack: NMPIPL10                Proc: TCPIP
Save Time: 03/06/19 12:52:43                Operator: PHK
Description:
Trace Start Time: 03/06/19 11:13:09                Packets: 400
Trace Last Time : 03/06/19 12:51:01
Local IP: 9.30.240.21
Remote IP: 9.30.238.6
Local Port: 175                Remote Port: 2339                Protocol: TCP
Interface Name: ALL
Command ==>
F1=Help                F3=Return                F4=Save Trace                F6=Roll
                                F12=Cancel

```

Figure 60. Saved Packet Trace Details Panel (Viewing Saved Data)

From the Saved Packet Trace Details panel, you can press F4 (View Trace) to display the content of this trace.

If the trace was saved in CTRACE format, the **CTRACE Format** field is set to Y and the **Data Set Name** field indicates the data set in which the trace was saved.

If you press F4, the FMTPACKT Session Detail Report, which is shown in [Figure 61 on page 77](#), displays the content of this trace.


```

CNMKWIND OUTPUT FROM  FMTPACKT Session Detail Report                LINE 0 OF 269
*----- Top of Data -----*
BNH773I NUMBER OF PACKETS: 221 , MISSED BUFFERS: 0 , TCPNAME: TCPIP
z/OS TCP/IP Packet Trace Formatter, Copyright IBM Corp. 2000, 2009; 2009.028

**** 2019/09/09
      No packets required reassembly

=====
Interface Table Report
Index Count Link                Address
   2    221 LOOPBACK            9.42.45.101

UDP Sessions Report
   1 sessions found
-----
221 UDP packets summarized
Local IP address:                9.42.45.101
Local port, app:                  161 snmp
Remote IP address:               9.42.45.101
Remote port, app:                 5823
Protocol:                        UDP
FirstTimeStamp: 2019/09/09 10:59:02.698435
LastTimeStamp:  2019/09/09 10:59:02.800651
Duration:                00:00:00.102216
Statistics:              Inbound, Outbound
Packets:                  0,      221
Fragments:                0,      0 ( 0%), ( 0%)
Bytes sent:                0,    10657
Time spent sending: 00:00:00.0, 00:00:00.1 ( 0%), ( 100%)
Time spent fragments: 00:00:00.0, 00:00:00.0 ( 0%), ( 0%)
Throughput:                0,    101.887 K/s
Largest segment size:      0,      83
Average segment size:      0,      48
Smallest segment size:     0,      45
Direction change:         0,      0
UDP Flags IO              Delta      Time  DatLn  RcdNr  Inf  Ip_id  Froff  Drop
      0 00:00:00.000000 10:59:02.698435    47  41638   2   73F6    0    0
      0 00:00:00.000588 10:59:02.699023    47  41640   2   73F8    0    0
      0 00:00:00.000426 10:59:02.699450    47  41642   2   73FA    0    0
TO SEE YOUR KEY SETTINGS, ENTER 'DISPFK'
CMD==>

```

Figure 61. FMTPACKT Session Detail Report

OSA Packet Tracing

When you select OSATRACE, the OSATRACE Control panel (FKXK2A30), which is shown in [Figure 62 on page 77](#), is displayed.

```

FKXK2A30      OSATRACE Control SYSTCPOT ACTIVE      for NVDomain: LOCAL
                                           z/OS      : V1R12

Service Point/Stack: TVT2007 TCPNAME: TCPIP7
OPKTS: ACTIVE      On Task: AUTOOPKT GTF: NO

Start Time:                                           Writer: *NONE*

Options: 1-START  2-STOP  3-VIEW PACKETS

  OSA Port  Stat/ Length Data      Record      Time  Discard  Nofilter
           Auth
- OSA1      OFF   224   1024      2147483647 10080 EXCEPTION NONE
           UNKNOWN 0      0      0      0
- OSAA      ON    224   1024      2147483647 10080 EXCEPTION ALL
           LOGICAL 0      172   12    164
           NEW    224   1024      2147483647 10080 EXCEPTION NONE

Command ==>
F1=Help          F3=Return  F4=Stop SYSTCPOT F5=Refresh  F6=Roll
F7=Backward      F8=Forward  F9=Filters      F10=PKTS Management F12=Cancel

```

Figure 62. OSATRACE Control Panel

Starting and Stopping an OSA Packet Trace

Starting and stopping OSA packet traces involves the following tasks:

- “Managing the z/OS Communication Server OSA Packet Trace” on page 78
- “Managing the NetView OSA Packet Trace Collection Process” on page 78

Managing the z/OS Communication Server OSA Packet Trace

If SYSTCPOT is not shown as ACTIVE on the OSATRACE Control panel (FKXK2A30), type any character next to at least one interface (for example, OSAA) and press F4 (Start SYSTCPOT) to start packet tracing by the SYSTCPOT component of z/OS Communications Server. After OSA packet tracing starts, SYSTCPOT is shown as ACTIVE, the status of the interface changes to ON, and the options change.

Note: The OSA SNMP subagent must be active to control OSA traces.

To set optional filters or to see more detail for an interface, press F9 (Filters) on the OSATRACE Control panel (FKXK2A30) to go to the OSATRACE Filters panel (FKXK2A31), which is shown in Figure 63 on page 78. Make any changes that are needed, press F4 to update the filters, and then press F3 to return.

FKXK2A31	OSATRACE	Filters	SYSTCPOT	ACTIVE	for NVDomain: LOCAL
Service Point/Stack: TVT2007			Proc: TCPIP7		z/OS : V1R12
OSA Port Name: OSAA			Clear Filters: NO (YES/NO)		

Protocol	Ethernet Type	Port	Device ID	VLAN ID	Mac Address
TCP		00020			
		00021			

Command ==>
F1=Help
F3=Return
F4=Update Filters
F6=Roll
F12=Cancel
F8=IP Addresses

Figure 63. OSATRACE Filters Panel

To set optional IP address filters for the interface from the OSATRACE Filters panel (FKXK2A31), press F8 (IP Addresses) to go to the OSATRACE Filters panel (FKXK2A32). After you make your changes, press F3 until you return to the OSATRACE Control panel (FKXK2A30).

On the OSATRACE Control panel (FKXK2A30), note the Options line:

Options: 1-START 2-STOP 3-VIEW PACKETS

To start an OSA packet trace filter for an OSA port name, type 1 next to the OSA port name that you want to trace, and press Enter. To stop an OSA packet trace filter, type 2 next to the OSA port name, and press Enter. To view the packets that were traced, type 3 next to the OSA port name, and press Enter; see “Viewing OSA Packet Trace Data” on page 79.

Managing the NetView OSA Packet Trace Collection Process

To manage the packet collection process (the PKTS task), press F10 (PKTS Management). The NetView PKTS Management panel (FKXK2A33), which is shown in Figure 64 on page 79, is displayed. Use this panel to start and stop the collection of packet data. Make any changes that are needed, and press F3 to return to the OSATRACE Control panel (FKXK2A30).

Note: Instead of using the NetView PKTS Management panel, you can use the PKTS command or statements in the CNMSTUSR or CxxSTGEN member; see [“Defining Packet Trace Data Collection”](#) on page 61.

```

FKXK2A33          NetView PKTS Management          PKTS Status: ACTIVE
                   for PSOURCE=OSA                 Domain:LOCAL
Start with Storage Size:      M

_ Stop
_ Stopcoll
Define TCPName: TCPIP7      OPID: AUT00PKT
_ Purge
  Portname *
    Time: Start *
        End   *

Command ==>
F1=Help                      F3=Return                      F6=Roll
                             F12=Cancel

```

Figure 64. NetView PKTS Management Panel for OSA Packet Trace

Viewing OSA Packet Trace Data

Use the Display OSA Packet Control panel (FKXK2A34), which is shown in Figure 65 on page 79, to specify additional filtering criteria. This panel is displayed by typing 3 next to the OSA port name to be traced on the OSATRACE Control panel (FKXK2A30).

```

FKXK2A34          Display OSA Packet Control          LOCAL
Service Point/Stack: TVT2007  Proc: TCPIP7
OSA Port Name: OSAA

Time: Start *
      End   *

MaxRecs: 1  1-Last  100      Truncate: 65535
          2-First

Command ==>
F1=Help                      F3=Return      F4=View Packets      F6=Roll
                             F8=Extended Options  F12=Cancel

```

Figure 65. Display OSA Packet Control Panel

To display packet trace data, set any display options and press F4 (View Packets). Pressing F4 opens the OSA TRACE PACKETS SUMMARY panel (FKXK2A36); see [Figure 66 on page 80](#).

Note: You can request a more specific trace report by pressing F8 (Extended Options), which opens the Display OSA Packets Control Extended Options panel (FKXK2A35). On that panel, you can set options for detailed packet data.

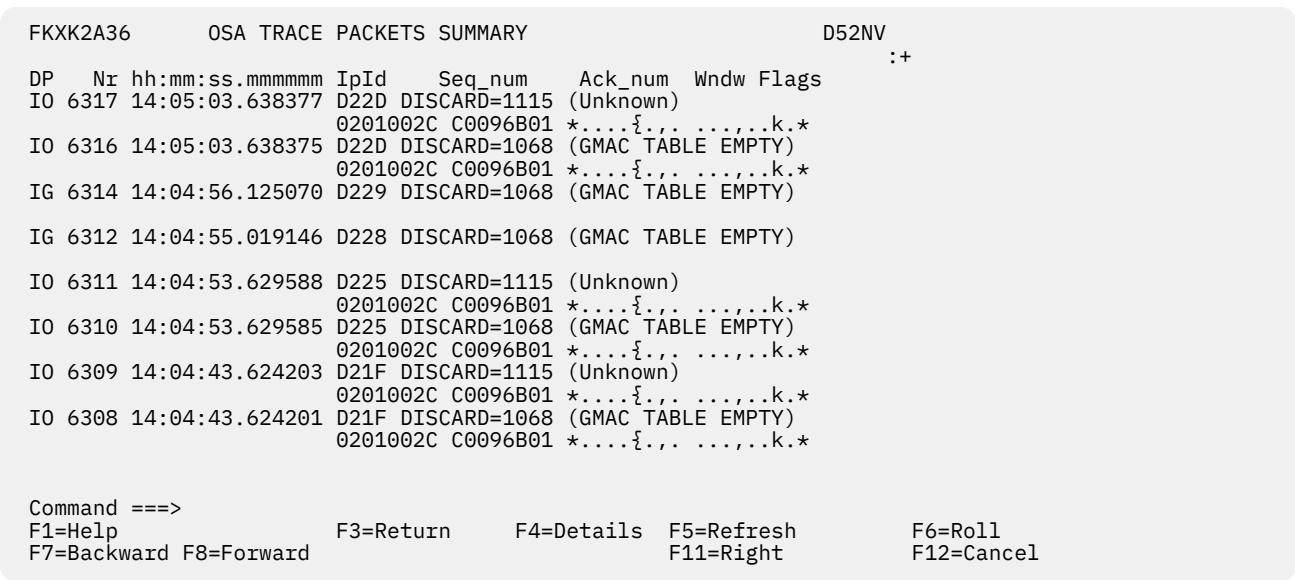


Figure 66. OSA TRACE PACKETS SUMMARY Panel

The OSA TRACE PACKETS SUMMARY panel (FKXK2A36) provides summary packet trace information for the packets requested. To navigate the panel, press F11 (Right) to scroll to the right or F8 (Forward) to scroll forward.

From the OSA TRACE PACKETS SUMMARY panel (FKXK2A36 or FKXK2A37), select a packet and press F4 (Details). The details of the selected packet are displayed in the Packet Detail window, which is shown in [Figure 67 on page 80](#).

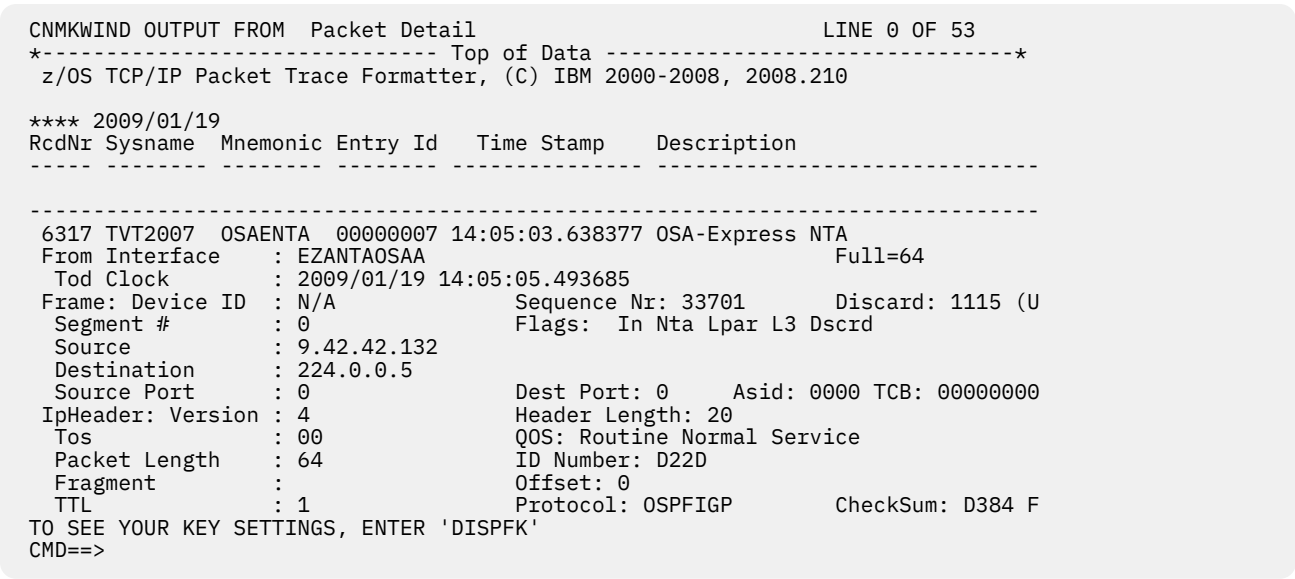


Figure 67. Packet Detail for the Selected OSA Packet

Component Tracing

When you select CTRACE for a stack with no scheduled tracing, the TCP/IP for 390 CTRACE Control panel (FKXK2A12), which is shown in [Figure 68 on page 81](#), is displayed. For a stack with active or delayed tracing, the TCP/IP for 390 CTRACE Control panel (FKXK2A10), which is shown in [Figure 72 on page 84](#), is displayed. For information about the fields on the panel, press F1 to see the online help.

FKXK2A12	TCP/IP for 390 CTRACE Control	SYSTCPIP	A55NV
Service Point/Stack: TVT2009		Proc: TCPIP	Domain: LOCAL
Delay Start Until: YYYY-MM-DD-HH.MM.SS		On Task: JOEF_____	
Set a Timer for HH.MM.SS		Writer: CTTCP__	
_ LISTS (IPAddrS, Ports, Job Names, ASIDs)			
Command ==>			
F1=Help	F2=Main Menu	F3=Return	F4=Start Trace F10=Options
			F6 =Roll F12=Cancel

Figure 68. CTRACE Control Panel (FKXK2A12)

Before you begin a trace, provide the following information on the TCP/IP for 390 CTRACE Control panel (FKXK2A12):

- You can specify IP addresses, IP ports, job names, and address space identifiers (ASIDs) that are to be traced. To do that, select the LISTS field; see [“Specifying IP Addresses, IP Ports, Job Names, and Address Space Identifiers for the Component Trace” on page 81.](#)
- You must specify options for the trace. To do that, press F10; see [“Specifying Options for the Component Trace” on page 82.](#)
- You must decide if the trace is to begin immediately or is to be scheduled for a later date or time. This cannot be done until the trace options are specified. See [“Starting or Scheduling the Component Trace” on page 83.](#)

Specifying IP Addresses, IP Ports, Job Names, and Address Space Identifiers for the Component Trace

When you select the LISTS field, the TCP/IP for 390 CTRACE Control panel (FKXK2A11) displays the IP addresses and submasks component trace filters currently set for this stack. If the CTRACE status is ACTIVE, all component trace filters currently set are collected and displayed. When the CTRACE status is in DELAY mode, all global variables for component trace filters that are set for a delayed start are displayed. When the trace is inactive, this panel looks similar to the one in [Figure 69 on page 82](#), and you can edit the IPADDRS/Mask(Prefix) fields. For information about the fields on the panel, press F1 to see the online help.

FKXK2A14	TCP/IP for 390 CTRACE Control				SYSTCPIP	A55NV	
OPTIONS:							
- ALL	-	ALLMIN	-	ACCESS	-	AFP	- ARP
- CLAW	-	CONFIG	-	CSOCKET	-	DLC	- EID
- ENGINE	-	FIREWALL	-	ICMP	-	IN	- INIT
- INTERNET	-	IOCTL	-	IPADDR	-	IPSEC	- LATCH
- LCS	-	MESSAGE	-	MINIMUM	-	MISC	- ND
- NONE	-	OETCP	-	OEUDP	-	OPCMDS	- OPMSG
- PASAPI	-	PFS	-	PFSMIN	-	PING	- POLICY
- PORT	-	QUEUE	-	RAW	-	ROUTE	- RW
- SERIAL	-	SMTP	-	SNMP	-	SOCKAPI	- SOCKET
- STORAGE	-	SYSTEM	-	TC	-	TCP	- TCPMIN
- TELNET	-	TELNVAM	-	TIMER	-	TN	- UD
- UDP	-	VTAM	-	VTAMDATA	-	WORKUNIT	- XCF
Command ==>							
F1=Help		F2=Main Menu		F3=Return		F6 =Roll	
						F12=Cancel	

Figure 71. CTRACE Control Panel (FKXK2A14)

Starting or Scheduling the Component Trace

After you select options on the TCP/IP for 390 CTRACE Control panel (FKXK2A14), you can start a trace on the TCP/IP for 390 CTRACE Control panel (FKXK2A12) in any of these ways:

- To start the trace immediately, press F4 (Start Trace).
- To delay the start of the trace, specify the date or date and time that tracing is to begin in the **Delay Start Until** field. The time must be specified in a 24-hour format. If the date portion YYYY-MM-DD is omitted and the input HH.MM.SS is earlier than the current time, the trace starts on the next day at the input time. For example, if the current time on the system clock is 9:33 a.m. and you specify 07.45.00 as the time to begin component tracing, the trace is scheduled to begin the following day at 7:45 a.m.
- To specify how long the trace is to run, specify a time in the **Set a Timer for** field. If you leave the **Set a Timer for** field blank, the trace runs until it is manually stopped.

If you want the trace to run on another task, enter any valid operator ID (as defined in DSIOPF) in the **On Task** field, which is valid only for delayed traces.

Note: If you specify another operator ID, that operator must be logged on at the time the trace runs.

The **Writer** field contains the source JCL to create an external writer where trace data is stored. The writer must be established before the trace runs. For more information about creating source JCL for an external writer, see *z/OS MVS Diagnosis: Tools and Service Aids*. The writer name for component tracing can be customized. Copy the following statement to the CNMSTUSR or CxxSTGEN member, and then change it to the appropriate name. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*. If the NetView program has been started, issue the RESTYLE COMMON command to pick up the change.

```
COMMON.EZLTCPcTRACEwriter = new_CTTCP // AON TCP component writer name
```

After you finish setting options, press F4 to begin or schedule the trace. The trace can take several minutes to run because of system processing. The trace might also begin a couple of minutes past the specified time, depending on system processing. When an immediate trace is successfully scheduled, a message similar to the following message is displayed in the IPTrace Control Center panel (FKXK2A01):

```
FKX400I CTRACE SCHEDULED FOR SP TVT2009 BY OPERATOR JOEF
```

For a delayed trace, the FKX401I message is displayed.

Displaying an Active or Delayed Component Trace

To view the details of an active or delayed component trace, select CTRACE from the IPTrace Control Center panel (FKXK2A01). The TCP/IP for 390 CTRACE Control panel (FKXK2A10), which is shown in [Figure 72 on page 84](#), is displayed.

```
FKXK2A10      TCP/IP for 390 CTRACE Control SYSTCPIP      A55NV
Service Point/Stack: TVT2009      Proc: TCPIP      Domain: LOCAL
Start Time: 2007-10-04-14:19:18      Duration: NA
Timer ID: NONE      Writer: CTTCP

Active Options:
ALL

_  LISTS (IPAddrs, Ports, Job Names, ASIDs )

Command ==>
F1=Help      F2=Main Menu      F3=Return      F4=Stop Trace      F6 =Roll
F12=Cancel
```

Figure 72. CTRACE Control Panel (FKXK2A10)

Stopping an Active or Canceling a Delayed Component Trace

To stop an active trace or to cancel a delayed trace, press F4 (Stop Trace) on the TCP/IP for 390 CTRACE Control panel (FKXK2A10), which is shown in [Figure 72 on page 84](#). When you press F4, the system can take several minutes to respond because of system processing.

When an active trace is successfully stopped, the FKX403I message is displayed. When a delayed trace is successfully canceled, the DSI205I message is displayed.

Chapter 8. Managing Connections

The NetView program provides both real-time and historical connection information, including stack name, local and remote addresses and ports, start time, end time and termination code (for connections that ended), sent and received byte and segment counts, retransmit counts, and information about connection state, interface, host, TN3270, and application transparent transport layer security (AT-TLS), if applicable. Data is available both in a readable form and in a form for programming use. Host name translation and IPv4 or IPv6 addresses are supported. In addition, because of the cross-domain capabilities of the NetView program, users can view connection data at remote z/OS hosts.

The NetView program collects and displays dynamic virtual IP address (DVIPA) information including the relationship to TCP/IP connections that use a DVIPA. For more information about DVIPA data collection, including DVIPA connections, see [Chapter 6, “Managing Dynamic Virtual IP Addresses,” on page 45](#).

Collecting Connection Data

You can use the NetView program to collect and query TCP/IP connection data. This function is available only if you have enabled data collection for TCP/IP connections.

The TCPCONN START command starts a long-running process to collect data that relates to the start and end of connections from z/OS Communications Server for the specified TCP/IP stack. To start collecting connection data, use the TCPCONN START command. (You can also configure the NetView program to start data collection automatically during startup; see *IBM Z NetView Installation: Configuring Additional Components* for more information.) The data-collection process persists as an autotask, which is defined by either a FUNCTION.AUTOTASK.TCPCONN statement in the CNMSTYLE member or a TCPCONN DEFINE command.

When a connection ends, data about the connection is collected and correlated with the connection start data. The TCPIPCOLLECT.TCPCONN subtower must be enabled to collect inactive connection data. The TCPCONN STOP command stops the data collection. The TCPCONN QUERY command queries the data that is collected using the TCPCONN START command.

The NetView program collects real-time connection data for any active connection specified by the TCPCONN.KEEP statements in the CNMSTYLE member. In addition, data is stored on DASD for inactive (historical) connections specified by TCPCONN.DASD statements in the CNMSTYLE member. After you have started data collection, you can use the TCPCONN QUERY command to retrieve the collected data based on filtering criteria you specify. For example, you can view all connections to a particular remote host.

Information that is provided by the TCPCONN QUERY command contains all connection data for inactive connections. However, only a subset of this information is displayed for active connections. Therefore, when you need active connection data, use the QUERYACT function instead of the QUERY function.

You can display both active and inactive data by using NetView workspaces in the Tivoli Enterprise Portal. Collection of active connection data, which is based on the TCPCONN QUERYACT command, is controlled by the TEMA.CONNACT subtower and the associated TEMA.CONNACT statements in the CNMSTYLE member. Collection of inactive connection data, which requires the TCPIPCOLLECT.TCPCONN subtower to be enabled, is controlled by the TEMA.CONINACT subtower and the associated TEMA.CONINACT statements in the CNMSTYLE member.

For more information about defining TCP/IP connection management, see *IBM Z NetView Installation: Configuring Additional Components*. For more information about the TCPIPCOLLECT.TCPCONN subtower and the TEMA subtowers, see *IBM Z NetView Installation: Getting Started*. For more information about the FUNCTION.AUTOTASK statement, see the *IBM Z NetView Administration Reference*. For the syntax of and detailed information about the TCPCONN command, see the online help.

You can also collect information about DVIPA connections. For more information about using DVIPAs, see [Chapter 6, “Managing Dynamic Virtual IP Addresses,” on page 45](#).

Viewing Connection Data

You can view information about connections by using NetView workspaces in the Tivoli Enterprise Portal. Two NetView workspaces display information about connections, one with data for active connections and another with data for inactive connections. For information about using these workspaces, see the *IBM Z NetView User's Guide: NetView Enterprise Management Agent*.

You can also view connection status information in a 3270 panel interface. To use the panel interface, issue the IPSTAT command from the NetView command line. For more information about using the IPSTAT panel interface, see [“Checking Connection Status \(IPSTAT\)”](#) on page 88. For the syntax of and detailed information about the IPSTAT command, see the online help.

You can view detailed information about connections by issuing NetView commands from a 3270 command line. To view information about active connections, use either the TCPCONN QUERYACT command or the CNMSTCPC sample. To view information about inactive connections, use either the TCPCONN QUERY command or the CNMSTCPC sample. The output from the TCPCONN QUERYACT and TCPCONN QUERY commands is not formatted and is intended for use in application programs. To view this data in a format that can be read, use the CNMSTCPC sample. For more information about the CNMSTCPC sample, see the *IBM Z NetView User's Guide: NetView*. For the syntax of and detailed information about the TCPCONN command, see the online help.

Table 4 on page 86 lists the command that can be used to collect TCP/IP connection data.

Table 4. TCP/IP Connection Data Collector Command			
Command	Type of Data Collected	Sample to View Data	Messages Returned by the Collector
TCPCONN	TCP/IP connection information	CNMSTCPC	BNH772I or BNH775I

This data collector command is intended for use in application programs. When this data collector command completes successfully, it returns the data in a multiline message. To view the data in a format that can be read, use the CNMSTCPC sample. To display this data in a different format, modify the CNMSTCPC sample or create your own application to format the data that is returned by the TCPCONN command. You can use input parameters with the sample to specify the data to collect. The input parameters for the sample are the same as those for the TCPCONN command. For information about the returned messages, the command parameters, or other information about the data collector commands, see the IBM Z NetView online message and command help.

Notes:

1. Because the data records can be quite wide, the sample is based on the WINDOW command. You can scroll to the right to view the data that is not currently visible in a 3270 session.
2. This sample is also used by the Z NetView Enterprise Management Agent take action commands.

You can also view information about DVIPA connections, which are a subset of all TCP/IP connections. To view DVIPA connection information, you can use the DVIPA Connections workspace, the DVIPCONN command, or the CNMSDVPC sample. For more information about using DVIPAs, see [Chapter 6, “Managing Dynamic Virtual IP Addresses,”](#) on page 45.

Viewing Encryption Data

NetView program utilizes the z/OS Encryption Readiness Technology (zERT) function provided by Communications Server (z/OS 2.3 or higher) to collect cryptographic security attributes of active TCP/IP connections that are protected using the TLS/SSL, SSH and IPsec cryptographic network security protocols. This information is available in NetView workspaces in the Tivoli Enterprise Portal. The NetView Active Connections workspace contains a table summary of connections by encryption protocol. Links

from the summary table or individual connection records contain links to a workspace that displays detailed information about encryption protocols in use for a connection, including encryption-related algorithms, links to certificates, IP filtering rules and more. For information about using these workspaces, see the *IBM Z NetView User's Guide: NetView Enterprise Management Agent*.

You can view detailed connection encryption information by issuing NetView commands from a 3270 command line. You can also generate encryption information in the form of NetView messages by using the CONNSEC QUERY command. This output is not formatted and is intended for use in application programs. To view this data in a format that can be read, use the CNMSZERT sample. For more information about the CNMSZERT sample, see the *IBM Z NetView User's Guide: NetView*. For the syntax of and detailed information about the CONNSEC command, see the online help or *IBM Z NetView Command Reference Volume 1 (A-N)*. [Table 5 on page 87](#) lists the command that can be used to collect TCP/IP connection data.

<i>Table 5. TCP/IP Encryption Data Collector Command</i>			
Command	Type of Data Collected	Sample to View Data	Messages Returned by the Collector
CONNSEC	TCP/IP encryption information	N/A	BNH932I

This command is intended for use in application programs. When the command completes successfully, it returns the data in a multiline message. To display this data, create an application to format the data that is returned by the CONNSEC command. You can use input parameters on the CONNSEC command to specify the data to collect. For information about the returned messages, the command parameters, or other information about the command, see the *IBM Z NetView online message and command help*. This data collector command is intended for use in application programs. When this data collector command completes successfully, it returns the data in a multiline message. To view the data in a format that can be read, use the CNMSZERT sample. To display this data in a different format, modify the CNMSZERT sample or create your own application to format the data that is returned by the CONNSEC command. You can use input parameters with the sample to specify the data to collect. The input parameters for the sample are the same as those for the CONNSEC command. For information about the returned messages, the command parameters, or other information about the data collector commands, see the *IBM Z NetView online message and command help*.

Monitoring Connections

NetView automation can monitor the connections to any application without needing a socket number, for example, a printer application that is connected over TCP/IP. Policy-based thresholds can be defined to detect the following conditions:

- No activity over a specified time interval
- Less than a minimum number of bytes sent over a specified time interval
- More than a maximum number of bytes sent over a specified time interval

Policy-based actions can be any of the following actions:

- Ignore the threshold; that is, take no action.
- Generate a message and notify the appropriate personnel according to a notification policy.
- Break the connection.

The following policy statements must be included in the specified member, and the IPMGT tower in the CNMSTYLE member must be enabled.

- IPCONN defines the connection thresholds and actions (FKXCFG01 member).
- AUTOOPS defines the automation task for monitoring (CNMIPMGT member).
- ACTMON defines the monitoring interval (CNMIPMGT member).

- NOTIFY defines the types of notifications (CNMIPMGT member).

Checking Connection Status (IPSTAT)

Note: This function no longer requires AON. The information about this function is included in both *IBM Z NetView IP Management* and *IBM Z NetView User's Guide: Automated Operations Network*.

Users who connect to your z/OS environment can encounter various problems. You can diagnose these problems by using the capabilities of the connection management function:

- Determine connection endpoints
- Identify the type of connection (such as TN3270, FTP, or SMTP)
- Determine if a connection is stopped
- Drop a connection
- Run TCP/IP problem determination commands, such as PING and TRACERTE, to determine the cause of session problems
- Query sessions on multiple z/OS hosts and multiple TCP/IP stacks
- Access the SNMP command menu
- Display detailed information about specified Telnet connections
- Cause specified ports not to accept any new Telnet connections
- Cause previously quiesced ports to begin accepting Telnet connections

To access the connection management function, issue the IPSTAT command. A panel similar to the one in [Figure 73 on page 88](#) is displayed.

FKXK2200 TCP/IP for z/OS Connection Status NTV70

Enter TCP/IP address or HOSTNAME:

Service Point	System	IP Address	Host Name
- NMPIPL10	TCPIP	9.42.45.10	nmpipl10.tivlab.raleigh.ibm.com
- NMPIPL10_B	TCPIP		nmpipl10-b.tivlab.raleigh.ibm.co
- NMP101	TCPIP	9.42.45.101	nmp101.tivlab.raleigh.ibm.com
- NMP190	TCPIP	9.42.45.190	NMP190.TIVLAB.RALEIGH.IBM.COM

Command ==>
 F1=Help F2=Main Menu F3=Return F6=Roll
 F7=Backward F8=Forward F9=Filters F10=Details F12=Cancel

Figure 73. Connection Status: Main Panel

You can set filters for viewing connections by pressing F9 (Filters). For information about setting filters, see [“Setting Connection Management Filters” on page 91](#).

To see the details about a stack, select a stack by placing the cursor next to the stack and press F10 (Details). After you view the details, return to the main connection status panel by pressing F3 (Return).

Displaying the Connections for a Stack

From the main connection status panel (FKXK2200), display one or more stacks and the number of connections for each by following these steps:

1. Type a TCP/IP address or host name (or a wildcard, for example, an asterisk (*)). Use of wild cards is the same as for the TCPCONN command; for more information, see the online help for the TCPCONN command.
2. Select one or more stacks by placing any non-blank character next to each stack.
3. Press Enter.

If, for example, you type an asterisk (*) for the TCP/IP address and select the first stack (NMPIPL10), a panel similar to the one in [Figure 74 on page 89](#) is displayed.

Notes:

1. Depending on how you have customized your IPPORT policy definitions, this panel might not show all active sessions. For additional information, see IPPORT in the *IBM Z NetView Administration Reference*.
2. The IBM and Cisco TN3270 servers do not support the use of a wildcard (*) in any part of the **IP Address** field. If you try to use a wildcard with a TN3270 server, the FKX962I message is displayed.

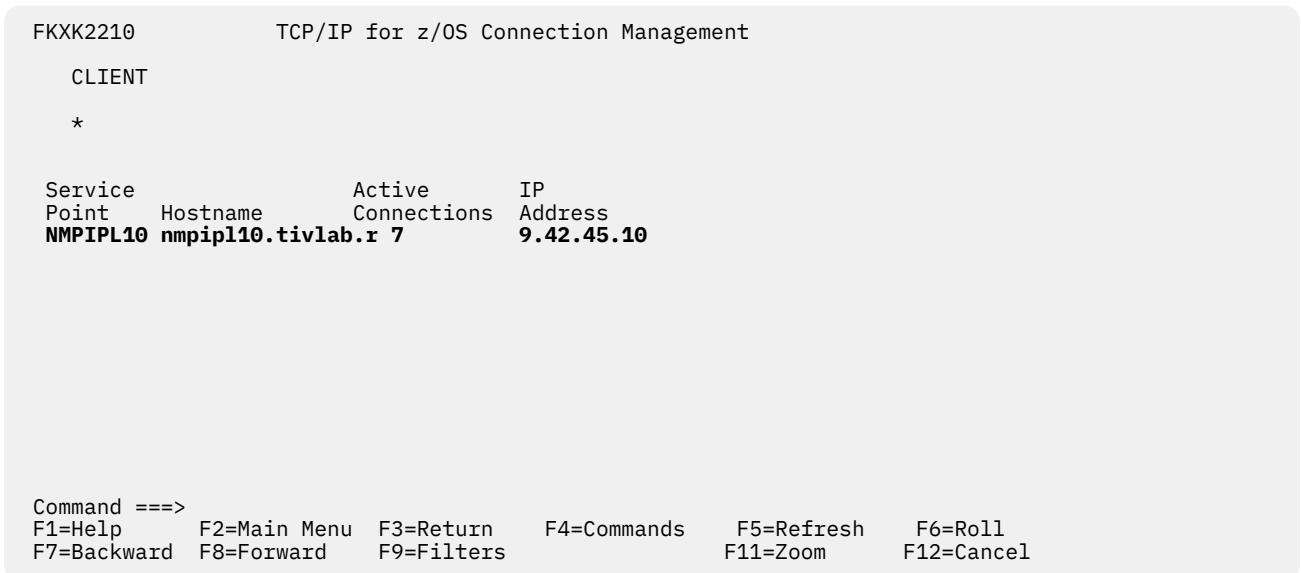


Figure 74. Connection Management: Selected Stacks

To display a list of connections for a specific stack, select the stack by placing the cursor next to it and press F11 (Zoom). For example, if you select NMPIPL10 on the panel shown in [Figure 74 on page 89](#), a panel similar to the one in [Figure 75 on page 90](#) is displayed. If you press F4 (Commands) next to a local port (in this example, Local Port 1030), you can display connection information, which is described in [“Displaying Connection Information” on page 90](#).

Figure 75. Connection Management: Connections for Selected Stack

To see the information for a connection, including the LU and application, and a list of commands that you can issue for the connection, select a connection by placing the cursor next to it and press F4 (Commands). If you select local port 1030 on the TCP/IP for z/OS Connection Management panel (FKXK2220), which is shown in [Figure 75 on page 90](#), and press F4, a panel that is similar to the one in [Figure 76 on page 90](#) is displayed.

Figure 76. Connection Management: Connection Information and Commands

Note: For options 5, 6, 9, and 10, the resources must support SNMP requests.

```

CNMKWIND OUTPUT FROM  TCPIP CONNECTION DETAILS                                LINE 0 OF 22
*----- Top of Data -----*
ibmMvsTcpConnectionInSegs.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 14
ibmMvsTcpConnectionOutSegs.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 13
ibmMvsTcpConnectionInOctets.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 492
ibmMvsTcpConnectionOutOctets.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 492
ibmMvsTcpConnectionAge.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = Wrong Type (sh
ibmMvsTcpConnectionLastActivity.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 67150
ibmMvsTcpConnectionResourceName.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = TCPIP
ibmMvsTcpConnectionResourceId.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 39807
ibmMvsTcpConnectionSockOpt.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = "C0 00 "
ibmMvsTcpConnectionPolicyAction.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 =
ibmMvsTcpConnectionPolicyRule.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 =
ibmMvsTcpConnectionServerResrcId.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 26
ibmMvsTcpConnectionApplName.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = NT70TS02
ibmMvsTcpConnectionLuName.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = NTC7001
ibmMvsTcpConnectionLogMode.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = NSX32702
ibmMvsTcpConnectionProto.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = "02 "
ibmMvsTcpConnectionTtlsPolStat.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 1
ibmMvsTcpConnectionTtlsConnStat.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 1
ibmMvsTcpConnectionTtlsSslProt.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 = 4
ibmMvsTcpConnectionTtlsNegCipher.1.4.201.2.10.11.23.1.4.201.2.10.11.1030 =
TO SEE YOUR KEY SETTINGS, ENTER 'DISPFK'
CMD==>

```

Figure 77. Connection Details Output

Press F3 to return to the Commands panel. To see the commands that can be issued for the LU or the application, press F4 (LU Cmds) or F5 (APPL Cmds), respectively. To close any of the command panels, press F12.

Determining If a Connection Is Stopped

To determine if a connection has stopped, press F5 to refresh the connection information in the TCP/IP for z/OS Connection Management panel (FKXK2220), shown in [Figure 75 on page 90](#). Check the send and receive columns. If, after you refresh the panel, the send and receive columns for the connection have not changed, the connection might be stopped.

To drop the connection, press F4 to display the command panel ([Figure 76 on page 90](#)) and issue the command to drop the connection. The following actions are taken:

1. A VARY TCPIP DROP command is issued for the connection.

Note: For the DROP option to work properly, the operator issuing the DROP command must be authorized to issue the VARY TCPIP DROP command.

2. The connection list is refreshed.
3. A message similar to the following message is displayed:

```
FKX611I SESSION 1006 WAS SUCCESSFULLY DROPPED
```

Setting Connection Management Filters

Displaying the connection management information can result in a large volume of information for you to review to find a particular resource or set of resources. With the connection management filters, you can limit the display to only the resources of particular interest. You can set connection management filters from a panel; see [“Using the Connection Management Filters Panel” on page 91](#). You can also set connection management filters from the command line; see [“Using a Command” on page 93](#).

Using the Connection Management Filters Panel

To display the TCP/IP for z/OS Connection Management Filters panel (FKXKCSF0), which is shown in [Figure 78 on page 92](#), press PF9 (Filters). The current filter settings are displayed on the panel.

Using a Command

You can use the FKXECSF command to set the connection management filters. If you enter the command with no parameters, the filters panel is displayed; see [“Using the Connection Management Filters Panel”](#) on page 91.

An example of the FKXECSF command follows. For more information about the command, see the online help.

```
FKXECSF DISIP=201.*
```

Launching IP Packet Trace

From the TCP/IP for z/OS Connection Management panel (FKXK2221), which is described in [“Displaying Connection Information”](#) on page 90, you can launch packet trace by entering 11 (Packet Trace). This runs the PKTTRACE command, which starts a packet trace filter to trace the IP addresses and ports associated with the connection that you selected.

```
FKXK2221          TCP/IP for z/OS Connection Management
CLIENT - - - - - > Service Point - - - - - > CONNECTION
              NMP101                      23
          9.42.45.10          9.42.45.101          1043
.....
Client : IP Address          9.42.45.10          :
Port   : Port                23                  :
23     : Connection ID       0000253A             :
       : LU                  :                   :
       : APPL                :                   :
       : Send                 63                   :
       : Receive              2250                  :
       : Send Window          32754                 :
       : Conn ResourceName    PHK                   :
       :                      :                   :
       :                      :                   :
       :                      :                   :
       :                      :                   :
       : F1=Help              :                   :
       :                      :                   :
.....
Command ==>
F1=Help      F2=Main Menu    F3=Return          F6=Roll
F7=Backward  F8=Forward     F9=Filters          F12=Cancel
```

Figure 80. TCP/IP for z/OS Connection Management Panel (FKXK2221)

In the Display Packet Control panel (FKXK2A24) that is displayed here, as called from the IPSTAT FKXK2220 or FKXK221C panel (see [Figure 79](#) on page 92), the IP addresses and ports are carried over from the selected connection. In this example, the **Infrc Name** field shows ALL, which indicates that all packets from all interfaces for this call are to be collected. From the Display Packet Control panel (FKXK2A24), you can use any of the trace display and formatting functions that are provided by the IPTRACE dialog.

```

FKXK2A24          Display Packet Control          LOCAL
                                                         NMPIPL

Stack: TCPIP                      Infc Name: ALL
LAddr *
RAddr *
Portnum: *          LPort: *          RPort: *          Protocol: - ALL (default)
                                                         - TCP
                                                         - UDP
Start Time: *          - ICMP
End Time:  *          - OSPF
                                                         - (Number)

MaxRecs: 1  1-Last  100          Truncate: 65535
              2-First

Command ==>
F1=Help  F2=Save Packets  F3=Return  F4=View Packets          F6=Roll
              F8=Extended Opts          F10=Analyze          F12=Cancel

```

Figure 81. Display Packet Control Panel (FKXK2A24)

When you exit the IPTRACE panels, you are asked whether you want to stop the trace. If you specify **Y** (Yes), the packet trace filter is stopped. If you specify **N** (No), the packet trace filters are left active, and you must either stop them manually or stop them by using the IPTRACE panels.

Chapter 9. Discovering IP Network Topology with the MultiSystem Manager IBM Tivoli Network Manager Agent

The MultiSystem Manager topology manager transfers the topology and status information that is discovered by IBM Tivoli OMNIBus and Network Manager about the IP resources in your network, including resources running in a z/OS environment, and stores the information in Resource Object Data Manager (RODM). After the information is in RODM, you can view your network resources from the NetView management console. Topology correlation automatically ties together resources that are managed by different types of topology functions such as IBM Tivoli Network Manager and open topology functions. Topology correlation is provided for MultiSystem Manager topology functions, for the NetView SNA Topology Manager, and for customer or vendor applications that use the Graphic Monitor Facility host subsystem data model. This chapter describes how to manage your IP networks using the views and command support provided by the MultiSystem Manager IBM Tivoli Network Manager agent.

The MultiSystem Manager IBM Tivoli Network Manager agent extracts IP topology information from the IBM Tivoli Network Manager topology database about the network resources and relationships that are discovered by Tivoli Network Manager and loads the information into Resource Object Data Manager (RODM). The resources that are discovered include subnetwork resources (such as hosts, routers, and subnetwork connections) and resources that are either z/OS resources or directly connected to a z/OS resource.

To use the MultiSystem Manager IBM Tivoli Network Manager agent to discover IP resources, follow these steps. For more information about the MultiSystem Manager IBM Tivoli Network Manager agent, see the `msm_nm_ip_readme_en.html` file. This file can be viewed from the Z NetView installation DVD or from a workstation where the agent is installed.

- Install and configure IBM Tivoli OMNIBus and Network Manager in your distributed environment.
- Use the Tivoli Integrated Portal to create network views that are to be used to populate RODM.
- Install and configure the MultiSystem Manager IBM Tivoli Network Manager agent on the Tivoli Network Manager workstation. The latest MultiSystem Manager IBM Tivoli Network Manager agent is available from the Z NetView support Web site.
- Install and configure an IBM Tivoli Network Manager SNMP gateway for each Z NetView program that is to receive trap information.
- Ensure that Z NetView SNMP trap automation task is active.
- Ensure that the MSM.ITNM tower is enabled and that the AUTOMSM autotask, which is used for alert automation, is active.
- Ensure that the host and agent can communicate using IP.
- If appropriate, set up security for the MultiSystem Manager IBM Tivoli Network Manager agent; for more information, see the *IBM Z NetView Security Reference*.

After you initialize network topology and status in RODM, the NetView management console provides views of IP networks. Using the NetView management console pull-down menus, you can navigate among the views to isolate failures and to send commands to resolve problems. This section explains how to use the views and command support that are provided by the MultiSystem Manager IBM Tivoli Network Manager agent to manage your IP networks. For information about using the NetView management console, see *IBM Z NetView User's Guide: NetView Management Console*.

IP View Objects

The following IP objects are displayed in IP views:

- Aggregate objects that represent the following resources:
 - A group of IP networks
 - A single IP network
 - A user-defined logical group of IP resources
 - Subnetworks
 - Routers
 - Bridges
 - Hubs
 - Hosts (distributed and mainframe)
 - Switches
- Real objects that represent the following resources:
 - Topology agents
 - Hosts
 - Interfaces
 - Endpoint (IP address)

Note: Mainframe TCP/IP hosts are highlighted in the NetView management console with an icon that is distinct from distributed TCP/IP hosts. The Other Data field in the Resource Properties window contains additional z/OS information about mainframe TCP/IP hosts, such as the sysplex name.

[Figure 82 on page 97](#) shows the geometric shapes and icons for the various types of objects that are displayed in IP views.

Note: The NetView management console, by default, displays icons in views. To display geometric shapes instead of icons, click **View > Show Icons**.

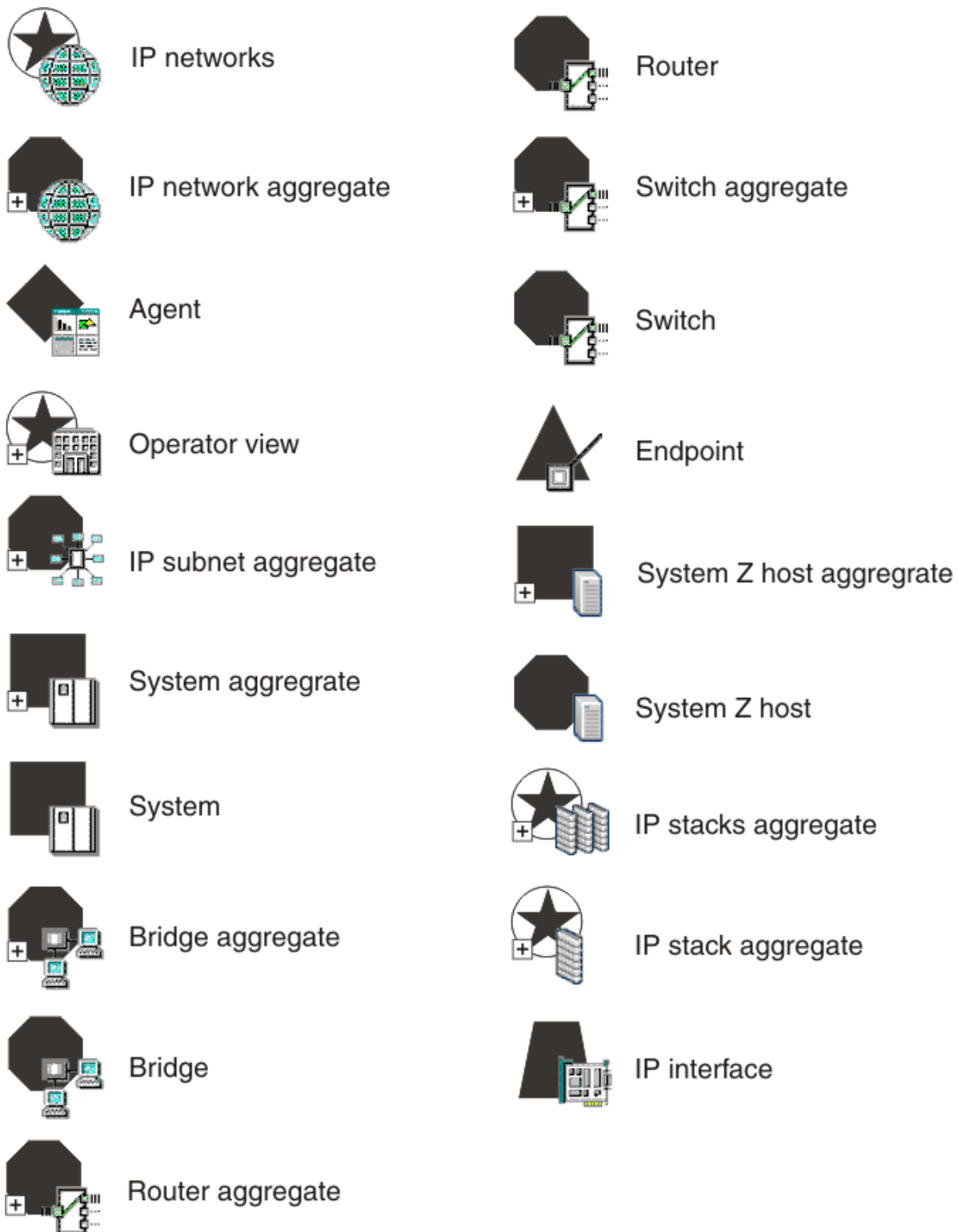


Figure 82. Types of IP View Objects for the IBM Tivoli Network Manager Agent

Finding Resources

To display resources, access the Locate Resource window. From this window, you can locate an adapter by its MAC address and other IP resources by their names. You can also locate a resource by its display name.

Navigating Network Views

To navigate the NetView management console views, begin by selecting the default MultiSystem Manager network view, **MultiSysView**, which is shown in [Figure 83](#) on [page 98](#).

Notes:

1. The following GETTOPO command, sent to a topology agent with an IP host name of Server1, was used to build the NetView management console views:

```
GETTOPO ITNMRES, SP=Server1
```

2. The information in this section is based on the default views that are created by MultiSystem Manager and user-defined views that are created in IBM Tivoli Network Manager. For more information about creating NetView management console views, see *IBM Z NetView Installation: Configuring Graphical Components*.

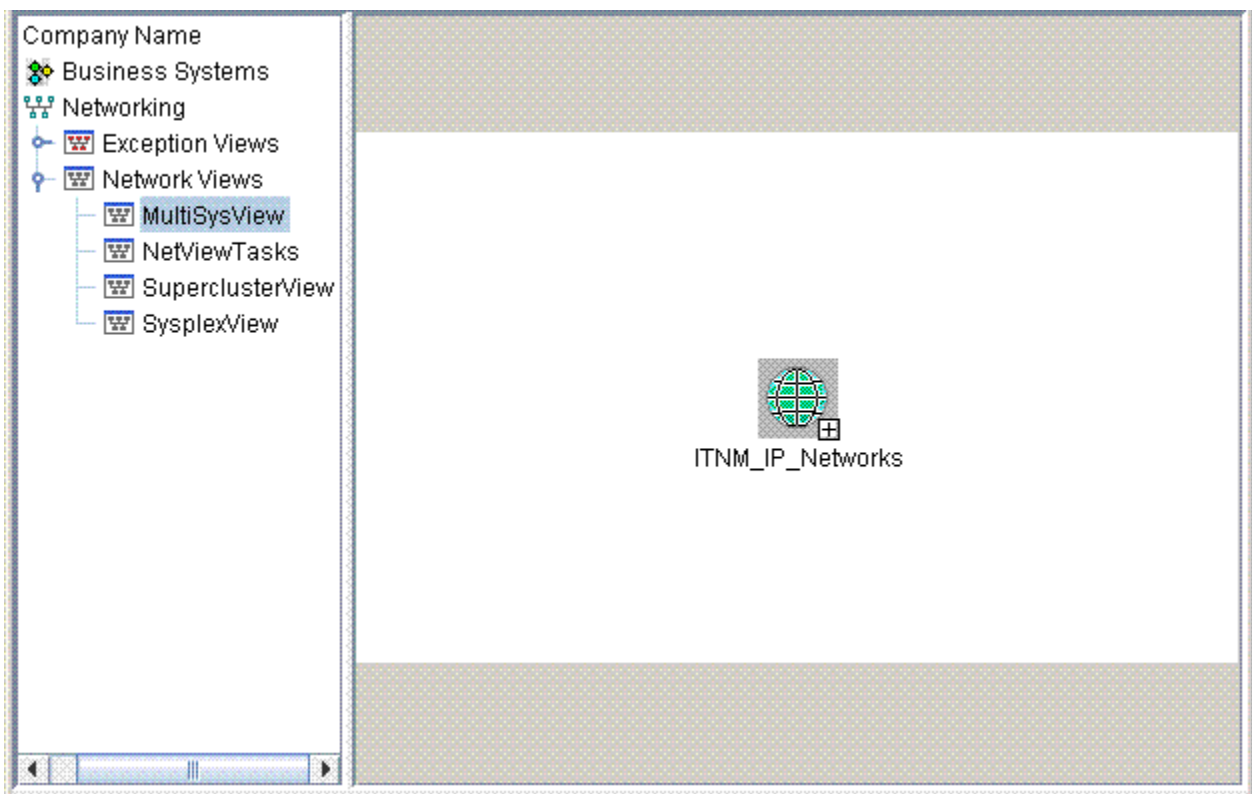


Figure 83. NetView Management Console Default Network View (MultiSysView): ITNM_IP_Networks

MultiSysView consists of an object (a NetView management console cluster aggregate object) that represents all the IP networks that are defined in the Network Manager topology database, discovered by the IBM Tivoli Network Manager agent, and stored in RODM. This aggregate object is named ITNM_IP_Networks and the resource type is *IP networks*. From MultiSysView, you can get more detailed views that display your IP networks.

Viewing IP Networks

IP views consist of the following levels:

IP Networks View

A single view that shows all your IP networks

User-defined Views

Views showing collections of user-defined IP managed resources

IP Subnetwork Views

Views showing the resources within subnetworks

Router Views

Views showing the interfaces and endpoints within a router

Bridge Views

Views showing the interfaces and endpoints within a bridge

Hub Views

Views showing the interfaces and endpoints within a hub

Host Views

Views showing the interfaces and endpoints within a host

Switch Views

Views showing the interfaces and endpoints within a switch

IP Networks View

To display the IP networks view with the NetView management console, double-click the ITNM_IP_Networks object that is shown in [Figure 83 on page 98](#). The resulting view, which is similar to the view in [Figure 84 on page 99](#), shows the IP network that is included in the aggregate object ITNM_IP_Networks.

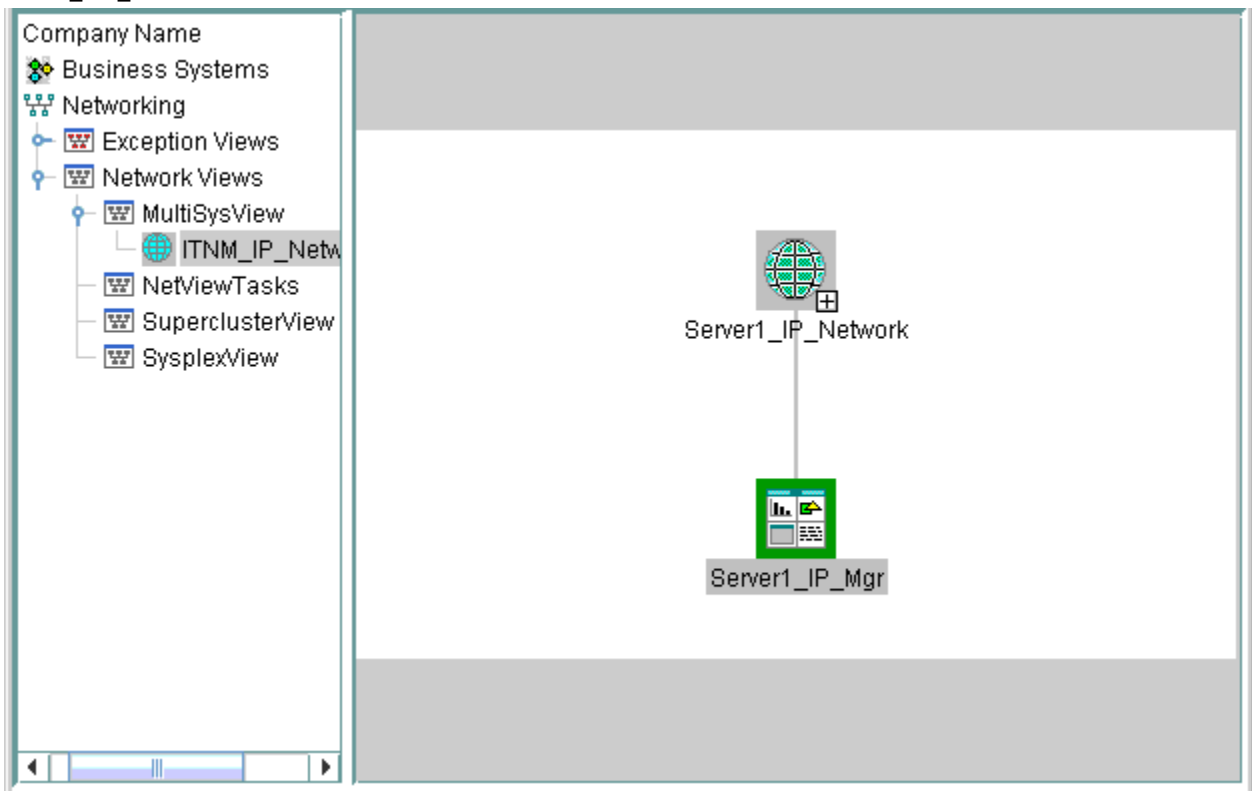


Figure 84. ITNM_IP_Networks IP Networks View

This view contains a single IP network named Server1_IP_Network. MultiSystem Manager displays a separate IP network for each agent from which topology is gathered.

Each IP network is represented by two connected icons:

- A node aggregate object, an *IP network aggregate*
- A real object, an *agent*

The Server1_IP_Network aggregate object that is shown in [Figure 84 on page 99](#) represents the managed IP resources that IBM Tivoli Network Manager on the Server1 system discovered. The real object Server1_IP_Mgr represents the MultiSystem Manager IBM Tivoli Network Manager agent. The

name of the topology agent is composed of the host name (Server1) where the topology agent is installed and the type of network the agent is managing (IP). To display additional information about the topology agent, select **Resource Properties**.

User-defined Views

To view the managed IP resources in the Server1 network, double-click the `Server1_IP_Network` object that is shown in Figure 84 on page 99. The resulting view, which is shown in Figure 85 on page 100, contains the NCOMS operator view object, which is a user-defined view.

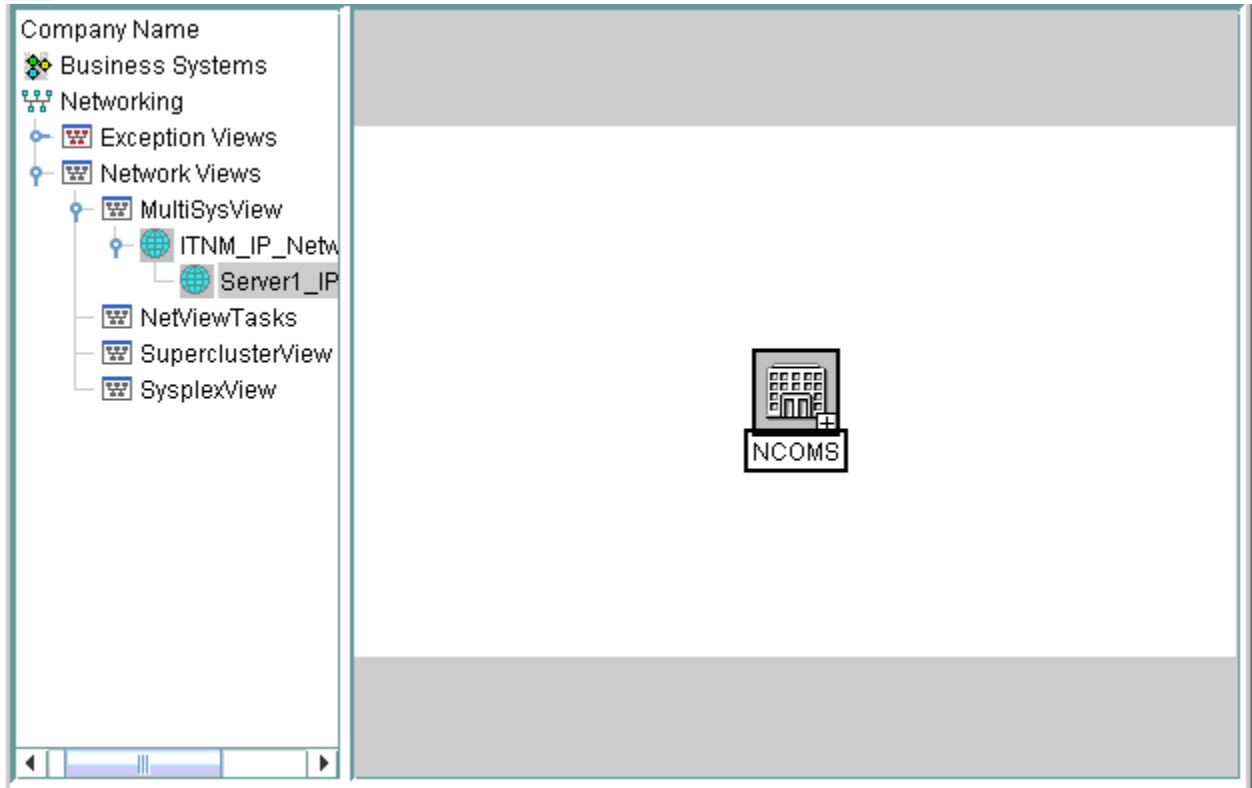


Figure 85. ITNM_IP_Networks User-Defined View: NCOMS Object

Double-click the NCOMS object that is shown in Figure 85 on page 100. The resulting view, which is shown in Figure 86 on page 101, shows the Subnets and Device_Classes operator view objects.

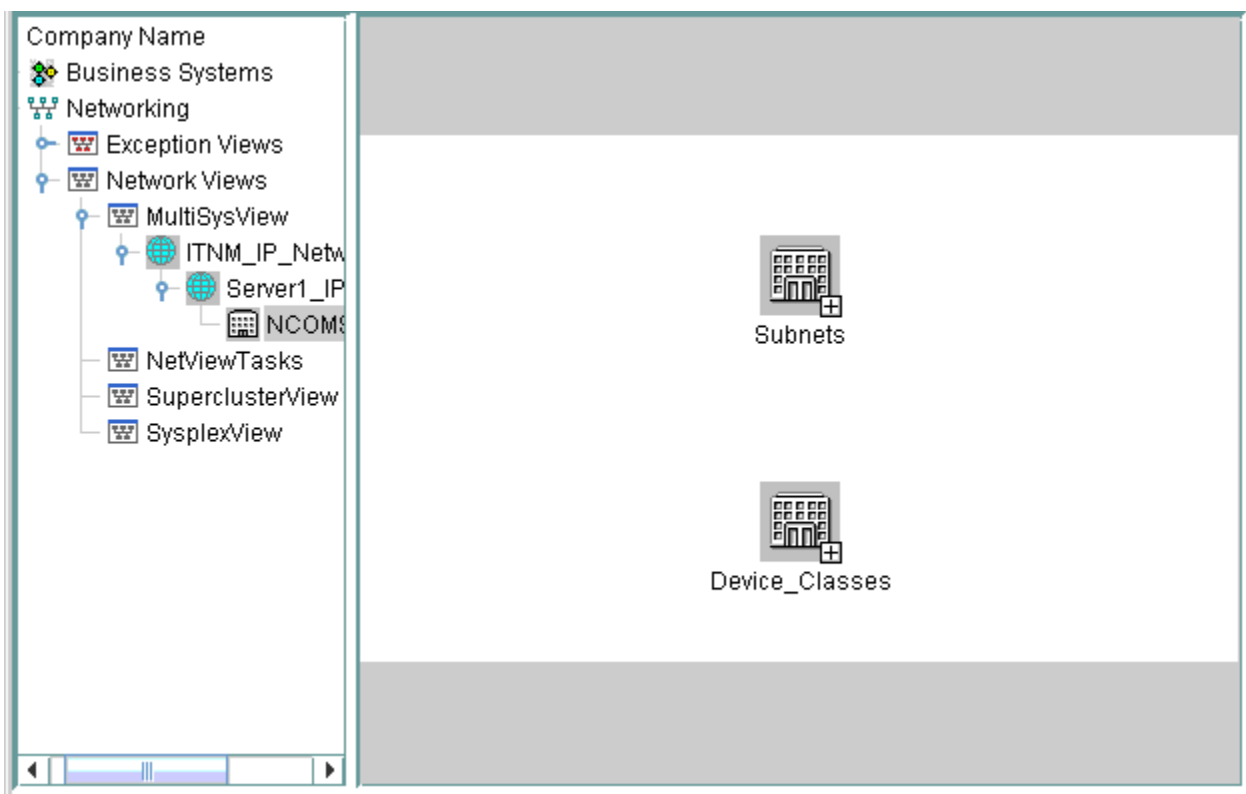


Figure 86. ITNM_IP_Networks User-Defined View: Subnets and Device_Classes Objects

Double-click the Subnets object that is shown in Figure 86 on page 101. The resulting view, which is similar to the view in Figure 87 on page 101, shows the fe80___/10 and 9.0.0.0_/8 operator view objects.

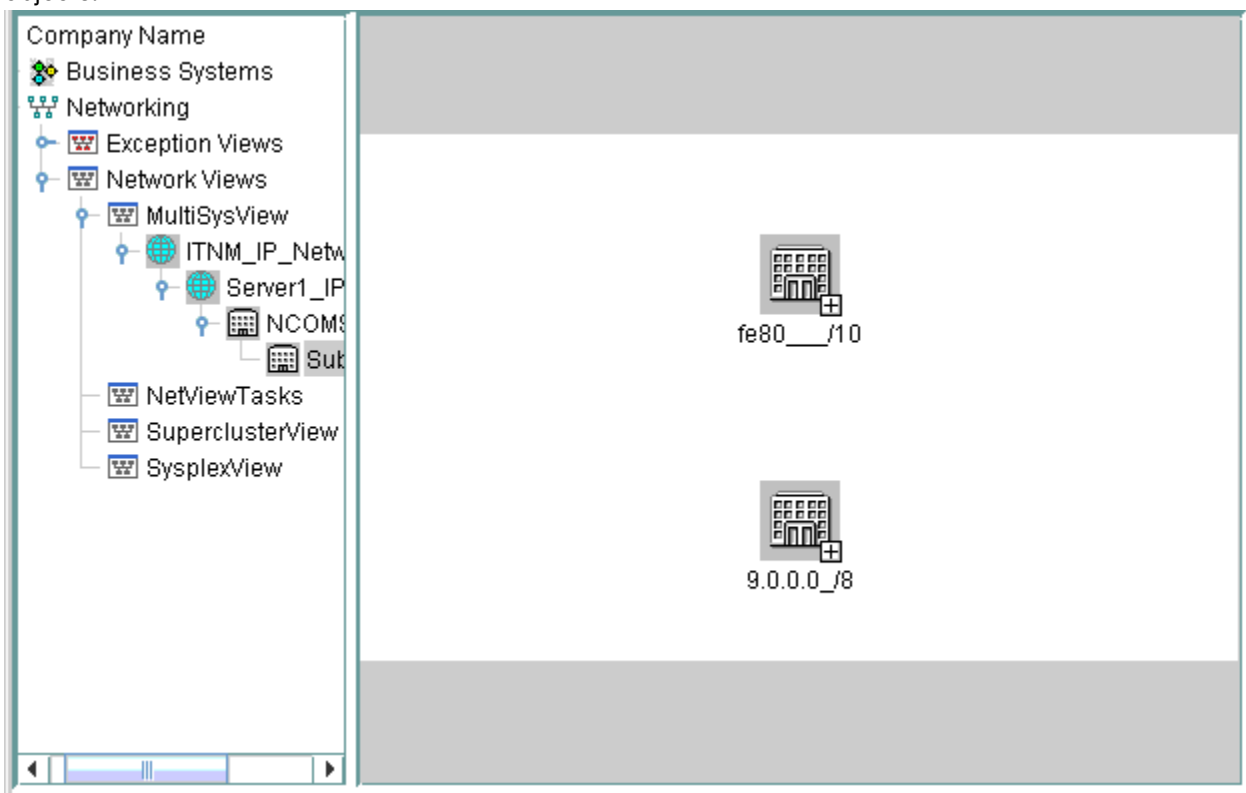


Figure 87. ITNM_IP_Networks User-Defined View: fe80___/10 and 9.0.0.0_/8 Objects

Double-click the 9.0.0.0_/8 object that is shown in [Figure 87](#) on page 101. The resulting view, which is similar to the view in [Figure 88](#) on page 102, shows the 9.0.0.0/255.0.0.0 IP networks object.

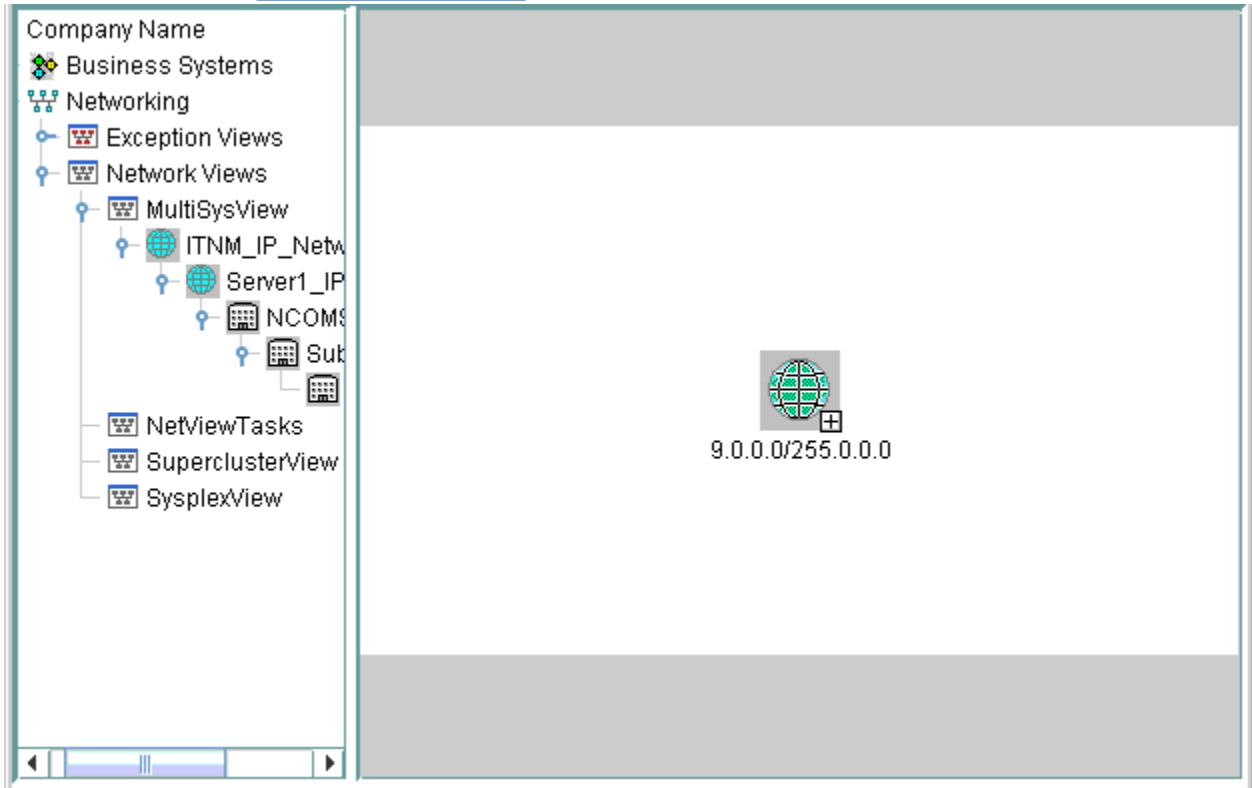


Figure 88. ITNM_IP_Networks User-Defined View: 9.0.0.0/255.0.0.0 Object

IP Subnetwork Views

To see the IP subnetwork view, double-click the 9.0.0.0/255.0.0.0 object that is shown in [Figure 88](#) on page 102. The resulting view, which is similar to the view that is shown in [Figure 89](#) on page 103, displays the `nmp196.tivlab.raleigh.ibm.com` object.

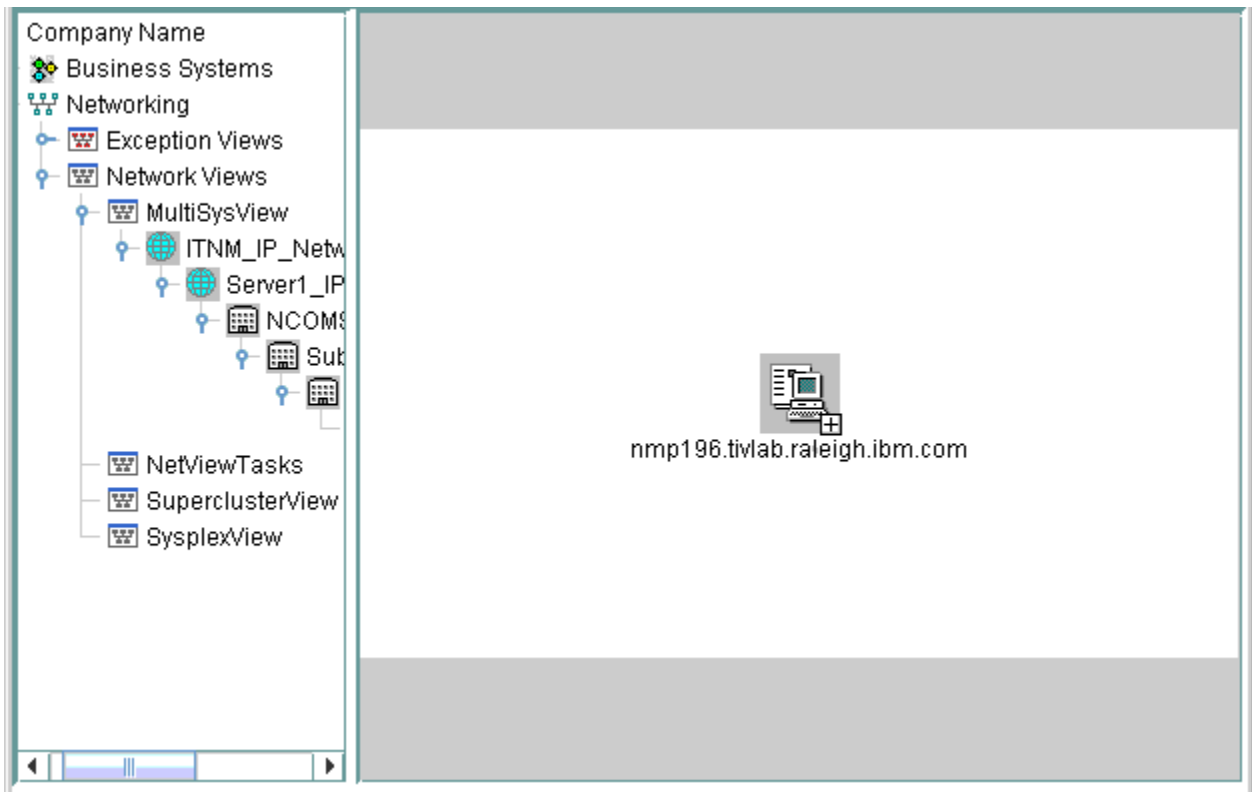


Figure 89. ITNM_IP_Networks IP Subnetwork View

Router, Bridge, Hub, Host, and Switch Views

To see the host view, double-click the `nmp196.tivlab.raleigh.ibm.com` object that is shown in Figure 89 on page 103. The resulting view, which is similar to the view in Figure 90 on page 104, shows the `nmp196.tivlab.raleigh.ibm.com` host system and the IP interfaces and endpoints that are associated with it.

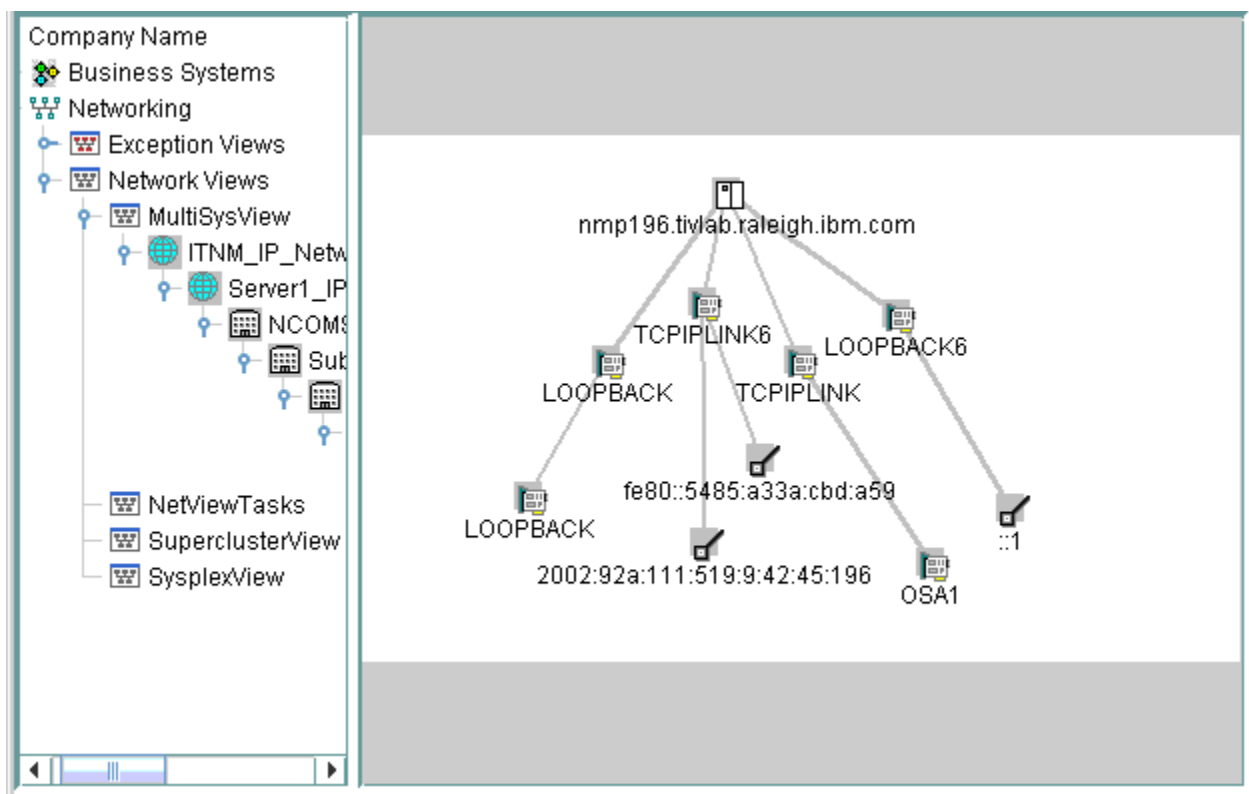


Figure 90. ITNM_IP_Networks Host View

Chapter 10. Monitoring and Managing IP Resources

You can monitor and manage IP resources with the following functions:

- PING command
- TRACERTE command
- SNMP management
- IP resource management
- Critical port monitoring

You can also manage SNA over IP.

You can access these functions from the NetView IP Management Functions menu panel, which is shown in [Figure 91 on page 105](#). To access the menu panel, use the NETVIP command.

```
CNM4NVIP          NetView IP Management Functions Menu

Type the number or move the cursor to a function and press Enter

  1. Ping a device (PING)
  2. Trace the route to a device (TRACERTE)
  3. Check TCP connection status (IPSTAT)
  4. Work with IP traces (IPTRACE)
     for SP: -----
  5. Manage IP Active Monitoring (IPMAN)
  6. Issue SNMP commands (NVSNNMP)
  7. Manage Sysplex
  8. Manage DVIPA
  9. Check the status of an IP port (TESTPORT)
 10. Show EE information for a VTAM resource (DIS PATH)

Command ===>
F1=Help          F3=Return          F6=Roll
                  F12=Cancel
```

Figure 91. NetView IP Management Functions Menu Panel

Using PING

Note: This function no longer requires AON. The information about this function is included in both *IBM Z NetView IP Management* and the *IBM Z NetView User's Guide: Automated Operations Network*.

Pinging is used to test connectivity to an IP host and can often be useful in determining if a resource can be reached. To ping a resource, issue the PING or MVSPING command. These commands provide the same function.

Note: PING is also available from the menus on the NetView management console.

When you issue a PING or MVSPING command without any parameters, the panel that is shown in [Figure 92 on page 106](#) is displayed.

```

FKXK2100      MVS TCP/IP Automation: Ping from a Service Point  CNM01

Host Name or IP Address
-----

Service Point Name _____ (? for Selection list)

Ping Count      4__
Ping Timeout    3__
Ping Length     16__

Command ==>
F1=Help      F2=Main Menu  F3=Return

F6=Roll
F12=Cancel

```

Figure 92. TCP/IP Automation: Ping Panel

The resource name can be an IP host name or an IP address. If no service point name is specified, the TCP/IP policy definitions for the service point associated with the resource are searched. Optionally, you can change the ping count, the ping timeout, and the ping length.

If you issue a PING or MVSPING command for a particular resource from an NCCF command line, the results are correlated and displayed from the command line rather than in panels.

For the syntax of and detailed information about the PING and MVSPING commands, see the online help.

Using TRACERTE

Note: This function no longer requires AON. The information about this function is included in both *IBM Z NetView IP Management* and *IBM Z NetView User's Guide: Automated Operations Network*.

The TRACERTE command is used to trace the routes of data packets to a specified IP host from the IP stack on the host on which the NetView program is running. Use this command to determine connectivity with or routing to a particular endpoint, roundtrip times between the NetView and target hosts, and routers along the way. The TRACERTE command is useful in problem determination, for example, in troubleshooting lost packets.

Note: The TRACERTE command is also available from NetView management console menus.

To access TRACERTE functions, enter the TRACERTE command. The panel that is shown in [Figure 93 on page 107](#) is displayed. For information about the fields on this panel, press F1 to see the online help.

```
FKXK2300  MVS TCP/IP Automation: Trace Route from a service point  CNM01
```

```
Host Name or IP Address:
```

```
-----
```

```
Service Point Name  _____  (? for Selection list)
```

```
Max      30
```

```
Try      3
```

```
Port     33434
```

```
Wait     5
```

```
Debug    2          ( 1=Y , 2=N )
```

```
Skip DNS  1          ( 1=Y , 2=N )  
lookup
```

```
Command ==>
```

```
F3=Ret  F4=Fndprev  F5=Rptfnd  F6=Roll  F7=Back  F8=Forward  F12=Cancel
```

Figure 93. TCP/IP Automation: Trace Route Panel

The resource name can be an IP host name or an IP address. If a stack name is not specified, the stack definitions in the CNMPOLCY member are searched for the specified IP address or host name.

Figure 94 on page 107 shows output from a TRACERTE command for a workstation with an IP address of 1.23.45.678:

```
FKXKLWN2  OUTPUT FROM TCP/IP 390 TRACE ROUTE  Line 1 of 6
```

```
Trace route to 1.23.45.678 (1.23.45.678)
```

```
 1 (1.23.45.6)      2 ms 0 ms 0 ms  
 2 (1.23.45.66)     3 ms 3 ms 3 ms  
 3 (1.23.45.67)     8 ms 9 ms 11 ms  
 4 (1.23.45.678)   13 ms 11 ms 10 ms
```

```
Command ==>
```

```
F3=Ret  F4=Fndprev  F5=Rptfnd  F6=Roll  F7=Back  F8=Forward  F12=Cancel
```

Figure 94. TCP/IP Automation: Trace Route Output

For the syntax of and detailed information about the TRACERTE commands, see the online help.

Using SNMP Management

Note: This function no longer requires AON. The information about this function is included in both *IBM Z NetView IP Management* and the *IBM Z NetView User's Guide: Automated Operations Network*.

To use SNMP management functions, issue the NVSNMP or SNMP command. The NVSNMP command provides a panel interface. The SNMP command, which provides the WALK, BULKWALK, GET, GETBULK, GETNEXT, INFORM, and TRAP operations, is intended more for use in application programs.

When you issue the NVSNMP command, the panel that is shown in Figure 95 on page 108 is displayed. For information about the fields on the panel, press F1 to see the online help.

Figure 95. TCP/IP SNMP Menu Panel

If you specify a community name, it is used for the resulting SNMP request. If you do not specify a community name, the stack definitions are searched for the community name. If no community name is found, the z/OS default name `public` is used. If a community name is not defined for the stack, then the default name defined to z/OS Communications Server IP is used. The community name can be defined in the TCP390 definition for the associated stack where the SNMP request is being issued. For more information, see the TCP390 definition in the *IBM Z NetView Administration Reference*.

Using the Get Command

Figure 96. TCP/IP SNMP Commands Panel: Get Command

108 IBM Z NetView: IP Management


```

FKXK2519          TCP/IP for 390 SNMP Commands          NTV70

Resource: LOCAL
Password(Community)
TCP/IP Stack: LOCAL      Comm

MIB Variables:
-----
: Options:
:
:   Debug
:   Retry 2_
:   Timeout 3_
:   Bulk
:       Max Repetitions 10
:       non-Repetitions 0_
:
: Additional options:
: -----
: -----
: -----
: -----
: .....
```

Command ==>
 F1=Help F2=Main Menu F3=Return
 F6=Roll
 F12=Cancel

Figure 97. TCP/IP SNMP Commands Panel: Options

The options panel shows the SNMP options and the system settings. These settings can be defined in the policy definitions. Type an X next to each option that you want to override. If the system definition is different from the SNMP default, the fields are automatically selected.

Note: If **Bulk** is selected using F9 (Options), the command is changed to GETBULK.

Using the Set Command

If you select the Set command for option **1** in the TCP/IP SNMP Menu Panel, the panel that is shown in Figure 98 on page 109 is displayed.

```
FKXK251S      TCP/IP for 390 SNMP Commands      CNM01
Resource: LOCAL
Password(Community)
TCP/IP Stack: LOCAL          Command: Set
MIB Variable                      Type
Value
sysName.0_-----
NMPIPL10 Test system -----
-----
-----
-----
-----
-----
-----
-----
-----
-----
Command ==>
F1=Help       F2=Main Menu   F3=Return     F6=Roll
               F9=Options    F12=Cancel
```

Figure 98. TCP/IP SNMP Commands Panel: Set Command

For the **Set** command, the **Type** and **Value** fields are added to the panel. **Type** is used to override the definition type of the MIB variable. **Value** is used to specify the new value of the MIB variable. To specify optional parameters for this command, press F9 (Options) to display the SNMP options panel.

Using the Walk Command

If you select the Walk command for option **1** in the TCP/IP SNMP Menu Panel, the panel that is shown in [Figure 99](#) on [page 110](#) is displayed. This command returns the values of all variables in the MIB tree that is identified by the specified MIB variable.

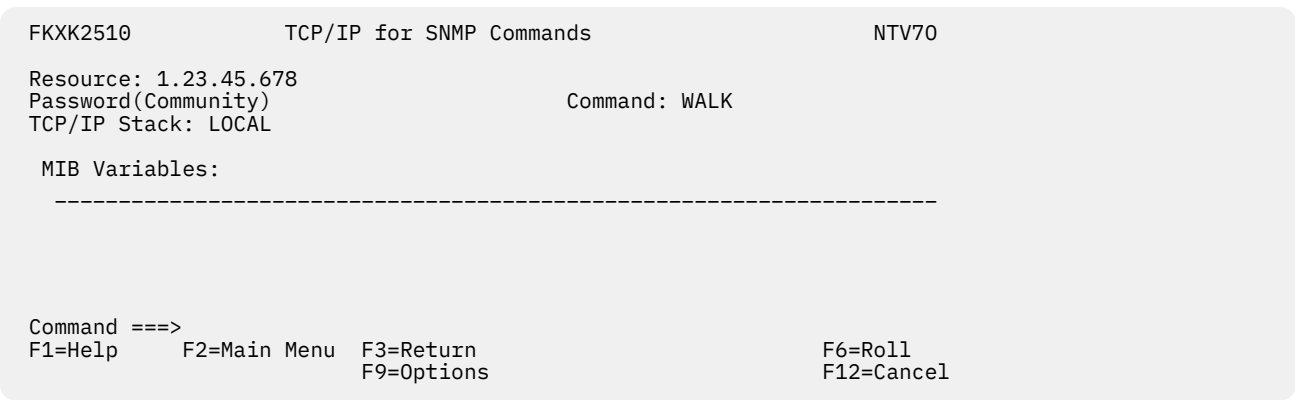


Figure 99. TCP/IP SNMP Commands Panel: Walk Command

For a **Walk** command, only one MIB variable can be specified.

Note: If **Bulk** is selected using F9 (Options), the command is changed to BULKWALK.

Using the Group Command

Figure 100 on page 110 is the resulting panel if you select **2** (the Group Menu option) in the TCP/IP SNMP Menu panel.

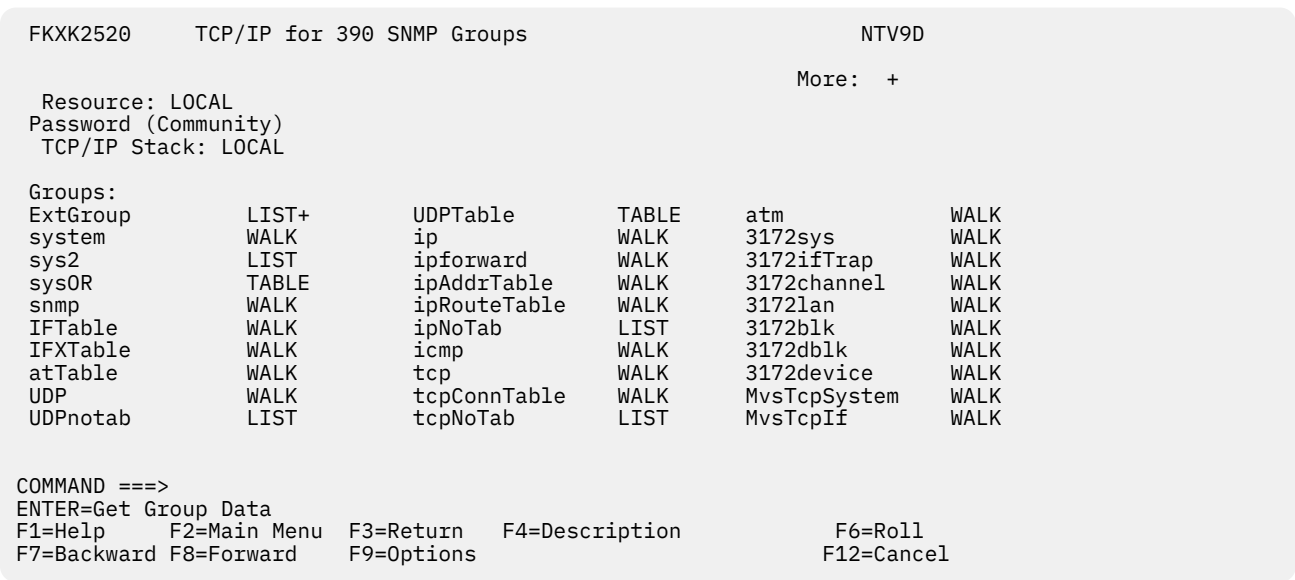


Figure 100. TCP/IP SNMP Groups Panel

The SNMP Group panel displays the groups that are defined in DSIPARM sample FKXSNMP. To display more information about the group, tab to the group and press F4. To display the SNMP options, press F9. For more information about creating MIB groups, see [Appendix B, “Customizing the SNMP Group Definition File \(FKXSNMP\),”](#) on page 159.

Figure 101 on page 111 displays the description of the group UDPnotab, and has a type of LIST. Note the four MIB variables that are displayed when you use the UDPnotab group.

```

FKXK2524      TCP/IP for 390 SNMP Groups Description      NTV70

Group: UDPhotab      Type: LIST

Abstract:
  UDP group MIBs without the UDP Table

MIB Variables:
  udplnDatagrams.0
  udpNoPorts.0
  udplnErrors.0
  udpOutDatagrams.0

Command ==>
F1=Help      F2=Main Menu  F3=Return      F6=Roll
F9=Options  ENTER=Get Group Data  F12=Cancel

```

Figure 101. TCP/IP SNMP Groups Description Panel: LIST

Figure 102 on page 111 displays the description of the group system and has a type of WALK.

```

FKXK2524      TCP/IP for 390 SNMP Groups Description      NTV70

Group: system      Type: WALK

Abstract:
  System group MIB variables for SNMP v1 or SNMP v2 including OR Table
  The definition of this group can be found in:
  RFC1907 for v2 or RFC1450 for v1.
MIB Variables:      FULL Name:SYSTEM

Command ==>
F1=Help      F2=Main Menu  F3=Return      F6=Roll
F9=Options  ENTER=Get Group Data  F12=Cancel

```

Figure 102. TCP/IP SNMP Groups Description Panel: WALK

Figure 103 on page 111 displays the description of the group UDPTable and has a type of TABLE.

```

FKXK2524      TCP/IP for 390 SNMP Groups Description      NTV70

Group: UDPTable      Type: TABLE

Abstract:
  UDP group MIBs with the UDP Table

MIB Variables:      INDEX MIB: 1.3.6.1.2.1.7.5
  udpLocalAddress
  udpLocalPorts

Command ==>
F1=Help      F2=Main Menu  F3=Return      F4=Description      F6=Roll
F9=Options  ENTER=Get Group Data  F12=Cancel

```

Figure 103. TCP/IP SNMP Groups Description Panel: TABLE

Using Extended SNMP Groups

If you need more flexibility than SNMP Groups provides, use extended SNMP Groups. These groups are different in that you can use them to code generic MIB objects (using the LIST+ statement), which are to be appended with group variables obtained from a pop-up panel when this group is selected.

To use extended SNMP groups, select option **2** on the SNMP Menu panel. In the TCP/IP SNMP Groups Panel that is displayed, move the cursor to LIST+ Group name (ExtGroup) and press Enter.

The panel in [Figure 104 on page 112](#) is displayed.

Note: To use the LIST+ function you must first customize the FKXSNMP member in DSIPARM offline. For information about customizing the FKXSNMP member, see [Appendix B, “Customizing the SNMP Group Definition File \(FKXSNMP\),” on page 159](#).

```
FKXK252V      TCP/IP for 390 SNMP Group Extensions      NTV9D

Resource: LOCAL
Password (Community)
TCP/IP Stack: LOCAL

Groups :
ExtGroup : PLEASE ENTER AN INTERFACE NUMBER
system :
sys2 : 2
sysOR :
snmp :
IFTable :
IFXTable :
atTable :
UDP :
UDPnotab :
WALK

COMMAND ==>
F1=Help      F2=Main Menu  F3=Return  F6=Roll      F12=Cancel
```

Figure 104. TCP/IP SNMP Group Extensions Panel

On the SNMP Group Extensions panel, type 2 to display details for interface 2 and press Enter. Listed MIB variable information for interface adapter number 2 is collected and displayed as [Figure 105 on page 112](#).

```
CNMKWIND Output from  SNMP GET to LOCAL                      LINE 0 OF 20
*-----TOP of DATA -----*
START OF DATA
ifName.2 = LOOPBACK
ifLinkUpDownTrapEnable.2 = 1
::::::::::::::::::
END OF DATA

TO SEE YOUR KEY STTINGS, ENTER 'DISPLAY'
CMD==>
```

Figure 105. SNMP GET to LOCAL Output

Using IP Resource Management

Note: This function no longer requires AON. The information about this function is included in both *IBM Z NetView IP Management* and the *IBM Z NetView User's Guide: Automated Operations Network*.

This section provides information about managing IP resources that are defined using the IP Resource Management panels.

Note: Some of the IP Resource Management panels use SNMP requests to collect data. The Community name used for those requests is defined on the TCP390 definition for the associated stack where the request is being issued. For more information, see the TCP390 definition statement in the *IBM Z NetView Administration Reference*.

IP Resource Management Main Panel

You can control monitoring of IP Resources from the IP Resource Management main panel, displayed in [Figure 106 on page 113](#). To display this panel, issue the IPMAN command.

You can select the following monitoring functions in the IP Resource Management main panel:

- ADD/START
- DISPLAY/CHANGE
- DELETE
- START
- STOP

FKXK2700		TCP/IP for 390 Resource Management			REFRESH: 0	
Select an IP Management Active Monitoring command and press ENTER						
1=ADD/START	2=DISPLAY/CHANGE	3=DELETE	4=START	5=STOP		
Resource	Resource Type	TCP/IP Stack	Actmon Definition	M	Status	
- **NEW**	IPHOST					
- **NEW**	IPPORT					
- LOOP1026	IPPORT	NMP101		R	DOWN	
- NMPIPL10	IPHOST	NMP101		A	NORMAL	
- LOOP1024	IPPORT	NMP101		A	NORMAL	
- TN3270	IPPORT	NMP101		A	NORMAL	
- NMPIPL10	TCP390	NMPIPL10		A	NORMAL	
- NMP101	TCP390	NMP101		A	NORMAL	
- NMP217	TCP390	NMP217		A	NORMAL	
- NMPIPL10V6	TCP390	NMPIPL10V6		R	UNKNOWN	
- TELNETAS	IPTELNET			N	NORMAL	
Command ==>						
F1=Help	F2=Main Menu	F3=Return	F4=Commands	F5=Refresh	F6=Roll	
F7=Backward	F8=Forward	F9=Display Opts	F10=Connections	F12=Cancel		

Figure 106. TCP/IP Resource Management Main Panel

Note the templates that are named ****NEW****, one for IPHOST and one for IPPORT. These are the most used resource types. Select either template to add a resource by using the required fields for the type in the FKXK2760 panel, described in [“IP Resource Management Add Panel” on page 114](#).

The following commands can be issued directly from the IP Resource Management main panel by typing the command number next to the resource name:

- 1 Display the Add/Change Panel in add mode. When the resource is added, monitoring is started. For additional information, see [“IP Resource Management Add Panel” on page 114](#).

2

Display the Add/Change panel in change mode. For additional information, see [“IP Resource Management Change Panel” on page 115](#).

3

Deletes the entry only from the in-store control file and ends all monitoring.

4

Start monitoring for the resource.

5

Stop monitoring for the resource.

Note: For IP Port Monitoring, all ports under a defined stack are always actively monitored at the start of the NetView program. They cannot be stopped or started individually.

The following fields are displayed for each IP resource defined in the control file:

Resource

The name of the resource you are monitoring.

Resource Type

The resource type can be TCP390, IPHOST, IPINFC, IPNAMESRV, IPPORT, IPTELNET, IPTN3270, or OSAPORT.

TCP/IP Stack

When the name in this field is the same as the resource name, it is a stack. This status indicates whether the TCP/IP stack specified on the TCPNAME parameter of the TCP390 statement is active. It does not indicate whether the IP address as specified on the IPADDR statement or the host name as specified on the HOSTNAME statement match the primary interface address or host name currently in use by the TCP/IP stack.

Actmon Definition

If a reference to an ACTMON definition exists, that name is displayed in this field.

M

The monitoring field contains the current active monitoring status. This list shows the monitoring status:

A

Active monitoring

R

Recovery monitoring

N

No monitoring.

Status

The status field contains the status. These are the possible status indicators:

- NORMAL (GREEN)
- DOWN (RED)
- DEGRADED (PINK)
- THRESH (YELLOW)
- UNKNOWN (BLUE)

REFRESH

The IP Resource Manager main panel can be automatically refreshed, by changing the value of this field, from 0 (no refresh) to 59 minutes.

IP Resource Management Add Panel

Select ADD (option 1) on the IP Resource Management main panel to display the following panel ([Figure 107 on page 115](#)):

FKXK2760		TCP/IP for 390 Resource Management	
Update the allowable fields, Press F4 to ADD resource and Start Monitoring		CMD Options X - Fixed Field R-Required Field 1 - Change Field 2 - Delete Field	
CMD	KEYWORD	VALUE	
R	RESNAME		
X	RESTYPE	IPPORT	
X	OPTION	IP390	
R	SP	NMP217	
-	PORT	8008	
-	PROTOCOL	TCP	
-	TCPNAME	T530EENV	
-	FORMAT	PORT	
-	ACTMON	IPPORT	
-	DESC	"NetView Web Browser Socket"	
-	STATUS	NORMAL	
-	INTVL	00:10	
Command ==>			
F1=Help	F2=Main Menu	F3=Return	F4=SUBMIT UPDATE
F7=Backward	F8=Forward	F9=Add a field	F6=Roll F12=Cancel

Figure 107. TCP/IP Resource Management Add Panel

With the Add panel, you can add resources dynamically into the in-store control file. When the resource is added, proactive monitoring is started for the resource. There is a delay before the monitoring field (M column) is updated on the main panel. Use F5 to refresh the panel until the change is displayed.

The IP Resource Management Add panel contains the following fields:

Command

A fixed field or a field that is updated using one of the following command options:

- 1**
Change Value for keyword.
- 2**
Delete keyword and its value.
- R**
Indicates field is required. This option is set by the program.
- X**
Indicates the field cannot be changed. This option is set by the program.

Keyword

Specifies the keyword name as set in the control file.

Value

Specifies the current value of the keyword under most circumstances. The value is cleared for ADD operations if a new value is required.

Notes:

- Keywords marked with an X cannot be updated. In cases where multiple field relationships exist, not all required keywords are marked with an R.
- Values for keywords are not syntax checked. Entering incorrect data can cause unpredictable results.

Changes are validated before a page forward or backward attempt. When R required fields are accepted their command is changed to X or fixed field, automatically.

To submit the changes, press F4. To add a new keyword-value pair press F9 to display the Add keyword window.

IP Resource Management Change Panel

Selecting Option 2 (change) on the IP Resource Management main panel displays the panel displayed in [Figure 108 on page 116](#).

```

FKXK2760          TCP/IP for 390 Resource Management

Update the allowable fields, Press F4      CMD Options
to Change the settings                    X - Fixed Field      R-Required Field
                                          1 - Change Field
                                          2 - Delete Field

CMD  KEYWORD      VALUE
-----
X    RESNAME      NMPIPL10
X    RESTYPE      IPHOST
X    OPTION        IP390
-    SP           NMPIPL10
-    ACTMON        ALLHOSTS
-    IPADDR        1.23.45.67
-    HOSTNAME      NMPIPL10
-    INTVL         00:15
-    STATUS        NORMAL
-    FORMAT        PING

Command ==>
F1=Help      F2=Main Menu  F3=Return      F4=SUBMIT UPDATE      F6=Roll
F7=Backward  F8=Forward   F9=Add a field  F12=Cancel

```

Figure 108. TCP/IP Resource Management Change Panel

IP Resource Management Filters Panel

The IP Resource Management Filters panel can be displayed by entering IPMANSSF with no parameters or by pressing F9 on the IP Resource Management main panel.

```

FKXK2710          IP Resource Management Filters          NTVE1

The current filter and sort settings are displayed. Change the settings
and press ENTER to activate.

Type the data to be displayed in one or more fields.
Resource      Resource      TCP/IP      Status
Name          Type          Stack Name
*             *             *             *
Type an action code to define the logical operator for the search:
1  1 - OR    2 - AND

Type numbers from 1 to 4 to define the column order for sorting.
Resource      Resource      TCP/IP      Status
Name          Type          Stack Name
1             2             3             4
Type an action code to define the sort order.
1  1 - Ascending order  2 - Descending order

Command ==>
F1=Help          F3=Return      F6=Roll
F9=Defaults      F12=Cancel

```

Figure 109. IP Resource Management Filters Panel

Use this panel to select the resources that you want to display. The settings selected are saved on a task basis and apply in future queries. You can specify filter criteria for exact names or names starting with specific characters, if the criteria is followed by an asterisk (*). The following fields can be filtered:

Resource Name

Specify criteria for the resource name.

Resource Type

Specify criteria for the resource type.

TCP/IP

Specify criteria for the TCP/IP stack name.

Status

Specify criteria based on resource status.

For filter options, specify the relationship between these options as 1 for OR or 2 for AND.

For sort options, specify 1–4 in any order. The four fields are then sorted based on the specified sort order. Indicate whether these sorts are to be done in ascending or descending order by entering a 1 or 2 in the last input field.

Pressing Enter causes the filter and sort entries to be validated and saved for use by the IP Resource Manager.

Pressing F9 restores the default filter and sort settings.

Press F3 to exit or F12 to cancel.

IP Resource Management Command Window

To display the COMMANDS window, press F4 from the IP Resource Management main panel or the SNMP Details panel. The information displayed depends on the resource type of the resource that has been selected.

```
FKXK2740          TCP/IP for 390 Resource Management

Select a command and press Enter

Resource          Resource Type : COMMANDS (based on Resource Type) :
-----          -
- **NEW**          IPHOST      : 1- Ping                      :
- **NEW**          IPPORT      : 2- Tracerte                  :
- LOOP1026         IPPORT      : 3- Connection Management    :
- NMPIPL10         IPHOST      : 4- SNMP                      :
- LOOP1024         IPPORT      : 5- Display Timers           :
- TN3270           IPPORT      : 6- SNMP Details Panel       :
- NMPIPL10         TCP390      :                               :
- / NMP101         TCP390      :                               :
- NMP217           TCP390      :                               :
- NMPIPL10V6       TCP390      :                               :
- TELNETAS         IPTELNET    :                               :

Command ==>
F1=Help      F2=Main Menu  F3=Return

F6=Roll
F12=Cancel
```

Figure 110. TCP/IP Resource Management Panel: Commands Window

Monitoring Critical Ports

The NETSTAT command displays all resources that are currently active. However, the NETSTAT command can sometimes indicate that a resource is active when the port for that resource refuses a connection. Run the TESTPORT command to check a port that looks active but might be refusing connections because it is inactive. Such ports are known as *hung listeners*.

The TESTPORT command provides additional function for monitoring critical ports. When a port is found to be hung, the BNH838W message is issued. This message can be used for automation, which enables recovery. If the port is not hung, the BNH839I message is issued.

If the TESTPORT command is issued without any parameters, it uses settings that are defined by COMMON.IPPORTMON statements in the CNMSTUSR or CxxSTGEN member. With these statements, you can specify the IP port to be monitored, the IP address associated with the IP port, and how frequently the IP port is to be monitored. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*. For information about the COMMON.IPPORTMON statements, see the *IBM Z NetView Administration Reference*.

You can also start the monitoring of a port that is not defined in the COMMON.IPPORTMON statements by using the TESTPORT command to specify the port to monitor and the interval at which to monitor it. For syntax of and detailed information about the TESTPORT command, see the online help.

Managing SNA over IP

With the Enterprise Extender technology, you can transport SNA traffic over an IP network. This technology routes SNA path information units (PIUs) over Advanced Peer-to-Peer Networking nodes using high-performance routing (HPR) and across IP using User Datagram Protocol (UDP). The routing that is provided by Enterprise Extender is more complex than the routing for SNA traffic only. Additional information about the routing to session partners might be needed for problem determination.

The NetView DIS command provides additional data for this routing. Although the information is primarily for local Enterprise Extender-connected resources, some information is also provided about other HPR-connected resources and about physical units (PUs) that are associated with an active, Enterprise Extender-connected logical unit (LU). TRACERTE information and EEDIAG analysis for these resources are displayed when appropriate. Operators can browse the data that is provided to discover congestion or broken links.

Using the NLDM SESS command, operators can select SNA sessions that traverse a given resource. Because a resource does not have to be a session endpoint, operators can see what sessions traverse an adjacent link station (ALS), which is shown as CNRnnnnn. In addition, the session monitor can display all SNA sessions across an Enterprise Extender connection.

For the syntax of and detailed information about the DIS and NLDM SESS commands, see the online help. For an example of how the Enterprise Extender technology can be used in a z/OS environment, see the IBM Redbooks® publication *Enterprise Extender Implementation Guide*.

Chapter 11. Basic IP Services

This chapter describes the basic IP services that are provided by the base NetView program. These commands provide control capabilities for managing IP resources and can be issued directly from the NetView command line or in REXX procedures and other automation routines. For the syntax of and detailed information about these commands, see the online help.

IPLOG

The IPLOG command sends a message to the syslog daemon on a remote host for processing. Standard UNIX protocol for the syslog daemon is used. The remote host must have a syslog server active for the command to work.

For information about the NetView syslog server, see [Chapter 15, “Using IP Servers,” on page 155](#). For the syntax of and detailed information about the IPLOG command, see the online help.

PING

The PING and MVSPING commands are used to test whether a host in an IP network can be reached by sending Internet Control Message Protocol (ICMP) messages (known as echo requests) to the host and reporting the responses (known as echo replies).

For information about using PING and MVSPING, see [“Using PING” on page 105](#). For the syntax of and detailed information about the PING and MVSPING commands, see the online help.

REXEC

The REXEC command is used to send a command over IP to a remote host for processing and display the resulting output. The standard UNIX RSH protocol is used. The remote host must have an REXEC server listening at the specified or default port for the command to work.

For information about the NetView REXEC server for use with UNIX commands, see [Chapter 15, “Using IP Servers,” on page 155](#). For the syntax of and detailed information about the REXEC command, see the online help.

RMTCMD

The RMTCMD command processor is used to send system, subsystem, and network commands to a remote NetView host for processing. The responses to these commands are returned to the RMTCMD issuer.

For the syntax of and detailed information about the RMTCMD command, see the online help.

RSH

The RSH command is used to send a command over IP to a remote host for processing. The output can be displayed as line-mode output or in a panel that is placed on the NetView roll stack. If the remote host supports it, additional commands can be issued from the panel where the output is displayed.

For information about the NetView RSH server, see [Chapter 15, “Using IP Servers,” on page 155](#). For the syntax of and detailed information about the RSH command, see the online help.

SNMP

The SNMP and NVSNMP commands are used to send requests to SNMP agents to retrieve or set information in the management information bases (MIBs) maintained by those agents.

For information about using SNMP and NVSNMP, see [“Using SNMP Management” on page 107](#). For the syntax of and detailed information about the SNMP and NVSNMP commands, see the online help.

SOCKET

The SOCKET command is used to request IP services, to retrieve information about TCP/IP stacks and to create TCP/IP applications within the NetView program.

For information about using the SOCKET command, see [Chapter 13, “Using Z NetView Socket,” on page 127](#). For the syntax of and detailed information about the SOCKET command, see the online help.

TN3270

The TN3270 command is used to log in to remote TCP/IP-connected systems, either from the NetView command line or from the NetView management console. The TN3270 command establishes a Telnet 3270 session with the specified host. Only Telnet 3270 protocol is supported. The resulting session is placed on the roll stack and can be rolled using the roll key specified on the command or using the default. The session can be ended with the specified (or defaulted) endkey.

For the syntax of and detailed information about the TN3270 command, see the online help.

TRACERTE

The TRACERTE command is used to trace the routes of IP data packets to a specified host from the IP stack on the NetView host. This command detects connectivity with the specified host, the actual routing to that host, the roundtrip times between the NetView program and that host, and the routers along the way.

For information about using TRACERTE, see [“Using TRACERTE” on page 106](#). For the syntax of and detailed information about the TRACERTE command, see the online help.

Part 4. Applications for IP Network Management

Chapter 12. Handling Events

The NetView program can handle the following types of events:

- SNMP traps
- Event Integration Facility (EIF) events
- Common Base Event Specification events
- SNA alerts

It can also respond to intrusions detected by Intrusion Detection Services (IDS) of z/OS Communications Server.

Handling SNMP Traps

The NetView program can send and receive SNMP traps, which are unsolicited SNMP messages from network resources that provide notification about significant events in the network.

Sending SNMP Traps

You can use the SNMP TRAP command to send SNMP traps to an SNMP manager; for detailed information about the SNMP TRAP command, see the online help.

The NetView Event/Automation Service (E/AS) can convert NetView alerts into SNMP traps and forward the trap data to an SNMP manager. The E/AS performs the function of an SNMP subagent and sends the converted alert data to an SNMP agent for eventual forwarding to an SNMP manager.

You can also use the CNMSMSGT sample to build and send an SNMPv1 or SNMPv2c trap. It uses information that is provided in global variables or defaults to build a trap. It sends the trap datagram, by using the SOCKET command to access the z/OS Communications Server API, to one or more trap destinations that are provided by common global variables or defaults. For more information, see the sample.

Receiving SNMP Traps

SNMP traps can be received by SNMP trap automation and by Event/Automation Service (E/AS). Because SNMP trap automation and E/AS use the same port, they cannot both run at the same time. If both are running, the one that tries to use the port second fails.

An SNMP trap automation task can receive and automate SNMPv1, SNMPv2c, and SNMPv3 traps in both IPv4 and IPv6 networks. Whenever a complete SNMP trap is received, whether over Transmission Control Protocol (TCP) or User Datagram Protocol (UDP), an SNMP trap automation task builds a CP-MSU (control point management services unit) and passes it to NetView automation. Vectors within the CP-MSU contain the data from the trap. You can use multiple SNMP trap automation tasks to distribute the SNMP trap automation workload.

An SNMP trap automation task is configured by setting common global variables that are unique to a specific task. These variables are read by an SNMP automation task. If an SNMP trap automation task receives encrypted SNMPv3 traps, the applicable pass phrase or key information must be provided in an SNMP trap automation task configuration file so that the SNMP trap automation task can decrypt and authenticate the trap data. For more information about configuring the SNMP trap automation task, see the *IBM Z NetView Automation Guide*.

E/AS, which runs under UNIX System Services (USS) or an MVS started task, can also receive traps. E/AS can convert SNMPv1 traps from SNMP agents into alerts and forward the alerts to the NetView program through the Alert Receiver PPI mailbox.

If the E/AS trap-to-alert conversion (TRAPALRT) function is using the default configuration, it creates an alert for each SNMP trap that it receives. The alerts are sent to the NetView alert receiver and processed by the hardware monitor, which can record the alerts in the hardware monitor database and can pass the alerts to automation. An SNMP trap automation task simply passes SNMP trap CP-MSUs to automation. If you want to record in the hardware monitor database the alerts that contain data from SNMP traps that are received by an SNMP trap automation task, you can use the CNMSALRT sample.

The CNMSALRT sample, when driven by the automation of an SNMP trap CP-MSU, creates a CP-MSU with an alert major vector that contains SNMP trap data that was copied from the SNMP trap CP-MSU and sends the new CP-MSU to the NetView alert receiver. The NetView alert receiver passes the CP-MSU with the alert major vector to the hardware monitor. Depending on the recording filter settings, the hardware monitor can record the alert in the hardware monitor database and can pass it to automation. For details about driving the sample with automation and about the information that the sample puts into the alert major vector of the CP-MSU that it builds, see the sample.

Handling Event Integration Facility Events

The NetView program can send and receive Event Integration Facility (EIF) events by using Event/Automation Service (E/AS). For information about configuring E/AS, see *IBM Z NetView Customization Guide*. For information about enabling E/AS, see *IBM Z NetView Installation: Configuring Additional Components*. For more information about EIF events, see the *IBM Tivoli Netcool/OMNIbus Event Integration Facility Reference*.

Sending EIF Events

E/AS can convert NetView alerts and messages into EIF events and forward the event data to an event manager such as the Tivoli Netcool/OMNIbus program or the Tivoli Enterprise Console®.

Receiving EIF Events

E/AS can convert EIF events from an event sender into alerts before forwarding the alerts to the NetView program through the Alert Receiver PPI mailbox.

Handling SNA Alerts

The NetView program can send and receive SNA alerts over IP.

Sending SNA Alerts

You can use the CNMSMSIP sample to send a CP-MSU (control point management services unit) that contains an SNA alert.

The following REXX example code builds a complete CP-MSU that contains an alert major vector and stores the CP-MSU in a REXX variable named altxt.

```
/* Ensure that all length values are correct within the alert and
   CP-MSU, or hardware monitor might not recognize the data. */

altxt =          '00DA121200D60000'X /* CP-MSU + alert MV */
altxt = altxt || '0B9200000AB00000000001'X /* Generic alert SV */

altxt = altxt ||,
'44100014110303000000E0EC9C2D440E2'X ||,
'A8A2A3859440A9'X ||,
'2D11040804'X ||,
'F0F5F0F4F0F01906E389A596938940D5'X ||,
'85A3E58985A64086969940A961D6E209'X ||,
'08F5F6F9F7C5D5E5'X /* Alert sender product set ID
                        subvector */

altxt = altxt ||,
'2C10000F11090300000090EA495929596'X ||,
'A6951A110C0E02F0F0F0F0F0F0F0F0'X ||,
```



```

'F0F0F00906A495929596A695'X          /* Reported resource product
                                         set ID subvector          */

altxt = altxt || '0F0109100901071105100004200000'X /* Date/time SV */
altxt = altxt || '0493FE00'X             /* Probable cause SV      */
altxt = altxt || '0A960401100004810000'X /* Failure cause SV       */
altxt = altxt || '1005'X ||,
'0E100009E2C5D5C4D9E3E3D90081'X        /* Hierarchy/resource list SV */

altxt = altxt || '2A31'X ||,
'0602000000000512C5D5E40321111A30'X ||,
'SENDRTTR-generated alert'              /* Self-defining Text Message SV*/

```

To send the alert over IP to the DSIRTTR task running within a NetView program, use a command similar to the following command, where 10.10.45.26 is the IP address of the host on which the target NetView program is running. You can specify an IPv6 address if both source and target NetView programs are enabled for IPv6 networking by using the IPv6Env statement.

```

'PIPE VAR altxt |',
'NETV CNMSMSIP 10.10.45.26 4021 TCPIP |',
'CONS'

```

For more information about using the CNMSMSIP sample, see the sample.

Receiving SNA Alerts

The DSIRTTR (alert receiver) task receives alerts over a TCP/IP connection and passes them to the hardware monitor. For information about initializing the DSIRTTR task, see *IBM Z NetView Installation: Configuring Additional Components*.

The example in [“Sending SNA Alerts” on page 124](#) shows how to create and send an SNA alert. If the hardware monitor at the target NetView program receives and records this alert and no filter prevents the display of the alert, information similar to the following information is shown:

- In the ALERTS-DYNAMIC (NPDA-30A) panel:

```

DOMAIN RESNAME  TYPE TIME  ALERT DESCRIPTION:PROBABLE CAUSE
NTV90 SENDRTTR SP   15:14 OPERATOR NOTIFICATION:UNDETERMINED

```

- In the ALERTS-STATIC (NPDA-30B) panel:

```

SEL# DOMAIN RESNAME  TYPE TIME  ALERT DESCRIPTION:PROBABLE CAUSE
( 1) NTV90 SENDRTTR SP   15:14 OPERATOR NOTIFICATION:UNDETERMINED

```

- In the EVENT DETAIL (NPDA-43S) panel:

```

SENDRTTR-generated alert

```

Responding to Intrusions

Intrusion Detection Services (IDS) of z/OS Communications Server enables the detection of attacks and the application of defensive mechanisms on the z/OS server. It provides support for scan detection and reporting; attack detection, reporting and prevention; and traffic regulation for TCP connections and UDP receive queues. In z/OS Communications Server, policies are used to specify what events are to be detected under what circumstances and what action to take. For complete information about IDS, see the z/OS Communications Server publications.

In conjunction with the Intrusion Detection Services of z/OS Communications Server, the NetView program can provide the following kinds of automated responses to an intrusion:

- Notification. Send an e-mail to security administrators, an alert to the NetView console, or a message to designated NetView operators.
- Commands. Issue UNIX, NetView, or z/OS commands to collect more data or to take other actions.

- **Statistics.** Collect statistics and generate TRMDSTAT reports to send by e-mail to security administrators. These reports give a consolidated view of the log messages that are written out by the Traffic Regulation Management daemon (TRMD).

To enable intrusion detection, define the IDS subtower on the IPMGT tower statement; for more information about tower statements, see *IBM Z NetView Installation: Getting Started*. To define the policies and command environment to be used when an IDS event occurs, use IDS statements in the CNMSTIDS member. For information about the IDS statements, see *IBM Z NetView Administration Reference*. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*.

Chapter 13. Using Z NetView Socket

You can use the NetView SOCKET command to develop TCP/IP applications. The SOCKET function provides the following advantages:

- It provides a NetView command interface with the z/OS Communications Server TCP/IP macro application programming interface. Because most NetView SOCKET commands are asynchronous requests, NetView tasks can continue processing their message queues and stop requests.

Note: It is also an alternative to the z/OS Communications Server REXX SOCKET (RXSOCKET) function and z/OS C/C++ UNIX sockets.

- It can be used by operators to perform the following functions:
 - Resolve host names, IP addresses, or service names (GETADDRINFO, GETNAMEINFO, GETHOSTBYNAME, GETHOSTBYADDR)
 - Retrieve a host name (GETHOSTNAME)
 - Retrieve a host ID (GETHOSTID)
 - Obtain sockets for communication and type a TCP/IP application, command by command
- It can be used in command procedures to create the following kinds of TCP/IP applications in an IPv4 network, an IPv6 network, or both.
 - Datagram (or User Datagram Protocol, UDP) sender, receiver, or both
 - Stream (or Transmission Control Protocol, TCP) data client, server, or both
 - Raw sockets sender, receiver, or both

Note: The sample NetView security definitions (the CNMSCAT2 and CNMSAF2 samples) do not allow users to obtain raw sockets (SOCKET TYPE=SOCKET SOCKTYPE=RAW). NetView operators who need to run raw socket applications based on the SOCKET command must be permitted to do so.

- It uses the NetView base TCP/IP services, which provide trace entries in the NetView trace if the TCP option of the TRACE command is specified. It also provides entries in the NetView trace about its own asynchronous TCP/IP request completions if the TCP option of the TRACE command is specified.

Generally, the keywords for the SOCKET command map to the keywords for the EZASMI macro. For information about the IP sockets API and request completion errors (errno), see the *z/OS Communications Server IP Sockets Application Programming Interface Guide and Reference*. You can use the SOCKET command to implement the same TCP/IP application programming concepts that are described in that manual, although not all of the EZASMI macro functions are implemented in the SOCKET command.

To prevent unauthorized TCP/IP hosts from connecting to the NetView program by using the NetView socket interface, or, for a server application, to prevent unauthorized clients from maintaining a connection with the server, you can use the SOCACC command. For more information, see the *IBM Z NetView Security Reference*.

For detailed information about the SOCKET command, see the online help.

Using the SOCKET Interface

This section describes general information about using the SOCKET interface.

Initializing a Socket Interface

Before a socket interface can be used to retrieve information or to implement a TCP/IP application (whether the SOCKET command is entered on a command line or used in a command procedure), it must be initialized for the task. That is done by using the SOCKET TYPE=INIT command.

In addition, the socket interface must have affinity with a TCP/IP stack. The user can either specify the TCP/IP stack name or allow the SOCKET command to use the default TCP/IP stack name of TCPIP. For example, if the socket interface is to have affinity with a TCP/IP stack named TCPIPX, the socket interface can be initialized by using the following command:

```
SOCKET TYPE=INIT TCPNAME=TCPIPX
```

Only one socket interface at a time can be initialized by using the SOCKET command on a task.

Either of the following messages indicates that a socket interface is initialized and that you can use other SOCKET commands to retrieve information or to implement a TCP/IP application:

```
BNH600I SOCKET INTERFACE INITIALIZED WITH 50 SOCKETS ON TCP/IP TCPIPX
```

```
BNH623I SOCKET INTERFACE HAS ALREADY BEEN INITIALIZED ON TCP/IP TCPIPX
```

Ending a Socket Interface

When finished using the socket interface, the user can stop it by using the SOCKET TYPE=TERM command. If the socket interface was active, the following message is displayed:

```
BNH605I SOCKET INTERFACE HAS TERMINATED
```

If sockets are allocated when SOCKET TYPE=TERM is issued, they are all closed by TCP/IP. Also, if the task upon which the socket interface was initialized is ended (stopped or logged off) or abends, the socket interface automatically stops.

Making Synchronous and Asynchronous Requests

Synchronous SOCKET command requests have a TYPE value of GETADDRINFO, GETHOSTBYADDR, GETHOSTBYNAME, GETNAMEINFO, INIT, or TERM. Synchronous SOCKET commands have the following characteristics:

- The task is blocked until the command finishes, that is, the task cannot process message queues or a stop request. For that reason, be careful in a TCP/IP application when using synchronous requests that might take an unusually long time to complete, such as resolver calls (all of the synchronous requests EXCEPT INIT and TERM).
- When SOCKET is called in a NETVIEW stage of PIPE, you do not have to use a CORRWAIT stage to capture the command output.

The remaining SOCKET command requests are asynchronous. Asynchronous requests have the following characteristics:

- The task can process its message queues while the SOCKET command request is outstanding. If SOCKET is used in a command procedure, the low priority message queue is not serviced.
- The task can immediately obey a stop request, such as from the NetView STOP and CLOSE commands.
- When SOCKET is called in a NETVIEW stage of PIPE, you must use a CORRWAIT stage to capture the command output. If CORRWAIT times out before the SOCKET command request finishes, the TCP/IP application does not necessarily know the state of the request; that is, it might not be finished. A good practice is to try to cancel such a request; for information about canceling SOCKET command requests, see [“Canceling Outstanding Asynchronous Requests” on page 131](#).

For more information about these commands, see the online help.

Making Global and Socket-specific Requests

As mentioned previously, some SOCKET commands are synchronous, while the rest are asynchronous. SOCKET commands also make either global or socket-specific requests.

A global request is not associated with a particular socket, whereas a socket-specific request is. Therefore, a socket-specific request requires that the SOCKID keyword be provided with the SOCKET command.

Although the following SOCKET requests look as if they are socket-specific or need a socket descriptor, they are global:

- SOCKET TYPE=SOCKET obtains a socket and does not require one to perform its action.
- SOCKET TYPE=SELECT waits for events that happen on zero or more sockets. It does not operate upon or require a single socket.
- SOCKET TYPE=TAKESOCKET takes a socket that was given, perhaps by some other process. When the command is issued, it is not operating upon a particular socket. The socket that is specified by using SOCKID identifies the socket that is to be taken.

SOCKET TYPE=SOCKET and SOCKET TYPE=TAKESOCKET requests do not start with a socket allocated within the current instance of socket interface, but a successful request produces a socket. A SOCKET TYPE=SELECT request waits for activity on zero or more sockets but does not require the use of any sockets to do so.

Other SOCKET requests are global and either do not require or do not permit the SOCKID keyword, such as the following commands:

- SOCKET TYPE=GETHOSTID obtains the host identifier from TCP/IP.
- SOCKET TYPE=GETHOSTNAME obtains a host name from TCP/IP.
- SOCKET TYPE=GETADDRINFO obtains host address and service characteristics if a host, a service, or both a host and a service are identified.

At a given time for an active socket interface on a task, only one global request can be outstanding, and, for each allocated socket, only one socket-specific request can be outstanding.

Retrieving Information

You can use the SOCKET command to retrieve information from the resolver or information about TCP/IP. This section provides some examples. Note that all of the requests are global and that SOCKET TYPE=GETHOSTNAME and SOCKET TYPE=GETHOSTID are also asynchronous.

Example 1. Request resolution of a host name:

```
SOCKET TYPE=GETADDRINFO HOSTNAME=MYHOST6
```

If the resolver learns that this host name maps to an IPv6 address, messages similar to the following messages might be displayed:

```
BNH641I HOST ADDRESS IS 3FF0:9:100:5:FF:AAA:BBBB:CCCC
BNH791I ADDRESS FAMILY IS INET6
BNH792I PORT NUMBER IS 0
BNH793I SOCKET TYPE IS STREAM
```

Example 2. Request resolution of a different host name:

```
SOCKET TYPE=GETADDRINFO HOSTNAME=MYHOST4
```

If the resolver learns that this host name maps to an IPv4 address, the following messages might be displayed:

```
BNH641I HOST ADDRESS IS 10.12.45.27
BNH791I ADDRESS FAMILY IS INET
BNH792I PORT NUMBER IS 0
BNH793I SOCKET TYPE IS STREAM
```

Example 3. Resolve a host name and a service name. The command is prefixed with NETVASIS because the service name lookup is case-sensitive.

```
NETVASIS SOCKET TYPE=GETADDRINFO HOSTNAME=MYHOST4 SERVICE=snmp
```

If the resolver has access to a SERVICES file that maps service names to port numbers, the following messages might be displayed:

```
BNH641I HOST ADDRESS IS 10.12.45.27
BNH791I ADDRESS FAMILY IS INET
BNH792I PORT NUMBER IS 161
BNH793I SOCKET TYPE IS DATAGRAM
```

Example 4. Determine a host name for a specific IP address:

```
SOCKET TYPE=GETNAMEINFO ADDRESS=10.12.45.27
```

If the resolver determines the host name for the specified IP address, the following messages might be displayed:

```
BNH639I HOST NAME IS myhost4.city.company.com
BNH796I SERVICE NAME IS 0
```

Example 5. Determine a host name and service name for a specified IP address and port number:

```
SOCKET TYPE=GETNAMEINFO ADDRESS=3FF0:9:100:5:FF:AAA:BBBB:CCCC PORT=512
```

If the resolver determines the host name for the specified IP address and a SERVICES file entry maps TCP port 512 to the name of a service, the following messages might be displayed:

```
BNH639I HOST NAME IS myhost6.city.company.com
BNH796I SERVICE NAME IS exec
```

Note that, if the name of a service that is using a UDP port is of interest, add INFOFLAG=NI_DGRAM to the command.

Example 6. Determine the host name defined for the TCP/IP stack with which the socket interface has affinity:

```
SOCKET TYPE=GETHOSTNAME
```

If a host name is defined in a TCPIP.DATA file for the specified TCP/IP stack, the following message might be displayed:

```
BNH639I HOST NAME IS MYHOST4
```

Example 7. Determine the host ID (possibly the "home" IP address) defined for the TCP/IP stack:

```
SOCKET TYPE=GETHOSTID
```

If a host ID can be determined, the following message might be displayed:

```
BNH637I HOST IDENTIFIER IS 10.12.45.27
```

Waiting for Activity on Sockets

Particularly when implementing an application that serves multiple clients or manages multiple sockets simultaneously, you might not want to block the application that is waiting for activity on just one socket (such as by issuing SOCKET TYPE=ACCEPT for a server listening socket when no connection requests are available to be accepted).

To wait for activity on any sockets managed by a TCP/IP application, use a SOCKET TYPE=SELECT command. To wait for activity on the sockets, any or all of read, write, and exception lists can be passed to the SOCKET TYPE=SELECT command. You can also specify a timeout value so that the command ends after a specified time, even if no activity occurred on any socket.

For example, suppose that a TCP/IP application has a listening (server) socket 0 and a socket 1 representing the server connection to a client and has issued the SOCKET TYPE=GIVESOCKET command to pass socket 2 to another TCP/IP application. To wait for another connection request, data from the client, and the other TCP/IP application to take the socket, respectively, and to wait no more than 15 seconds for the command to finish, use the following command:

```
SOCKET TYPE=SELECT READ=(0,1) EXCEPT=2 TIMEOUT=15
```

If data from the client becomes available before 15 seconds elapses, the SOCKET TYPE=SELECT command finishes with a message that is similar to the following message:

```
BNH610I SOCKET 1 READY FOR 'READ'
```

If multiple sockets had activity, a BNH610I message is issued for each socket that had activity, with the message variable containing a value for each type of event that occurred for the socket.

If 15 seconds elapses without any activity for any socket passed to SOCKET TYPE=SELECT, the SOCKET TYPE=SELECT command finishes with the following message:

```
BNH609I SELECT REQUEST HAS TIMED OUT
```

Note: Because of the limitations of the command parsing services that are used by the SOCKET command, the value of each of the READ, WRITE, and EXCEPT keywords cannot exceed 255 characters (including the parentheses that delimit a list of socket descriptors). If the number of sockets monitored for a type of activity becomes large enough to exceed the value length limit, the SOCKET command ends with a syntax error.

This error can be prevented in a couple of ways. One way is to limit the list to the descriptors that fit in the list and to close any others. Another way is to create multiple lists and rotate calls of SOCKET TYPE=SELECT through the different lists, perhaps using a relatively small TIMEOUT value so that no socket has to wait very long for its activity to be processed.

Handling TCP/IP Errors

When a TCP/IP request made by the SOCKET command on behalf of the user fails, the resulting message depends on the kind of request and on what TCP/IP returns.

A failed global request for which TCP/IP returns an error number (errno) generally results in the following message:

```
BNH604I requestname REQUEST FAILED, ERRNO = errno
```

A failed socket-specific request for which TCP/IP returns an error number usually results in the following message:

```
BNH613I requestname REQUEST ON SOCKET socket FAILED, ERRNO = errno
```

A global request for which TCP/IP returns only a return code (no error number) results in the following message:

```
BNH638I requestname REQUEST FAILED, RETURN CODE retcode
```

Canceling Outstanding Asynchronous Requests

To cancel an outstanding asynchronous global request, use the following command:

```
SOCKET TYPE=CANCEL
```

To cancel an outstanding asynchronous socket-specific request, also specify SOCKID, as shown in the following example. Note that the SOCKET TYPE=CANCEL command itself is also asynchronous.

```
SOCKET TYPE=CANCEL SOCKID=0
```

A successful result for the SOCKET TYPE=CANCEL command is indicated by a message that is similar to the following message:

```
BNH608I CANCEL REQUEST COMPLETED SUCCESSFULLY
```

A canceled SOCKET TYPE=SELECT request is indicated by a message that is similar to the following message:

```
BNH604I SELECT REQUEST FAILED, ERRNO = 1009
```

If a request has finished but the NetView program has not yet returned the results to the TCP/IP application, the SOCKET TYPE=CANCEL command can still finish successfully without changing the completion results of the original request.

Giving and Taking Sockets

To help distribute the workload for managing sockets, TCP/IP applications can give sockets to other TCP/IP applications (possibly on other tasks or address spaces) and can take sockets from other TCP/IP applications (possibly on other tasks or address spaces).

For example, suppose that socket 1 was created by a TCP server application to communicate with a client somewhere in the network, and now another task in this NetView instance is supposed to handle the communication with that client. In this example, the job name is NETVIEW and the "taking" task has already initialized a socket interface by using a SOCKET TYPE=INIT command.

To take the socket, the "taking" task must be able to identify the "giving" task; therefore, the "giving" TCP server should process the following command.

```
SOCKET TYPE=GETCLIENTID
```

A successful result is indicated by a message that is similar to the following message:

```
BNH603I CLIENT IDENTIFIER: NAME NETVIEW, TASK 007475D1
```

The information in the BNH603I message must be made available to the "taking" task. The "giving" task then processes the following command:

```
SOCKET TYPE=GIVESOCKET SOCKID=1
```

A successful result is indicated by a message that is similar to the following message:

```
BNH614I GIVESOCKET REQUEST ON SOCKET 1 COMPLETED SUCCESSFULLY
```

Because the JOBNAME keyword was not specified with the SOCKET TYPE=GIVESOCKET command, only a unit of work in the NETVIEW job can take the socket. Because the TASK keyword was not specified with SOCKET TYPE=GIVESOCKET, then any task in the NETVIEW job can take the socket.

Next, to take socket 1 from NetView task 007475D1, the "taking" task might process the following command:

```
SOCKET TYPE=TAKESOCKET SOCKID=1 JOBNAME=NETVIEW TASK=007475D1
```

A successful result is indicated by a message that is similar to the following message:

```
BNH606I TAKESOCKET REQUEST COMPLETED SUCCESSFULLY. SOCKET 8 HAS BEEN ALLOCATED
```

If the "giving" task processes the SOCKET TYPE=SELECT EXCEPT=1 command, the completion of the SOCKET TYPE=TAKESOCKET command by the "taking" task results in an EXCEPT event for socket 1. At that point, the "giving" task closes socket 1, and the "taking" task uses socket 8 in its own socket interface to manage the communication with the client.

Notes:

1. You can use the SOCKET TYPE=TAKESOCKET command to take a socket given by a TCP/IP application that is not based on the SOCKET command provided that the request used to give the socket authorizes the socket interface running in the NetView program to do so.
2. To take an AF_INET6 socket by using the SOCKET TYPE=TAKESOCKET command, ensure that FAMILY=INET6 is specified with SOCKET TYPE=TAKESOCKET.

3. A TCP/IP application that is not based on the SOCKET command can take a socket given by a TCP/IP application that is based on the SOCKET command, provided that the SOCKET TYPE=GIVESOCKET command authorizes it to do so and that the address family that is specified by the "taking" application matches the address family of the socket that is given.

Using the SOCKET Command to Create TCP/IP Applications

You can use the SOCKET command to create TCP/IP applications. You can create the following types of applications:

- Datagram applications; see [“Creating Datagram Applications” on page 133](#).
- Stream applications; see [“Creating Stream Applications” on page 137](#).

Creating Datagram Applications

Datagram sockets have the following characteristics:

- Datagram sockets are connectionless. No data is exchanged between sending and receiving UDP protocol layers until your application issues the first send call.
- The UDP protocol layer does not implement reliability functions. Data can be lost, duplicated, or corrupted, and datagrams can arrive out of order. If you need reliability, you can either add reliability code to your client and server programs or use TCP protocols.
- UDP sockets have a one-to-one relationship between send and receive calls.
- Datagrams are sent as independent packets. The size of a datagram is limited to the number of bytes that can be sent in a single transaction.

This section shows the commands that you can use in a datagram socket application. You can run each of these SOCKET commands by using the NETV stage of PIPE and, because these commands are all asynchronous, they require a subsequent CORRWAIT stage to return results to the PIPE.

Obtaining a Datagram Socket

To obtain a datagram socket by using the SOCKET command, use the following command:

```
SOCKET TYPE=SOCKET SOCKETTYPE=DATAGRAM
```

A successful result is indicated by a message that is similar to the following message:

```
BNH606I SOCKET REQUEST COMPLETED SUCCESSFULLY. SOCKET 0 HAS BEEN ALLOCATED
```

Because FAMILY is not specified and FAMILY=INET is the default (set the address family to IPv4), socket 0 can then be used to send or receive, or both send and receive, datagrams in an IPv4 network.

To obtain a datagram socket that can be used in an IPv6 network, use the following command:

```
SOCKET TYPE=SOCKET SOCKETTYPE=DATAGRAM FAMILY=INET6
```

A successful result is indicated by the same message as for FAMILY=INET, for example:

```
BNH606I SOCKET REQUEST COMPLETED SUCCESSFULLY. SOCKET 1 HAS BEEN ALLOCATED
```

Binding a Datagram Socket

If your application is expected to receive datagrams and a particular port number is to be used and made public, the socket can be bound to that port by using commands such as the following commands:

- If the datagram socket (in this case, socket 1) was obtained by using the SOCKET TYPE=SOCKET SOCKETTYPE=DATAGRAM FAMILY=INET command, use the following command:

```
SOCKET TYPE=BIND SOCKID=1 ADDRESS=0.0.0.0 PORT=9000
```

- If the datagram socket was obtained by using the SOCKET TYPE=SOCKET TYPE=DATAGRAM FAMILY=INET6 command, use the following command:

```
SOCKET TYPE=BIND SOCKID=1 ADDRESS=: : PORT=9000
```

Or, to limit the sending and receiving of datagrams to IPv4 network interfaces, use the following command:

```
SOCKET TYPE=BIND SOCKID=1 ADDRESS=0.0.0.0 PORT=9000
```

A successful result is indicated by a message that is similar to the following message:

```
BNH614I BIND REQUEST ON SOCKET 1 COMPLETED SUCCESSFULLY
```

A datagram sender does not have to process an explicit SOCKET TYPE=BIND command to send datagrams, although it is allowed.

A socket can be bound to a specific interface IP address to limit the interface over which the socket can send or receive datagrams.

If you want TCP/IP to choose the port number to which the socket is bound, specify PORT=0 on the SOCKET command, as shown in the following commands:

- If the datagram socket was obtained by using the FAMILY=INET option, use the following command:

```
SOCKET TYPE=BIND SOCKID=1 ADDRESS=0.0.0.0 PORT=0
```

- If the datagram socket was obtained by using the FAMILY=INET6 option, use the following command:

```
SOCKET TYPE=BIND SOCKID=1 ADDRESS=: : PORT=0
```

Or, to limit the sending and receiving of datagrams to IPv4 network interfaces, use the following command:

```
SOCKET TYPE=BIND SOCKID=1 ADDRESS=0.0.0.0 PORT=0
```

```
SOCKET TYPE=BIND SOCKID=1 ADDRESS=: : PORT=0
```

A successful result is indicated by a message that is similar to the following message:

```
BNH614I BIND REQUEST ON SOCKET 1 COMPLETED SUCCESSFULLY
```

To see the number of the port that is selected and bound to the socket by TCP/IP when PORT=0 is specified, use the following command:

```
SOCKET TYPE=GETSOCKNAME SOCKID=1
```

A message that is similar to one of the following messages is returned:

- For a socket that is bound to 0.0.0.0 (ADDRESS=0.0.0.0), a message that is similar to the following message is returned:

```
BNH635I SOCKET 1 IS BOUND TO 0.0.0.0 PORT 1399
```

- For a socket that is bound to : : (ADDRESS=: :), a message that is similar to the following message is returned:

```
BNH635I SOCKET 1 IS BOUND TO 0:0:0:0:0:0:0:0 PORT 1400
```

Receiving a Datagram

To receive a datagram on socket 0, use the following command:

```
SOCKET TYPE=RCVFROM SOCKID=0
```

A successful result is indicated by messages that are similar to the following messages:

```
BNH619I SOCKET 0 RECEIVED 1 BYTES OF DATA FROM 0:0:0:0:0:0:1 PORT 10000
BNH621I Y
```

In this case, a single character (upper case EBCDIC "Y") was received from a datagram sender bound to the IP address and port shown in message BNH619I.

As many BNH621I messages are used as necessary to present all the received data to the TCP/IP application. As many as 240 bytes of data can be contained in a BNH621I message line (beginning after the message ID and one blank). The SOCKET command does not convert the code set of any data that is returned in a BNH621I message. If needed, you can use the XLATE stage of PIPE to perform ASCII-to-EBCDIC or EBCDIC-to-ASCII data conversion.

A datagram that is to be received on a socket can satisfy only one SOCKET TYPE=RCVFROM request. If the receive size limit (MAXLEN value) is too small to hold the entire datagram, the datagram is truncated to the receive size limit. TCP/IP configuration can also affect the amount of data that can be received; for more information, see the *z/OS Communications Server IP Configuration Reference*.

Sending a Datagram

To send a datagram by using socket 0, use the following command:

```
PIPE LIT /ABC/ | NETV SOCKET TYPE=SENDTO SOCKID=0 ADDRESS=10.12.45.27 PORT=9000 |
CORRWAIT 5 | CONS
```

This command sends the EBCDIC characters "ABC" to a datagram receiver at IP address 10.12.45.27, port 9000. A successful result is indicated by a message that is similar to the following message:

```
BNH617I SOCKET 0 SENT 3 BYTES OF DATA
```

When sending data, the PIPE command provides more than just the correlation of output from an asynchronous command. It also provides the data to be sent in a collection of one or more data buffers that the SOCKET command subsequently collects into one datagram to be sent. In the command previously shown, the PIPE is providing the literal data "ABC" in one data buffer to be sent.

Because of the nature of the datagram protocol (UDP), there is no certainty that the data actually made it to the receiver or, if multiple datagrams are sent, that the order was maintained. In addition, the sender has no knowledge whether the receiver even exists. The application must have mechanisms to ensure correct data delivery.

Closing a Socket

To close a datagram socket, such as socket 0, use the following command:

```
SOCKET TYPE=CLOSE SOCKID=0
```

A successful result is indicated by a message that is similar to the following message:

```
BNH614I CLOSE REQUEST ON SOCKET 0 COMPLETED SUCCESSFULLY
```

Creating a Simple Datagram Sender or Receiver

This section shows the SOCKET commands that you can use to implement simple datagram sender and datagram receiver applications. More complex applications can use additional SOCKET commands to gather information, set socket options, and perform other functions.

Simple Datagram Sender

The following steps implement a simple datagram sender application. The basic steps are to obtain a datagram socket, send a datagram, and then close the datagram socket.

1. Initialize a socket interface with the following command:

```
SOCKET TYPE=INIT TCPNAME=stackname
```

2. Obtain a datagram socket with one of the following commands:

```
SOCKET TYPE=SOCKET SOCKTYPE=DATAGRAM FAMILY=INET6  
SOCKET TYPE=SOCKET SOCKTYPE=DATAGRAM FAMILY=INET
```

3. Send a datagram with one of the following commands where *x* is the socket number of the socket that was obtained and where *data_source* might be, for example, "VAR *varname*" or "STEM *stemname* . | COLLECT" or "LIT */data/*". A PIPE command is used because it can provide data in the manner that is required by the SOCKET TYPE=SENDTO command,

```
PIPE data_source | NETV SOCKET TYPE=SENDTO SOCKID=x ADDRESS=ipv6_addr PORT=nnnnn  
| CORR 5 | CONS  
PIPE data_source | NETV SOCKET TYPE=SENDTO SOCKID=x ADDRESS=ipv4_addr PORT=nnnnn  
| CORR 5 | CONS
```

4. Close the datagram socket with the following command:

```
SOCKET TYPE=CLOSE SOCKID=x
```

5. End the socket interface with the following command:

```
SOCKET TYPE=TERM
```

The CNMSMSGT sample contains an example of a datagram sender. This example also illustrates what can generally be done when SOCKET command requests fail, such as canceling a request for which no messages are returned. For more information, see [“Sending SNMP Traps” on page 123](#).

Simple Datagram Receiver

The following steps implement a simple datagram receiver application. The basic steps are to obtain a datagram socket, listen for datagrams sent to port *nnnnn*, receive one datagram, and close the datagram socket.

1. Initialize the socket interface with the following command:

```
SOCKET TYPE=INIT TCPNAME=stackname
```

2. Obtain a datagram socket with one of the following commands:

```
SOCKET TYPE=SOCKET SOCKTYPE=DATAGRAM FAMILY=INET6  
SOCKET TYPE=SOCKET SOCKTYPE=DATAGRAM FAMILY=INET
```

3. Bind the datagram socket to an IP address and port with one of the following commands, where *x* is the socket number of the socket that was obtained. To accept datagrams regardless of the interface over which they are sent, use : : (IN6ADDR_ANY) for a socket that is obtained by using FAMILY=INET6 or 0.0.0.0 (INADDR_ANY) for a socket that is obtained by using FAMILY=INET.

```
SOCKET TYPE=BIND SOCKID=x ADDRESS=ipv6_addr PORT=nnnnn  
SOCKET TYPE=BIND SOCKID=x ADDRESS=ipv4_addr PORT=nnnnn
```

4. Wait for activity with the following command. A read event for the socket is that data is available to be received.

```
SOCKET TYPE=SELECT READ=x
```

5. Receive the datagram with the following command. If the application is to continue waiting for data, it can issue the SOCKET TYPE=SELECT command again. When the application is ready to stop, it can proceed to the next step.

```
SOCKET TYPE=RCVFROM SOCKID=x
```

6. Close the datagram socket with the following command:

```
SOCKET TYPE=CLOSE SOCKID=x
```

7. Stop the socket interface with the following command:

```
SOCKET TYPE=TERM
```

Creating Stream Applications

Stream (TCP) sockets have the following characteristics:

- Stream sockets are connection-oriented.
- Stream sockets provide reliability; that is, they guarantee that data is delivered in the order sent and without duplication. In addition, built-in flow control prevents data overruns. The burden of transferring the data reliably is handled by TCP/IP, rather than by the application.
- Stream sockets do not have a one-to-one relationship between send and receive calls. When a send or receive is attempted, not all of the data to be sent or received is necessarily processed in a single call.
- No boundaries are imposed on the data; that is, the data is treated as a stream of bytes. Because of that, communicating processes must agree on mechanisms for distinguishing information. Usually, the process that is sending information sends a number that specifies the length of the data, followed by the data itself. The process receiving information reads the length and then accepts the data, looping until it receives all of the data. Because no data boundaries exist, multiple concurrent read or write socket calls of the same type on the same stream socket yield unpredictable results.

This section shows the commands that you can use in a stream socket application.

Obtaining a Stream Socket

To obtain a stream socket to be used as a client or server only in an IPv4 network (AF_INET), use the following command:

```
SOCKET TYPE=SOCKET SOCKTYPE=STREAM FAMILY=INET
```

A successful result is indicated by a message that is similar to the following message:

```
BNH606I SOCKET REQUEST COMPLETED SUCCESSFULLY. SOCKET 0 HAS BEEN ALLOCATED
```

If the socket is for a client or server that might also be used in an IPv6 network (AF_INET6), use the following command:

```
SOCKET TYPE=SOCKET SOCKTYPE=STREAM FAMILY=INET6
```

A successful result is indicated by a message that is similar to the following message:

```
BNH606I SOCKET REQUEST COMPLETED SUCCESSFULLY. SOCKET 1 HAS BEEN ALLOCATED
```

You can use an AF_INET6 socket in both IPv4 and IPv6 networks unless the IPV6_V6ONLY socket option is set to limit the socket to IPv6 networks; for more information, see the TYPE=SETSOCKOPT description in the SOCKET command online help.

The socket descriptor can also be chosen by the user, as shown in the following command:

```
SOCKET TYPE=SOCKET SOCKTYPE=STREAM FAMILY=INET6 NEWSOCK=5
```

A successful result is indicated by a message that is similar to the following message:

```
BNH606I SOCKET REQUEST COMPLETED SUCCESSFULLY. SOCKET 5 HAS BEEN ALLOCATED
```

Binding a Stream Socket

While a stream socket that is used as a TCP client can be explicitly bound, it is more typical for a stream socket that is used as a TCP server. In that way, the server port is fixed and can be published for use by the client applications.

Also, while a TCP server can be limited to processing connection requests received over specific interfaces, it is more typical for such a server to process connection requests regardless of the interfaces over which they are sent. To receive these connection requests, the TCP server binds its socket to IP address 0.0.0.0 (referred to as INADDR_ANY) if only IPv4 networking is supported or to IP address :: (compressed text presentation form of 0:0:0:0:0:0:0:0, referred to as IN6ADDR_ANY) if IPv6 networking is supported.

To bind AF_INET socket 0 to the INADDR_ANY IP address and port 9000, use the following command:

```
SOCKET TYPE=BIND SOCKID=0 ADDRESS=0.0.0.0 PORT=9000
```

A successful result is indicated by a message that is similar to the following message:

```
BNH614I BIND REQUEST ON SOCKET 0 COMPLETED SUCCESSFULLY
```

To bind AF_INET6 socket 1 to the IN6ADDR_ANY IP address and port 10000, use the following command:

```
SOCKET TYPE=BIND SOCKID=1 ADDRESS=:: PORT=10000
```

A successful result is indicated by a message that is similar to the following message:

```
BNH614I BIND REQUEST ON SOCKET 1 COMPLETED SUCCESSFULLY
```

As with datagram sockets, a stream socket can be bound to a port chosen by TCP/IP by using the following command:

```
SOCKET TYPE=BIND SOCKID=1 ADDRESS=:: PORT=0
```

To see the number of the port to which a particular socket is bound, use the following command:

```
SOCKET TYPE=GETSOCKNAME SOCKID=1
```

If the stream socket (in this case, socket 1) was bound by using a SOCKET command that specified PORT=0, a message that is similar to the following message is returned:

```
BNH635I SOCKET 1 IS BOUND TO 0:0:0:0:0:0:0:0 PORT 1401
```

Listening (Server)

For a stream socket to serve clients, TCP/IP must be told that the socket is to receive connection requests from clients by using the SOCKET TYPE=LISTEN command. A socket for which SOCKET TYPE=LISTEN is issued becomes a "listening socket". For a socket that was previously bound (such as socket 0), use the following command:

```
SOCKET TYPE=LISTEN SOCKID=0
```

A successful result is indicated by a message that is similar to the following message:

```
BNH614I LISTEN REQUEST ON SOCKET 0 COMPLETED SUCCESSFULLY
```

TCP/IP can now present a connection request to the socket.

Connecting (Client)

A stream socket becomes a TCP client by connecting to a server that is listening for connection requests. To connect an AF_INET or AF_INET6 socket 0 to a server at host 10.12.45.27 bound to port 10000 and listening for connection requests, use the following command:

```
SOCKET TYPE=CONNECT SOCKID=0 ADDRESS=10.12.45.27 PORT=10000
```

A successful result is indicated by a message that is similar to the following message:

```
BNH611I SOCKET 0 CONNECTED TO 10.12.45.27 PORT 10000
```

To connect an AF_INET6 socket 1 to a server on the same host bound to port 10000 and listening for connection requests, use the following command:

```
SOCKET TYPE=CONNECT SOCKID=1 ADDRESS>:::1 PORT=10000
```

A successful result is indicated by a message that is similar to the following message:

```
BNH611I SOCKET 1 CONNECTED TO 0:0:0:0:0:0:1 PORT 10000
```

Accepting a Connection Request (Server)

A server can anticipate a connection request in the following ways:

- The SOCKET TYPE=SELECT command, where *x* is the socket descriptor for the listening socket:

```
SOCKET TYPE=SELECT READ=x
```

When a connection request arrives, the command finishes with a message that is similar to the following message:

```
BNH610I SOCKET x READY FOR 'READ'
```

- The SOCKET TYPE=ACCEPT command, which cannot finish until a connection request arrives. For listening socket 0 to accept a connection request, use the following command:

```
SOCKET TYPE=ACCEPT SOCKID=0
```

A successful result is indicated by a message that is similar to the following message. This message indicates that a connection request came from a client in an IPv6 network and that socket 1 can be used for communication with that client.

```
BNH612I SOCKET 0 ACCEPTED CONNECTION FROM 0:0:0:0:0:0:1 PORT 1032. SOCKET 1  
HAS BEEN ALLOCATED
```

If the server is notified that another connection request has arrived (another READ event for listening socket 0), this time from a client in the IPv4 network, the same command is used again to accept the connection:

```
SOCKET TYPE=ACCEPT SOCKID=0
```

A successful result is indicated by a message that is similar to the following message:

```
BNH612I SOCKET 0 ACCEPTED CONNECTION FROM 10.12.45.27 PORT 1034. SOCKET 2  
HAS BEEN ALLOCATED
```

Socket 2 can then be used for communication with that client.

Determining a Connection Peer

For a client or server application to see information about a connection partner (peer) that a particular socket has, use a command that is similar to the following command:

```
SOCKET TYPE=GETPEERNAME SOCKID=6
```

If socket 6 is the server socket used to handle communication with a client socket, a message that is similar to the following message might be returned:

```
BNH636I SOCKET 6 PEER IS 0:0:0:0:0:0:1 PORT 1032
```

If a client socket 4 was also created by using the SOCKET command and is the connection partner, you can use the following command:

```
SOCKET TYPE=GETPEERNAME SOCKID=4
```

A message that is similar to the following message might be returned:

```
BNH636I SOCKET 4 PEER IS 0:0:0:0:0:0:1 PORT 10000
```

Sending a Stream

Whether the application is a client or server, and assuming that a stream socket is available (because either a client processing the SOCKET TYPE=SOCKET command or a server processing the SOCKET TYPE=ACCEPT command caused a new stream socket to be allocated), data can be sent by using a command such as the following command (where *data_source* might be, for example, "VAR *varname*" or "STEM *stemname*. | COLLECT" or "LIT */data/*"):

```
PIPE data_source | NETV SOCKET TYPE=SEND SOCKID=x | CORR 5 | CONS
```

The PIPE command is used because it can provide data in the manner that is required by the SOCKET TYPE=SEND command. Because this is a stream application, not all the data might be sent with a single call of SOCKET TYPE=SEND. The application needs to check the output of the SOCKET TYPE=SEND command to see if all bytes were sent; if not, the application can attempt to send again, starting with the first previously unsent byte.

For example, if a stream of data ("AFDAFD") is to be sent by using socket 2, use a command that is similar to the following command:

```
PIPE LIT /AFDAFD/ | NETV SOCKET TYPE=SEND SOCKID=2 | CORR 5 | CONS
```

A successful result is indicated by a message that is similar to the following message:

```
BNH617I SOCKET 2 SENT 6 BYTES OF DATA
```

In this example, all the data is being sent in one SOCKET TYPE=SEND command. However, if fewer than 6 bytes were sent, the sending application can create a new stream beginning with the first byte of data not yet sent and issue the SOCKET TYPE=SEND command again.

The wait time of 5 seconds for the CORRWAIT (CORR) stage is arbitrary. You can set a longer or shorter wait time, or perhaps make it adjustable, depending on system and network conditions.

Receiving a Stream

Whether the application is a client or server, and assuming that a stream socket is available (because either a client processing the SOCKET TYPE=SOCKET command or a server processing the SOCKET TYPE=ACCEPT command caused a new stream socket to be allocated), data can be received by using the following command, where socket 2 is specified:

```
SOCKET TYPE=RECV SOCKID=2
```

A successful result is indicated by messages that are similar to the following messages:

```
BNH620I SOCKET 2 RECEIVED 6 BYTES OF DATA  
BNH621I AFDAFD
```

The BNH620I message indicates how many bytes of data were received by this SOCKET TYPE=RECV command. Following the BNH620I message are as many BNH621I messages as are required to present all of the received data. As many as 240 bytes of data can be contained in a BNH621I message line (beginning after the message ID and one blank). The SOCKET command includes the received data as is in the BNH621I messages; that is, it does no data conversion between code sets.

Because this is stream data, a single SOCKET TYPE=RECV request might not receive all of the data that was sent. In that case, the application can prepare to receive additional data. One method is to include this socket in a READ list that is passed to a SOCKET TYPE=SELECT command; this method is recommended because it does not block all other requests on the socket while it is being processed. If additional data is to be received by that socket, SOCKET TYPE=SELECT produces a BNH610I message for

a READ event on that socket almost immediately. The application can then issue SOCKET TYPE=RECV again to either get more data (or learn that the connection was stopped).

If other sockets are passed to SOCKET TYPE=SELECT and read events happen for them, the SOCKET TYPE=SELECT output also contains BNH610I messages for the other sockets.

Another method is to issue another SOCKET TYPE=RECV for the socket; however, this request blocks all others on the socket until the SOCKET TYPE=RECV command finishes (either data arrives or the connection is stopped) or it is canceled (by a socket-specific SOCKET TYPE=CANCEL command).

Ending a Connection

When a client or server determines that no more data is to be exchanged and the connection is to be stopped, the client or server processes a command similar to the following command (for example, if socket 2 is being used to communicate with the connection partner), where HOW=BOTH indicates that both send and receive operations are to be stopped:

```
SOCKET TYPE=SHUTDOWN SOCKID=2 HOW=BOTH
```

A successful result is indicated by a message that is similar to the following message:

```
BNH614I SHUTDOWN REQUEST ON SOCKET 2 COMPLETED SUCCESSFULLY
```

Closing a Socket

When all operations with a client or server socket have finished, the socket can be closed to free the socket and the associated resources. For example, to close client socket 2 for which the connection was previously ended, use the following command:

```
SOCKET TYPE=CLOSE SOCKID=2
```

A successful result is indicated by a message that is similar to the following message:

```
BNH614I CLOSE REQUEST ON SOCKET 2 COMPLETED SUCCESSFULLY
```

Creating a Simple Client or Concurrent Server

This section shows the SOCKET commands that you can use to implement stream applications, both a simple client and a simple concurrent server. More complex applications can use additional SOCKET commands to gather information, set socket options, and perform other functions.

Simple Client

The following steps implement a simple client. The basic steps are to connect to a server, send data to it, and end the connection.

1. Initialize the socket interface with the following command:

```
SOCKET TYPE=INIT TCPNAME=stackname
```

2. Obtain a stream socket with one of the following command:

```
SOCKET TYPE=SOCKET SOCKTYPE=STREAM FAMILY=INET6  
SOCKET TYPE=SOCKET SOCKTYPE=STREAM FAMILY=INET
```

3. Connect to a server at the specified IP address and port with one of the following commands:

```
SOCKET TYPE=CONNECT SOCKID=x ADDRESS=ipv6_addr PORT=nnnnn  
SOCKET TYPE=CONNECT SOCKID=x ADDRESS=ipv4_addr PORT=nnnnn
```

4. Send data to the server by using the following command (where *data_source* might be, for example, "VAR *varname*" or "STEM *stemname* . | COLLECT" or "LIT /*data*/"). A PIPE command is used because it can provide data in the manner required by the SOCKET TYPE=SEND command.

```
PIPE data_source | NETV SOCKET TYPE=SEND SOCKID=x | CORR 5 | CONS
```

Because this is a stream application, not all of the data might be sent with a single call of SOCKET TYPE=SEND. The application needs to check the output of the SOCKET TYPE=SEND command, and, if not all of the bytes were sent, the application needs to send again, starting with the first previously unsent byte.

5. End the connection (both directions of the data flow) with the following command:

```
SOCKET TYPE=SHUTDOWN SOCKID=x HOW=BOTH
```

6. Close the socket with the following command:

```
SOCKET TYPE=CLOSE SOCKID=x
```

7. End the socket interface with the following command:

```
SOCKET TYPE=TERM
```

The CNMSMSIP sample contains an example of a simple TCP client. This example sends a CP-MSU (control point management services unit) that contains an SNA alert. For more information, see [“Sending SNA Alerts”](#) on page 124.

Simple Concurrent Server

The following steps implement a simple concurrent server that receives data from clients:

1. Initialize the socket interface with the following command:

```
SOCKET TYPE=INIT TCPNAME=stackname
```

2. Obtain a datagram socket with one of the following commands:

```
SOCKET TYPE=SOCKET SOCKTYPE=STREAM FAMILY=INET6  
SOCKET TYPE=SOCKET SOCKTYPE=STREAM FAMILY=INET
```

3. Bind the socket to an IP address and port with one of the following commands. To receive connection requests regardless of the interface over which they are sent, bind the socket to :: (IN6ADDR_ANY) for a socket that is obtained by using FAMILY=INET6 or 0.0.0.0 (INADDR_ANY) for a socket that is obtained by using FAMILY=INET.

```
SOCKET TYPE=BIND SOCKID=x ADDRESS=ipv6_addr PORT=nnnnn  
SOCKET TYPE=BIND SOCKID=x ADDRESS=ipv4_addr PORT=nnnnn
```

4. Establish the socket as a *listening socket*, that is, a socket that is ready to receive connection requests, with the following command:

```
SOCKET TYPE=LISTEN SOCKID=x
```

5. Wait for activity (READ event) on the listening socket with the following command. A READ event for the listening socket is a connection request from a client. When a READ event occurs, the application can proceed to the next command.

```
SOCKET TYPE=SELECT READ=x
```

6. Accept a connection request with the following command. If this succeeds, socket *y* is created for the server to use to handle communication with the client. In this example, the client is expected to send data, so the application can proceed to the next command.

```
SOCKET TYPE=ACCEPT SOCKID=x
```

7. The following command waits for READ events for both the listening socket and the socket that is used for communication with the client. A READ event for socket x is another connection request. A READ event for socket y might be that data is available to be received or that the connection has ended.

```
SOCKET TYPE=SELECT READ=( $x,y$ )
```

8. If the server awakens for a READ event for socket y , use the following command. Because this is a stream application, a single SOCKET TYPE=RECV command might not receive all the data that was sent by the client. A good practice is to use the SOCKET TYPE=SELECT command again (step “7” on page 143), listening for events on both sockets.

```
SOCKET TYPE=RECV SOCKID= $y$ 
```

9. If the SOCKET TYPE=RECV command finishes with the following message, this indicates that the client closed the connection:

```
BNH618I  RECV REQUEST ON SOCKET  $y$  FAILED. CONNECTION CLOSED
```

In this case, the server can close the socket with the following command and then wait for activity again (step “5” on page 142):

```
SOCKET TYPE=CLOSE SOCKID= $y$ 
```

10. However, if the server is responsible for ending the connection, the server can issue the following commands to end the connection and close the socket:

```
SOCKET TYPE=SHUTDOWN SOCKID= $y$   
SOCKET TYPE=CLOSE SOCKID= $y$ 
```

11. If the server has no other sockets for communication with clients, it can wait for the next connection request with a SOCKET TYPE=SELECT command (step “5” on page 142) or, if the socket interface is to be ended, go to the next step.
12. When the socket interface is to be ended, use the following commands to close the listening socket and stop the socket interface:

```
SOCKET TYPE=CLOSE SOCKID= $x$   
SOCKET TYPE=TERM
```

Chapter 14. Verifying and Converting IP Addresses

This section contains Programming Interface information.

The NetView program provides functions that you can use to verify whether an IP address is valid and to convert an IP address from one form to another so that you can use it in socket calls to z/OS or in automation. Conversion might be from one text presentation form to another text presentation form, from a text presentation form to binary, or from binary to a text presentation form.

In REXX programs, you can use the IPXLATE function to validate an IP address in presentation form and convert it to another format, either a standard format or a compressed format. For detailed information about the IPXLATE function to process IP addresses, see *IBM Z NetView Programming: REXX and the NetView Command List Language*.

In HLL command procedures, you can use the CNMIPXL (CNMIPXLATE) HLL service routine to validate a presentation form IP address and to convert it to another format, either a standard format or a compressed format. You can also use this service routine to convert an IP address in binary form to a presentation form of the IP address or to convert an IP address from text presentation to binary form. For detailed information about the CNMIPXL (CNMIPXLATE) service routine, see *IBM Z NetView Programming: PL/I and C*.

The following topics describe these functions:

- [“IPXLATE\(\) REXX Function” on page 145](#)
- [“Cnmipxl C Macro” on page 147](#)
- [“CNMIPXLATE NetView PL/I Macro” on page 151](#)

IPXLATE() REXX Function

You can use the IPXLATE() REXX function to verify that a string is a valid presentation form of an IP address. If the string is a valid presentation form of an IP address, you can also convert the string to another presentation form of that IP address. The following examples show how you can use the IPXLATE() function.

IPXLATE() Example: Verification

You can use the IPXLATE() function to verify whether a string is a presentation form of an IP (IPv4 or IPv6) address. To do that, use the VERIFY option:

```
IPXLATE('VERIFY','10.163.17.1')
```

Because the specified string is a presentation form of an IP address, the following result string is returned:

```
0
```

Suppose that the string to be verified is not a presentation form of an IP address, as shown here:

```
IPXLATE('VERIFY','yourhost.yourcompany.com')
```

In this case, a non-zero result string such as the following string is returned:

```
25
```

IPXLATE() Example: Conversion of One Presentation Form to Another

You can use the IPXLATE() function to convert one presentation form of an IP address to another presentation form of the IP address. One reason to do this is to obtain an IP address with a consistent format that can be passed to some form of security verification, such as the AUTHCHKX() REXX function. To convert the 010.163.017.001 string to another presentation form, use the STANDARD option:

```
IPXLATE('STANDARD','010.163.017.001')
```

Because this string is a presentation form of an IPv4 address, the following result string is returned:

```
0 10.163.17.1
```

In this result string, 0 indicates a successful conversion, and the rest of the result string (10.163.17.1) is the returned presentation form of the IP address.

Suppose that you want to convert the following IPv6 address:

```
IPXLATE('STANDARD','0000:0000:0000:0000:0000:0000:0000:0001')
```

The following result string is returned:

```
0 0:0:0:0:0:0:0:1
```

When presenting an IPv6 address, in particular, you might want to remove a large group of consecutive zeros and show the address in compressed form. To convert an IPv6 address to the compressed form, use the COMPRESS option:

```
IPXLATE('COMPRESS','0000:0:0000:0:0000:0:0:0001')
```

For this conversion request, the following result string is returned:

```
0 ::1
```

To decompress a compressed form IPv6 address, use the STANDARD option:

```
IPXLATE('STANDARD','::1')
```

The following result string is returned:

```
0 0:0:0:0:0:0:0:1
```

Because an IPv4-mapped IPv6 address essentially maps an IPv4 address into an IPv6 network (making it usable with an AF_INET6 socket, for example), you can use the IPXLATE() function to convert this form to a standard presentation form IPv4 address, for example:

```
IPXLATE('STANDARD','::FFFF.10.163.17.1')
```

The following result string is returned:

```
0 10.163.17.1
```

If the string to be converted is not a presentation form of an IP address, the result string has only a return code indicating the problem. Suppose that you request the following conversion:

```
IPXLATE('STANDARD','yourhost.yourcompany.com')
```

Because this string is not a presentation form of an IP address, the following result string is returned:

```
25
```

Cnmipxl C Macro

You can use the Cnmipxl C macro to request IP address translation and verification services. This macro sets up a call to the Cnmipxl() function and passes to it the address of Hlbptr, which has the address of the DSIHLB control block, and any user-specified parameters. The DSICCALL header in the SCNMMAC1 data set shows how this works. The following examples illustrate several uses of the Cnmipxl macro for IP address translation and verification service.

Cnmipxl Example: Verification

You can use the Cnmipxl macro to verify whether a string passed is a presentation form of an IP (IPv4 or IPv6) address:

```
void *pInData;
int lenInData;
void *ppInData = (void *)&(pInData);
char MAYBE_PREP_IPV4[] = "10.163.17.1";
void *pOutData;
int lenOutData;
void *ppOutData = (void *)&(pOutData);

pInData = (void *)&(MAYBE_PREP_IPV4[0]);
lenInData = strlen((char *)pInData);

/* Checks to see if the string that was passed is a valid IP address. The */
/* two output parameters, ppOutdata and lenOutData, are ignored when the */
/* function is PCHECK. */

Cnmipxl(PCHECK,
        ppInData,
        lenInData,
        ppOutData,
        lenOutData);
```

If the string to be checked is a presentation form of IP address, the Hlbrc field has a value of 0 (CNM_GOOD). If the string is not a presentation form of an IP address, the Hlbrc field has a non-zero value.

Cnmipxl Examples: Conversion of One Presentation Form to Another

You can use the Cnmipxl macro to convert a presentation form of IP address to another presentation form of IP address. You might want to ensure that an IP address is in a standard format, perhaps for use in authorization checking, or in a compressed format.

Upon completion, if the string is a valid presentation form of IP address and the result fits in the output area, the Hlbrc field has a value of 0 (CNM_GOOD). If the string is a valid presentation form of IP address but the output area is too small, the Hlbrc field has a value of 40 (CNM_DATA_TRUNC), the output area is filled, and the Hlbleng field contains the actual length that is required to hold the output. If the string is not a valid presentation form of IP address, the Hlbrc field has another non-zero value.

Cnmipxl Example 1: Conversion of One Presentation Form to Another

The following example requests the conversion of one presentation form to another:

```
void *pInData;
int lenInData;
void *ppInData = (void *)&(pInData);
char MAYBE_PREP_IPV4[] = "010.163.017.001";
void *pOutData;
int lenOutData;
void *ppOutData = (void *)&(pOutData);
char outPres[45]; /* Room for maximum size presentation form IP address */

pInData = (void *)&(MAYBE_PREP_IPV4[0]);
lenInData = strlen((char *)pInData);
pOutData = (void *)&(outPres[0]);
lenOutData = sizeof(outPres);

/* Convert the input presentation form IP address to another presentation */
```

```

/* form IP address. With the P2PV4 conversion, IPv4, IPv4-mapped IPv6, and */
/* IPv4-compatible IPv6 addresses are all converted to the dotted decimal */
/* presentation form of an IPv4 address (with leading zeros removed). IPv6 */
/* addresses are converted to the standard colon-hex forms. */

Cnmipxl(P2PV4,
        ppInData,
        lenInData,
        ppOutData,
        lenOutData);

```

Upon completion, the Hlbrc field has a value of 0, outPres contains "10.163.17.1", and the Hlbleng field has a value of 11:

Cnmipxl Example 2: Conversion of One Presentation Form to Another

The following example also requests the conversion of one presentation form to another:

```

void *pInData;
int lenInData;
void *ppInData = (void *)&(pInData);
char MAYBE_PREP_IPV6[] = "3F10:0000:0000:0000:010:163:017:0001";
void *pOutData;
int lenOutData;
void *ppOutData = (void *)&pOutData;
char outPres[45]; /* Room for maximum size presentation form IP address */

pInData = (void *)&(MAYBE_PREP_IPV6[0]);
lenInData = strlen((char *)pInData);
pOutData = (void *)&(outPres[0]);
lenOutData = sizeof(outPres);

/* Convert the input presentation form IP address to the compressed */
/* presentation form IP address. With the P2PCOMP conversion, IPv4 */
/* addresses are returned in the standard dotted decimal form, and */
/* IPv4-mapped IPv6, IPv4-compatible IPv6, and IPv6 addresses are returned in */
/* the compressed forms (if possible). */

Cnmipxl(P2PCOMP,
        ppInData,
        lenInData,
        ppOutData,
        lenOutData);

```

Upon completion, the Hlbrc field has a value of 0, outPres contains "3F10::10:163:17:1", and the Hlbleng field has a value of 17.

Cnmipxl Examples: Conversion of Presentation Form to Binary

You can use the Cnmipxl macro to convert a presentation form of IP address to binary (network byte order).

Cnmipxl Example 1: Conversion of Presentation Form to Binary

The following example requests the conversion of a presentation form IPv4 address to a 4-byte binary IPv4 address:

```

void *pInData;
int lenInData;
void *ppInData = (void *)&(pInData);
char MAYBE_PREP_IPV4[] = "10.163.17.1";
char binOut[16];
void *pOutData;
int lenOutData;
void *ppOutData = (void *)&pOutData;

pInData = (void *)&(MAYBE_PREP_IPV4[0]);
lenInData = strlen((char *)pInData);
pOutData = (void *)&(binOut[0]);
lenOutData = sizeof(binOut);

/* Request conversion of a prospective presentation form IPv4 address to a */
/* 4-byte binary IPv4 address (P2NV4). If successful, the first 4 bytes of */
/* the binOut field contain the IPv4 address. This binary IPv4 address is */
/* appropriate for use in the sin_addr field of a sockaddr_in structure */

```



```

/* that is used with AF_INET z/OS UNIX C/C++ sockets. */
Cnmipxl(P2NV4,
        ppInData,
        lenInData,
        ppOutData,
        lenOutData);

```

Upon completion, the Hlbrc field has a value of 0 (CNM_GOOD) if the conversion succeeded and a non-zero value if it did not. If the output area is not large enough to hold the result, the output area is filled, the Hlbrc field has a value of 40 (CNM_DATA_TRUNC), and the Hlbleng field has a value of the actual length that is required to hold the result.

Cnmipxl Example 2: Conversion of Presentation Form to Binary

The following example requests the conversion of a presentation form IPv4 address to a 16-byte binary IPv6 address:

```

/* Request conversion of a prospective presentation form IPv4 address to a 16-byte binary IPv6 address (P2NV6). If successful, the binOut field contains a binary IPv4-mapped IPv6 address that is appropriate for use in the sin6_addr field of a sockaddr_in6 structure that is used with AF_INET6 z/OS UNIX C/C++ sockets. */
pInData = (void *)&MAYBE_PRE_IPV4[0];
lenInData = strlen((char *)pInData);
pOutData = (void *)&binOut[0];
lenOutData = sizeof(binOut);

Cnmipxl(P2NV6,
        ppInData,
        lenInData,
        ppOutData,
        lenOutData);

```

Cnmipxl Examples: Conversion of Binary to Presentation Form

You can use the Cnmipxl macro to convert an IP address in binary (network byte order) to a presentation form.

The binary (network byte order) IP address that is passed must be either 4 bytes in length (indicating an IPv4 address) or 16 bytes in length (indicating an IPv6, IPv4-mapped, or IPv4-compatible address). Any length value other than those two is invalid input to the Cnmipxl() macro, and the results are that the Hlbrc field has a non-zero value and no conversion occurs.

If the binary (network byte order) IP address is valid, but the output area is not large enough to hold the presentation form result, then Cnmipxl() fills the output area with what can be placed there, the Hlbrc field has a value of 40 (CNM_DATA_TRUNC), and the Hlbleng field contains the actual length that is required to hold the presentation form of the IP address.

Cnmipxl Example 1: Conversion of Binary to Presentation Form

The following example requests conversion of a binary IP address to a standard presentation form. Upon completion, outData contains a presentation form dotted decimal IP address, namely "10.163.17.1", the Hlbleng field has a value of 11, and the Hlbrc field has a value of 0.

```

void *pInData;
int lenInData;
void *ppInData = (void *)&(pInData);
unsigned int BIN_IPV4 = 0x0AA31101;
char charOut[45]; /* Room for the maximum size presentation form */
void *pOutData;
int lenOutData;
void *ppOutData = (void *)&pOutData;

pInData = (void *)&BIN_IPV4;
lenInData = sizeof(BIN_IPV4);
pOutData = (void *)&charOut[0];
lenOutData = sizeof(charOut);

/* Request conversion of a binary (network byte order) IP address to a presentation form */

```

```

/* standard presentation form (N2PSTD). If successful, the charOut field */
/* contains a string that represents the IP address. Because the input */
/* binary IP address is a 4-byte value, it must be an IPv4 address and, */
/* therefore, the presentation form should be dotted decimal. */

Cnmipxl(N2PSTD,
        ppInData,
        lenInData,
        ppOutData,
        lenOutData);

```

Cnmipxl Example 2: Conversion of Binary to Presentation Form

The following example requests conversion of an IPv6 address. Upon completion, outData contains "4BBE:4DB5:0:0:FOCA:530F:1049:4AC", the Hlbleng field has a value of 32, and the Hlbrc field has a value of 0. Notice that the standard form ensures that leading zeros are removed.

```

unsigned int BIN_IPV6[4] = {0x4BBE4DB5,
                           0x00000000,
                           0xF0CA530F,
                           0x104904AC};

pInData = (void *)&BIN_IPV6[0];
lenInData = 4 * sizeof(unsigned int);

/* Request conversion of a binary (network byte order) IP address to the */
/* standard presentation form (N2PSTD). If successful, the charOut field */
/* contains a string that represents the IP address. Because the input */
/* binary IP address is a 16-byte value that does not contain the special */
/* values for IPv4-mapped or IPv4-compatible IPv6 addresses, it must be an */
/* IPv6 address and, therefore, the presentation form should be colon-hex. */

Cnmipxl(N2PSTD,
        ppInData,
        lenInData,
        ppOutData,
        lenOutData);

```

Cnmipxl Example 3: Conversion of Binary to Presentation Form

The following function call requests a compressed presentation form. Upon completion, outData contains "4BBE:4DB5::FOCA:530F:1049:4AC", the Hlbleng field has a value of 29, and the Hlbrc field has a value of 0. Note the double-colon (::) that signifies the compression of multiple groups of zeros in the IPv6 address.

```

Cnmipxl(N2PCOMP,
        ppInData,
        lenInData,
        ppOutData,
        lenOutData);

```

Cnmipxl Example 4: Conversion of Binary to Presentation Form

The following function call requests the conversion of a binary IPv4-mapped IPv6 address to the standard (text) presentation form by using the N2PSTD option:

```

unsigned int BIN_IPV6[4] = {0x00000000,
                           0x00000000,
                           0x0000FFFF,
                           0xAA31101};

pInData = (void *)&BIN_IPV6[0];
lenInData = 4 * sizeof(unsigned int);

```

Suppose that you pass that binary IP address and use the N2PSTD function in the following way. Upon completion, outData contains "10.163.17.1", the Hlbleng field has a value of 11, and the Hlbrc field has a value of 0. This is because the IPv4-mapped IPv6 address is essentially an IPv4 address and the standard presentation form of an IPv4 address is a string representing dotted decimal.

```

Cnmipxl(N2PSTD,
        ppInData,
        lenInData,

```

```
ppOutData,
lenOutData);
```

CNMIPXLATE NetView PL/I Macro

You can use the CNMIPXLATE PL/I macro to request IP address translation and verification services. The macro sets up a call to an entry named CNMIPXL and passes to it both HLBPTR (the address of DSIHLB) and any user-specified parameters. The DSIPHLLS member in the SCNMMAC1 data set shows how this works. The following examples show several ways to use the IP address translation and verification service.

CNMIPXLATE Example: Verification

You can use the CNMIPXLATE macro to verify whether a string passed to it is really a presentation form of an IP (IPv4 or IPv6) address. Upon completion, if the string is a presentation form of an IP address, the HLBRC field has a value of 0 (CNM_GOOD). If the string is not a presentation form of an IP address, the HLBRC field has a non-zero value.

```
DCL pInData PTR;
DCL lenInData FIXED BINARY(31,0);
DCL IN_ADDR1 CHAR(15) INIT('010.163.017.001');

pInData = ADDR(IN_ADDR1);
lenInData = LENGTH(IN_ADDR1);

CNMIPXLATE TYPE(PCHECK)
           INDATA(pInData)
           INLEN(lenInData);
```

CNMIPXLATE Examples: Conversion of One Presentation Form to Another

You can use the CNMIPXLATE PL/I macro to convert a presentation form of IP address to another presentation form of IP address. You might want to ensure that the IP address is in a standard format, perhaps for use in authorization checking, or a compressed format.

Upon completion, the HLBRC field has a value of 0 (CNM_GOOD), if the string is a valid presentation form of IP address and the result fits in the output area. If the string is a valid presentation form of IP address but the output area is too small, the HLBRC field has a value of 40 (CNM_DATA_TRUNC), the output area is filled, and the HLBLENG field contains the actual length that is required to hold the output. If the string is not a valid presentation form of IP address, the HLBRC field has another non-zero value.

CNMIPXLATE Example 1: Conversion of One Presentation Form to Another

The following example converts one presentation form to another. Upon completion, the HLBRC field has a value of 0, outPres contains the string "10.163.17.1", and the HLBLENG field has a value of 11.

```
DCL pInData PTR;
DCL lenInData FIXED BINARY(31,0);
DCL MAYBE_PRE_IPV4 CHAR(15) INIT('010.163.017.001');
DCL pOutData PTR;
DCL lenOutData FIXED BINARY(31,0);
DCL outPres CHAR(45); /* Room for maximum size presentation form IP address */

pInData = ADDR(MAYBE_PRE_IPV4);
lenInData = LENGTH(MAYBE_PRE_IPV4);
pOutData = ADDR(outPres);
lenOutData = LENGTH(outPres);

/* Convert the input presentation form IP address to another presentation */
/* form IP address. With the P2PV4 conversion, IPv4, IPv4-mapped IPv6, and */
/* IPv4-compatible IPv6 addresses are all converted to the dotted decimal */
/* presentation form of an IPv4 address (with leading zeros removed). IPv6 */
/* addresses are converted to the standard colon-hex form. */

CNMIPXLATE TYPE(P2PV4)
           INDATA(pInData)
           INLEN(lenInData)
```

```

OUTDATA(pOutData)
OUTLEN(lenOutData);

```

CNMIPXLATE Example 2: Conversion of One Presentation Form to Another

The following example converts a presentation form to a compressed presentation form. Upon completion, the HLBRC field has a value of 0, outPres contains the string "3F10::10:163:17:1", and the HLBLENG field has a value of 17.

```

DCL pInData PTR;
DCL lenInData FIXED BINARY(31,0);
DCL MAYBE_PRES_IPV6 CHAR(36) INIT('3F10:0000:0000:0000:010:163:017:0001');
DCL pOutData PTR;
DCL lenOutData FIXED BINARY(31,0);
DCL outPres CHAR(45); /* Room for maximum size presentation form IP address */

pInData = ADDR(MAYBE_PRES_IPV6);
lenInData = LENGTH(MAYBE_PRES_IPV6);
pOutData = ADDR(outPres);
lenOutData = LENGTH(outPres);

/* Convert the input presentation form IP address to the compressed */
/* presentation form IP address. With the P2PCOMP conversion, IPv4 */
/* addresses are returned in the standard dotted decimal form, and */
/* IPv4-mapped IPv6, IPv4 compatible IPv6, and IPv6 addresses are returned in */
/* the compressed forms (if possible). */

CNMIPXLATE TYPE(P2PCOMP)
            INDATA(pInData)
            INLEN(lenInData)
            OUTDATA(pOutData)
            OUTLEN(lenOutData);

```

CNMIPXLATE Examples: Conversion of Presentation Form to Binary

You can use the CNMIPXLATE macro to convert a presentation form of IP address to binary (network byte order). Upon completion, the HLBRC field has a value of 0 (CNM_GOOD) if the conversion succeeded and a non-zero value if it did not. If the output area is not large enough to hold the result, the output area is filled, the HLBRC field has a value of 40 (CNM_DATA_TRUNC), and the HLBLENG field is set to the actual length that is required for holding the result.

CNMIPXLATE Example 1: Conversion of Presentation Form to Binary

The following example requests conversion of a presentation form IPv4 address to a 4-byte binary IPv4 address:

```

DCL pInData PTR;
DCL lenInData FIXED BINARY(31,0);
DCL MAYBE_PRES_IPV4 CHAR(11) INIT('10.163.17.1');
DCL binOut CHAR(16);
DCL pOutData PTR;
DCL lenOutData FIXED BINARY(31,0);

pInData = ADDR(MAYBE_PRES_IPV4);
lenInData = LENGTH(MAYBE_PRES_IPV4);
pOutData = ADDR(binOut);
lenOutData = LENGTH(binOut);

/* Request conversion of a prospective presentation form IPv4 address to a */
/* 4-byte binary IPv4 address (P2NV4). If successful, the first 4 bytes of */
/* the binOut field contain the IPv4 address. This binary IPv4 address is */
/* appropriate for use in the sin_addr field of a sockaddr_in structure that */
/* is used with AF_INET z/OS UNIX C/C++ sockets, if an interlanguage call */
/* between PL/I and C/C++ is used to gain access to those services. */

CNMIPXLATE TYPE(P2NV4)
            INDATA(pInData)
            INLEN(lenInData)
            OUTDATA(pOutData)
            OUTLEN(lenOutData)

```

CNMIPXLATE Example 2: Conversion of Presentation Form to Binary

The following example requests conversion of a presentation form IPv4 address to a 16-byte binary IPv6 address:

```
/* Request conversion of a prospective presentation form IPv4 address to a  */
/* 16-byte binary IPv6 address (P2NV6). If successful, the binOut field  */
/* contains a binary IPv4-mapped IPv6 address, appropriate for use in the  */
/* sin6_addr field of a sockaddr_in6 structure that is used with AF_INET6  */
/* z/OS UNIX C/C++ sockets.  */
pInData = ADDR(MAYBE_PREP_IPV4);
lenInData = LENGTH(MAYBE_PREP_IPV4);
pOutData = ADDR(binOut);
lenOutData = LENGTH(binOut);

CNMIPXLATE TYPE(P2NV6)
            INDATA(pInData)
            INLEN(lenInData)
            OUTDATA(pOutData)
            OUTLEN(lenOutData);
```

CNMIPXLATE Examples: Conversion of Binary to Presentation Form

You can use the CNMIPXLATE macro to convert an IP address in binary to a presentation form.

The binary (network byte order) IP address that is passed must be either 4 bytes in length (indicating an IPv4 address) or 16 bytes in length (indicating an IPv6, IPv4-mapped, or IPv4-compatible address). Any length value other than these two is invalid input to the CNMIPXL macro, meaning that the HLBRC field has a non-zero value and no conversion occurs.

If the binary (network byte order) IP address is valid, but the output area is not large enough to hold the presentation form result, then CNMIPXL fills the output area with what can be placed there, the HLBRC field has a value of 40 (CNM_DATA_TRUNC), and the HLBLEN field contains the actual length that is required to hold the presentation form of the IP address.

CNMIPXLATE Example 1: Conversion of Binary to Presentation Form

The following example requests the conversion of a binary (network byte order) IP address to the standard presentation form. Upon completion, outData contains a presentation form dotted decimal IP address, namely "10.163.17.1", the HLBLEN field has a value of 11, and the HLBRC field has a value of 0.

```
DCL pInData PTR;
DCL lenInData FIXED BINARY(31,0);
DCL BIN_IPV4 CHAR(4) INIT('0AA31101'X);
DCL charOut CHAR(45); /* Room for the maximum size presentation form */
DCL pOutData PTR;
DCL lenOutData FIXED BINARY(31,0);

pInData = ADDR(BIN_IPV4);
lenInData = LENGTH(BIN_IPV4);
pOutData = ADDR(charOut);
lenOutData = LENGTH(charOut);

/* Request conversion of a binary (network byte order) IP address to the  */
/* standard presentation form (N2PSTD). If successful, the charOut field  */
/* contains a string that represents the IP address. Because the input  */
/* binary IP address is a 4-byte value, it must be an IPv4 address and,  */
/* therefore, the presentation form should be dotted decimal.  */

CNMIPXLATE TYPE(N2PSTD)
            INDATA(pInData)
            INLEN(lenInData)
            OUTDATA(pOutData)
            OUTLEN(lenOutData);
```

CNMIPXLATE Example 2: Conversion of Binary to Presentation Form

The following example requests the conversion of an IPv6 address. Upon completion, outData contains "4BBE:4DB5:0:0:FOCA:530F:1049:4AC", the HLBLENG field has a value of 32, and the HLBRC field has a value of 0. Note that the standard form ensures that leading zeros are removed.

```
DCL BIN_IPV6 CHAR(16) INIT('4BBE4DB500000000F0CA530F104904AC'X);

pInData = ADDR(BIN_IPV6);
lenInData = LENGTH(BIN_IPV6);

/* Request conversion of a binary (network byte order) IP address to the      */
/* standard presentation form (N2PSTD).  If successful, the charOut field      */
/* contains a string that represents the IP address.  Because the input        */
/* binary IP address is a 16-byte value that does not contain the special    */
/* values for IPv4-mapped or IPv4-compatible IPv6 addresses, it must be an    */
/* IPv6 address and, therefore, the presentation form should be colon-hex.    */
/*                                                                            */

CNMIPXLATE TYPE(N2PSTD)
            INDATA(pInData)
            INLEN(lenInData)
            OUTDATA(pOutData)
            OUTLEN(lenOutData);
```

CNMIPXLATE Example 3: Conversion of Binary to Presentation Form

Suppose that the function call requests a compressed presentation form, as in the following example. Upon completion, outData contains "4BBE:4DB5::FOCA:530F:1049:4AC", the HLBLENG field has a value of 29, and the HLBRC field has a value of 0. Note that the double-colon (::) indicates the compression of multiple groups of zeros in the IPv6 address.

```
CNMIPXLATE TYPE(N2PCOMP)
            INDATA(pInData)
            INLEN(lenInData)
            OUTDATA(pOutData)
            OUTLEN(lenOutData);
```

CNMIPXLATE Example 4: Conversion of Binary to Presentation Form

Suppose that the binary IP address to be converted is an IPv4-mapped IPv6 address.

```
DCL BIN_IPV6 CHAR(16) INIT('00000000000000000000FFFF0AA31101'X);
```

Suppose that you pass that binary IP address and use the N2PSTD function call, as in the following example. Upon completion, outData contains "10.163.17.1", the HLBLENG field has a value of 11, and the HLBRC field has a value of 0. This is because the IPv4-mapped IPv6 address is essentially an IPv4 address and the standard presentation form of an IPv4 address is a string that represents dotted decimal.

```
CNMIPXLATE TYPE(N2PSTD)
            INDATA(pInData)
            INLEN(lenInData)
            OUTDATA(pOutData)
            OUTLEN(lenOutData);
```

Chapter 15. Using IP Servers

The NetView program supplies several TCP/IP services that are provided as server and client functions. Server and client functions are available for the REXEC, RSH, and syslog services. The REXEC and RSH services provide remote command processing support. The syslog service provides remote logging.

The DSIRXEXC task and REXEC statements in the CNMSTYLE member are used to initialize the REXEC server. To prevent unauthorized TCP/IP hosts from connecting to the NetView program by using the REXEC server, use the JAVAACC command; for more information, see the *IBM Z NetView Security Reference*.

The DSIRSH task and RSH statements in the CNMSTYLE member are used to initialize the RSH server. To set the security access for the RSH server, use the DSIRHOST sample; for more information, see the sample.

The DSIIPLOG task and IPLOG statements in the CNMSTYLE member are used to initialize the syslog server. To grant permission to a remote host to send syslog messages to the NetView syslog server, use the REGIP command; for more information, see the online help.

For more information about configuring these TCP/IP services, see *IBM Z NetView Installation: Configuring Additional Components*. For more information about the REXEC, RSH, and IPLOG statements, see the *IBM Z NetView Administration Reference*. For information about changing CNMSTYLE statements, see *IBM Z NetView Installation: Getting Started*.

Appendix A. Discovery Manager Data

Table 6 on page 157 lists the discovery manager data that is collected by using the CNMEERSC command, automation and sampling, and configuration and status commands.

Table 6. Discovery Manager Data Collection							
	System ^{“1”} on page 157	Application	Stack	Interface	OSA Channel and Port	HiperSockets	Telnet Server and Port
Rediscovery Command							
CNMEERSC ENTITY=ALL	x	x	x	x	x	x	x
CNMEERSC ENTITY=APPL		x					
CNMEERSC ENTITY=STACK			x	x	x	x	x
CNMEERSC ENTITY=SYSTEM	x						
Automation and Sampling ^{“2”} on page 157							
Application sampling		x					
Interface sampling			x	x	x	x	
Stack up/down automation			x	x	x	x	x
Telnet server sampling			x				x
Telnet server up/down automation			x				x
Commands ^{“3”} on page 157							
HIPERSOC			x	x	x	x	
IFSTAT			x	x			
NVSTAT		x					
OSAPORT			x	x	x	x	
STACSTAT			x				
TELNSTAT			x				x
TNPTSTAT			x				x
Notes: 1. System includes the sysplex, the z/OS image, and the coupling facility. 2. For automation and sampling, interrelated data is collected to construct the data for the requested data type. Only the requested data type is updated. 3. For commands, interrelated data is collected to construct the data for the requested data type. Only the requested data type is displayed.							

Appendix B. Customizing the SNMP Group Definition File (FKXSNMP)

Note: This function no longer requires AON. The information about this function is included in both *IBM Z NetView IP Management* and the *IBM Z NetView User's Guide: Automated Operations Network*.

Use the following rules for creating new entries in the SNMP group definition (FKXSNMP) file:

- The Group name must be from 1 to 15 characters and must start in column 1.
- The Group name cannot be duplicated.
- There must be at least one space between the Group Name, the GROUP, the Group type, and the base MIB for Table type Groups.
- There can be up to three lines of abstract definition for a Group. The abstract lines can be up to 72 positions and must start with a question mark (?) in column 1.
- The Abstract lines for a Group must follow the GROUP statement for the group.
- Valid Group types are:
 - LIST
 - LIST+
 - TABLE
 - WALK

A **LIST** group type must include the EXACT MIB variable names to be collected.

A **LIST+** group works almost the same as a LIST type Group, but enables the definition of variable data.

The LIST+ group enables you to specify a variable field to be appended to the list of MIB objects in the group. This enables a single group definition to be used for various MIB object groups. For example, a group can contain objects that relate to a specific interface number. If you use traditional LIST type groups, you need multiple groups, one to define each interface. A LIST+ group can be defined to ask prompt for an interface number, when selected, enabling only one group definition to be needed. LIST+ adds keywords that are used to set up the variable data. All of these keywords must start in column 1.

PANELINPUT

Defines this as a LIST+ group

PANELCONST

A user-customizable field that is displayed in the input panel and must be delineated with double quotation marks ("")

PANELVAR

An input field where the data is collected from the screen, for example:

```
PANELCONST "PLEASE ENTER AN INTERFACE NUMBER:
"PANELVAR "___"
```

Displays as:

```
PLEASE ENTER AN INTERFACE NUMBER:
```

```
---
```

VAR keywords in LIST and LIST+ groups indicate the starting of varbind lists. This helps in parsing in UNIX. VAR must start in column 1.

A **TABLE** group type must have a base variable to start the search. This is the index variable for the table. A Table group type does not need to have variables listed. The table is Walked and all variables in the table are collected. MIB variables listed in this group are for documentation only.

A **WALK** group type must be a known group name (defined in an existing RFC). A walk command is issued against the name. MIB variables listed in this group are for documentation only. Walk groups, as defined in some RFCs, might be much larger than the 15-character limit. To work with this limit, the Full Name for the Walk Group might be entered on the Group line after the WALK type (for example: groupname GROUP WALK veryLongGroupName).

Comments must have an asterisk (*) in column 1.

Notices

This information was developed for products and services offered in the U.S.A.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not give you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing
IBM Corporation
North Castle Drive
Armonk, NY 10504-1785
U.S.A.

For license inquiries regarding double-byte (DBCS) information, contact the IBM Intellectual Property Department in your country or send inquiries, in writing, to:

Intellectual Property Licensing
Legal and Intellectual Property Law
IBM Japan, Ltd.
19-21, Nihonbashi-Hakozakicho, Chuo-ku
Tokyo 103-8510, Japan

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law:

INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement might not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM Web sites are provided for convenience only and do not in any manner serve as an endorsement of those Web sites. The materials at those Web sites are not part of the materials for this IBM product and use of those Web sites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Licensees of this program who wish to have information about it for the purpose of enabling: (i) the exchange of information between independently created programs and other programs (including this one) and (ii) the mutual use of the information which has been exchanged, should contact:

IBM Corporation
224A/101
11400 Burnet Road

Austin, TX 78758
U.S.A.

Such information may be available, subject to appropriate terms and conditions, including in some cases payment of a fee.

The licensed program described in this document and all licensed material available for it are provided by IBM under terms of the IBM Customer Agreement, IBM International Program License Agreement or any equivalent agreement between us.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs. You may copy, modify, and distribute these sample programs in any form without payment to IBM for the purposes of developing, using, marketing, or distributing application programs conforming to IBM's application programming interfaces.

Each copy or any portion of these sample programs or any derivative work, must include a copyright notice as follows:

© (your company name) (year). Portions of this code are derived from IBM Corp. Sample Programs. © Copyright IBM Corp. _enter the year or years_. All rights reserved.

Programming Interfaces

This publication primarily documents information that is NOT intended to be used as Programming Interfaces of IBM Z NetView. This publication also documents intended Programming Interfaces that allow the customer to write programs to obtain the services of IBM Z NetView. This information is identified where it occurs, either by an introductory statement to a chapter or section or by the following marking:

```
Programming Interface information
End of Programming Interface information
```

Trademarks

IBM, the IBM logo, and [ibm.com](http://www.ibm.com)® are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at <http://www.ibm.com/legal/copytrade.shtml>.

Adobe is a trademark of Adobe Systems Incorporated in the United States, and/or other countries.

Linux is a registered trademark of Linus Torvalds in the United States, other countries, or both.

Microsoft and Windows are trademarks of Microsoft Corporation in the United States, other countries, or both.

UNIX is a registered trademark of The Open Group in the United States and other countries.

Other product and service names might be trademarks of IBM or other companies.

Privacy policy considerations

IBM Software products, including software as a service solutions, (“Software Offerings”) may use cookies or other technologies to collect product usage information, to help improve the end user experience, to tailor interactions with the end user or for other purposes. In many cases no personally identifiable information is collected by the Software Offerings. Some of our Software Offerings can help enable you to collect personally identifiable information. If this Software Offering uses cookies to collect personally identifiable information, specific information about this offering’s use of cookies is set forth below.

This Software Offering does not use cookies or other technologies to collect personally identifiable information.

If the configurations deployed for this Software Offering provide you as customer the ability to collect personally identifiable information from end users via cookies and other technologies, you should seek your own legal advice about any laws applicable to such data collection, including any requirements for notice and consent.

For more information about the use of various technologies, including cookies, for these purposes, See IBM’s Privacy Policy at <http://www.ibm.com/privacy> and IBM’s Online Privacy Statement at <http://www.ibm.com/privacy/details> the section entitled “Cookies, Web Beacons and Other Technologies” and the “IBM Software Products and Software-as-a-Service Privacy Statement” at <http://www.ibm.com/software/info/product-privacy>.

Index

A

accessibility [xvi](#)
Advanced Peer-to-Peer Networking [118](#)

B

books
 see publications [xiii](#)
bridge views, IP [99](#), [103](#)
BULKWALK command (SNMP) [110](#)

C

CLOSE command [128](#)
CNMDVIPP data set [46](#)
CNMDVIPS data set [46](#)
CNMEDRCL command [45](#)
CNMEERSC command [21](#)
Cnmipxl
 examples
 conversion [147–149](#)
 verification [147](#)
 using [147](#)
CNMIPXL
 using in HLL command procedures [145](#)
CNMIPXLATE
 examples
 conversion [151–153](#)
 verification [151](#)
 using [151](#)
 using in HLL command procedures [145](#)
CNMSAF2 sample [127](#)
CNMSALRT sample [124](#)
CNMSCAT2 sample [127](#)
CNMSDDCR sample [48](#)
CNMSDVIP sample [48](#)
CNMSDVPC sample [48](#)
CNMSDVPH sample [48](#)
CNMSDVST command [48](#)
CNMSDVST sample [46](#), [48](#)
CNMSHIPR sample [22](#)
CNMSIFST sample [22](#)
CNMSMSGT sample [123](#), [136](#)
CNMSMSIP sample [124](#), [142](#)
CNMSNVST sample [22](#)
CNMSOSAP sample [22](#)
CNMSPLEX sample [48](#)
CNMSSTAC sample [22](#)
CNMSTARG sample [48](#)
CNMSTCPC sample [86](#)
CNMSTNST sample [22](#)
CNMSTPST sample [22](#)
CNMSTYLE
 COMMON.IPPORTMON statements [117](#)
 TESTPORT statements [117](#)
CNMSVPRT sample [48](#)

COLLCTL command [21](#), [45](#)
commands
 BULKWALK (SNMP) [110](#)
 CLOSE [128](#)
 CNMEDRCL [45](#)
 CNMEERSC [21](#)
 CNMSDVST [48](#)
 COLLCTL [21](#), [45](#)
 data collector
 discovery manager [22](#)
 DVIPA [48](#)
 TCP/IP connection [86](#)
 DIS [118](#)
 DVIPALOG [46](#)
 DVIPCONN [48](#)
 DVIPDDCR [48](#)
 DVIPHLTH [48](#)
 DVIPPLEX [48](#)
 DVIPSTAT [48](#)
 DVIPTARG [48](#)
 FKXECSEF [93](#)
 FMTPACKT [61](#)
 GET (SNMP) [108](#)
 GETTOPO [98](#)
 GROUP (SNMP) [110](#)
 HIPERSOC [22](#)
 IFSTAT [22](#)
 IPLOG [119](#)
 IPMAN [113](#)
 IPSTAT [86](#)
 IPTRACE [61](#), [62](#)
 JAVAACC [155](#)
 LIST [16](#)
 LISTTRC [75](#)
 MVSPING [119](#)
 NETSTAT [117](#)
 NVSNNMP [107](#), [120](#)
 NVSTAT [22](#)
 OSAPORT [22](#)
 PING [105](#), [119](#)
 PIPE [135](#)
 PKTS [61](#)
 PKTTRACE [93](#)
 PLEXCTL [16](#)
 REGIP [155](#)
 RESTYLE [17](#), [21](#), [45](#)
 REXEC [119](#)
 RMTCMD [20](#), [119](#)
 RSH [119](#)
 SESS [118](#)
 SET (SNMP) [109](#)
 SNMP [107](#), [120](#)
 SOCKET [120](#), [123](#), [127](#)
 STACSTAT [22](#)
 START [16](#)
 STOP [16](#), [128](#)
 TCPCONN [85](#), [86](#), [89](#)

commands (*continued*)

- TELNSTAT [22](#)
- TESTPORT [117](#)
- TN3270 [120](#)
- TNPTSTAT [22](#)
- TRACERTE [106](#), [120](#)
- TRAP (SNMP) [123](#)
- VARY TCPIP DROP [91](#)
- VIPAROUT [48](#)
- WALK (SNMP) [109](#)
- WINDOW [23](#), [49](#), [86](#)

COMMON.IPPORTMON statements [117](#)

Communications Server, z/OS

- IP sockets API [127](#)

component tracing

- displaying [84](#)
- scheduling [80](#)

CONNSEC [86](#)

conventions

- typeface [xvii](#)

D

data collectors

- discovery manager [22](#)
- DVIPA [48](#)
- TCP/IP connection [86](#)

directory names, notation [xvii](#)

DIS command [118](#)

distributed DVIPA

- statistics [46](#)
- workload balancing [46](#)

Distributed DVIPA Connection Routing workspace

- about [49](#)

distributed DVIPA server health information [48](#)

Distributed DVIPA Server Health workspace

- about [50](#)

Distributed DVIPA Targets workspace

- about [51](#)

DSIIPLOG task [155](#)

DSIRHOST sample [155](#)

DSIRSH task [155](#)

DSIRXEXC task [155](#)

DVIPA

- connections [48](#)
- definition and status [48](#)
- distributed
 - connection routing information [48](#)
 - server health information [48](#)
 - statistics [46](#), [48](#)
- distributed targets [48](#)
- events [45](#)
- rediscovery algorithm [46](#)
- sysplex distributors [48](#)
- updates [45](#)

DVIPA Connections workspace

- about [52](#)

DVIPA Definition and Status workspace

- about [53](#)

DVIPA Stack Summary workspace

- about [56](#)

DVIPA Sysplex Distributors workspace

- about [54](#)

DVIPA.Event.Delay timer [45](#)

DVIPA.Mast.Disc.Delay timer [45](#)

DVIPALOG command [46](#)

DVIPCONN command [48](#)

DVIPDDCR command [48](#)

DVIPHLTH command [48](#)

DVIPPLEX command [48](#)

DVIPSTAT autotask [46](#)

DVIPSTAT command [48](#)

DVIPTARG command [48](#)

E

Enterprise Extender technology [118](#)

environment variables, notation [xvii](#)

Extended SNMP Groups [112](#)

F

FKXECSE command [93](#)

FKXSNMP [159](#)

FMTPACKT command [61](#)

G

GET command (SNMP) [108](#)

GETTOPO command [98](#)

GROUP command (SNMP) [110](#)

H

high-performance routing (HPR) [118](#)

HIPERSOC command [22](#)

HiperSockets

- information

- command [22](#)

- sample [22](#)

HiperSockets Configuration and Status workspace

- about [23](#)

host views, IP [99](#), [103](#)

hub views, IP [99](#), [103](#)

I

IBM Tivoli Network Manager

- view objects [95](#)

IDS (Intrusion Detection Services) [125](#)

IFSTAT command [22](#)

Intrusion Detection Services (IDS) [125](#)

IP

- finding resources [97](#)

- navigating network views [98](#)

IP addresses

- CNMIPXL [145](#)

- CNMIPXLATE [145](#)

- conversion [145](#)

- IPXLATE [145](#)

- verification [145](#)

IP networks

- viewing [98](#)

IP networks view, IP [98](#), [99](#)

IP packet tracing

- displaying [66](#)

IP servers

IP servers (*continued*)

- about [155](#)
- REXEC [155](#)
- RSH [155](#)
- syslog [155](#)

IP subnetwork views, IP [99](#)

IP trace

- accessing [62](#)
- component
 - scheduling [80](#)
- introduction [62](#)
- IP packet
 - displaying [66](#)
- OSA packet
 - displaying [79](#)

IPLOG command [119](#)

IPMAN command [113](#)

IPSTAT command [86](#)

IPTRACE command [61](#), [62](#)

IPXLATE

- examples
 - conversion [146](#)
 - verification [145](#)
- using [145](#)
- using in REXX programs [145](#)

J

JAVAACC command [155](#)

L

LIST command [16](#)

LIST+ group [159](#)

listener, hung [117](#)

LISTTRC command [75](#)

M

manuals

- see publications [xiii](#)

MVSPING command [119](#)

N

NETSTAT command [117](#)

NetView Applications workspace

- about [24](#)

NetView domains

- information [22](#)

network views, navigating

- IP [98](#)

notation

- environment variables [xvii](#)
- path names [xvii](#)
- typeface [xvii](#)

NVSNMP command [107](#), [120](#)

NVSTAT command [22](#)

O

online publications

- accessing [xv](#)

OSA

- port information
 - command [22](#)
 - sample [22](#)

OSA Channels and Ports workspace

- about [25](#)

OSA packet tracing

- displaying [79](#)

OSAPORT command [22](#)

P

path names, notation [xvii](#)

PING command [105](#), [119](#)

PIPE command [135](#)

PKTS command [61](#)

PKTTRACE command [93](#)

PLEXCTL command [16](#)

ports

- critical, monitoring [117](#)

publications

- accessing online [xv](#)
- IBM Z NetView [xiii](#)
- ordering [xvi](#)

R

rediscovery algorithm, DVIPA [46](#)

REGIP command [155](#)

RESTYLE command [17](#), [21](#), [45](#)

REXEC command [119](#)

REXEC server [155](#)

REXEC statements [155](#)

RMTCMD command [20](#), [119](#)

router views, IP [99](#), [103](#)

RSH command [119](#)

RSH server [155](#)

RSH statements [155](#)

S

samples

- CNMSAF2 [127](#)

- CNMSALRT [124](#)

- CNMSCAT2 [127](#)

- CNMSDDCR [48](#)

- CNMSDVIP [48](#)

- CNMSDVPC [48](#)

- CNMSDVPH [48](#)

- CNMSDVST [46](#), [48](#)

- CNMSHIPR [22](#)

- CNMSIFST [22](#)

- CNMSMSGT [123](#), [136](#)

- CNMSMSIP [124](#), [142](#)

- CNMSNVST [22](#)

- CNMSOSAP [22](#)

- CNMSPLEX [48](#)

- CNMSSTAC [22](#)

- CNMSTARG [48](#)

- CNMSTCPC [86](#)

- CNMSTNST [22](#)

- CNMSTPST [22](#)

- CNMSVPRT [48](#)

- samples (*continued*)
 - DSIRHOST [155](#)
 - used by Z NetView Enterprise Management Agent [22](#), [48](#), [86](#)
- security
 - sockets [127](#)
- SESS command [118](#)
- session packet data
 - saving [72](#)
- SET command (SNMP) [109](#)
- SNA over IP [118](#)
- Sniffer [66](#)
- SNMP BULKWALK command [110](#)
- SNMP command [107](#), [120](#)
- SNMP GET command [108](#)
- SNMP GROUP command [110](#)
- SNMP group definition file [159](#)
- SNMP SET command [109](#)
- SNMP TRAP command [123](#)
- SNMP traps, DVIPA [45](#)
- SNMP WALK command [109](#)
- SOCKET command [120](#), [123](#), [127](#)
- sockets
 - Communications Server IP sockets API [127](#)
 - SOCKET command [127](#)
 - TCP/IP applications [127](#)
 - trace entries in NetView trace [127](#)
- Stack Configuration and Status workspace
 - about [26](#)
- STACSTAT command [22](#)
- START command [16](#)
- statements
 - COMMON.IPPORTMON [117](#)
 - IPLOG (syslog) [155](#)
 - IPLOG statements [155](#)
 - REXEC [155](#)
 - RSH [155](#)
 - TCP/IP services
 - syslog [155](#)
- STOP command [16](#), [128](#)
- subnetwork views, IP [102](#)
- switch views, IP [99](#), [103](#)
- syslog server [155](#)
- sysplex
 - view nodes [29](#)
- sysplex distributors, DVIPA [48](#)
- sysplex monitoring messages [45](#)

T

- tasks
 - DSIIPLOG [155](#)
 - DSIRSH [155](#)
 - DSIRXEXC [155](#)
- TCP/IP services
 - about [155](#)
 - REXEC [155](#)
 - RSH [155](#)
- TCP/IP stack interfaces
 - information [22](#)
- TCP/IP stacks
 - information [22](#)
- TCP/IP trace
 - accessing [62](#)

- TCP/IP trace (*continued*)
 - component
 - displaying [84](#)
 - scheduling [80](#)
 - introduction [62](#)
 - IP packet
 - displaying [66](#)
 - OSA packet
 - displaying [79](#)
 - TCPCONN command [85](#), [86](#), [89](#)
 - Telnet Server Configuration and Status workspace
 - about [27](#)
 - Telnet server ports
 - information [22](#)
 - Telnet servers
 - information [22](#)
 - TELSTAT command [22](#)
 - TESTPORT command
 - critical port monitoring [117](#)
 - hung listener detection [117](#)
 - port monitoring [117](#)
 - Tivoli
 - user groups [xvi](#)
 - Tivoli Software Information Center [xv](#)
 - TN3270 command [120](#)
 - TNPTSTAT command [22](#)
 - trace data, IP
 - saving [72](#), [73](#)
 - session packet data [72](#)
 - UDP session report [73](#)
 - TRACERTE command [106](#), [120](#)
 - traces
 - socket [127](#)
 - TRAP command (SNMP) [123](#)
 - typeface conventions [xvii](#)

U

- UDP session report
 - saving [73](#)
- user group, NetView [xvi](#)
- user groups
 - NetView [xvi](#)
 - Tivoli [xvi](#)
- user-defined views, IP [98](#), [100](#)

V

- variables, notation for [xvii](#)
- VARY TCPIIP DROP command [91](#)
- view nodes, sysplex [29](#)
- views, IP
 - bridge [99](#), [103](#)
 - host [99](#), [103](#)
 - hub [99](#), [103](#)
 - IP networks [98](#), [99](#)
 - IP subnetwork [99](#), [102](#)
 - router [99](#), [103](#)
 - switch [99](#), [103](#)
 - user-defined [98](#), [100](#)
- VIPA routes [48](#)
- VIPA Routes workspace
 - about [55](#)

VIPADYNAMIC TCPIP profile updates [45](#)
VIPAROUT command [48](#)

W

WALK command (SNMP) [109](#)
WINDOW command [49](#), [86](#)
WINDOWS command [23](#)
workspaces

- Distributed DVIPA Connection Routing
 - about [49](#)
- Distributed DVIPA Server Health
 - about [50](#)
- Distributed DVIPA Targets
 - about [51](#)
- DVIPA Connections
 - about [52](#)
- DVIPA Definition and Status
 - about [53](#)
- DVIPA Stack Summary
 - about [56](#)
- DVIPA Sysplex Distributors
 - about [54](#)
- HiperSockets Configuration and Status
 - about [23](#)
- NetView Applications
 - about [24](#)
- OSA Channels and Ports
 - about [25](#)
- Stack Configuration and Status
 - about [26](#)
- Telnet Server Configuration and Status
 - about [27](#)
- VIPA Routes
 - about [55](#)

Z

zERT [86](#)



SC27-2855-03

